Indian Statistical Institute Series



Rituparna Sen
Sourish Das

Computational Finance with R

Springer

Indian Statistical Institute Series

Editors-in-Chief

Abhay G. Bhatt, Indian Statistical Institute, New Delhi, India

Ayanendranath Basu, Indian Statistical Institute, Kolkata, India

B. V. Rajarama Bhat, Indian Statistical Institute, Bengaluru, India

Joydeb Chattopadhyay, Indian Statistical Institute, Kolkata, India

S. Ponnusamy, Indian Institute of Technology Madras, Chennai, India

Associate Editors

Arijit Chaudhuri, Indian Statistical Institute, Kolkata, India

Ashish Ghosh, Indian Statistical Institute, Kolkata, India

Atanu Biswas, Indian Statistical Institute, Kolkata, India

B. S. Daya Sagar, Indian Statistical Institute, Bengaluru, India

B. Sury, Indian Statistical Institute, Bengaluru, India

C. R. E. Raja, Indian Statistical Institute, Bengaluru, India

Mohan Delampady, Indian Statistical Institute, Bengaluru, India

Rituparna Sen, Indian Statistical Institute, Bengaluru, Karnataka, India

S. K. Neogy, Indian Statistical Institute, New Delhi, India

T. S. S. R. K. Rao, Indian Statistical Institute, Bengaluru, India

The **Indian Statistical Institute Series,** a Scopus-indexed series,publishes high-quality content in the domain of mathematical sciences, bio-mathematics, financial mathematics, pure and applied mathematics, operations research, applied statistics and computer science and applications with primary focus on mathematics and statistics. Editorial board comprises of active researchers from major centres of the Indian Statistical Institute.

Launched at the 125th birth Anniversary of P.C. Mahalanobis, the series will publish high-quality content in the form of textbooks, monographs, lecture notes, and contributed volumes. Literature in this series are peer-reviewed by global experts in their respective fields, and will appeal to a wide audience of students, researchers, educators, and professionals across mathematics, statistics and computer science disciplines.

Rituparna Sen · Sourish Das

Computational Finance with R

Springer

Rituparna Sen
Applied Statistics Unit
Indian Statistical Institute
Bengaluru, Karnataka, India

Sourish Das
Department of Mathematics
Chennai Mathematical Institute
Siruseri, Tamil Nadu, India

ISSN 2523-3114 ISSN 2523-3122 (electronic)
Indian Statistical Institute Series
ISBN 978-981-19-2007-3 ISBN 978-981-19-2008-0 (eBook)
https://doi.org/10.1007/978-981-19-2008-0

Mathematics Subject Classification: 60H35, 62G32, 62M10, 62P05, 65C30, 91B28, 91B30

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023
This work is subject to copyright. All rights are solely and exclusively licensed by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.
The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.
The publisher, the authors, and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Cover photo: Reprography & Photography Unit, Indian Statistical Institute, Kolkata, India

This Springer imprint is published by the registered company Springer Nature Singapore Pte Ltd.
The registered company address is: 152 Beach Road, #21-01/04 Gateway East, Singapore 189721, Singapore

Preface

This book will prepare readers to execute the quantitative and computational needs of the finance industry. The quantitative methods are explained in detail with examples from real financial problems like option pricing, risk management, portfolio selection, etc. Codes are provided in R programming language to execute the methods. Tables and figures, often with real data, illustrate the codes. References to related work are intended to aid the reader to pursue the areas of specific interest in further detail. The comprehensive background with economic, statistical, mathematical, and computational theory strengthens the understanding.

The primary audience is graduate students in statistics, mathematics, financial engineering and researchers interested in the application of statistics in finance. This book should also be accessible to advanced undergrads. Practitioners implementing quantitative analysis in the finance industry will benefit as well.

This book is based on the lecture notes that the authors have used at Chennai Mathematical Institute (CMI) and Indian Statistical Institute (ISI). In CMI, these classes are attended by master's students in applications of mathematics pursuing specialization in finance and analytics. In ISI, they are attended by master's students in statistics with finance specialization. The selection of topics has also been influenced by Dr. Das's experience in developing and delivering the financial risk management systems at SAS. The opportunity to discuss the use of statistical methods in the financial industry with practitioners and colleagues has helped our thinking about the statistical techniques and their applications.

This book should be accessible to graduate students and others of varying backgrounds. Exposure to undergraduate calculus, probability, and statistics is required. No prior knowledge of economics or finance is needed. This book is set up with examples that use real-life data with R implementation to visualize the theory. This will help the reader to develop the holistic understanding of the theory and implementation of financial statistics. More than 50 examples are distributed over the book, and their purpose varies from the simple application of formulas to extending the ideas to the next level.

This book covers all aspects of computation, namely numerical, simulation, and statistical, in a single volume. Numerical procedures, their advantages, applications

in finance and execution in R, are presented in Part I. Despite the advantages, it is not always possible to solve financial problems numerically. In such cases, simulation methods are very useful. These are presented in Part II. The final part concentrates on statistical methods. These enable the reader to train the economic models to real data, test the suitability of the model, and forecast important quantities like risk. The basic statistical topics of descriptive, inferential, multivariate, and time series analysis are presented with their applications in finance. A whole chapter is devoted to the quantification of risk and another to high-frequency data. Two chapters expose the reader to cutting-edge machine learning techniques. Two topics related to simulation, namely Bayesian Monte Carlo and resampling methods, are included in the third section as they require some basic statistical knowledge. Necessary theory of mathematical finance and extreme value, as well as an introduction to R, is presented in the Appendix.

Several people have influenced this book in various ways, and it is a pleasure to express our thanks to them here. We owe a particular debt to the former director of CMI, Rajeeva Karandikar. We have also benefited from our discussions with Ananya Lahiri, Swaminathan, B. V. Rao, and Sreejata Banerjee. We thank all the students who helped and found errors in the lecture notes from which this book has evolved.

Bengaluru, India Rituparna Sen
Chennai, India Sourish Das

Contents

Part I Numerical Methods

1 Preliminaries 3
1.1 Algorithms 3
1.1.1 Iterative Algorithms 4
1.1.2 Error 5
1.1.3 Stability 6
1.2 Interpolation of Functions 8
1.2.1 Polynomial Interpolation 8
1.2.2 Spline Approximation 9
1.3 Exercises 13

2 Vectors and Matrices 15
2.1 Dimension of a Vector Space 15
2.2 Solving Linear Equations 17
2.2.1 Condition Number 19
2.2.2 Iterative Procedures 20
2.3 Eigenvalues and Eigenvectors 21
2.4 Cholesky Factorization 23
2.5 Exercises 23

3 Solving Nonlinear Equations 25
3.1 Find Implied Volatility 25
3.2 Bisection Algorithm 28
3.3 Newton's Method 30
3.4 Secant Method 31
3.5 Fixed Point Method 32
3.6 R Functions for Root Finding 32
3.7 Exercises 32

4 Numerical Integration 33
4.1 Introduction 34
4.2 Newton–Cotes Formulas 35
4.2.1 Error 36
4.2.2 Trapezoidal Rule 37
4.2.3 Simpson's Rule 38
4.3 Gaussian Quadrature 41
4.4 Exercises 46

5 Numerical Differentiation 47
5.1 Introduction 47
5.2 Differentiation Via Interpolation 49
5.3 Method of Undetermined Coefficients 51
5.4 Exercises 52

6 Numerical Methods for PDE 53
6.1 Introduction 53
6.2 Finite Difference Methods 55
6.2.1 Explicit Scheme 57
6.2.2 Implicit Scheme 57
6.2.3 Crank–Nicolson Scheme 59
6.3 Solution to Black–Scholes PDE by Transformation to Heat Equation 59
6.4 Error 63
6.5 Exercises 64

7 Optimization 65
7.1 Linear Programming 66
7.2 Quadratic Programming 70
7.3 Other Methods 74
7.4 Exercises 75

Part II Simulation Methods

8 Monte Carlo Methods 79
8.1 Generating Sequence of Uniform[0, 1] Random Numbers 80
8.1.1 Linear Congruential Generator 81
8.1.2 Combining Generators 83
8.2 General Sampling Methods 84
8.2.1 Inverse Transform 85
8.2.2 Accept–Reject Method 88
8.3 Exercises 91

9 Lattice Models 93
9.1 Option Pricing with Binomial Lattices 93
9.2 Binomial Option Price as Approximate Solution to Black–Scholes Equation 96

9.3 Specific Models . . . 97
9.4 n-Period Binomial Option Pricing Formula . . . 98
9.5 Simulating from Binomial Asset Pricing Model . . . 101
9.6 Log-Normal Distribution as Limit of BAPM . . . 102
9.7 Pricing American Options with R . . . 103
9.8 Beyond Binomial . . . 105
9.9 Exercises . . . 106

10 Simulating Brownian Motion . . . 109
10.1 Generating Normal Random Variables and Vectors . . . 110
10.1.1 Inverse Transform Method . . . 110
10.1.2 Box–Muller Method . . . 111
10.1.3 Generating Multivariate Normal . . . 112
10.2 Simulation of Brownian Motion . . . 114
10.2.1 Random Walk Construction . . . 115
10.2.2 Brownian Bridge Construction . . . 117
10.2.3 Principal Components Construction . . . 118
10.3 Geometric Brownian Motion . . . 120
10.4 Example: Path-Dependent Options . . . 121
10.5 Multiple Dimensions . . . 123
10.6 Exercises . . . 125

11 Variance Reduction . . . 127
11.1 Control Variates . . . 127
11.1.1 Multiple Controls . . . 130
11.1.2 Nonlinear Controls and Delta Method . . . 131
11.2 Antithetic Variates . . . 132
11.3 Importance Sampling . . . 133
11.4 Stratified Sampling . . . 137
11.4.1 Optimal Allocation . . . 139
11.4.2 Stratifying a Poisson Process . . . 140
11.4.3 Stratifying the Binomial Lattice . . . 142
11.5 Exercises . . . 142

Part III Statistical Methods

12 Descriptive Statistics . . . 147
12.1 Financial Data . . . 148
12.2 Graphical Representation of Qualitative Data . . . 148
12.3 Numerical Representation of Quantitative Data . . . 150
12.4 Graphical Representation of Quantitative Data . . . 152
12.5 Assessing Normality . . . 154
12.6 Measures of Relative Position . . . 155
12.7 Stylized Facts . . . 157
12.7.1 Gain-Loss Asymmetry . . . 158
12.7.2 Aggregational Gaussianity . . . 158

12.7.3 Heavy Tails . . . 159
12.8 Exercises . . . 160

13 Inferential Statistics . . . 163
13.1 Estimation . . . 166
13.1.1 Methods of Estimation . . . 166
13.1.2 Properties of Estimators . . . 171
13.2 Interval Estimation . . . 172
13.3 Hypothesis Testing . . . 175
13.4 Exercises . . . 177

14 Bayesian Computation . . . 179
14.1 Introduction to Bayesian Inference . . . 179
14.2 Poisson Model for Count Data . . . 180
14.3 Bayesian Inference with Monte Carlo for Any Model . . . 187
14.4 Markov Chain Monte Carlo . . . 190
14.4.1 Markov Chain . . . 191
14.4.2 Markov Chain Monte Carlo Integration . . . 192
14.4.3 How MCMC Works? . . . 196
14.5 Exercises . . . 202

15 Resampling . . . 203
15.1 Jackknife . . . 204
15.2 Bootstrap . . . 206
15.2.1 Parametric Bootstrap . . . 207
15.2.2 Bootstrap of Portfolio Returns . . . 208
15.3 Cross-Validation . . . 210
15.4 Exercises . . . 211

16 Statistical Risk Analysis . . . 213
16.1 Value at Risk . . . 214
16.1.1 Gaussian Estimation . . . 215
16.1.2 Modified and Robust Estimation . . . 216
16.1.3 Historical Simulation . . . 218
16.1.4 EVT-Based Estimation . . . 219
16.2 Expected Shortfall . . . 221
16.3 Other Measures of Risk . . . 222
16.4 Ratios to Compare Portfolios . . . 224
16.5 Back-Testing VaR Models . . . 226
16.6 Exercises . . . 228

17 Supervised Learning . . . 229
17.1 Framework for Statistical Learning . . . 229
17.1.1 Bayesian Decision Theoretic Framework . . . 230
17.1.2 Learning with Empirical Risk Minimization . . . 232
17.2 Regression and Classification . . . 233
17.3 Supervised Learning and Generalized Linear Models . . . 234

17.4 Regression . . . 236
17.5 Logistic Regression . . . 241
17.5.1 Sigmoid Curve Behavior of Logistic Regression . . . 242
17.5.2 Non-monotonic Relation with Logistic Regression . . . 243
17.6 Discriminant Analysis . . . 245
17.7 Tree Structured Model . . . 251
17.7.1 Decision Tree . . . 252
17.7.2 Random Forest . . . 254
17.8 Exercises . . . 255

18 Multiple Variables . . . 257
18.1 Capital Market Line . . . 258
18.2 Systematic Risk: Beta . . . 261
18.2.1 CAPM and Beta Using R . . . 263
18.2.2 Security Market Line . . . 265
18.2.3 Can Beta Be Negative? . . . 266
18.2.4 Beta Hedging . . . 267
18.2.5 Achieving Target Beta . . . 268
18.2.6 Effect of Outlier on Beta . . . 268
18.3 Measuring Active Return: Alpha . . . 269
18.4 Pricing Theory and System of Linear Equations . . . 270
18.4.1 Factor Models and Risk Matrix . . . 271
18.4.2 Benefit of Diversification . . . 272
18.4.3 Solving Factor Model as System of Equation . . . 273
18.4.4 Stress Testing . . . 275
18.5 Summary . . . 277
18.6 Exercises . . . 278

19 Time Series . . . 279
19.1 Components . . . 279
19.2 Stationary Time Series . . . 282
19.2.1 White Noise . . . 282
19.2.2 Moving Average Models . . . 284
19.2.3 Autoregressive Models . . . 284
19.2.4 ARMA Models . . . 286
19.2.5 ARIMA Models . . . 287
19.3 Fitting ARIMA Model to Data . . . 287
19.4 Forecasting . . . 289
19.5 Exercises . . . 292

20 High-Frequency Data . . . 293
20.1 Infill Asymptotics in Geometric Brownian Motion . . . 295
20.2 Microstructure Noise . . . 298
20.3 Asynchronicity . . . 300
20.4 Exercises . . . 303

21 Unsupervised Learning 305
21.1 Introduction 305
21.2 Dimension Reduction 305
21.2.1 Multidimensional Scaling 306
21.2.2 Principal Component Analysis 310
21.3 Clustering 312
21.3.1 K-means Clustering 313
21.3.2 Hierarchical Clustering 315
21.4 Discussion 317
21.5 Exercises 318

Appendix A: Basics of Mathematical Finance 319

Appendix B: Introduction to R 329

Appendix C: Extreme Value Theory (EVT) in Finance 345

Bibliography 349

Index 353

About the Authors

Rituparna Sen is Associate Professor at the Applied Statistics Division, Indian Statistical Institute, Bangalore Centre, Karnataka, India. Earlier, she was Assistant Professor at the University of California at Davis from 2004–2011. With a Ph.D. in statistics from the University of Chicago, USA, she has been internationally recognized for her outstanding contributions to the applications of statistical theory and methods in finance and for her initiative and leadership in research, teaching, and mentoring in this area. She is editor of the journal Applied Stochastic Models in Business and Industry and on the editorial board of several other journals. Rituparna is an elected member of the International Statistical Institute. She has been awarded the Young Statistical Scientist Award by the International Indian Statistical Association in the Applications category and the Best Student Paper Award by the American Statistical Association section on the Statistical Computing and Women in Mathematical Sciences award by Technical University of Munich, Germany.

Sourish Das is Associate Professor of mathematics at Chennai Mathematical Institute (CMI), Tamil Nadu, India. At CMI, he teaches data science courses, including statistical finance using R and Python. His research interests are in Bayesian methodology, machine learning on big data in statistical finance, and environmental statistics. He did his Ph.D. in statistics from the University of Connecticut and postdoctoral work at Duke University, USA. He was awarded the UK Commonwealth Rutherford Fellowship to visit the University of Southampton, UK. He was awarded the Best Student Research Paper by the American Statistical Association section on Bayesian statistics.

Part I
Numerical Methods

Chapter 1
Preliminaries



We start with some preliminaries in computation to introduce the reader to the basic concepts and ideas. The first distinction between theoretical derivations and computation is that in the latter we are seeking a specific numerical solution as opposed to proof of existence. For that purpose we need to have a set of clear instructions that can be followed step by step to arrive at a solution, often an approximate one. Such a set of instructions is called *algorithm*. The following is an example of an algorithm.

Example 1.0.1. Suppose we want to find the median of a set of n numbers, we use the following algorithm.

1. Sort the numbers in order of magnitude.
2. Check if n is odd or even.
3. If n is odd, median is the value at the $(n+1)/2$-th position.
4. If n is even, median is the average of the values at the $n/2$-th and $n/2+1$-th positions.

Of course the first step of sorting will require another algorithm.

1.1 Algorithms

We now discuss some specific issues related to algorithms that will be required for the rest of the developments in the book.

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/978-981-19-2008-0_1.

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

R. Sen and S. Das, *Computational Finance with R*, Indian Statistical Institute Series,
https://doi.org/10.1007/978-981-19-2008-0_1_1

1.1.1 Iterative Algorithms

Example 1.0.1 is a direct method where the exact answer can be arrived at in a predetermined finite number of steps. However, many problems do not have such direct methods of solution. Consider the following example.

Example 1.1.1. A bond is a financial contract that pays a fixed amount, called the par value, at a fixed time, called term. Some bonds pay fixed amounts, called coupons, at intermediate fixed times. There is a purchase price for this contract at time zero. By the fundamental theorem of arbitrage pricing (See Appendix A), the purchase price should equal the discounted total price of the future payoffs. The latter quantity is also called the present value. The discount rate r is called the effective rate. The aim is to find the effective rate of a bond of term n years with par value P, purchase price A, and coupon payments $B_1, \ldots, B_n$ at the end of each year. Converting the total payoff to present value and equating that to the price, we have the following equation,

$$A = \sum_{k=1}^{n} B_k (1+r)^{-k} + P(1+r)^{-n}.$$

We are interested in finding the value of r. All other quantities in the equation can be observed from the market. If we denote $x = (1+r)^{-1}$, then this is a polynomial equation. $r = 1 - 1/x$ will be the effective rate if x is a solution of this equation. However, a fundamental theorem in algebra states that there is no direct method for computing the roots of a polynomial of degree 5 or greater. So we cannot solve this problem directly when n is greater than 4.

When direct methods are absent or too time-consuming, we turn to iterative methods. Let $x*$ denote the exact solution to the problem. In an iterative procedure, we start with an approximate solution x_0. Then we proceed with a sequence of solutions $x_1, x_2, \ldots$ where x_k is a function of $x_0, x_1, \ldots, x_{k-1}$ for every $k = 0, 1, 2, \ldots$. The function has to be constructed in such a way that this sequence converges to $x*$. The choices to make are the initial point x_0 as well as the iterative function.

Time complexity: The speed of convergence has to be taken under consideration when choosing a particular algorithm over another. The amount of time taken by an algorithm to execute as a function of the input is known as time complexity. This measure is used to rank algorithms that give equal level of precision.

At every step k of the algorithm, the difference between $x*$ and x_k is the error. We choose to stop either after a fixed number of steps, or when the error is smaller than a pre-specified value. Instead of specifying the error in terms of the solution, we can also specify it in terms of the problem. For example if the problem involves the value of a function f, the error can be specified as the distance between $f(x*)$ and $f(x_k)$.

Of course we do not know $x*$. As an approximation we use the difference between two successive values, that is x_k and x_{k-1}. In summary, an iterative procedure will take longer time than a direct procedure and will give only an approximate solution, the degree of which can be chosen by the user. Note that in order to obtain higher precision, a longer time will be required by the algorithm.

Example 1.1.2. Given $\epsilon = 0.005$, find an approximate solution to the equation $3x^2 - 16x + 5 = 0$ with error not greater than ϵ.
We are looking for the solution of $f(x) = 0$ for $f(x) = 3x^2 - 16x + 5$. Since $f(x)$ is continuous, we know from intermediate value theorem that if we can find two values a and b with $a < b$ such that $f(a)$ and $f(b)$ have opposite signs, then there will be at least one root of f between a and b. In this case, observe that $f(0) = 5 > 0$ and $f(1) = -4 < 0$. So there is a root of f in the interval $(0, 1)$. We now use the bisection algorithm to find the approximate root. The iterative step of the bisection algorithm goes as follows:

- Set c as the midpoint (hence the name bisect) of a and b.
- Check if the sign of f changes between a and c. If so, set b to c. Otherwise, set a to c.

Thus after every iteration we have an interval of length half of the previous one with a root lying inside the interval. Iterate until the length of the interval is less than the pre-specified tolerance ϵ. The following is the `R`-code that executes this algorithm and produces the solution `0.3359375` in `8` steps. In this case the exact solution is 1/3.

```
f<-function(x) 3*x^2-16*x+5
a<-0
b<-1
e<-0.005
while(b-a>e){
  c<-(a+b)/2
  if(f(a)*f(c)<0){b<-c}else{a<-c}}
sol=b
sol

## [1] 0.3359375
```

1.1.2 Error

There are various errors in numerical solutions. Below we discuss some sources of error that one needs to be aware of when using a computational procedure to arrive

at a solution. Some of these can be controlled to some extent, and we shall discuss this next. The basic lesson is that in most cases one cannot expect to obtain an exact solution as is possible in theoretical solutions.

Rounding Error: Computers can only perform integer arithmetic. So any number has to be represented in terms of integers and this involves approximation error. This process is called rounding, and it replaces a number with its nearest machine representable number. This is inherent to numerical solutions, and the only way to reduce it is to increase the number of bits used to represent a number.

Truncation Error: An algorithm or a mathematical procedure might involve the limit of an infinite process. For example evaluation of the exponential function $\exp(x) = \sum_{n=0}^{\infty} x^n/n!$ involves an infinite sum. But in practice we have to stop with $\sum_{n=0}^{N} x^n/n!$ for some finite N. In order to obtain the exact solution to the problem in Example 1.1.2 the iterative step needs to be executed infinite number of times. Obviously in practice these procedures need to be terminated in finite time. This introduces the truncation error. This type of error can be reduced by making algorithms more efficient and running them longer.

Example 1.1.3. Taylor expansion: Suppose a function f and its derivatives of successive orders $f', \ldots, f^{(n)}$ are continuous on $[a, b]$ and $f^{(n+1)}$ exists on (a, b). Let $x_0 \in [a, b]$. Then for every $x \in [a, b]$, there exists a ξ between x_0 and x with

$$f(x) = \sum_{k=0}^{n} \frac{f^{(k)}(x_0)}{k!}(x - x_0)^k + R_n(x), \tag{1.1}$$

where $R_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!}(x - x_0)^{(n+1)}$ is the remainder. This is the truncation error.

Discretization Error: The third source of error arises from discrete approximation to continuous systems. For example we may be interested in deriving the numerical solution of a differential equation $x' = f(x)$ with a given initial condition. We approximate this differential equation by a difference equation $(x(t + h) - x(t))/h = f(x(t))$. This discretization step leads to error in the solution. We shall discuss this kind of error in detail in Chap. 6 where we deal with numerical solutions of PDEs.

1.1.3 Stability

The other aspect of an algorithm that we need to consider is stability. An algorithm is stable if small changes in the input lead to small changes in the output, otherwise it is unstable.

A related concept is the condition of the problem. The latter is not something that we can address as it is inherent to the problem itself. But it is necessary to be aware of the situation. A problem is ill-conditioned if a small perturbation of the problem results in a large change in the solution. Such problems are very common in finance in the form of inverse problem. See Hein and Hofmann (2003) for an example in the context of recovering the implied volatility from option prices observed with noise.

Example 1.1.4. Suppose we want to solve the following linear equations simultaneously

$$\begin{aligned} x - y &= 1, \\ x - 0.99y &= 1.01. \end{aligned}$$

The solution is $x = 2$, $y = 1$. Now we change the value 1.01 to 1.02. The solution to the new problem is $x = 3$, $y = 2$. Thus a change of $0.01/1.01 \approx 1\%$ in the statement of the problem leads to a 50% change in case of x and 100% in case of y. Thus, this problem is ill-conditioned.

For problems related to solution of simultaneous linear equations $Ax = b$, the condition of the problem is related to the condition number $\kappa(A)$ of the matrix A. The definition of $\kappa(A)$ relative to a norm $\|\cdot\|$ is $\kappa(A) = \|A\|\|A^{-1}\|$. A matrix A is ill-conditioned if $\kappa(A)$ is much larger than one. Some common matrix norms are

$$\begin{aligned} \|A\|_1 &= \max_{1 \le j \le n} \sum_{i=1}^{n} |a_{ij}|, \\ \|A\|_2 &= \sqrt{\rho(A^T A)}, \end{aligned}$$

where ρ denotes the maximum eigenvalue. We shall discuss these concepts in detail in Chap. 2.

In general, the condition number is defined as the relative error in output divided by the relative error in input. So if we are evaluating a function f at x_0 with a perturbation of size ϵ, then the condition number is

$$\left| \frac{f(x_0 + \epsilon) - f(x_0)}{f(x_0)} \frac{x_0}{\epsilon} \right| \approx \left| \frac{x_0 f'(x_0)}{f(x_0)} \right|.$$

Now we are ready to discuss the stability issue of an algorithm. An algorithm is stable if all the steps involve well-conditioned problems.

Example 1.1.5. Suppose the problem is to evaluate $f(x) = \sqrt{1+x} - 1$ near $x = 0$. We want to verify if this is a well-conditioned problem. The steps involved are $x_1 = 1 + x$, $x_2 = \sqrt{x_1}$, $x_3 = x_2 - 1$. Note that x_2 is close to 1 and the condition number of the third step is $|x_2/(x_2 - 1)|$ which is going to infinity. Hence the third step is ill-conditioned and the algorithm is unstable.

1.2 Interpolation of Functions

Often there is an underlying continuous function that we are interested in, but the observations are on a discrete set of points. Take, for example, the yield curve. An yield curve represents the relationship between spot rates of zero-coupon securities and their term to maturity. We need the value of this function at all maturities in order to discount cash flows appropriately. However, in practice the interest rates are available for only a finite number of maturities. In such situations, we need to interpolate to obtain approximate values of the function at all points.

1.2.1 Polynomial Interpolation

The most common method of interpolation is polynomial interpolation. Given any continuous function f on an interval $[a, b]$, there exist polynomials P that can be as close to f as desired. Taylor polynomials of Example 1.1.3 come to mind as our first choice as they agree closely with a given function at a specific point. But these do not provide a good approximation over an entire interval. One set of polynomials that satisfies this condition is the Lagrange polynomials.

Suppose $x_i \in [a, b]$, $i = 0, 1, \ldots, n$, are distinct points in $[a, b]$. The Lagrange polynomial p_n is a polynomial of degree $\leq n$ such that $p_n(x_i) = f(x_i)$, $i = 0, 1, \ldots, n$. p_n can be constructed explicitly as

$$p_n(x) = \sum_{i=0}^{n} L_i(x) f(x_i), \tag{1.2}$$

where L_i is a polynomial of degree n satisfying $L_i(x_j) = 0$, $j \neq i$, $L_i(x_i) = 1$. This results in

$$L_i(x) = \prod_{j \neq i} \left(\frac{x - x_j}{x_i - x_j} \right), \quad i = 0, 1, \ldots, n. \tag{1.3}$$

p_n is called linear interpolation if $n = 1$ and quadratic interpolation if $n = 2$. Suppose f is $n + 1$ times differentiable on $[a, b]$. Then it holds that

$$f(x) = p_n(x) + \frac{f^{(n+1)}(\xi)}{(n+1)!}(x - x_0)\dots(x - x_n), \quad \text{where} \quad \xi = \xi(x) \in (a, b). \tag{1.4}$$

This gives the magnitude of the approximation error at $x = \alpha$ as

$$f(\alpha) - p_n(\alpha) = \frac{f^{(n+1)}(\xi)}{(n+1)!}(\alpha - x_0)\dots(\alpha - x_n),$$

where $\xi = \xi(\alpha) \in (a, b)$.

Example 1.2.1. Find the Lagrange polynomial for the following points $(x, f(x)) : (1, 0), (-1, -3)$ and $(2, -4)$. Here $n = 2$.

$L_0(x) = \prod_{j\neq 0}\left(\frac{x-x_j}{x_i-x_j}\right) = \left(\frac{x+1}{1+1}\frac{x-2}{1-2}\right) = -\frac{1}{2}(x+1)(x-2)$

$L_1(x) = \prod_{j\neq 1}\left(\frac{x-x_j}{x_i-x_j}\right) = \left(\frac{x-1}{-1-1}\frac{x-2}{-1-2}\right) = \frac{1}{6}(x-1)(x-2)$

$L_2(x) = \prod_{j\neq 2}\left(\frac{x-x_j}{x_i-x_j}\right) = \left(\frac{x-1}{2-1}\frac{x+1}{2+1}\right) = \frac{1}{3}(x-1)(x+1)$

$$\begin{aligned} p_2(x) &= L_0 \times 0 + L_1 \times (-3) + L_2 \times (-4) \\ &= -\frac{1}{2}(x-1)(x-2) - \frac{4}{3}(x-1)(x+1) \\ &= -\frac{1}{6}(x-1)(3x - 6 + 8x + 8) \\ &= -\frac{11}{6}x^2 + \frac{3}{2}x + \frac{1}{3}. \end{aligned}$$

1.2.2 Spline Approximation

A drawback of polynomial interpolation is that p_n tends to oscillate wildly when n is large. This implies that $p_n(x)$ may be a poor approximation to $f(x)$ if x is not close to the interpolating points. An alternative method is spline interpolation where the interval $[a, b]$ is divided into a set of subintervals $[x_i, x_{i+1}]$, $i = 0, 1, \dots, n - 1$, and on each subinterval a different polynomial is constructed to approximate the function f.

The simplest spline to approximate a function f is the linear spline P which is a piecewise linear function. In particular, P agrees with f at each node x_i and is a straight line in each interval $[x_i, x_{i+1}]$. Linear splines are easy to construct but are not differentiable at the nodes $x_1, x_2, \dots, x_{n-1}$.

The differentiability issue can be resolved by considering cubic splines. A function S is called a natural cubic spline if it is the combination of n cubic polynomials S_i such that they agree with the given function on the nodes and their first and second derivatives match. Mathematically, S needs to satisfy the following conditions.

1. $S(x) = S_i(x)$ for x in $[x_i, x_{i+1}]$ and $i = 0, 1, \ldots, n-1$
2. $S_i(x_i) = f(x_i)$ and $S_i(x_{i+1}) = f(x_{i+1})$ for $i = 0, 1, \ldots, n-1$
3. $S'_{i+1}(x_{i+1}) = S'_i(x_{i+1})$ for $i = 0, 1, \ldots, n-2$
4. $S''_{i+1}(x_{i+1}) = S''_i(x_{i+1})$ for $i = 0, 1, \ldots, n-2$
5. $S''(x_0) = S''(x_n) = 0$.

Example 1.2.2. Construct a cubic spline interpolation for $(-1, 1), (0, 2), (1, -1)$.
Here $n = 2, x_0 = -1, x_1 = 0, x_2 = 1, f(-1) = 1, f(0) = 2, f(1) = -1$.
Let

$$S(x) = \begin{cases} S_0(x) = ax^3 + bx^2 + cx + d & \text{for} \quad x \in [-1, 0] \\ S_1(x) = ex^3 + fx^2 + gx + h & \text{for} \quad x \in [0, 1] \end{cases}$$

We have, from the given conditions on cubic spline,

$$S_0(-1) = 1, S_0(0) = 2, S_1(0) = 2, S_1(1) = -1,$$
$$S'_1(0) = S'_0(0),$$
$$S''_1(0) = S''_0(0), S''(-1) = S''(1) = 0.$$

Thus we have the following system of equations.

$$\begin{aligned} -a + b - c + d &= 1 \\ d &= 2 \\ h &= 2 \\ e + f + g + h &= -1 \\ g &= c \\ 2f &= 2b \\ 6e + 2f &= 0 \\ -6a + 2b &= 0. \end{aligned}$$

Solving these we get $a = -1, b = -3, c = -1, d = 2, e = 1, f = -3, g = -1, h = 2$. Hence the spline approximation to f is

$$S(x) = \begin{cases} -x^3 - 3x^2 - x + 2 & \text{for} \quad x \in [-1, 0] \\ x^3 - 3x^2 - x + 2 & \text{for} \quad x \in [0, 1] \end{cases}$$

It can be shown that S_i is given by

$$S_i(x) = \frac{c_i}{6h_i}(x_{i+1} - x)^3 + \frac{c_{i+1}}{6h_i}(x - x_i)^3 + p_i(x_{i+1} - x) + q_i(x - x_i), \quad (1.5)$$

$$\text{where} \quad p_i = \left(\frac{f(x_i)}{h_i} - \frac{c_i h_i}{6}\right), \quad q_i = \left(\frac{f(x_{i+1})}{h_i} - \frac{c_{i+1} h_i}{6}\right)$$

and $c_1, \ldots, c_{n-1}$ satisfy the linear equations:
$h_{i-1}c_{i-1} + 2(h_{i-1} + h_i)c_i + h_i c_{i+1} = u_i$,
where $u_i = 6(d_i - d_{i-1})$, $d_i = (f(x_{i+1}) - f(x_i))/h_i$, $h_i = x_{i+1} - x_i$ for $i = 1, 2, \ldots, n-1$. The R code to derive the constants for the Example 1.2.2 is given below.

```
x = c(-1,0,1)
f = c(1,2,-1)
h = c(0,0)
d = h
p = h
q = h
c = c(0,0,0)
for(i in 1:2){
  h[i] = x[i+1]-x[i]
  d[i] = (f[i+1]-f[i])/h[i]
}
u = 3*(d[2]-d[1])
c[2] = u/(h[2]+h[1])
for(i in 1:2){
  p[i] = f[i]/h[i]-c[i]*h[i]/6
  q[i] = f[i+1]/h[i]-c[i+1]*h[i]/6
}
```

This gives $h = (1, 1)$, $c = (0, -6, 0)$, $p = (1, 3)$, $q = (3, -1)$. Putting these in the form of Eq. 1.5, we get the same solution as before.

Below we present an R code to compare the polynomial and spline methods using built-in functions from the R library polynorm. For values of $x = 1, 2, \ldots, 9$, we generate $f(x)$ as random normal observations. The points and the two interpolated curves are presented in Fig. 1.1. Note that the polynomial fluctuates wildly in between the observed values.

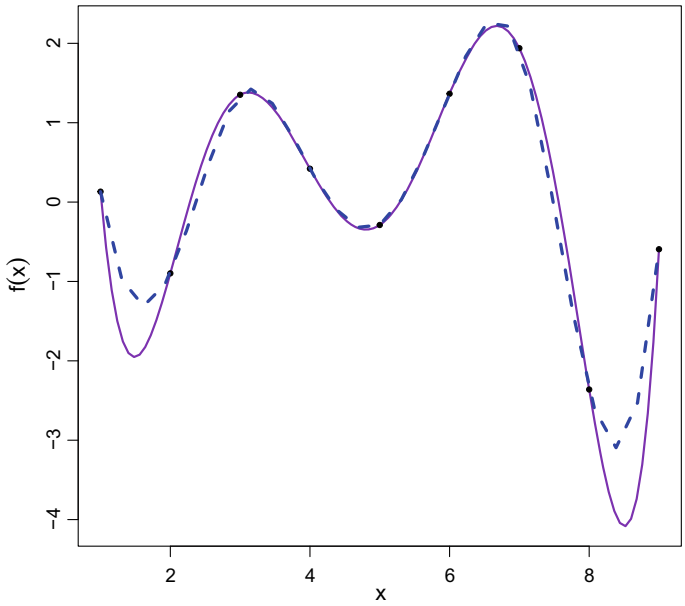


Fig. 1.1 Observed points of a function with fitted polynomial (purple) and spline (blue) approximation

```
require(graphics)
n = 9
x = 1:n
set.seed(1010)
y = rnorm(n)
## polynomial fit
library(polynom)
g = poly.calc(x,y)
f = as.function(g)
curve(f(x),1,9,lwd=2,col='purple'
      ,ylab = expression(f(x)))
points(x, y, ylim=c(-3,3),pch=20)
## Spline approximation
lines(spline(x, y), col="blue",lwd=3,lty=2)
```

1.3 Exercises

1. Set up the equation for the effective rate of a three-year bond with par value ₹1000 that is selling for ₹900 and pays ₹10 in coupons annually.
2. Use the bisection algorithm to solve the equation obtained in problem 1.
3. Find the Taylor expansion of the function $\log(1+x)$ around $x = 0$. Given that $\log(1.5) = 0.4054651$, find the truncation error for a Taylor expansion of order 3.
4. Verify the solution of Example 1.2.2 from Eq. 1.5 and the constants obtained from the R code.
5. Find the cubic spline approximation of a function that takes values $1, 2, -1, 1$ at the points $0, 1, 3$ and 6 respectively.

Chapter 2
Vectors and Matrices



In financial analysis we are often dealing with multiple securities, like stocks, bonds, options, etc. There are also multiple market scenarios to consider like recession, boom, growth, stable state, etc. Corresponding to each of these, there are different prices that we need to consider. Relations between these quantities can be explored conveniently using vectors, matrices, and linear algebra. So we recall some linear algebra in this chapter and discuss the associated numerical methods.

2.1 Dimension of a Vector Space

The first thing that we shall consider is the space spanned by k vectors $\{u_1, \ldots, u_k\}$ in E. This space is the collection of all linear combinations $\sum_{i=1}^{k} a_i u_i$ where $a_i \in \mathbb{R}$. We are interested in the dimension of this vector space, that is the maximum number of linearly independent vectors in the collection $\{u_1, \ldots, u_k\}$.

Consider a market that has n possible scenarios. For a given security, the payoffs in these scenarios can be written as a vector in $E = \mathbb{R}^n$. Suppose we can find a set A of securities such that the dimension of the vector space spanned by their payoffs is n. Then these payoffs form a basis of E and any vector in E can be expressed as a linear combination of these. Thus, given a new asset with payoff vector x, one can get the exact same payoffs by investing in the assets in A with properly chosen proportion of capital invested in each. In financial terms, we say that we can build a replicating portfolio for the new asset using the given set of securities.

The most simple basis in E is the canonical basis denoted $\{e_1, \ldots, e_n\}$ where e_i has its i-th component as one and all others as zero. These e_i are known as Arrow–Debreu securities or pure contingent securities and pay one unit in a given state of nature and zero otherwise. For any vector $b = (b_1, \ldots, b_n)^T \in E$ that denotes the payoff of a portfolio in the n possible states of nature, we can write $b = \sum_{i=1}^{n} b_i e_i$.

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/978-981-19-2008-0_2.

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

R. Sen and S. Das, *Computational Finance with R*, Indian Statistical Institute Series,
https://doi.org/10.1007/978-981-19-2008-0_2

Now suppose we have a set of k securities such that the dimension of the vector space spanned by their payoffs $(u_1, \ldots, u_k)$ is $r < n$. A new asset with payoff b can be replicated in the original securities iff b belongs to the space spanned by $(u_1, \ldots, u_k)$. This happens iff dimension of the vector space spanned by $(u_1, \ldots, u_k, b)$ is also r.

The above discussion demonstrates the various situations in finance, where we want to find the dimension of the space spanned by a set of vectors $\{u_1, \ldots, u_k\}$ in $E = \mathbb{R}^n$. One way to do that is to construct the $n \times k$ matrix $A = (u_1 u_2 \ldots u_k)$ with the given vectors as columns. The dimension of the space spanned by the vectors equals the rank of the matrix A. One way of finding the rank of a matrix is through **QR decomposition**. We can decompose a matrix A into product of two matrices as $A = QR$, where Q is an orthogonal matrix and R is an upper triangular matrix. Such representation is known as QR decomposition.

There are several methods for actually computing the QR decomposition. One such method is the **Gram–Schmidt process**. This is described as follows. Let $A = [u_1 \ldots u_k]$. We construct $Q = [q_1 \cdots q_n]$ by the following algorithm that is terminated when $a_j = 0$.

$$\begin{aligned} a_1 &= u_1 & q_1 &= a_1/\|a_1\| \\ a_2 &= u_2 - <u_2, q_1> q_1 & q_2 &= a_2/\|a_2\| \\ &\vdots \\ a_i &= u_i - <u_i, q_1> q_1 - \ldots <u_i, q_{i-1}> q_{i-1} & q_i &= a_i/\|a_i\| \\ &\vdots \end{aligned}$$

Here $<x, y>$ denotes the vector product of x and y and $|x| = \sqrt{<x, x>}$ denotes the norm of x. If the rank r of A is less than n, then a_j will be zero for $j = r + 1$. Thus the rank of A is the largest j for which a_j is nonzero. We input with any set of vectors that are orthogonal to $q_1, \ldots q_n$ for the remaining columns of Q. The triangular matrix R has elements $R_{i,j} = <u_j, q_i>$ for $j \geq i$ and zero otherwise.

The function `qr` in `R` performs the QR decomposition. Below is an `R` code that generates a 4×3 matrix with inputs as random normal variables and performs its QR decomposition to find the rank.

```
#define a matrix for this example
set.seed(1010)
M = matrix(data = rnorm(12), ncol = 3)
round(M,2)
```

```
##       [,1]  [,2]  [,3]
## [1,]  0.13 -0.29 -0.59
## [2,] -0.90  1.36 -0.54
## [3,]  1.35  1.94 -0.20
## [4,]  0.42 -2.36 -0.83
```

```
#run the function qr()
```

```
qr(M)$rank

## [1] 3
```

Example 2.1.1. Let us consider three assets in a four scenario market with payoffs given by the following matrix

$$\begin{pmatrix} u_1 & u_2 & u_3 \\ -2 & -1 & 0 \\ 1 & 3 & 2 \\ -1 & -1 & 1 \\ 0 & 6 & -5 \end{pmatrix}$$

where the (i, j)-th element denotes the payoff of the j-th asset in the i-th scenario. Can you build a portfolio using these that will have payoff vector $b = (1\,2\,3\,4)^T$?

```
u = matrix(c(-2,1,-1,0,-1,3, -1,6, 0,2,1,-5), nrow=4)
qr(u)$rank

## [1] 3

b = c(1,2,3,4)
ub = cbind(u,b)
qr(ub)$rank

## [1] 4
```

Running the above code we find that the rank of u is 3 and that of $(u\ b)$ is 4. So b does not belong to the space spanned by the columns of A. This implies that we cannot build such a portfolio with payoff vector b.

2.2 Solving Linear Equations

Consider the same situation as in Example 2.1.1 but with a new payoff vector $b = (3\ 3\ 2\ 1)^T$. Running the code in the example, we get that the rank of $(A\ b)$ is now 3. This means we can indeed build a portfolio with the three given assets to yield a payoff b. How do we find the weights in the portfolio? In terms of matrix operations, we need to find a 3×1 matrix x such that $Ax = b$. This is a system of linear equations.

When $b < -Ax$ with only A and b given, vector x is the solution of the linear equation system. Here A is an $n \times k$ matrix and b is an $n \times 1$ vector, that is, we have n equations in k unknowns. In the above example we have 4 equations in 3 unknowns.

A solution exists iff rank of A is same as rank of $[A : b]$. The solution is unique iff rank of A is m. Without loss of generality we can assume A to be full column rank. Otherwise, some columns of A can be written as linear combinations of others and we can remove these without changing the space spanned by the columns. In practice, we can put zero in the corresponding positions of the solution x.

If in addition, A is a square matrix then the solution can be obtained in R with the following command.

```
solve(A, b)
```

In case A is not square, we can use the QR decomposition described earlier. The algorithm goes as follows:

1. Factor A as $A = QR$
2. Compute $y = Q^T b$
3. Solve $Rx = y$ by back substitution.

This works, because Q is orthogonal and hence $Q^{-1} = Q^T$. Note, that by the assumption of A being full column rank R is a square matrix. Now for a square upper triangular matrix R, an equation of the form $Rx = y$ can be solved easily by back substitution. This means start from the bottom row. That involves exactly one unknown. Find the value and plug in to the next higher row, which adds one more unknown, and so on.

Now we get back to the problem of finding the portfolio weights in Example 2.1.1 with payoff vector $b = (3\ 3\ 2\ 1)^T$. The can be solved by the above procedure using the following R code.

```
b = c(3,3,2,1)
ub = cbind(u,b)
qr(ub)$rank

## [1] 3

qr.solve(u,b)

## [1] -2  1  1
```

Running the above code we find the weights as $(-2\ 1\ 1)$. Other decomposition methods like LU factorization can also be used for solving a system of linear equations. The **LU decomposition** involves writing a matrix $A = LU$ where L is lower triangular and U is upper triangular. Once this is done, the system of equations $Ax = b$ can be solved as follows.

$$
\begin{aligned}
Ax &= b \\
LUx &= b \\
Ly &= b \quad \text{use forward substitution to obtain } y \\
Ux &= y \quad \text{use backward substitution to obtain } x.
\end{aligned}
$$

2.2.1 Condition Number

Solving linear systems may be a difficult task with certain matrices. In order to see why, we need to consider the condition number of the matrix. Before doing so, we must introduce preliminary concepts related to the norms of vectors and matrices. Intuitively, the norm measures the overall size of the elements in a vector or matrix.

The norm $\|x\|$ of a vector x in $\mathbb{R}^n$ is a function from $\mathbb{R}^n$ to $\mathbb{R}$ such that

- $\|x\| \geq 0$ for every x with equality only when $x = 0$.
- $\|cx\| = |c|\|x\|$ for all $c \in \mathbb{R}$ and all $x \in \mathbb{R}^n$.
- $\|x + y\| \leq \|x\| + \|y\|$ for all $x, y \in \mathbb{R}^n$.

The most common vector norms are

- $\|x\|_1 = \sum_{i=1}^{n} \mid x_i \mid$, known as the L_1 norm.
- $\|x\|_2 = (\sum_{i=1}^{n} x_i^2)^{1/2}$, known as the L_2 or Euclidean norm.
- $\|x\|_\infty = \max \mid x_i \mid$, known as the L_∞ norm.

Similarly we define norm of matrices such that they follow the same properties. For an $m \times n$ matrix A, the common norms are

- $\|A\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^{m} \mid a_{ij} \mid$.
- $\|A\|_2 = \max \frac{\|Ax\|_2}{\|x\|_2}$, known as the spectral norm.
- $\|A\|_F = (\sum_{i=1}^{m} \sum_{j=1}^{n} a_{ij}^2)^{1/2}$, known as the Frobenius norm.
- $\|A\|_\infty = \max_{1 \leq i \leq m} \sum_{j=1}^{n} \mid a_{ij} \mid$.

Now we are ready to analyze the error in solving a system of linear equations. We use the spectral norm for matrices and Euclidean norm for vectors from now on without explicitly writing it, but most of the results can be generalized to other norms. From the definition of spectral norm,

$$\|Ax\|_2 \leq \|A\|_2 \|x\|_2.$$

Suppose we are trying to solve the system $Ax = b$ and there is a small perturbation in b, say δb. The question of interest is the amount of error δx this introduces in the solution x. The new equation can be written as $A(x + \delta x) = b + \delta b$. Thus, when A is non-singular, we have

$$Ax = b \text{ and } A\delta x = \delta b,$$
$$\|b\| = \|Ax\| \leq \|A\|\|x\| \text{ and } \|\delta x\| = \|A^{-1}\delta b\| \leq \|A^{-1}\|\|\delta b\|.$$

Thus

$$\frac{\|\delta x\|}{\|x\|} \leq \|A\|\|A^{-1}\| \frac{\|\delta b\|}{\|b\|}$$

The condition number $K(A) = \|A\| \cdot \|A^{-1}\|$ gives an upper bound on the ratio of the relative error $\|\delta x\|/\|x\|$ in the solution to the relative perturbation $\|\delta b\|/\|b\|$. Generally speaking, the higher the condition number, the more difficult it is to solve a linear system. If a problem is ill-conditioned, we attempt to find a transformation, possibly nonlinear, so that the transformed problem is well-conditioned.

Note that $\|A^{-1}\| = \left(\min \frac{(\|Ax\|_2)}{\|x\|_2}\right)^{-1}$. Thus for non-square matrix A, the condition number is defined as

$$\kappa(A) = \left(\max \frac{(\|Ax\|_2)}{\|x\|_2}\right)\left(\min \frac{(\|Ax\|_2)}{\|x\|_2}\right)^{-1}. \tag{2.1}$$

For the matrix A in Example 2.1.1, we can find the condition number in `R` using the command `kappa(A, exact=TRUE)` to be 4.297, which is not too large. So the problem is well-conditioned.

2.2.2 *Iterative Procedures*

The decomposition methods of solving linear equations are computationally costly. For example both QR and LU decomposition require n^3 computations for a square matrix of order n. For large n, this becomes very time consuming. An alternative approach is to follow an iterative procedure. Here we proceed as follows. The equation to solve is $Ax = b$. We write A as $M + N$ where M is invertible and the inverse can be obtained with little computation. Then the equation translates to

$$Mx = -Nx + b \quad \text{or equivalently} \quad x = -M^{-1}Nx + M^{-1}b.$$

If we denote $-M^{-1}N$ by C and $M^{-1}b$ by d, then we have

$$x = Cx + d.$$

We solve this iteratively by putting

$$x^{(k)} = Cx^{(k-1)} + d$$

and stop when $\|x^{(k)} - x^{(k-1)}\|$ is smaller than a pre-specified level.

We state the following result without proof.

> The iterative procedure above converges iff the determinant of C is less than one.

One popular choice of M is the diagonal part of A. That is, $M = \text{diag}(A)$. This is called the Gauss–Jacobi algorithm. Element-wise, this translates to

$$
\begin{aligned}
M_{ii} &= A_{ii} \\
M_{ij} &= 0 \quad \text{for} \quad i \neq j \\
N_{ii} &= 0 \\
N_{ij} &= A_{ij} \quad \text{for} \quad i \neq j \\
C_{ii} &= 0 \\
C_{ij} &= -A_{ij}/A_{ii} \quad \text{for} \quad i \neq j \\
d_i &= b_i/A_{ii}.
\end{aligned}
$$

This gives

$$x_i^{(k)} = -\sum_{j \neq i} A_{ij}/A_{ii} x_j^{(k-1)} + b_i/A_{ii}. \tag{2.2}$$

2.3 Eigenvalues and Eigenvectors

Eigenvalues of a matrix need to be calculated in several applications, particularly in statistical analysis of financial data that we shall describe in part III. In particular, we need them for principal component analysis and factor analysis in multivariate methods. In multivariate time series, testing for cointegration requires eigenvalues. The condition number used in Eq. (2.1) is the ratio of the largest to the smallest eigenvalue.

For a square matrix A of order n, a scalar λ is an eigenvalue with corresponding eigenvector b, which is a vector, if $Ab = \lambda b$. In order to find the eigenvalues, we solve the equation $\det(A - \lambda I) = 0$ where det stands for the determinant. This is a polynomial equation of order n and has to be solved numerically as described in Chap. 3. Thus we obtain n roots of the equation. For each root λ, we find the eigenvector b by solving $Ab = \lambda b$ or $(A - \lambda I)b = 0$.

The solution of the polynomial is time consuming and not very efficient. So we look for alternative methods. For example, often we are interested only in the highest eigenvalue and the corresponding eigenvector. This is the case of principal component or factor analysis. We employ the following procedure called the power method. Take a random vector u_0 and compute the following unit vectors $u_1, u_2, \ldots,$ iteratively

$$x_1 = Au_0, u_1 = \frac{x_1}{\|x_1\|_2}, x_2 = Au_1, u_2 = \frac{x_2}{\|x_2\|_2}, \ldots, x_i = Au_{i-1}, u_i = \frac{x_i}{\|x_i\|_2}, \ldots$$

This sequence will almost always converge to an eigenvector $u*$ corresponding to the highest eigenvalue of A. The eigenvalue is then computed as $\lambda* = u*^T Au*$.

Example 2.3.1. Let us consider the matrix

$$A = \begin{pmatrix} 1 & 0 & 2 \\ 0 & 3 & 1 \\ 2 & 1 & 5 \end{pmatrix}$$

We start with the vector $u = (2\ 3\ 4)^T$ and use the following R code iteratively

```
x = A%*%u
u = x/sqrt(sum(x^2))
```

After 5 iterations, the distance between the actual eigenvector corresponding to the highest eigenvalue of A and the iterative value of u in the L_∞ norm is 0.005. The discrepancy in the eigenvalue is 1.1×10^{-4}.

Another quantity of interest in applications is the determinant, which is the product of eigenvalues. For this we can appeal to the QR decomposition described in Sect. 2.1. Note that the determinant of the product of two matrices is the product of determinants. In this case the given matrix A is the product of two matrices Q and R. Also, the determinant of an orthogonal matrix is 1 and that of a triangular matrix is the product of its diagonal elements. Since Q is orthogonal, the determinant of A is the product of the diagonal elements of the triangular matrix R.

Next we describe an iterative algorithm to obtain all eigenvalues using the QR factorization of matrix A.

Algorithm 2.1

1. Start with $A_0 = A$ and perform the QR decomposition $A_0 = Q_0 R_0$
2. Define $A_1 = R_0 Q_0$ and perform a QR decomposition $A_1 = Q_1 R_1$
3. Iterate at the k-th step as $A_k = R_{k-1} Q_{k-1}$ and perform a QR decomposition $A_k = Q_k R_k$
4. Stop when A_k is approximately triangular.

It can be shown that under fairly general conditions (see Charles et al. (1996) for details), A_k converges to a triangular matrix $A*$. Note that

$$A_k = Q_{k-1}^T Q_{k-1} R_{k-1} Q_{k-1} = Q_{k-1}^T A_{k-1} Q_{k-1}.$$

Since Q_{k-1} is orthogonal, the eigenvalues of A_k and A_{k-1} are same. The eigenvalues of the triangular matrix $A*$ are the diagonal elements. Thus we obtain the eigenvalues of A.

In the following R code, the function `eigen(Sm)` calculates the eigenvalues and eigenvectors of a symmetric matrix *Sm* and saves it in the list `ev`. `ev $val` is the vector of eigenvalues of *Sm* and `ev $ vec` is the matrix of corresponding

eigenvectors. For large matrices, it is better to avoid computing the eigenvectors if they are not needed by using the option `only.values=TRUE`

```
ev = eigen(Sm)
evals = eigen(Sm, only.values = TRUE)$values
```

2.4 Cholesky Factorization

A representation of a symmetric positive definite matrix Σ as AA^T with A lower triangular is a Cholesky factorization of Σ. We shall use this method in Chap. 10 for generating multivariate normal random vectors. Considering a $d \times d$ covariance matrix Σ, we need to solve

$$\begin{pmatrix} A_{11} & & & \\ A_{21} & A_{22} & & \\ \vdots & \vdots & \ddots & \\ A_{d1} & A_{d2} & \dots & A_{dd} \end{pmatrix} \begin{pmatrix} A_{11} & A_{21} & \dots & A_{d1} \\ & A_{22} & \dots & A_{d2} \\ & & \ddots & \vdots \\ & & & A_{dd} \end{pmatrix} = \Sigma. \tag{2.3}$$

Traversing the Σ_{ij} by looping over $j = 1, \dots, d$ and then $i = j, \dots, d$ produces a set of equations. Exactly one new entry of the A matrix appears in each equation. Thus we can solve for the individual entries sequentially to obtain the Cholesky factorization explicitly as follows:

$$A_{ij} = (\Sigma_{ij} - \sum_{k=1}^{j} A_{ik}A_{jk})/A_{jj}, \quad j < i,$$

$$A_{ii} = \sqrt{\Sigma_{ii} - \sum_{k=1}^{i-1} A_{ik}^2}.$$

Note that one problem with Cholesky factorization is that if one of the A_{ii} is very small, then the division in the first step becomes unstable leading to floating point error. In such cases, QR decomposition is preferred, even though it takes more time.

2.5 Exercises

1. Consider the same situation as in Example 2.1.1 but with a new payoff vector $b = c(1\ 4\ 1\ 1)^T$. Determine if one can build a portfolio using the three given securities that will have payoff b. If so, find the portfolio weights.

2. For an iterative procedure of solving a system of linear equations, take M to be the lower triangular part of A along with the diagonal. Show that the iterative step in Eq. (2.2) simplifies to

$$x_i^{(k)} = -\sum_{j=1}^{i-1} A_{ij}/A_{ii}x_j^{(k)} - \sum_{j=i+1}^{n} A_{ij}/A_{ii}x_j^{(k-1)} + b_i/A_{ii}.$$

3. Start with the matrix A in Example 2.3.1. Use the following R code to obtain the QR decomposition of A and execute the Algorithm 2.1 iteratively to find the eigenvalues of A.

```
QR = qr(A)
A = qr.R(QR)%*%qr.Q(QR)
```

Now find the eigenvector b by solving $(A - \lambda I)b = 0$ with each computed eigenvalue λ.

Chapter 3
Solving Nonlinear Equations



Calibration of financial models corresponds to the problem of finding the roots of a function. These might appear directly, as in the example of finding implied volatility in Sect. 3.1 or indirectly. Some examples of indirect problems are maximizing the likelihood (see Sect. 13.1) or minimizing a loss function, as in method of least squares in Chap. 17. Nonlinear equation solving is a central component in many numerical procedures.

3.1 Find Implied Volatility

Suppose the price of one share of a particular stock at time t is denoted by P_t. Then the ratio P_t/P_s, for $s < t$ is the return over the period (s, t). One of the central problems of mathematical finance is to find the volatility, since it is not an observed quantity and has to be inferred from observed market prices of stocks or derivatives. Volatility is a measure of dispersion of prices of a particular asset over a time period. Assuming that the distribution of returns does not change with time, volatility is the standard deviation of log returns for unit time. The estimate of volatility that is derived from the option price is called implied volatility.

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/978-981-19-2008-0_3.

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

R. Sen and S. Das, *Computational Finance with R*, Indian Statistical Institute Series,
https://doi.org/10.1007/978-981-19-2008-0_3

Let us consider the simplest case of a European CALL option. This is a legal contract that gives the holder the right, but not the obligation, to buy one share of a particular stock at a fixed price, called strike, at a fixed time, called maturity. Another kind of option is a PUT option, which is similar to a call option except, here the right is to sell instead of buy.

The traded price of the option P can be observed from the market. The Black–Scholes formula, see Appendix, gives the price p_0 of this option under certain model assumptions in terms of the current stock price S_0, time to maturity T, strike price K, interest rate r and volatility σ as follows:

$$p_0 = S_0\Phi(d_1) - \exp(-rT)K\Phi(d_2) \tag{3.1}$$
$$\text{with} \quad d_1 = d_2 + \sigma\sqrt{T}, \quad d_2 = \frac{\log(S_0/K) + (r - 1/2\sigma^2)T}{\sigma\sqrt{T}}.$$

All the quantities in the formula are known except for the volatility. So we can consider p_0 as a function of σ. The implied volatility is defined as the value of σ for which $p_0(\sigma) - p = 0$. Thus we have to solve this equation to obtain the implied volatility. The function for computing the Black–Scholes price of an option is as follows:

```
blackscholes = function(S, X, rf, T, sigma, CallPut=0){
  # S : Spot price
  # X : Strike price
  # rf: Risk free rate of interest
  # T : Time to maturity
  # sigma : volatility

  # CallPut : (default Call option)
  #            0 for Call option
  #            1 for Put option
  #

  d1 = (log(S/X)+(rf-sigma^2/2)*T)/(sigma*sqrt(T))
  d2 = d1 - sigma * sqrt(T)
  if(CallPut == 0){
    value = max(S*pnorm(d1) - X*exp(-rf*T)*pnorm(d2), 0)
  }else{
    value = max(X*exp(-rf*T) * pnorm(-d2) - S*pnorm(-d1), 0)

  }
return(value)
}
```

At any fixed point of time, there are multiple traded options on the same stock with different maturities and strikes. In general the implied volatility obtained from these different options are not equal. Plotted as a function of the strike price K, the implied volatility designs a u-shaped curve that is called volatility smile.

Table 3.1 Strike and price of call option in Apple stock when price of Apple stock is 201.74

Strike	235	210	215	245	250	225	265	220	200	205
Price	3.15	10.20	8.35	1.77	1.30	4.94	0.57	6.50	15.30	12.75

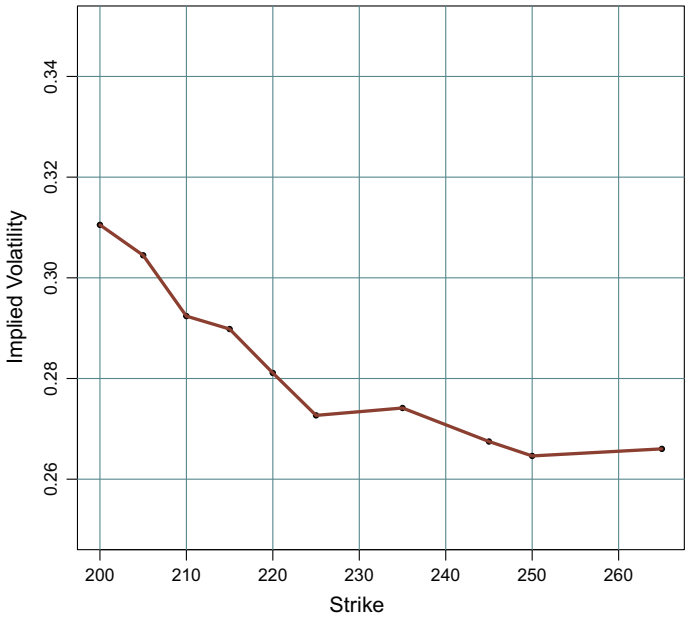


Fig. 3.1 Implied volatility of options on Apple stock as a function of strike when price of Apple stock is 201.74.

Example 3.1.1. We obtained market price of call options on Apple stock on Aug 15, 2019 that expire on Dec 20, 2020. The strike (K) and market price (p) of the options are given in Table 3.1. We only considered those options that traded more than 30 units.
The stock price (S_0) is \$201.74 and the time to expiration is 127 d. We take the risk free interest rate to be 5%. Under these conditions, the following R code computes the implied volatility and plots it against the strike. The result is presented in Fig. 3.1. The function `implied_bisect` is explained and displayed in Sect. 3.2.

```
# Current spot price
S=201.74
# All available Strike prices
K=c(235, 210, 215, 245, 250, 225, 265, 220, 200, 205)
# Time to maturity
```

```
T=127/365
# Risk free rate of interest
r = 0.05
# Last traded call price
C=c(3.15, 10.2, 8.35, 1.77, 1.3, 4.94, 0.57, 6.5, 15.3, 12.75)
# Vector for calculating the implied volatility
sig = rep(0,10)
for(i in 1:10){
sig[i] = implied_bisect(S,K[i],0,T,0,C[i],.0001,1)}
j = order(K)

plot(NULL,ylim=c(0.25,0.35)
     ,xlim=c(min(K),max(K))
     ,xlab="Strike"
     ,ylab="Implied Volatility")
grid(col='cadetblue4',lty=1)
points(K,sig,pch=20)
lines(K[j],sig[j],lwd=3,col='coral4')
```

3.2 Bisection Algorithm

We have a function $f : \mathbb{R} \to \mathbb{R}$ of a variable x. The problem is to find a root of f, that is, a value x such that $f(x) = 0$.

The bisection algorithm is the simplest way to find the root of a general continuous function and was introduced in Example 1.2. We describe it in greater detail here with application to finding the implied volatility and then describe some more advanced techniques.

Suppose f is continuous and we can find numbers a and b with $a < b$ such that $f(a)$ and $f(b)$ have opposite signs. Then the intermediate value theorem (IVT) tells us that there is a root of f between a and b. The bisection method uses this result repeatedly to compute a root. This is a slow iterative method, but is guaranteed to converge as long as we can find the initial interval.

We start with the interval (a, b). Without loss of generality we shall assume here that $f(a) < 0 < f(b)$. The iterative step of the algorithm goes as follows:

- Consider $c = (a + b)/2$, the midpoint of $[a, b]$.
- If $f(c) = 0$, we are done.
- If $f(c) < 0$, then $f(c)$ and $f(b)$ have opposite signs. By IVT, there is a root of f between c and b.
- Otherwise, there is a root of f between a and c.

Thus, starting with the interval (a, b), we reduced it to either (a, c) or (c, b), each of which has half the length of the original interval. This method is repeated until the length of the interval is less than a pre-specified value. Thus we obtain a sufficiently small interval around the root, in other words, we find the root with a small degree of approximation. An alternative way to specify the stopping criterion is by putting

an upper bound on difference between the values of f at the ends of the interval. Since f is continuous, it is guaranteed that this condition will also be met eventually, although the number of steps is random and can be very large.

We are interested in finding the implied volatility. That is, we want to find the value σ that solves the equation $p_0(\sigma) - p = 0$. We can do this using the bisection algorithm. We first need the starting upper and lower bounds for σ. An obvious lower bound is zero. For the upper bound, we start with 1 and iteratively increase to powers of 2 after checking for the sign of $f(a) \times f(b)$. Once an upper bound is obtained, we go with the bisection algorithm described above. Below is the R code to execute this.

The inputs of this function are C, r, T, K, S which are the call price, interest rate, expiration, strike, and stock price respectively. In the code we have put a check for the condition $C + \exp(-rT)K \geq S$ and display an error message when this is violated. Violation of this condition implies that there is arbitrage (see Appendix) in the market. The binary input `CallPut` tells the function if the price is for a call or a put option.

```
implied_bisect = function(S, K, r, T, CallPut, C, tol, cond) {
# cond=1 if length of interval is used for tolerance, otherwise
# absolute value of the function at the midpoinr is used for tolerance.
if(CallPut==0 && C<S-K*exp(-r*T)|| CallPut!=0 && C<K*exp(-r*T)-S){
  guess=0
}else{#finding the upper bound
 lower=0
 flower=blackscholes(S, K, r, T,lower,CallPut)- C
 upper=1
 fupper=blackscholes(S, K, r, T,upper,CallPut)- C
  while(flower*fupper>0){
  upper=upper*2
  fupper=blackscholes(S, K, r, T,upper,CallPut)- C}
#updating to bisected interval
 guess=0.5*(upper+lower)
 fguess=blackscholes(S, K, r, T,guess,CallPut)- C
  if (cond==1)
   while(upper-lower>tol){
    if(fguess*flower<0)
    upper=guess
    else
    lower=guess
    end
   flower=blackscholes(S, K, r, T,lower,CallPut)- C
   fupper=blackscholes(S, K, r, T,upper,CallPut)- C
   guess=0.5*(upper+lower)
   fguess=blackscholes(S, K, r, T,guess,CallPut)- C}
  else
   while(abs(fguess)>tol){
    if(fguess*flower<0)
    upper=guess
    else
    lower=guess
    end
   flower=blackscholes(S, K, r, T,lower,CallPut)- C
   fupper=blackscholes(S, K, r, T,upper,CallPut)- C
```

```
    guess=0.5*(upper+lower)
    fguess=blackscholes(S, K, r, T,guess,CallPut)- C}
   end

}
return(guess)
}
```

Depending on the situation, other tolerance conditions may me relevant. The following example considers the relative error. One might also want to put a restriction on the number of iterative steps.

Example 3.2.1. **Finding equilibrium Price** Consider a typical problem in Microeconomics. We want to find the price p such that supply $S(p)$ of some item equals demand $D(p)$. What we are looking for is a zero of the excess demand function $f(p) = D(p) - S(p)$. An appropriate termination test could be $|D(p) - S(p)| < \delta D(p)$, i.e., demand minus supply is small with respect to demand. This is an example of 'relative' rather than 'absolute' condition.

3.3 Newton's Method

The bisection method described above requires very mild assumption on the function, namely continuity. However it does require two initial values a and b such that the function has opposite signs at those points, which may be difficult to obtain. It is also quite slow and therefore a poor method.

If the function has more smoothness, that is if it is differentiable, then we can use a faster method called Newton's method that also requires only one initial point. Newton's method approximates $f(x)$ by a succession of linear functions, and it approximates a zero of f with the zeros of the linear approximations. Staring with the Taylor expansion of f around x_0 and ignoring the remainder term, we get

$$f(x) = f(x_0) + (x - x_0)f'(x_0).$$

When x is close to the root of f, then $f(x)$ can be approximated by zero. Rearranging the terms, we obtain the iterative step of Newton's method as

$$x_1 = x_0 - f(x_0)/f'(x_0).$$

If f is smooth in the neighborhood of the root and we start at a point that is reasonably close to the root, then this method converges quite rapidly. However, Newton's method may not always converge. This happens when f is not smooth, or when f' is close to zero in the neighborhood of the solution. It may also converge to an incorrect value if the starting point is not chosen carefully.

In case of option prices, the derivative of the option price with respect to volatility is called vega. For the European option price under the Black–Scholes model with $S = K$ and $r = 0$, this equals

$$\nu = S\sqrt{T}\phi(\frac{\sigma\sqrt{T}}{2}),$$

where ϕ is the normal pdf. So iterative step is

```
guess = guess-call/vega
```

Example 3.3.1. **Internal Rate of Return** The internal rate of return is a metric used in financial analysis to estimate the profitability of potential investments. The internal rate of return is a discount rate that makes the net present value (NPV) of all cash flows equal to zero in a discounted cash flow analysis. It is the yield y which equates an investment's present value (PV) with its price P. Suppose the cash flows at times $1, \ldots, n$ are $C_1, \ldots C_n$ respectively. We want the value y such that

$$0 = f(y) = \frac{C_1}{(1+y)} + \frac{C_2}{(1+y)^2} + \frac{C_3}{(1+y)^3} + \cdots + \frac{C_n}{(1+y)^n} - P.$$

Assume a company is assessing the profitability of a project that requires \$250,000 in funding and is expected to generate \$50,000 in after-tax cash flows the first year and grow by \$30,000 for each of the next four years. Here $P = 250$, $n = 5$, $C_i = 50 + i \times 30, i = 0, 1, \ldots, 4$. The algorithm converges at 26.66%.

3.4 Secant Method

An idea that is similar to Newton's method but that does not require the computation of derivative is to keep track of two most recent $(x, f(x))$ and use the difference quotient.

$$\frac{f(x_{n+1}) - f(x_n)}{x_{n+1} - x_n}.$$

The secant method suffers the same convergence problems as Newton's method. The convergence speed depends on the trade-off between the time saved by replacing and avoiding the computation of the derivative versus the quality of the approximation of the derivative by the difference quotient.

3.5 Fixed Point Method

The fixed point method can be used to solve a nonlinear equation without the need for using derivatives or fixing an interval for the solution. Suppose we want to solve $f(x) = 0$. We need to express this algebraically in an equivalent form as $g(x) = x$ and then use the recursion $x_{n+1} = g(x_n)$ with a starting value x_0. In order to guarantee that the second equation has a unique solution, the function g should be a contraction mapping, that is, it should satisfy $|g'(x)| \leq L < 1$, where L is a constant.

3.6 R Functions for Root Finding

There are in-built functions in R for finding roots of a function. If the function is a polynomial, like $p(x) = a_0 + a_1 x + \cdots + a_n x^n$, these solutions can be found using the polyroot function. One needs to specify the vector of coefficients $a_0, a_1, \ldots, a_n$. For example, if we want to find the solutions of $1 + 2x + x^2 = 0$ then we use

```
polyroot(c(1, 2, 1))
```

For general functions, we can use the uniroot function. For example, in the above case we can write

```
f = function(x)  2 * x + x^2-2
uniroot(f, c(-2, 2))
```

The term `c(-2, 2)` specifies the range of values in which to find the roots. The method requires that the function for which we want to find the roots has opposite signs in the extremes of the specified interval. This allows the algorithm to improve the speed of convergence. It is also possible to specify the precision of the solution or the maximum number of iterations.

3.7 Exercises

1. Find the derivative, vega, of the option price with respect to volatility from the Black–Scholes price of European Call option in Eq. (3.1).
2. Write an R code to find the implied volatilities in Example 3.1.1 using Newton's method. Compare the results to Fig. 3.1.
3. Use the `polyroot` function in R to solve the problem in Example 3.3.1.
4. Show that the equation in Example 3.3.1 can be written as $x = (P - C_2 x^2 - C_3 x^3 - \cdots - C_n x^n)/C_1$. Taking the right-hand side as $g(x)$ with the values given in the example, verify whether the condition of convergence of the fixed point method is satisfied.
5. Solve the equation in Exercise 1.3.1 using the secant method in R.
6. Repeat the procedure of Example 3.3.1 with continuous compounding.

Chapter 4
Numerical Integration



Integration is a very common requirement in quantitative finance. In some cases integrals can be evaluated analytically, or at least in terms of standard functions whose values are known. For example in the case of option pricing under the Black–Scholes model, the integral can be expressed in terms of the normal distribution function. However, very often we encounter problems where the integral cannot be evaluated analytically. In such cases we need numerical techniques that we shall explore in this chapter. For one-dimensional integrals, these deterministic methods work quite well. For evaluating integrals in higher dimensional spaces, it may be more advantageous to use random sampling. We shall see these in the Chap. 8 as Monte Carlo methods.

Integrals frequently arise as expectations. For example, in economics one needs to evaluate the expected utility. Option pricing requires computing the expected value of the payoff under the risk-neutral measure, defined in Appendix A. For a random variable X with density $f_X(x)$, the expected value of a function $h(.)$ is given by $\int_{-\infty}^{\infty} h(x) f_X(x) \mathrm{d}x$, which is really an integral.

Numerical integration may be used to estimate probabilities. Suppose a random variable X has density $f_X(x)$, and we are interested in the probability of an event A that can be expressed in terms of X, then $P(A) = \int_{-\infty}^{\infty} I_A(x) f_X(x) \mathrm{d}x$, where $I_A(x)$ is the indicator function for event A. For example A could be the event that $X > 0$. Another popular application is in Bayesian statistics where the marginal distribution of the data has to be computed by integrating the likelihood with respect to the prior.

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/978-981-19-2008-0_4.

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

R. Sen and S. Das, *Computational Finance with R*, Indian Statistical Institute Series,
https://doi.org/10.1007/978-981-19-2008-0_4

4.1 Introduction

There are situations in which we define a function by an integral. A typical case is encountered in stochastic optimization and dynamic programming. Here we want to maximize or minimize the expectation of $g(X, z)$ which is a function of a control variable z and a random variable X.

$$H(z) = E(g(X, z)) = \int_{-\infty}^{\infty} g(x, z) f_X(x) \mathrm{d}x.$$

The value of the argument z where this extremum occurs is the optimal choice of z.

The question of interest is to find an approximate numerical value for a definite integral of a function f of a single variable x over the bounded interval $[a, b]$, that is,

$$I[f] = \int_a^b f(x) \mathrm{d}x$$

We start with the idea of the Riemann sum S of f over a partition $a = x_0 < x_1 < \cdots < x_n = b$ as

$$S = \sum_{i=1}^{n} f(x_i^*)(x_i - x_{i-1}) \quad x_i^* \in [x_{i-1}, x_i].$$

For a large class of functions the limit of this sum exists as the norm of the partition, given by $\max(x_i - x_{i-1})$, goes to zero. Such functions are said to be Riemann integrable. This limit does not depend on the specific choice of the points x_i^*.

A generalization of this is the quadrature formula given by

$$Q_n[f] = \sum_{j=0}^{n} w_j f(x_j),$$

where w_j are the weights and x_j are the nodes. The choice of these parameters will determine the error of approximation

$$E_n = I[f] - Q_n[f].$$

We shall discuss several methods for choosing these parameters and their associated errors. Just like the integral operator, any quadrature formula is also linear, that is

$$Q_n(f + g) = Q_n(f) + Q_n(g).$$

4.2 Newton–Cotes Formulas

The classical way to derive an interpolatory formula is to consider a set of equally spaced nodes:

$$x_j = a + jh,\ j = 0, 1, 2, \ldots, n,$$

where $h = (b - a)/n$; also let $f_j = f(x_j)$. This yields the set of Newton–Cotes quadrature formulas.

Given these $n + 1$ nodes, we now proceed to find the appropriate weights. One guiding principle is that we want the procedure to be exact for a class of simple functions, say polynomials, of upto a certain degree, called the order of approximation. In that case, the approach is to approximate the given function f by a polynomial, then propose the integral of that approximate polynomial as an approximation to the integral of the original function.

Let us consider the interpolating polynomial $p_n(x)$ using Lagrange polynomials of degree n, as introduced in Sect. 1.2.1. For a given set of distinct points x_j and numbers f_j, the Lagrange polynomial is the polynomial of lowest degree that assumes at each point x_j the corresponding value f_j. Given a set of $n + 1$ data points $((x_0, f_0), \ldots, (x_n, f_n))$ where no two x_j are the same, the interpolation polynomial in the Lagrange form is a linear combination

$$p_n(x) := \sum_{j=0}^{n} f_j L_j(x)$$

of Lagrange basis polynomials

$$L_j(x) := \prod_{\substack{0 \le m \le n \\ m \ne j}} \frac{x - x_m}{x_j - x_m} = \frac{(x - x_0)}{(x_j - x_0)} \cdots \frac{(x - x_{j-1})}{(x_j - x_{j-1})} \frac{(x - x_{j+1})}{(x_j - x_{j+1})} \cdots \frac{(x - x_n)}{(x_j - x_n)},$$

where $0 \le j \le n$. Note that $L_j(x_i) = I(i = j)$. So $p_n(x_i) = f_i \forall i$. That is, p_n is a polynomial of degree n that matches f at $n + 1$ points. If f is a polynomial of degree n or less, then $f(x) = p_n(x) \forall x$. Then we may compute the correct weights as follows:

$$\int_a^b f(x)\mathrm{d}x = \int_a^b p_n(x)\mathrm{d}x = \int_a^b \sum_{j=0}^{n} f_j L_j(x)\mathrm{d}x = \sum_{j=0}^{n} f_j \left(\int_a^b L_j(x)\mathrm{d}x \right) = \sum_{j=0}^{n} w_j f_j.$$

That is, $w_j = \int_a^b L_j(x)\mathrm{d}x$.

Example 4.2.1. Let us consider the function $f(x) = x\sin(x)$ in the interval $[0, \pi/4]$. The following R code can be used to obtain the Lagrange polynomials of order one and two. We use the function `poly_calc` in the `R`-library called `PolynomF`.

```
# install.packages("PolynomF")
library("PolynomF")

f = function(x) {
  return(x * sin(x))
}
f1 = poly_calc(c(0,pi/4), c(0,f(pi/4)))
f2 = poly_calc(c(0, pi/8, pi/4), c(0, f(pi/8), f(pi/4)))
plot(f,xlim=c(0,pi/4),ylab=expression(f(x))
     ,lwd=2,col='black')
curve(f2,add=TRUE,col="red",lwd=2,lty=2)
curve(f1,add=TRUE,col="blue",lwd=2)
```

We obtain the following polynomials as result of running this code.

$$p_2(x) = 0.05826008 * x + 0.8261373 * x^2/2,$$
$$p_1(x) = 0.7071068 * x.$$

The plot of the original functions and the two polynomials is presented in Fig. 4.1. Note that the second degree polynomial is quite a good approximation.

4.2.1 Error

Suppose f is $n+1$ times differentiable on $[a, b]$. Then it holds that

$$f(x) = p_n(x) + \frac{f^{(n+1)}(\xi_x)}{(n+1)!}(x - x_0)\dots(x - x_n),$$

where $\xi_x \in (a, b)$. Thus

$$\int_a^b f(x)\mathrm{d}x - \int_a^b p_n(x)\mathrm{d}x = \int_a^b \frac{f^{(n+1)}(\xi_x)}{(n+1)!}(x - x_0)\dots(x - x_n)\mathrm{d}x$$
$$= O((b-a)^{n+2}). \tag{4.1}$$

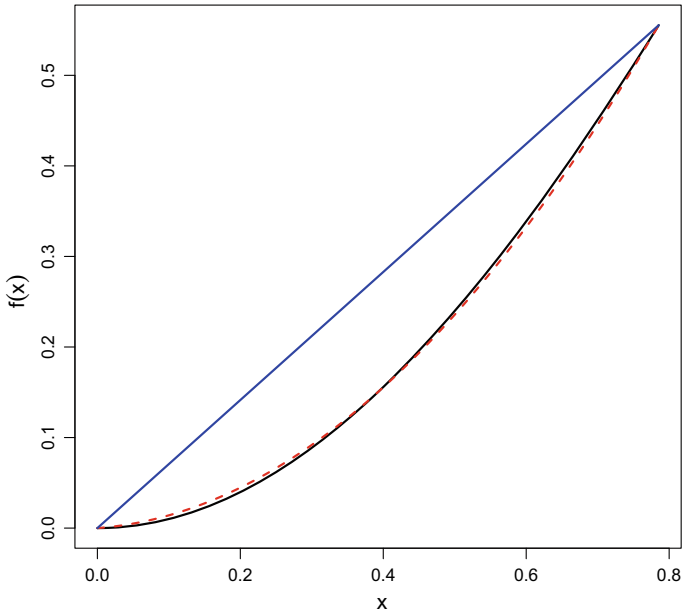


Fig. 4.1 Plot of the function $f(x) = x\sin(x)$ (black) and the linear (blue) and quadratic (red) approximations using Lagrange polynomials. Clearly the quadratic approximation performs far better than the linear approximation

We have used the notation $O(.)$ for order of convergence here. A function $g(x)$, is said to be of the order $O(x^k)$ at x_0 where k is that value for which $g(x)/x^k \to c$ as $x \to x_0$ and c is a constant. Here the left-hand side is a function of the length $b - a$ of the interval and the convergence is considered as that length goes to zero. In the above notation $x = (b - a)$ and $x_0 = 0$.

4.2.2 Trapezoidal Rule

A special case of Newton-Cotes is the trapezoid rule that is built on two nodes. In this case, the Lagrange polynomials are $L_0 = (x - x_1)/(x_0 - x_1)$ and $L_1 = (x - x_0)/(x_1 - x_0)$. The polynomial approximation to f is the straight line

$$p_1(x) = f_0 L_0(x) + f_1 L_1(x) = f_0 \frac{(x - x_1)}{(x_0 - x_1)} + f_1 \frac{(x - x_0)}{(x_1 - x_0)}. \tag{4.2}$$

A straightforward calculation yields $\int_{x_0}^{x_1} p_1(x) = (x_1 - x_0)\left(\frac{f_1 + f_0}{2}\right)$. This is the area of the trapezium given by $(x_0, 0)$, $(x_1, 0)$, (x_1, f_1), (x_0, f_0). We are approximating

the area under the curve f between the points x_0 and x_1 by that under the trapezium obtained by linearly interpolating between (x_1, f_1) and (x_0, f_0). This is a very crude approximation to the integral and is called the trapezoid rule.

Given a function on an interval, we can break the interval into m smaller subintervals, calculate the integral on each of these and then add up to get a composite formula. Applying this principle to the above simple trapezoid rule, we get the general trapezoidal rule:

$$Q_m[f] = h\left(\frac{1}{2}f_0 + \sum_{j=1}^{m-1} f_j + \frac{1}{2}f_m\right) \quad \text{where} \quad b - a = mh. \tag{4.3}$$

4.2.3 Simpson's Rule

We now introduce an alternative way of constructing approximations to definite integrals that are exact for polynomials upto a certain degree. Here we start with n nodes as before and construct a linear combination of the values of the function on those nodes, but keep the coefficients of this linear combination unspecified. We then construct a set of linear equations involving these coefficients by equating the value of the integral for function $f(x) = 1, x, x^2, \ldots, x^{n-1}$ with the corresponding linear combination. Consider the case of $n = 3$ nodes,

$$\int_0^1 f(x)\mathrm{d}x = w_0 f(0) + w_1 f(0.5) + w_2 f(1).$$

We obtain the following three linear equations in the three unknowns w_1, w_2, w_3.

$$\begin{aligned} 1 &= \int_0^1 1\mathrm{d}x = w_0 + w_1 + w_2 \\ 1/2 &= \int_0^1 x\mathrm{d}x = 1/2w_1 + w_2 \\ 1/3 &= \int_0^1 x^2\mathrm{d}x = 1/4w_1 + w_2, \end{aligned}$$

which yields $w_0 = 1/6$, $w_1 = 2/3$, $w_2 = 1/6$. Applying the same idea on the interval $[a, b]$, we get the quadratic approximation of Simpson's rule:

$$\int_a^b f(x)\mathrm{d}x = \frac{b-a}{6}\left(f(a) + 4f\left(\frac{a+b}{2}\right) + f(b)\right). \tag{4.4}$$

Example 4.2.2. Continuing with the function $f(x) = x\sin(x)$ in the interval $[0, \pi/4]$ presented in Example 4.2.1. We can integrate this theoretically and the integral is

$$\int_0^{\pi/4} x\sin(x)\mathrm{d}x = -x\cos(x) + \sin(x)\,|_0^{\pi/4} = \sin(\pi/4) - \pi/4\cos(\pi/4) = 0.1517464.$$

Below we present the R code for obtaining the approximate integral using the quadratic approximation of Simpson's rule.

```
f = function(x) {
  return(x * sin(x))
}
simpsons.rule = function(f, a, b) {
  h = (b - a) / 2
  x0 = a
  x1 = a + h
  x2 = b
  s = (h / 3) * (f(x0) + 4 * f(x1) + f(x2))
  return(s)
}
simpsons.rule(f, 0, pi/4)

## [1] 0.1513826
```

The value obtained is `0.1513826`.

We do the same calculation with another function $f(x) = x^2\exp(-x)$ on the interval $[0, 1]$ and also using the trapezoid rule. The results are summarized in Table 4.1. Note that in both cases Simpson's rule performs better than the trapezoid rule.

In Sect. 4.2.2, we presented the idea of dividing an interval into subintervals, approximating the area under the curve for each subinterval using a linear formula and then adding them up to arrive at the trapezoid rule. The same idea can be applied to the quadratic approximation to arrive at the general Simpson's rule. Suppose we divide the interval $[a, b]$ into m equal subintervals $(a + ih, a + (i + 2)h)$, where $i = 0, 2, \ldots, 2m - 2$ and $h = (b - a)/2m$. Then for each subinterval, we have the following from Eq. (4.4),

Table 4.1 Numerical integration of two functions using different methods

Function	Theoretical integral	Simpson's rule	Trapezoid rule
$\int_0^{\pi/4} x\sin(x)\mathrm{d}x$	0.1517464	0.1513826	0.1680593
$\int_0^{1} x^2\exp(-x)\mathrm{d}x$	0.1606028	0.1624017	0.1677862

$$\int_{a+ih}^{a+(i+2)h} f(x)dx = \frac{2h}{6}\left(f(a+ih) + 4f(a+(i+1)h) + f(a+(i+2)h)\right).$$

Adding these over i, we get the composite Simpson's rule as

$$\begin{aligned}\int_a^b f(x)\mathrm{d}x &= \frac{2h}{6}\left(f(a) + 4f(a+h) + f(a+2h) + f(a+2h) + 4f(a+3h)\right.\\ &\quad \left. + f(a+4h) + \cdots + f(a+(2m-2)h) + 4f(a+(2m-1)h) + f(a+2mh)\right)\\ &= \frac{h}{3}\left(f(a) + 4\sum_{j=1}^{m} f(a+(2j-1)h) + 2\sum_{j=2}^{m-1} f(a+2jh) + f(b)\right).\end{aligned}$$

Observe from Eq. (4.1) that the error in approximation of each subinterval is $O(h^{n+2})$ where n is the degree of the polynomial. For Simpson's rule $n = 2$. When we add up over m subintervals with $h = (b-a)/2m$, the total error is $O(m^{-3})$.

Below we present the code to compute the integral of a function using the composite Simpson's rule with different number of intervals. Using the function $f(x) = x\sin(x)$ we demonstrate in Fig. 4.2 the relation between the number of intervals and the total error. The first five errors are 3.64e − 04, 2.23e − 05, 4.38e − 06, 1.38e − 06, and 5.66e − 07. Note that the scale of the y-axis is error$^{-1/3}$, and the resulting plot is nearly linear. This shows that the error is indeed $O(m^{-3})$, where m is the number of subintervals.

```
library(latex2exp)
composite.simpson = function(f, a, b, m) {
  h = (b - a) / {2*m}
  xj = seq.int(a, b, length.out = 2*m + 1)
  xj = xj[-1]
  xj = xj[-length(xj)]
  n = length(xj)
  approx = (h / 3) * (f(a)
                      + 2 * sum(f(xj[seq.int(2, n, 2)]))
                      + 4 * sum(f(xj[seq.int(1, n, 2)]))
                      + f(b)
                      )
  return(approx)
}
g = rep(0,times=9)
g[1] = simpsons.rule(f, 0, pi/4)
for(i in 2:9){g[i] = composite.simpson(f, 0, pi/4,i)}
x = pi/4
u = -x*cos(x)+sin(x)
plot(1/(u-g)^(1/3), xlab="Number of sub-intervals"
     , ylab=TeX("$Error^{-1/3}$")
     , pch=20)
```

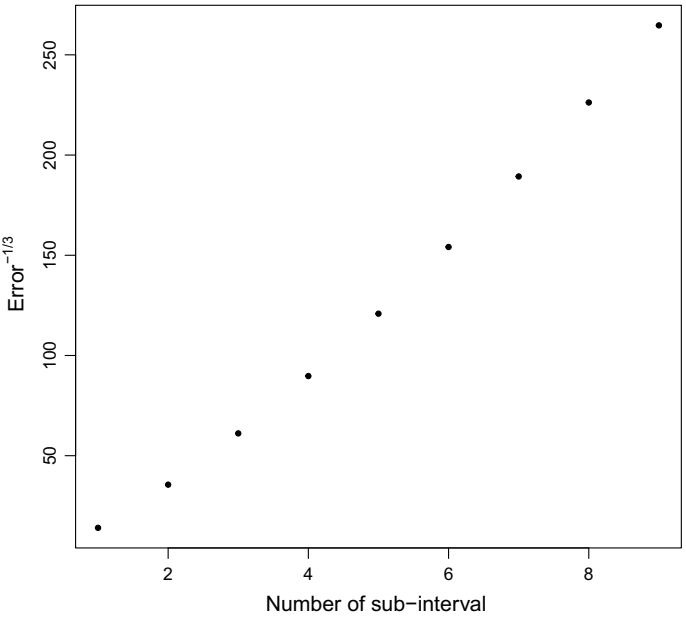


Fig. 4.2 Plot of error$^{-1/3}$ versus number of subintervals in the composite Simpson's method. The linearity of the plot confirms that the error is indeed $O(m^{-3})$

4.3 Gaussian Quadrature

The quality of approximation of a quadrature formula can be drastically improved by allowing, not only the weights, but also the nodes, to vary. The Gaussian quadrature formulae are developed to choose both nodes and weights in a way that the order is almost double of that of Newton–Cotes. More precisely, Gaussian quadrature using n nodes will provide the same order of approximation error as a Newton–Cotes quadrature with $2n - 1$ nodes.

For this purpose, we have to first choose a proper non-negative weight function $w(x)$. We illustrate the method with $w(x) = \exp(-x^2/2)$ and at the end of the section list other possible weight functions. We look for a quadrature formula like $\int_a^b w(x)f(x) = \sum_{i=1}^n w_i f(x_i)$ which is exact when f is a polynomial.

Next we discuss the choice of nodes. These are taken to be the roots of a polynomial of order n, selected within a family of orthogonal polynomials with respect to the inner product:

$$< f, g >= \int_a^b w(x)f(x)g(x)\mathrm{d}x.$$

It can be shown that a polynomial of degree n within that family has n distinct real roots. These are chosen as the nodes.

Table 4.2 Weights for quadrature formulae

Family	Weight	Range
Legendre	$w(x) = 1$	$-1 < x < 1$
Laguerre	$w(x) = \exp(-x)$	$0 < x < \infty$
Hermite	$w(x) = \exp(-x^2/2)$	$-\infty < x < \infty$
Chebyshev	$w(x) = (1 - x^2)^{-1/2}$	$-1 \leq x \leq 1$

The weights w_i are chosen as in the previous section so that the quadrature is exact for polynomials of degree n. However, since we have now chosen the nodes in an optimal way, these same weights will actually yield a quadrature formula with order $2n - 1$. To see this, define orthogonal polynomials Π_n recursively.

> Orthogonal polynomials are a sequence, Π_n, of sets of polynomials. For every n, elements of Π_n are polynomials of degree n. Each element is orthogonal to all the polynomials in Π_{n-1}.

Given a polynomial $f \in \Pi_{2n-1}$, we want to show that the quadrature based on n nodes and n weights is exact. Take a member q of Π_n. When f is divided by q, we obtain a quotient p and a remainder $r : f = qp + r$, where $p, r \in \Pi_{n-1}$.

$$\begin{aligned}
&\int_a^b w(x) f(x) \mathrm{d}x \\
&= \int_a^b w(x) p(x) q(x) \mathrm{d}x + \int_a^b w(x) r(x) \mathrm{d}x \\
&= 0 + \int_a^b w(x) r(x) \mathrm{d}x \quad (q \text{ is orthogonal to } p) \\
&= \textstyle\sum_{i=1}^n w_i r(x_i) \quad (\text{quadrature is exact for } r \in \Pi_{n-1}) \\
&= \textstyle\sum_{i=1}^n w_i f(x_i) \quad (x_i \text{ is zero of } q).
\end{aligned}$$

In summary the algorithm is as follows.

> **Algorithm 4.1**
>
> 1. Choose a weight function, e.g., Gaussian $w(x) = \exp(-x^2/2)$
> 2. Build a system of orthogonal polynomials wrt w. In particular $q \in \Pi_n$ is orthogonal to all polynomials in Π_{n-1}.
> 3. Find $\{x_i\}_{i=1}^n$, the zeros of q.
> 4. Find the weights $\{w_i\}_{i=1}^n$ to make the quadrature exact upto degree $n - 1$.

Some combinations of weights and families of polynomials are given in Table 4.2.

The only missing part in the above treatment is the procedure to generate a family of orthogonal polynomials $p_j(x)$. This may be achieved as follows. Set $p_0(x) = 1$. The polynomials are defined recursively as:

$$p_{j+1}(x) = (x - a_j)p_j(x) - b_j(p_{j-1}(x)), \quad j = 0, 1, 2, \ldots$$

$$\text{where} \quad a_j = \frac{< xp_j, p_j >}{< p_j, p_j >} \quad \text{and} \quad b_j = \frac{< p_j, p_j >}{< p_{j-1}, p_{j-1} >}.$$

Here coefficient b_0 is arbitrary and it can be set to 0. In the Gaussian case, where $w(x) = \exp(-x^2/2)$, applying the procedure above results in the following recursive procedure yielding a sequence of Hermite polynomials: $H_{j+l} = xH_j - jH_{j-1}$.

The following `R` code uses the function `legendre.quadrature` in the library `gaussquad` to find the Legendre quadrature approximation to the function $f(x) = x\sin(x)$ to find the integral on the interval $[0, \pi/4]$. The first five errors are `2.43e-04, -4.90e-07, 4.61e-10, -2.51e-13, and 1.94e-16`. Comparing these to the errors of Simpson's rule, the third degree Legendre polynomial beats all composite Simpson's rule results. This is expected as Simpson's rule is a polynomial approximation of second degree.

```
x = pi/4
u = sin(x)-x*cos(x)
library("gaussquad")
g<-rep(0,5)
for(i in 1:5){
  rules = legendre.quadrature.rules( i+1 )
  order.np1.rule = rules[[i+1]]
  g[i] = legendre.quadrature(f, order.np1.rule
                             , lower=0,upper=pi/4)
}
abs(u-g)

## [1] 2.429760e-04 4.900268e-07 4.608492e-10 2.506051e-
13 1.110223e-16
```

Finally we present two examples from economics where numerical integration is used. In the first example, the payoff of a portfolio is random and we want to find the expected utility of this portfolio for an investor. So the integral is over the possible payoff values. In the second example, we compute present value of discounted utility over infinite horizons for a monopolist faced with a particular demand curve. Here the integral is over the infinite time horizon in the future.

Example 4.3.1. **Expected utility of an investment** An investor holds one bond which will be worth 1 and equity whose value will be X, where $\log X \sim \mathcal{N}(\mu, \sigma^2)$ at the terminal time. So the total wealth at terminal time will be $1 + \exp(Z)$ where $Z \sim \mathcal{N}(\mu, \sigma^2)$. The investor has a power utility function given by $u(c) = c^{1+\gamma}/(1+\gamma)$. The expected utility is

$$\begin{aligned} U &= \frac{1}{\sqrt{2\pi\sigma^2}} \int_{-\infty}^{\infty} u(1+\exp(z)) \exp(-(z-\mu)^2/2\sigma^2) \mathrm{d}z \\ &= \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{\infty} u(1+\exp(\sigma x+\mu)) \exp(-x^2/2) \mathrm{d}x. \end{aligned}$$

This can be computed as

```
hermite.he.quadrature(u(1+exp(sigma*x+mu),rule)/sqrt(2*pi)
```

The certainty equivalent is $c = u^{-1}(U)$. This is a deterministic amount which has the same utility as the expected utility of the random terminal wealth. This means that the investor will have the same preference for the suggested portfolio as one with a bond that will be worth c. With $\mu = 0.15$ and $\sigma = 0.25$, the certainty equivalent for various values of γ can be computed as

γ	c
-0.5	2.1885
-1.1	2.1766
-2.0	2.1592
-5.0	2.1054

Example 4.3.2. **Total profit of monopolist** Suppose a monopolist faces a demand curve $D(p) = p^{-\eta}, \eta > 1$ and has unit cost of $m(t) = a + b\exp(-\lambda t)$ at time t. The monopolist will determine the amount to produce so that marginal cost(MC) and marginal revenue (MR) are equal. If we denote the quantity by x, then

$$\begin{aligned} x &= D(p) = p^{-\eta} \\ \text{Therefore} \quad p &= x^{-1/\eta} \\ \text{Total Revenue} &= xp = x^{1-1/\eta} \\ MR &= \frac{\mathrm{d}}{\mathrm{d}x} x^{1-1/\eta} = (1 - 1/\eta)\, x^{-1/\eta} \\ MC &= m(t). \end{aligned}$$

Equating MC to MR, we get the optimal amount to be

$$x = \left(\frac{\eta}{\eta - 1}\right)^{-\eta} (m(t))^{-\eta}.$$

Thus profit equals

$$x(p - m(t)) = \eta \left(\frac{\eta}{\eta - 1}\right)^{-\eta} m(t)^{1-\eta}.$$

Suppose the interest rate is r, then total discounted profits equal

$$\eta \left(\frac{\eta - 1}{\eta}\right)^{\eta - 1} \int_0^\infty \exp(-rt) m(t)^{1-\eta} \mathrm{d}t$$

With $a = 2, b = -1, \eta = 1.25, r = 0.05, \lambda = 5$, we get the profit at 0.68 using the following code. In this case the unit cost at time zero is $a + b = 1$.

```
f = function(t){exp(-0.05*t)*(2-exp(-5*t))^(1-1.25)}
for(i in 1:5){
  rules = laguerre.quadrature.rules( i+1 )
  order.np1.rule = rules[[i+1]]
  g[i]=laguerre.quadrature(f, order.np1.rule)
}
et = 1.25
et*(1-1/et)^(et-1)*g[5]

## [1] 0.6841649
```

4.4 Exercises

1. Let $f(x) = p_1(x) + E(x)$, where p_1 is the polynomial of degree one in the trapezoid rule. Show that $\mid E(x) \mid \leq f''(\xi) * (x-a)(x-b)/2$ with $\xi \in (a, b)$. Assume that $\mid f'' \mid \leq M$ is bounded. Then show that $\mid \int_a^b E(x)\mathrm{d}x \mid \leq = M/12(b-a)^3$.
2. For the function $f(x) = x^2 \exp(-x)$ on the interval [0, 1] show that the first two Lagrange polynomials are as follows

$$p_1(x) = 0.358093 * x,$$
$$p_2(x) = 0.1722329 * x + 0.2366443 * x^2.$$

   Obtain a plot like Fig. 4.1 for this function and comment on the quality of the approximations.
3. Verify the numbers given in Table 4.1 by writing an R code for evaluating integrals using the trapezoid rule.
4. For the function in problem 2, check if the error of the composite Simpson's rule is $O(m^{-3})$, where m is the number of subintervals.
5. Write the R code for obtaining the table in Example 4.3.1.

Chapter 5
Numerical Differentiation



Derivatives arise in finance in several situations. In option pricing, the hedge ratios are related to the derivatives, also known as Greeks. We have seen the need for one such Greek, namely vega, in Sect. 3.3. Another important application is sensitivity analysis. The general idea is to understand how a target variable, like stock price, is affected by an input variable like book value of the company. A particular sensitivity is bond duration, where the input variable is interest rate and the target variable is bond price.

Given any differentiable function, we can find its derivative analytically. However, in many situations, we might need or prefer to obtain a numerical approximation. Here are some examples:

- The exact formula, even if available in closed analytical form, may entail evaluation of complicated functions. In such cases numerical approximations might be simpler and quicker to obtain.
- It is often the case that the form of the underlying function is unknown, but given any argument, it can be evaluated. In such cases, even though there is an underlying differentiable function, we cannot find the analytical derivative.
- The most common case, particularly in financial applications, is the case of solving differential equations. The underlying function, which is the desired solution of the equation, is itself unknown. But values are known for some grid points and these can be used to evaluate the derivatives at grid points.

5.1 Introduction

We start with the definition of derivative

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}.$$

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

R. Sen and S. Das, *Computational Finance with R*, Indian Statistical Institute Series,
https://doi.org/10.1007/978-981-19-2008-0_5

Thus a natural approximation is to remove the limit and evaluate the right-hand side for one very small nonzero value of h. This gives us the **forward (backward) differencing** formula for numerical derivatives as

$$f'(x) = \frac{f(x+h) - f(x)}{h}, \tag{5.1}$$

when $h > (<)0$. In fact, when f is linear then this approximation is exact. For nonlinear functions there is an error arising from this approximation, which we shall now compute. Assuming f has two continuous derivatives, we can write the Taylor expansion of $f(x+h)$ about x as

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2} f''(\xi), \quad \xi \in (x, x+h).$$

Rearranging the terms, we get

$$f'(x) = \frac{f(x+h) - f(x)}{h} - \frac{h}{2} f''(\xi), \quad \xi \in (x, x+h). \tag{5.2}$$

The differencing formula given in Equation (5.1) is obtained from the exact formula (5.2) by ignoring the error term $-\frac{h}{2} f''(\xi)$. Thus the differencing formula comes with a truncation error as described in Sect. 1.1.2.

The interest is to find methods such that the truncation error is small. Of course, as $h \to 0$, the error decreases to zero. But we are interested in the speed at which the error goes to zero, also known as the rate of convergence. Recall the definition of order $O(.)$ introduced in Sect. 4.2.1. In the above formula, the error is $-\frac{h}{2} f''(\xi)$, that is of the order of $O(h)$ and the method is said to be of first order.

If f has three continuous derivatives, then we can obtain a more accurate approximation. For $h > 0$, we can write the Taylor expansions

$$\begin{aligned} f(x+h) &= f(x) + hf'(x) + \frac{h^2}{2} f''(x) + \frac{h^3}{6} f'''(\xi_1), \\ f(x-h) &= f(x) - hf'(x) + \frac{h^2}{2} f''(x) - \frac{h^3}{6} f'''(\xi_2). \end{aligned}$$

Here $\xi_1 \in (x, x+h)$ and $\xi_2 \in (x-h, x)$. Combining the above two equations we get

$$f'(x) = \frac{f(x+h) - f(x-h)}{2h} - \frac{h^2}{12}[f'''(\xi_1) + f'''(\xi_2)]. \tag{5.3}$$

Equation (5.3) gives another differencing formula, called **centered differencing** with truncation error given by

$$-\frac{h^2}{12}[f'''(\xi_1) + f'''(\xi_2)].$$

Since we have assumed that f has three continuous derivatives, the intermediate value theorem implies that there exists a point $\xi \in (x-h, x+h)$ such that

$$f'''(\xi) = \frac{1}{2}[f'''(\xi_1) + f'''(\xi_2)].$$

Hence

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h} - \frac{h^2}{6} f'''(\xi).$$

Thus centered differencing is a second-order approximation of the first derivative.

5.2 Differentiation Via Interpolation

In Sect. 4.2 we described the method of obtaining interpolating polynomials in order to execute numerical integration. The same idea of approximating a given function by a polynomial can be used for finding an approximate numerical derivative. For demonstration we use the Lagrange polynomials introduced in Sect. 1.2.1. The procedure is to fit an appropriate Lagrange polynomial of a chosen degree to the given polynomial and then to propose the derivative of the approximating polynomial as an approximation to the derivative of the given function.

Suppose we have values of f at points $x_0, \ldots, x_n \in (a, b)$ and we are interested in the derivative at $x = x_k$, which is one of the given points. The Lagrange form of the interpolation polynomial through these points is

$$p_n(x) = \sum_{j=0}^{n} f(x_j) L_j(x) \quad x \in (a, b).$$

Assuming that f is $n+1$ times differentiable, according to Eq. (1.4) the interpolation error is

$$f(x) - p_n(x) = \frac{1}{(n+1)!} f^{(n+1)}(\xi) \prod_{j=0}^{n} (x - x_j),$$

where $\xi \in (a, b)$. We then have:

$$f(x) = \sum_{j=0}^{n} f(x_j) L_j(x) + \frac{1}{(n+1)!} f^{(n+1)}(\xi) \prod_{j=0}^{n} (x - x_j).$$

If we take the derivative of the right-hand side with respect to x, we get

$$\sum_{j=0}^{n} f(x_j)L_j'(x) + \frac{1}{(n+1)!} f^{(n+1)}(\xi) \sum_{i=0}^{n} \prod_{j \neq i} (x - x_j).$$

When we plug in $x = x_k$, all the terms in the above sum over i become zero except the one where $i = k$ since there is one j that equals k rendering the product to be zero. Thus we get

$$f'(x_k) = \sum_{j=0}^{n} f(x_j)L_j'(x_k) + \frac{1}{(n+1)!} f^{(n+1)}(\xi) \prod_{j \neq k} (x_k - x_j). \tag{5.4}$$

This is known as the differentiation by interpolation formula. The second term is the error term. The first term is the approximate derivative. In order to evaluate this explicitly, we need to find $L_j'(x_k)$. From Eq. (1.3), we have the explicit form of $L_j(x)$. Taking the derivative and substituting $x = x_k$, we get

$$L_j'(x_k) = \begin{cases} \frac{1}{x_j - x_k} \prod_{i \neq j, i \neq k} \frac{x_k - x_i}{x_j - x_i} & j \neq k, \\ \sum_{i \neq k} \frac{1}{x_k - x_i} & j = k. \end{cases} \tag{5.5}$$

We now proceed to obtain the differentiation by integration formula (5.4) explicitly in the case where $n = 2$, $k = 1$ and the interpolation points are $(x - h, x, x + h)$. Since $k = 1$, we are trying to approximate $f'(x)$. The required derivative is $\sum_{j=0}^{2} f(x_j)L_j'(x_k)$. We evaluate L_j' for each j below using Eq. (5.5).

$$\begin{aligned} L_0'(x_1) &= \frac{1}{x_0 - x_1} + \frac{x_1 - x_2}{x_0 - x_2} = \frac{1}{-h} \frac{-h}{-2h} = -\frac{1}{2h}, \\ L_1'(x_1) &= \frac{1}{x_1 - x_0} + \frac{1}{x_1 - x_2} = \frac{1}{h} + \frac{1}{-h} = 0, \\ L_2'(x_1) &= \frac{1}{x_2 - x_1} + \frac{x_1 - x_0}{x_2 - x_0} = \frac{1}{h} \frac{h}{2h} = \frac{1}{2h}. \end{aligned}$$

Thus the approximate derivative is

$$f(x - h)\frac{1}{-2h} + f(x + h)\frac{1}{2h} = \frac{f(x + h) - f(x - h)}{2h}.$$

Note that this is same as the centered differencing formula.

Next we present an example where the derivative of an option price is of interest. For definition of option and related terms refer to Sect. 3.1.

Example 5.2.1. **Dual delta of option** The derivative of option price with respect to the strike, when all other factors like stock price, expiration, volatility, and interest rate are fixed, is known as the dual delta. When the interest rate is zero, then the negative of this quantity equals the probability of the option expiring in the money. That is

$$\frac{\partial C}{\partial K} = -P(S_T > K), \tag{5.6}$$

where C is the price of a call option with strike K, expiration T on a stock whose price at time t is S_t.
We find the dual delta at $K = 210$ using different methods described in this chapter for the data on option price on Apple stock for different strikes given in Table 3.1. We get the following results.

Method	Code	Result
Backward differencing	$(y[3] - y[2])/(x[3] - x[2])$	−0.51
Forward differencing	$(y[4] - y[3])/(x[4] - x[3])$	−0.37
Centered differencing	$(y[4] - y[2])/(x[4] - x[2])$	−0.44
Forward second order	$(-3 * y[3] + 4 * y[4] - y[5])/(x[5] - x[3])$	−0.37
Backward second order	$(-y[3] + 4 * y[2] - 3 * y[1])/(x[3] - x[1])$	−0.51

For definition of forward and backward differencing refer to Eq. (5.1). For centered differencing refer to Eq. (5.3)

5.3 Method of Undetermined Coefficients

In this approach, we approximate the derivative by a linear combination of the functional values at some pre-specified points. The coefficients of this linear combination are allowed to vary and will be determined later, hence the term undetermined. We take the Taylor expansion of the function at the pre-specified points and combine them according to the linear combination. Finally equating the coefficients of the derivatives on both sides of the equation, we obtain the values of the coefficients.

We shall now use this method to demonstrate how to obtain the approximation to the second derivative of f at x using three points $(x - h, x, x + h)$. We begin with writing the second derivative as a linear combination of the values of the function at the given points with unknown coefficients a, b, c.

$$f''(x) = af(x - h) + bf(x) + cf(x + h). \tag{5.7}$$

Now we find the Taylor series expansion of f at the given points as

$$f(x \pm h) = f(x) \pm hf'(x) + \frac{h^2}{2} f''(x) \pm \frac{h^3}{6} f'''(\xi_\pm),$$

where (assuming that $h > 0$) $x - h < \xi_- < x < \xi_+ < x + h$. Now we substitute these in (5.7) as

$$f''(x) = (a + b + c) f(x) + h(c - a) f'(x) + \frac{h^2}{2}(c + a) f''(x) + O(h^3).$$

Equating the coefficients of $f(x)$, $f'(x)$ and $f''(x)$ on both sides of the equation we obtain the linear system

$$\begin{aligned} a + b + c &= 0, \\ c - a &= 0, \\ c + a &= \frac{2}{h^2}. \end{aligned}$$

The system has the unique solution $a = 1/h^2, b = -2/h^2, c = 1/h^2$. Hence we obtain the familiar second-order approximation of the second derivative:

$$f''(x) = \frac{f(x + h) - 2f(x) + f(x - h)}{h^2} + O(h^3).$$

5.4 Exercises

1. Assuming that $f(x)$ has four continuous derivatives in the interval $(x - h, x + h)$, verify that the following is a second-order approximation of the second derivative

$$f''(x) \approx \frac{f(x + h) - 2f(x) + f(x - h)}{h^2}, \tag{5.8}$$

   with truncation error given by

$$-\frac{h^2}{12} f^{(4)}(\xi), \quad \xi \in (x - h, x + h).$$

2. Prove Eq. (5.5).
3. Prove Eq. (5.6).
4. Use $n = 2$ and $k = 0$ in (5.4) to obtain

$$f'(x) = \frac{-3f(x) + 4f(x + h) - f(x + 2h)}{2h} + \frac{f'''(\xi)}{3} h^2,$$

   which is a one-sided, second-order approximation of the first derivative.

Chapter 6
Numerical Methods for PDE



The time evolution of prices of different financial quantities is often represented as a partial differential equation (PDE) with independent variables being time and prices of some other, often underlying, assets. Under the Black Scholes model these PDE's can be solved analytically for European Call and Put options. However for other options, like barrier and lookback, such solutions may not be available. Also, if the model assumptions fail, then these analytic solutions are not valid. But general method of solution of PDEs may be applied. In this chapter we shall develop such general methods to solve PDE's numerically.

6.1 Introduction

Let $V(S_t, t)$ be the price of an option at time t when the share price of the underlying stock is S_t. See Appendix A.1 for background on mathematical finance that is used in what follows. Under the Black–Scholes set-up, we have a risk-less asset bond B_t and a risky asset stock S_t. They evolve as

$$\begin{aligned} \mathrm{d}B_t &= rB_t\mathrm{d}t \\ \mathrm{d}S_t &= \mu S_t\mathrm{d}t + \sigma S_t\mathrm{d}W_t, \end{aligned} \tag{6.1}$$

where r is the interest rate, μ is the drift, σ is the volatility and W is a standard Brownian motion(BM). We apply Ito's formula to the option price to get

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/978-981-19-2008-0_6.

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

R. Sen and S. Das, *Computational Finance with R*, Indian Statistical Institute Series,
https://doi.org/10.1007/978-981-19-2008-0_6

$$\begin{aligned} dV(S_t, t) &= \frac{\partial V}{\partial t}dt + \frac{\partial V}{\partial S}dS_t + \frac{1}{2}\frac{\partial^2 V}{\partial S^2}\sigma^2 S_t^2 dt \\ &= \frac{\partial V}{\partial t}dt + \frac{\partial V}{\partial S}\mu S_t dt + \frac{\partial V}{\partial S}\sigma S_t dW_t + \frac{1}{2}\frac{\partial^2 V}{\partial S^2}\sigma^2 S_t^2 dt. \end{aligned}$$

Consider the discounted option price $B_t^{-1}V(S_t, t)$. By the Fundamental Theorem of Arbitrage Pricing (see Appendix), the discounted option price must be a martingale under the risk-neutral measure. Also, under the risk-neutral measure $\mu = r$. We have, from the above,

$$d(B_t^{-1}V(S_t, t)) = d\left(\frac{1}{B_t}\right)V(S_t, t) + \frac{1}{B_t}dV(S_t, T).$$

For a martingale, the coefficient of the dt term has to be zero, otherwise there is a systematic drift. Thus we have,

$$-rV + \frac{\partial V}{\partial t} + rS_t\frac{\partial V}{\partial S} + \frac{1}{2}\sigma^2 S_t^2\frac{\partial^2 V}{\partial S^2} = 0. \tag{6.2}$$

This is called the Black–Scholes PDE and the boundary condition is given by the pay-off of the option. For example, for a European call option with maturity T and strike K, the boundary condition is $V(S_T, T) = (S_T - K)_+$. Thus we can obtain the Black Scholes price of an option as solution of the PDE (6.2).

Depending on the situation, we may have other PDE's and boundary conditions. Here we define new kinds of path-dependent options. Various other possibilities, particularly discrete time versions, are presented in Sect. 10.4.

Barrier options A typical example of a Barrier option is one that gets 'knocked out' if the underlying asset crosses a pre-specified level. For instance, a down-and-out call with barrier b, strike K, and expiration T has payoff $1\{\tau(b) > T\}(S(T) - K)_+$, where $\tau(b) = inf\{t : S(t) \leq b\}$ is the first time the price of the underlying asset drops below b (understood to be ∞ if $S(t) > b$ for all t) and $1\{\}$ denotes the indicator of the event in braces. This option has zero pay-off if the stock price goes below b at any time before T. If that does not happen then the payoff is same as that of a Call option. A down-and-in call has payoff $1\{\tau(b) \leq T\}(S(T) - K)_+$: it gets 'knocked in' only when the underlying asset crosses the barrier. Up-and-out and up-and-in calls and puts are defined analogously.

Lookback options Like Barrier options, Lookback options depend on extremal values of the underlying asset price. Lookback puts and calls expiring at T have payoffs $(\max_{\{0<u<T\}} S(u) - S(T))$ and $(S(T) - \min_{\{0<u<T\}} S(u))$ respectively. A Lookback call, for example, may be viewed as the profit from buying at the lowest price over $u \in (0, T)$ and selling at the final price $S(T)$.

For Barrier options with barriers at a and b, we have, in addition to the conditions of European options, the boundary conditions $V(a,T) = V(b,T) = 0$. For Lookback options V is a function of S_t, t and $M_t = \max\{S_u : 0 \le u \le t\}$, that is we have $V(S,t,M)$. There we have the additional boundary condition $\frac{\partial V}{\partial M}(S,t,S) = 0$. Following is an example in multiple dimensions with a basket option.

Example 6.1.1. **PDE for basket option** Suppose we have two assets whose prices S_1 and S_2 evolve according to geometric Brownian motion as in (6.1) with volatilities σ_1 and σ_2 with two BMs W_1 and W_2. Note that under the risk-neutral measure, the drifts are both r. There is a correlation ρ between the two BM's. Consider a basket option, that is, an option whose payoff depends on the prices of both of these assets. Then, by the above argument, we get the PDE

$$-rV + \frac{\partial V}{\partial t} + rS_1\frac{\partial V}{\partial S_1} + \frac{1}{2}\sigma_1^2 S_1^2\frac{\partial^2 V}{\partial S_1^2} + rS_2\frac{\partial V}{\partial S_2} + \frac{1}{2}\sigma_2^2 S_2^2\frac{\partial^2 V}{\partial S_2^2} + \rho\sigma_1\sigma_2 S_1 S_2\frac{\partial^2 V}{\partial S_1 \partial S_2} = 0. \quad (6.3)$$

Similar equations can be derived in higher dimensions for basket options on multiple assets.

Thus we have seen that PDE's arise often and quite naturally in option pricing. Analytical solutions can be obtained only in very few cases. Thus, to price and hedge options properly, we need to learn how to solve PDE's numerically.

The first step is to classify the PDE since the choice of numerical method will depend on the characteristics of the PDE. The order of a PDE is the highest order of derivatives involved. For example in Eq. (6.2), the highest order is the second derivative. So it a second order PDE. Also, Eq. (6.2) is linear in the derivatives, making it a linear PDE. The general form of a second order linear PDE of a function ϕ of x and y is given by

$$a\frac{\partial^2\phi}{\partial x^2} + b\frac{\partial^2\phi}{\partial x\partial y} + c\frac{\partial^2\phi}{\partial y^2} + d\frac{\partial\phi}{\partial x} + e\frac{\partial\phi}{\partial y} + f\phi + g = 0. \quad (6.4)$$

This is further classified as hyperbolic, parabolic, or elliptic depending on the sign of $b^2 - 4ac$ being positive, zero, or negative respectively. Note that the Black–Scholes PDE is parabolic.

6.2 Finite Difference Methods

We have a function $u : D \to \mathbb{R}$ with $D = [a,b] \times [0,\infty)$. The first argument of u denotes the space, e.g., price of an underlying asset and the second argument denotes time. We shall illustrate the finite difference methods with the following PDE, known as the diffusion equation or heat equation:

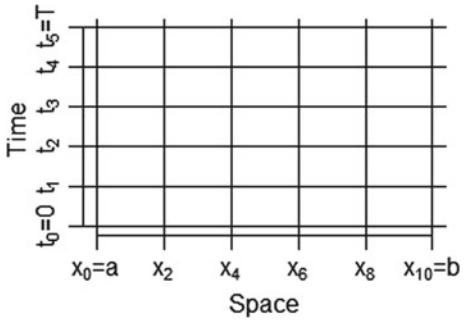


Fig. 6.1 Illustration of a grid

$$\frac{\partial u}{\partial t} = c^2 \frac{\partial^2 u}{\partial x^2}, \quad (x, t) \in D, \tag{6.5}$$

with the initial condition

$$u(x, 0) = u_0(x) \tag{6.6}$$

and the boundary conditions

$$u(a, t) = g_a(t) \quad \text{and} \quad u(b, t) = g_b(t), \tag{6.7}$$

where u_0, g_a, g_b are continuous functions. The diffusion Equation (6.5) with the initial Condition (6.6) and the boundary Conditions (6.7) is well-posed, i.e., there exists a unique solution that depends continuously on u_0, g_a and g_b.

For the finite difference method we proceed by choosing a set of discrete grid points as follows. Decide on the desired level of granularity on the state variable as $\Delta x = (b - a)/N$, where N is an integer. Similarly, for the time variable, choose an appropriate small enough Δt. The grid points are then (x_j, t_n), where $x_j = a + j\Delta x, j = 0, \ldots, N$ and $t_n = n\Delta t, n = 0, 1, \ldots$

Now the partial derivatives are approximated by the corresponding difference quotients. The partial derivatives with respect to the state variable are always approximated by central difference quotients, i.e.,

$$u_x \approx \frac{u_{j+1}^n - u_{j-1}^n}{2\Delta x} \quad \text{and} \quad u_{xx} \approx \frac{u_{j+1}^n - 2u_j^n + u_{j-1}^n}{(\Delta x)^2}$$

at a grid point (x_j, t_n), where $u_j^n = u(x_j, t_n)$. Here u_t can be approximated in different ways and accordingly we have different schemes namely: explicit, implicit, and Crank–Nicolson. We discuss these in further detail below. The explicit scheme requires fewer computations and is therefore faster. The Crank–Nicolson scheme has lower error as will be shown in Sect. 6.4.

6.2.1 Explicit Scheme

The first option is to approximate u_t by a forward difference quotient as follows

$$u_t \approx \frac{u_j^{n+1} - u_j^n}{\Delta t} \, at(j, n).$$

Then the difference equation corresponding to (6.5) at grid point (x_j, t_n) is

$$w_j^{n+1} = \lambda w_{j+1}^n + (1 - 2\lambda) w_j^n + \lambda w_{j-1}^n, \tag{6.8}$$

where $\lambda = c^2 \Delta t/(\Delta x)^2$. The initial condition is $w_j^0 = u_0(x_j), j = 0, \ldots, N$, and the boundary conditions are $w_0^n = g_a(t_n)$ and $w_N^n = g_b(t_n), n = 0, 1, \ldots$ The difference Equations (6.8) can be solved explicitly. We proceed from t_0 and move upwards in the grid in Fig. 6.1 as given by the following algorithm.

Algorithm 6.2.1

1. $t = t_0 : w_j^0 = u_0(x_j), j = 0, \ldots, N,$
2. $t = t_1 : w_0^1 = g_a(t_1),$
   $w_j^1 = \lambda(w_{j+1}^0 + (1 - 2\lambda) w_j^0 + \lambda w_{j-1}^0), j = 1, \ldots, N - 1,$
   $w_N^1 = g_b(t_1)$
3. $t = t_2 : w_0^2 = g_a(t_2),$
   $w_j^2 = \lambda(w_{j+1}^1 + (1 - 2\lambda) w_j^1 + \lambda w_{j-1}^1), j = 1, \ldots, N - 1,$
   $w_N^2 = g_b(t_2)$
4. Continue until a reasonably large value of t.

Note that at any point t_i in time, the values of w for the previous time point are used. These are already known from from the previous steps of the algorithm.

6.2.2 Implicit Scheme

The second option is to approximate u_t by a backward difference quotient as follows.

$$u_t \approx \frac{u_j^{n+1} - u_j^n}{\Delta t} \quad \text{at} \quad (j, n + 1).$$

Then the difference equation corresponding to (6.5) at grid point $(j, n + 1)$ is

$$-\lambda w_{j+1}^{n+1} + (1 + 2\lambda) w_j^{n+1} - \lambda w_{j-1}^{n+1} = w_j^n. \tag{6.9}$$

Now we cannot obtain an explicit solution as before, since w_j^{n+1} is not directly expressed in terms of the w^ns of the previous time step. Note that, as before, at $t = t_0$, $w_j^0 = u_0(x_j), j = 0, \ldots, N$. At $t = 1$, we have the following set of equations in matrix form

$$\begin{pmatrix} 1+2\lambda & -\lambda & & \\ -\lambda & 1+2\lambda & -\lambda & \\ & \vdots & \vdots & \\ & & -\lambda & 1+2\lambda \end{pmatrix} \begin{pmatrix} w_1^1 \\ w_2^1 \\ \vdots \\ w_{N-1}^1 \end{pmatrix} = \begin{pmatrix} w_1^0 \\ w_2^0 \\ \vdots \\ w_{N-1}^0 \end{pmatrix}.$$

We know the right-hand side from the previous step and need to solve for $(w_1^1, w_2^1, \ldots, w_{N-1}^1)^T$. Since the matrix is in tri-diagonal form it is simple to do an LU factorization using the Crout algorithm described below. The solution can then be obtained by the algorithm described in Sect. 2.2 for solving an LU system of linear equations.

Crout Algorithm The decomposition for $A = LU$, where A is a tri-diagonal matrix, L and U are lower and upper triangular respectively, has the following form.

$$\begin{pmatrix} a_{1,1} & a_{1,2} & 0 & 0 & \ldots \\ a_{2,1} & a_{2,2} & a_{2,3} & 0 & \ldots \\ 0 & a_{3,2} & a_{3,3} & a_{3,4} & \ldots \\ \vdots & \vdots & \vdots & \ldots & \\ \ldots & & 0 & a_{n,n-1} & a_{n,n} \end{pmatrix} = \begin{pmatrix} l_{1,1} & 0 & 0 & \ldots \\ l_{2,1} & l_{2,2} & 0 & \ldots \\ 0 & l_{3,2} & l_{3,3} & \ldots \\ \vdots & \vdots & \vdots & \ldots \\ \ldots & 0 & l_{n,n-1} & l_{n,n} \end{pmatrix} \begin{pmatrix} 0 & u_{1,2} & \ldots \\ 0 & u_{2,3} & \ldots \\ 0 & 0 & \ldots \\ \vdots & \vdots & \ldots \\ \ldots & 0 & 0 \end{pmatrix}.$$

The algorithm goes as follows.

$$\begin{aligned} l_{1,1} &= a_{1,1} \\ u_{1,2} &= \frac{a_{1,2}}{l_{1,1}}. \end{aligned}$$

For $i = 2, \ldots, n$,

$$\begin{aligned} l_{i,i-1} &= a_{i,i-1} \\ l_{i,i} &= a_{i,i} - l_{i,i-1} u_{i-1,i} \\ u_{i,i+1} &= \frac{a_{i,i+1}}{l_{i,i}}. \end{aligned}$$

6.2.3 Crank–Nicolson Scheme

The Crank–Nicolson scheme is the average of the explicit scheme at (j, n) and the implicit scheme at $(j, n+1)$. The resulting difference equation is

$$\frac{w_j^{n+1} - w_j^n}{\Delta t} = \frac{1}{2}c^2 \frac{w_{j-1}^n - 2w_j^n + w_{j+1}^n}{(\Delta x)^2} + \frac{1}{2}c^2 \frac{w_{j-1}^{n+1} - 2w_j^{n+1} + w_{j+1}^{n+1}}{(\Delta x)^2},$$

which on simplification becomes

$$-\frac{\lambda}{2}w_{j-1}^{n+1} + (1+\lambda)w_j^{n+1} - \frac{\lambda}{2}w_{j+1}^{n+1} = \frac{\lambda}{2}w_{j-1}^n + (1-\lambda)w_j^n + \frac{\lambda}{2}w_{j+1}^n. \quad (6.10)$$

The difference Equations (6.10), together with the initial and boundary conditions as before, can be solved using Crout algorithm as this is again a tri-diagonal system.

6.3 Solution to Black–Scholes PDE by Transformation to Heat Equation

In Sect. 6.2 we have described finite difference methods and demonstrated their use in solving the heat Equation (6.5). In finance we are interested in solving equations of the type (6.2), the Black–Scholes PDE. The latter can actually be transformed suitably to the heat equation. Here we shall first demonstrate that and then show how to execute the solution.

We apply the following transformations of variables

$$\tau = T - t,$$
$$x = \log S.$$

Now consider an option with value $V(S, t)$ when stock price is S at time t. Set $u(x, \tau) = V(S, t)$. Then

$$\frac{\partial V}{\partial t} = -\frac{\partial u}{\partial \tau},$$
$$S\frac{\partial V}{\partial S} = \frac{\partial u}{\partial x},$$
$$S^2\frac{\partial^2 V}{\partial S^2} = \frac{\partial^2 u}{\partial x^2} - \frac{\partial u}{\partial x}.$$

Using these results, the Black–Scholes PDE (6.2) is equivalent to

$$-\frac{\partial u}{\partial \tau} + r\frac{\partial u}{\partial x} + \frac{1}{2}\sigma^2\left(\frac{\partial^2 u}{\partial x^2} - \frac{\partial u}{\partial x}\right) - ru = 0. \quad (6.11)$$

The initial condition will depend on the payoff of the specific option. For example, for a call option it is $u(x,0) = V(S,T) = (\exp(x) - K)_+$. When $x \to \infty$, then $S = \exp(x) \to \infty$, implying $V(S,t) \approx S$. When $x \to -\infty$, then $S = \exp(x) \to 0$, implying $V(S,t) \to 0$. Thus the boundary conditions are

$$\begin{aligned} u(x,\tau) &\approx \exp(x) \quad \text{as} \quad x \to \infty, \\ u(x,\tau) &\to 0 \quad \text{as} \quad x \to -\infty. \end{aligned}$$

Now we shall define another transformation to get Eq. (6.11) into the form of the heat equation by eliminating the terms corresponding to u and $\frac{\partial u}{\partial x}$. Define

$$w(x,\tau) = \exp(\alpha x + \beta\tau)u(x,\tau),$$

for some parameters α and β to be chosen appropriately. Then, with $C = \exp(-\alpha x - \beta\tau)$, we have

$$\begin{aligned} u_\tau &= C(-\beta w + w_\tau), \\ u_x &= C(-\alpha w + w_x), \\ u_{xx} &= C(-\alpha(-\alpha w + w_x) + (-\alpha w_x + w_{xx})) = C(\alpha^2 w - 2\alpha w_x + w_{xx}). \end{aligned}$$

So (6.11) is equivalent to

$$\begin{aligned} C(-\beta w + w_\tau) = &\frac{1}{2}\sigma^2 C(\alpha^2 w - 2\alpha w_x + w_{xx}) \\ &+ (r - \frac{1}{2}\sigma^2)C(-\alpha w + w_x) - rCw. \end{aligned} \tag{6.12}$$

In order to cancel the w and w_x terms we need to have

$$\begin{aligned} -\beta &= -\frac{1}{2}\sigma^2\alpha^2 - (r - \frac{1}{2}\sigma^2)\alpha - r, \\ 0 &= \frac{1}{2}\sigma^2(-2\alpha) + r - \frac{1}{2}\sigma^2. \end{aligned}$$

This is equivalent to

$$\begin{aligned} \alpha &= \frac{1}{\sigma^2}(r - \frac{1}{2}\sigma^2), \\ \beta &= \frac{1}{2\sigma^2}(r - \frac{1}{2}\sigma^2)^2 + r. \end{aligned}$$

With this choice of α and β the Eq. (6.11) is equivalent to

$$w_\tau = \frac{1}{2}\sigma^2 w_{xx}, \tag{6.13}$$

with initial condition $w(x,0) = \exp(\alpha x)u(x,0) = \exp(\alpha x)(\exp(x) - K)_+$, and boundary condition $w(x,\tau) \to 0$ as $x \to -\infty$, $w(x,\tau) \approx \exp(\alpha x + \beta\tau)\exp(x)$ as $x \to \infty$.

Implementation

The following is the full algorithm for solving the Black–Scholes equation numerically.

Algorithm for solving Black–Scholes equation

1. Choose the interval $[a,b]$ and the number of points in the grid N, M. The resulting grid points are (x_j, τ_n) : $x_j = a + j \triangle x, j = 0, 1, \ldots, N$ and $\tau_n = n \triangle \tau, n = 0, 1, \ldots, M$.
2. Solve the heat equation with proper initial and boundary conditions and step size λ using the finite difference methods in Sect. 6.2.
3. If $w(x_j, \tau_M), j = 0, 1, \ldots, N$ are the solutions obtained at the final time step, then the option price at time t is $V(S_j, t) = \exp(-\alpha x_j - \beta(T - t))w(x_j, \tau_M) \quad j = 0, 1, \ldots, N$, where $S_j = \exp(x_j)$ and $T - t \equiv \tau M$.

The following is the R code for this execution with the European call option using the explicit scheme. For this particular option we can obtain the option price analytically and the function was presented as `blackscholes` in Sect. 3.1. We present the ratio of these two prices as a function of the strike K in Fig. 6.2. Note that the ratio is close to one which means the two methods give similar results. The ratio is not exactly one due to the error involved in numerical approximation. We discuss this error in Sect. 6.4.

```
r = 0.01
s = 0.3
s0 = 50
T = 1
f = function(z) max(z,0)
a = -8
b = 8
N = 1000
M = 1000
x = seq(a,b,(b-a)/N)
al = (r -s^2/2)/s^2
be = (r - s^2/2)^2/(2*s^2) + r
la = (s^2/2)*(1/M)/((b-a)/N)^2
price = K =rep(0,8)
pp = price
for(l in 1:8){
```

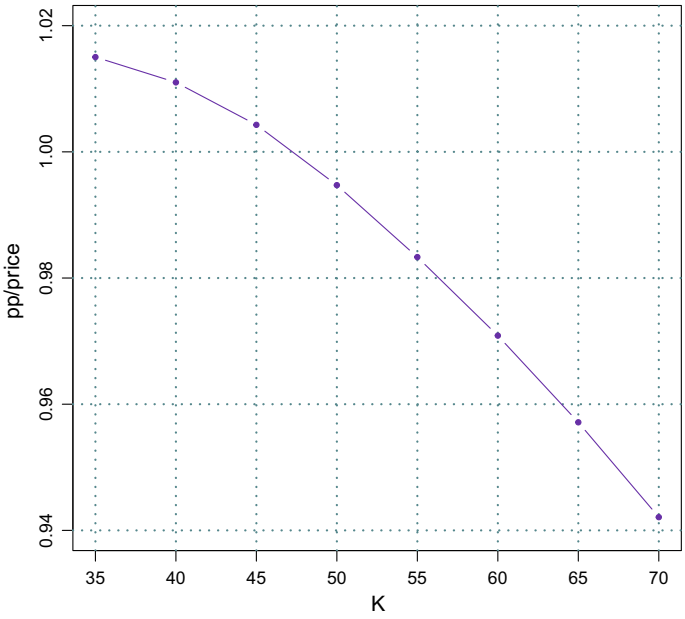


Fig. 6.2 Ratio of price of call option obtained numerically and analytically plotted against strike

```
  K[l] = 30+5*l
  w = matrix(0,M,N)
  for(j in 1:N){w[j,1] = exp(al*x[j])*f(exp(x[j])-K[l])}
  for(i in 2:M){
    w[1,i] = 0
    w[N,i] = exp(8)
    for (j1 in 2:(N-1)){
      w[j1,i] = la*w[j1-1,i-1]+(1-2*la)*w[j1,i-1]+la*w[j1+1,i-1]
    }
    jj = ceiling((log(s0)-a)/((b-a)/N))
    price[l] = w[jj,M]*exp(-al*x[jj]-be*T)
    pp[l] = blackscholes(s0,K[l],r,T,s,0)
  }
}

plot(NULL,ylim=c(0.94,1.02),xlim=c(K[1],K[8])
     ,ylab='pp/price',xlab='K')
grid(col='cadetblue4',lty=3,lwd=2)
points(K,pp/price,pch=20,col='purple3',type='b')
```

6.4 Error

In all the methods described in this chapter we have approximated a continuous problem by a discrete grid. We are interested in the truncation error obtained by substituting the exact solution of the continuous problem into the numerical scheme. A necessary condition for the convergence of the numerical solutions to the continuous solution is that the local truncation error (LTE) tends to zero as the step size goes to zero. In that case the method is said to be consistent. It can be shown that all three finite difference methods are consistent. For the heat equation, the explicit and implicit schemes have local truncation errors $O(\Delta t, (\Delta x)^2)$, while that of the Crank–Nicolson scheme is $O((\Delta t)^2, (\Delta x)^2)$. We now proceed to prove these results.

For the explicit scheme we get for the LTE at (j, n)

$$E_j^n = \frac{u(x_j, t_{n+1}) - u(x_j, t_n)}{\Delta t} - c^2 \frac{u(x_{j-1}, t_n) - 2u(x_j, t_n) + u(x_{j+1}, t_n)}{(\Delta x)^2}.$$

With the help of a Taylor expansion at (x_j, t_n) we find that

$$\frac{u(x_j, t_{n+1}) - u(x_j, t_n)}{\Delta t} = u_t(x_j, t_n) + O(\Delta t),$$

$$\frac{u(x_{j-1}, t_n) - 2u(x_j, t_n) + u(x_{j+1}, t_n)}{(\Delta x)^2} = u_{xx}(x_j, t_n) + O((\Delta x)^2). \tag{6.14}$$

Hence

$$E_j^n = u_t(x_j, t_n) - u_{xx}(x_j, t_n) + O((\Delta t), (\Delta x)^2)$$

where $u_t(x_j, t_n) - u_{xx}(x_j, t_n) = 0$ at the solution.

For the Crank–Nicolson method for LTE, we have

$$E_j^n = \frac{u(x_j, t_{n+1}) - u(x_j, t_n)}{\Delta t} - \frac{c^2}{2} \frac{u(x_{j-1}, t_n) - 2u(x_j, t_n) + u(x_{j+1}, t_n)}{(\Delta x)^2} - \frac{c^2}{2} \frac{u(x_{j-1}, t_{n+1}) - 2u(x_j, t_{n+1}) + u(x_{j+1}, t_{n+1})}{(\Delta x)^2}.$$

For the second and third terms, we have an expansion like Eq. (6.14). Therefore, together, they equal

$$-\frac{c^2}{2}(u_{xx}(x_j, t_n) + u_{xx}(x_j, t_{n+1})) + O((\Delta x)^2) = -\frac{c^2}{2}(2u_{xx}(x_j, t_n) + u_{xxt}(x_j, t_n)\Delta t) + O(\Delta t)^2 + O((\Delta x)^2). \tag{6.15}$$

The first term, upon Taylor expansion, gives

$$u_t(x_j, t_n) + u_{tt}(x_j, t_n)\frac{\Delta t}{2} + O(\Delta t)^2.$$

In the expression for E_j^n we add these terms and the coefficient of Δt is

$$\frac{1}{2}(u_{tt}(x_j, t_n) - c^2 u_{xxt}(x_j, t_n)) = 0,$$

since the original heat equation satisfies $u_t = c^2 u_{xx}$. Thus

$$E_j^n = u_t(x_j, t_n) - u_{xx}(x_j, t_n) + O((\Delta t)^2, (\Delta x)^2),$$

where $u_t(x_j, t_n) - u_{xx}(x_j, t_n) = 0$ at the solution.

6.5 Exercises

1. Prove Eq. (6.3).
2. Prove Eq. (6.15).
3. Apply the Crank–Nicolson scheme directly to the Black–Scholes PDE in Eq. (6.2), i.e., there is no transformation of variables and write out the resulting difference equations.
4. Write a program to solve the Black–Scholes PDE using the result of the previous exercise and the Crout algorithm. The inputs are the interest rate r, the volatility σ, the current time t, the expiry time T, the strike price X, the maximum price $S_{\max}$, the number of intervals N in $[0, S_{\max}]$, and the number of subintervals M in $[t, T]$. The output are the asset prices $S_j, j = 0, 1, \ldots, N$, at time t, and their corresponding European call and put prices.

Chapter 7
Optimization



Optimization refers to the minimization (or maximization) of a given objective function of several decision variables that satisfy functional constraints. Let f and g_i be functions from $\mathbb{R}^n$ to $\mathbb{R}$. Then, a generic optimization problem takes the following form:

$$\begin{aligned} & \min\ f(x) \\ g_i(x) &= b_i, \quad i \in \mathcal{E} \\ g_i(x) &\geq 0, \quad i \in \mathcal{I}. \end{aligned} \tag{7.1}$$

Here f is the objective function, x is a collection of n decision variables, and g_i are the functional constraints. If the set S of x satisfying the constraints g_i is non-empty, then the problem is said to be feasible. Any x belonging to S is a feasible solution. Here is an example of a non-feasible problem.

Example 7.0.1. Suppose $n = 2$ and we have the following constraints

$$\begin{aligned} x_1 + x_2 &= 1 \\ x_1 - x_2 &= 3 \\ x_1 &\geq 0 \\ x_2 &\geq 0. \end{aligned}$$

Here the first two equations have a single solution $(2, -1)$ which violates the last inequality.

If there exists a sequence $x_k \in S$ such that $f(x_k) \to -\infty$ as $k \to \infty$, then the problem is called unbounded. We try to find the optimal solution to bounded feasible problems.

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/978-981-19-2008-0_7.

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

R. Sen and S. Das, *Computational Finance with R*, Indian Statistical Institute Series,
https://doi.org/10.1007/978-981-19-2008-0_7

7.1 Linear Programming

In a linear programming (LP) problem, f and all the g_i's are linear functions. The standard form of an LP problem is given below:

$$\begin{aligned} \min\, & c^T x \\ Ax &= b, \\ x &\geq 0. \end{aligned} \tag{7.2}$$

Here c is an n dimensional vector and A is an $m \times n$ dimensional matrix. Thus there are m equality and n inequality constraints.

Note that any LP problem as in (7.1) with linear f and g_i can be reduced to the standard form. If original the problem is of maximization of $f(x)$, then it is equivalent to minimization of $f_1(x) = -f(x)$. Linear functions g_i for $i \in \mathcal{I}$ can be reduced to equalities by adding new surplus variables x_{n+i} that are non-negative, as in the following example.

Example 7.1.1. Suppose $n = 2$ and one constraint in the problem is $g(x) = 2x_1 + 3x_2 \geq 5$. Then we introduce $x_3 = 5 - (2x_1 + 3x_2)$. The new constraints become

$$\begin{aligned} 2x_1 + 3x_2 + x_3 &= 5 \\ x_3 &\geq 0. \end{aligned}$$

Together, these new constraints are equivalent to the original constraint $g(x) \geq 5$.

Before proceeding further with the theory and solution of LP problems, we present a practical example which reduces to an LP problem. The objective is to maximize profits with optimal allocation of resources under constraints.

Example 7.1.2. **(Optimal allocation of resources)** A plant can manufacture 4 types of products and can sell them at profits of s_i per unit of the i-th product for $i = 1, \ldots, 4$. The plant manager wants to maximize profits, but there are constraints of labor. Each product needs two types of labor, assembly and packaging. The time required to process one unit of product i with j-th type of labor is p_{ij} hours, for $i = 1, 2, 3, 4$ and $j = 1, 2$. For example, the first type of product takes p_{11} hours for assembly and p_{12} hours for packaging. The man-hours of available workers for the two types of labor per week are C_1 and C_2 respectively. The problem is to find the number of units of each type of product to be produced each week. Thus we want to find $x_1, \ldots x_4$, the amount of each product to be manufactured per week, to maximize $\sum_{i=1}^{4} s_i x_i$ subject to the two constraints $\sum_{i=1}^{4} p_{ij} x_i \leq C_j$ for $j = 1, 2$. Here

$$n = 4, c = \begin{pmatrix} s_1 \\ s_2 \\ s_3 \\ s_4 \end{pmatrix}, A = \begin{pmatrix} p_{11} & p_{21} & p_{31} & p_{41} \\ p_{12} & p_{22} & p_{32} & p_{42} \end{pmatrix}, b = \begin{pmatrix} C_1 \\ C_2 \end{pmatrix}.$$

This is in non-standard form with two inequalities and a maximization. This can be changed to standard form according to the procedure described above by taking the negative of the objection function and adding two extra variables.

For an LP problem in standard form as in (7.2) we define the dual problem as

$$\begin{aligned} &\max b^T y, \\ A^T y \ &\leq \ c. \end{aligned} \tag{7.3}$$

Then we have the following theorems which are often helpful in solving LP problems. We prove the first theorem and state the second theorem without proof. Right after the theorems we present an example that uses the dual set-up to derive the solution of an LP problem.

Theorem 7.1.1. *(Weak Duality Theorem) Let x be any feasible solution to the primal LP (7.2) and y be any feasible solution to the dual LP (7.3). Then*

$$c^T x \geq b^T y$$

Proof 7.1.1. Since $x \geq 0$ and $c - A^T y \geq 0$, the inner product of these two vectors must be non-negative.

$$x^T (c - A^T y) = x^T c - x^T A^T y = c^T x - b^T y$$

Theorem 7.1.2. *(Strong Duality Theorem) If the primal (dual) problem has an optimal solution x (y), then the dual (primal) problem has an optimal solution y (x) such that $c^T x = b^T y$.*

Example 7.1.3. Consider the following linear programming problem with 5 variables and two constraints:

$$\begin{aligned} \min \quad & 6x_1 + 5x_2 + 4x_3 + 5x_4 + 6x_5 \\ x_1 + x_2 + x_3 + x_4 + x_5 &\geq 3 \\ 5x_1 + 4x_2 + 3x_3 + 2x_4 + x_5 &\geq 14. \end{aligned}$$

Then the dual problem is

$$\begin{aligned} \max \quad & 3y_1 + 14y_2 \\ y_1 + 5y_2 &\leq 6 \\ y_1 + 4y_2 &\leq 5 \\ y_1 + 3y_2 &\leq 4 \\ y_1 + 2y_2 &\leq 5 \\ y_1 + y_2 &\leq 6. \end{aligned}$$

Observe by sketching the regions for y, that only the first and third inequalities are sufficient. This is because, the area enclosed by them is contained in that enclosed by the other equations under the constraints of non-negativity of the variables. The possible points where the maximum occurs are the points of intersection of these two lines and the axes. These points are respectively (0, 1.2), (1, 1), and (4, 0). The values of the objective function at these points are 16.8, 17, and 12 respectively. So the optimum y is (1, 1). Since the first and third inequalities are both satisfied, only the corresponding x values will be non-zero. Hence we solve the inequalities in the primal problem as equations with x_2, x_4 and x_5 as zero. This gives $x_1 = 2.5$ and $x_3 = 0.5$. Note that with the solution (2.5, 0, 0.5, 0, 0) the value of the objective function in the primal problem equals $6x_1 + 4x_3 = 17$ which agrees with the value of the objective function for the dual problem with the optimal solution. Hence the optimal solution of the primal problem is

$$(2.5, 0, 0.5, 0, 0).$$

The best known and most successful methods for solving LPs are interior-point methods (IPMs) and the simplex method. We shall not go into the details of these algorithms, but explain the general ideas.

Suppose we have an LP problem in standard form, possibly with some surplus variables. The feasible region is given by the equation $Ax = b$. If the rank of A is strictly less than the number of rows, then some variables are dropped to obtain a set of equations where A is full row rank. The simplex method then constructs a tableau of the following form,

$$\begin{pmatrix} 1 & c^T & 0 \\ 0 & A & b \end{pmatrix}.$$

From here the first task is to find a basic feasible solution. This entails doing row operations, multiplication and addition of rows, on the tableau to get to a point where for each row other than the first, there is exactly one element that is one and $(n-1)$ elements that are zeros. The elements that are one represent the set of variables that are nonzero in the feasible solution. The corresponding values in the last column are the values that these variables take in the feasible solution. The method then proceeds iteratively by changing the set of variables, which amounts to selecting a column/variable to be used as a pivot for which the element in the first row is positive. The procedure stops when all such elements are negative and that is the optimal solution. This is the general idea of the simplex method. The idea of the IPMs will be described in Sect. 7.2.

The `R` library `lpSolAPI` can be used to solve LP problems. We demonstrate this below with an R code for the problem in Example 7.1.3.

```
library(lpSolveAPI)
# Construct the optimization problem
lprec <- make.lp(2, 5)
set.column(lprec, 1, c(1,5))
set.column(lprec, 2, c(1,4))
set.column(lprec, 3, c(1,3))
set.column(lprec, 4, c(1,2))
set.column(lprec, 5, c(1,1))
## Setup the objective function
set.objfn(lprec, c(6,5,4,5,6))
set.constr.type(lprec, c(">=", ">="))
set.rhs(lprec, c(3,14))
# Reun the optimization
solve(lprec)

## [1] 0

get.objective(lprec)

## [1] 17
```

We conclude this section with another example. This is regarding the optimal combination of loans with different maturities, rates of interest, and payment scheme to minimize total interest paid.

Example 7.1.4. **Optimal combination of loans** A company needs to take a loan of amount L to be paid off in monthly installments over T months. There are k sources of loans available with interest rates r_i, minimum payment per month p_i, maximum borrowing amount m_i, and term $T_i \leq T$ for $i = 1, \ldots, k$. The company cannot afford to pay more than an amount M every month. We want to find the break-up of the amount L among the various sources, L_i, and the amount to be repaid to each source x_{it} at time t so that the total payment is minimized. Thus the objective function to be minimized is $\sum_{i=1}^{k} \sum_{j=1}^{T} x_{it}$. The constraints are as follows

$$\begin{aligned}
&\sum_{i=1}^{k} L_i = L \\
&\quad L_i \leq m_i \quad i = 1, \ldots, k \\
&\quad x_{it} \geq p_i, \quad t = 1, \ldots, T_i \quad i = 1, \ldots, k \\
&\sum_{i=1}^{k} x_{it} \leq M, \quad t = 1, \ldots, T \\
&L_i r_i^{T_i} = \sum_{s=1}^{T_i} x_{is} r_i^{T_i - s}.
\end{aligned}$$

The first equation fixes the total amount of the loan. The second one takes care of the upper bound on the amount to be borrowed from each source. The third one is about the minimum payment each month. The fourth one is about the maximum available funds for the company to pay each month and the last one is about the term of the loan with respect to interest rate.

There are very prominent examples of the theory of LP problems related to arbitrage and risk neutral measures as discussed in the Appendix.

7.2 Quadratic Programming

In a quadratic programming (QP) problem, f is a quadratic function and all the g_i's are linear functions. A typical QP problem is given below:

$$\begin{aligned}
&\min \ \tfrac{1}{2} x^T Q x + c^T x \\
Ax &= b, \\
x &\geq 0.
\end{aligned} \tag{7.4}$$

Here A, b, c are as in the LP setting and Q is an $n \times n$ dimensional symmetric matrix. The reason to take Q as symmetric is that, given any matrix B one can always write $x^T B x = \frac{1}{2} x^T (B + B^T) x$ and then put $Q = B + B^T$ to get a symmetric Q.

The most common use of quadratic programming in finance is in portfolio optimization. We want to construct a portfolio, that is, invest the total available wealth as a linear combination of several assets. For a fixed level of return, we want to adjust the portfolio weights so that the risk in minimized. According to Markowitz portfolio theory (Markowitz, 1952), the measure of return is taken as the expectation and that of risk is taken as the variance. If μ is the vector of expectations of several assets and Σ is the covariance matrix, then we want to find a vector x that minimizes $x^t \Sigma x$ subject to $\mu^T x$ is a specified constant. Clearly this is a QP problem as the constraint is linear and the objective function is quadratic.

A twice differentiable function of several variables is convex on a convex set if and only if its Hessian matrix of second partial derivatives is positive semidefinite on the interior of the convex set. Note that the Hessian matrix of the objective function is Q. Since the feasible set is defined by linear constraints, it is a convex set. Therefore, when Q is positive semidefinite, the QP (7.4) is a convex optimization problem. In this case we have the following theorem.

Theorem 7.2.1. *(Karush–Kuhn–Tucker) Suppose that x is a local optimal solution of the QP given in (7.4) so that it satisfies $Ax = b; x \geq 0$ and assume that Q is a positive semidefinite matrix. Then, there exist vectors y and s such that the following conditions hold:*

$$\begin{aligned} A^T y - Qx + s &= c \\ s &\geq 0 \\ x_i s_i &= 0 \quad \forall i. \end{aligned}$$

Furthermore, x is a global optimal solution.

By the KKT theorem, when Q is positive semidefinite, then a local solution of the QP problem is also the global solution. By Wolf's method one can then develop modifications of the simplex method that can also solve convex quadratic programming problems. We shall not discuss that approach here, but describe an alternative algorithm called Interior Point Method (IPM).

Note that adding the original constraints to the equations in the KKT theorem, we have the following set of equations in matrix form,

$$\begin{aligned} A^T y - Qx + s - c &= 0, \\ Ax - b &= 0, \\ XSe &= 0. \end{aligned} \tag{7.5}$$

under the constraints $x \geq 0$ and $s \geq 0$. Here X and S are diagonal matrices with elements x and s and e is n-dimensional vector of ones. Thus we have $2n + m$ equations in same number of unknowns. However the system is nonlinear because of presence of the product terms $x_i s_i$. We cannot apply the methods of solving nonlinear equations, like Newton's method, directly because of the non-negativity constraints. So we introduce a modification in Newton's method so that the iterative step is in

the direction of the gradient, but multiplied by an appropriate factor to satisfy the inequality constraints. This leads to the following algorithm.

Algorithm for Interior Point Method

1. Start with the set of solutions to the first two lines of Eq. (7.5). This can be obtained easily as these are linear equations. Pick a point (x_0, y_0, s_0) in this set that satisfies the inequality constraints.
2. Note that the Jacobian of the system (7.5) in x, y, s at the k-th step is

$$J_k = \begin{pmatrix} -Q & A^T & I \\ A & 0 & 0 \\ S_k & 0 & X_k \end{pmatrix}.$$

For the Newton method, the iterative step would be

$$\begin{pmatrix} x_k \\ y_k \\ s_k \end{pmatrix} = \begin{pmatrix} x_{k-1} \\ y_{k-1} \\ s_{k-1} \end{pmatrix} - (J_{k-1})^{-1} \begin{pmatrix} 0 \\ 0 \\ X_{k-1}S_{k-1}e \end{pmatrix}. \tag{7.6}$$

We find α_{k-1} so that by replacing $(J_{k-1})^{-1}$ with $\alpha_k(J_{k-1})^{-1}$ in the Eq. (7.6), the new solution satisfies the inequality constraints and that is the modified iteration step.

In the following example, we carry out a portfolio optimization exercise in the sense of (Markowitz, 1952) as introduced in the beginning of this section with the `edhec` dataset in the `PortfolioAnalytics` library of `R`.

Example 7.2.1. **Portfolio Optimization** The dataset consists of historical daily returns on 13 indices from February 1997 to November 2019. We use the following R code to compute the mean and covariance matrix and then solve the QP problem for values of expected return between 0 to 1 in steps of 0.001. The solution is obtained using the function `solve.QP` in the library `quadprog` of R. The resulting plot of expectation vs the minimum variance is presented in Fig. 7.1. Such a plot is known as the efficient frontier as all other portfolios lie below this curve. The optimal weights, for every fixed value of expectation, can also be obtained as an output of the solve.QP program. For example, for the value of mean equal to 1, the optimal weights are (0.00 5.43 0.00 0.00 69.61 0.00 7.61 0.00 0.00 96.95 27.85 15.61 0.00).

```
library(PortfolioAnalytics)
library(quadprog)
data(edhec)
#head(edhec)
```

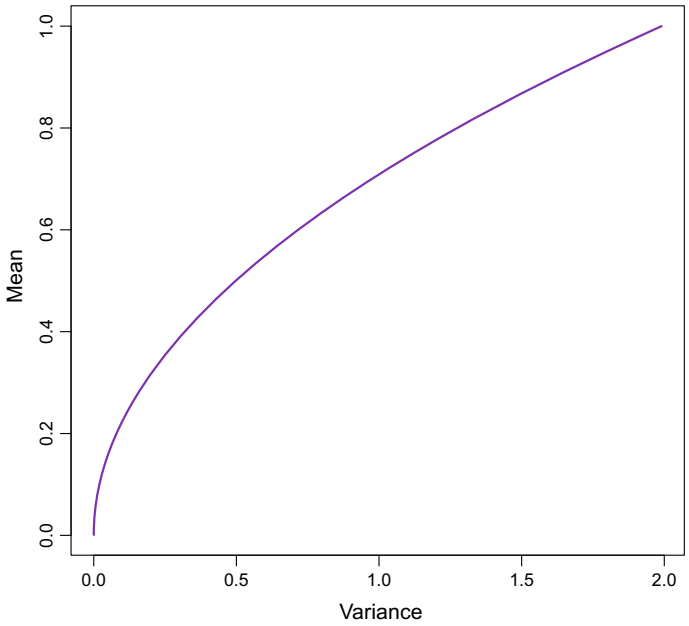


Fig. 7.1 Efficient frontier for Example 7.2.1

```
dim(edhec)

## [1] 275  13

x = rep(0,1000)
y = x
A = apply(edhec,2,mean)
n = length(A)
Amat1 = matrix(A,n,1)
Amat = cbind(Amat1, diag(rep(1,times=n),n))
Dmat = 2*var(edhec)
dvec = matrix(0,n,1)
for(i in 1:1000){
  y[i] = i*0.001
  bvec = matrix(c(y[i],rep(0,times=n)),n+1,1)
  S = solve.QP(Dmat,dvec,Amat,bvec)
  x[i] = S$value
}
plot(x,y,type="l", xlab="Variance", ylab="Mean"
     ,lwd=2,col='purple')
```

```
round(S$solution,2)

## [1]  0.00 5.43 0.00 0.00 69.61 0.00 7.61 0.00 0.00 96.95 27.85
15.61 ## [13] 0.00
```

7.3 Other Methods

In practice, decisions must often be made in a sequential manner and decisions made at an earlier time may affect the feasibility and performance of later decisions. To find optimal strategies one must consider current and future decisions simultaneously. These types of multi-stage decision problems are the typical settings where one employs dynamic programming (DP). Dynamic programming models and methods are based on Bellman's Principle of Optimality, namely, that for overall optimality in a sequential decision process, all the remaining decisions after reaching a particular state must be optimal with respect to that state. In other words, if a strategy for a sequential decision problem makes a suboptimal decision in any one of the intermediate stages, it cannot be optimal for the overall problem. This principle allows one to formulate recursive relationships between the optimal strategies of successive decision stages. The best-known and most common examples are the tree or lattice models (binomial, trinomial, etc.) used to describe the evolution of security prices, interest rates, volatilities, etc. and the corresponding pricing and hedging schemes. We shall discuss these in detail in Chap. 9.

Stochastic programming assumes that the uncertain parameters are random variables with known probability distributions. This information is then used to transform the stochastic program into a deterministic equivalent. For example, suppose we want to find an optimal portfolio where the risk measure is not the variance but the expected shortfall. The stochastic programming approach would involve replacing the unknown distribution of returns by the empirical distribution based on a finite number of observations. This leads to an approximate LP problem which can then be solved by the methods discussed above.

Robust optimization formulations also address problems with input uncertainty. In this approach, uncertainty in the inputs is described through uncertainty sets that contain all (or most) possible values that can be realized by the uncertain inputs. As opposed to the stochastic approach, where a probability distribution is considered on these inputs, robust optimization gives all the values equal importance and tries to obtain a solution that is optimal under each of the possible values. Any financial optimization problem with unknown inputs fits into this framework.

7.4 Exercises

1. Give an example of a two-variable unbounded LP problem.
2. Rewrite the LP problem of Example 7.2.1 in standard form.
3. Use the simplex method to solve the LP problem in Example 7.1.3.
4. Write an R code for the algorithm of the Interior Point Method.
5. Use the R code in the previous problem to solve the QP problem of Example 7.2.1 for $\mu = 1$ using the mean and covariance matrix of the edhec dataset.

Part II
Simulation Methods

Chapter 8
Monte Carlo Methods



In Chap. 4, we have discussed the need for evaluating integrals and described methods for doing this numerically. Let $g : D \to \mathbb{R}$ be a function on $D \subseteq \mathbb{R}^n$ and we are interested in evaluating

$$\int_D g(x)\mathrm{d}x. \tag{8.1}$$

We have seen in Chap. 4 that when $n = 1$, then this can be achieved by deterministic methods like quadrature to achieve errors of rate $O(m^{-k})$ where m is the number of subintervals and k depends on the method and other parameters. For example, for Simpson's rule with 3 nodes, $k = 3$. This procedure entails evaluation of the function at mk points. There are two drawbacks of this procedure

1. Although the order of the error is known, we do not have any bounds on the error as that will depend on the smoothness of the function. Thus the actual error can be arbitrarily large.
2. The procedure gets less efficient when we go to higher dimensions. For the same error rate in n dimensions, we shall have to evaluate the function at $m^n k$ points. If we evaluate at the same mk number of points, the error rate becomes $m^{-k/n}$.

Monte Carlo or simulation methods overcome both of these drawbacks. Firstly, we can give (at least approximate) bounds on the actual error and not only the rates. Secondly, the error rate does not change with the dimension and is always $\sqrt{m}$.

Suppose we are interested in evaluating the integral in Eq. (8.1). Let $X_1, \ldots, X_m$ be iid uniform random vectors in D. By the law of large numbers (LLN, see Chap. 13), with probability one

$$\overline{g(X)}_m = \frac{1}{m}(g(X_1) + \cdots + g(X_m)) \to \mathrm{E}(g(X_1)) = \frac{\int_D g(x)\mathrm{d}x}{\mathrm{Area}(D)}.$$

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/978-981-19-2008-0_8.

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

R. Sen and S. Das, *Computational Finance with R*, Indian Statistical Institute Series,
https://doi.org/10.1007/978-981-19-2008-0_8

Also, from the central limit theorem (CLT) in Chap. 13, we have

$$\sqrt{m}\frac{\overline{g(X)}_m - \mathrm{E}(g(X_1))}{\sqrt{\mathrm{Var}(g(X_1))}} \Longrightarrow \mathcal{N}(0, 1).$$

Using $\overline{g(X)}_m \mathrm{Area}(D)$ as an approximation for $\int_D g(x)\mathrm{d}x$ is the simplest Monte Carlo method for evaluating the integral and it achieves an error rate of $\sqrt{m}$, irrespective of the dimension of D. A bound can be put on the error using the cut-off points of the Gaussian distribution which is reasonably precise for large values of m.

Example 8.0.1. Suppose we want to integrate $g(x) = \cos(x)$ for x between zero and one. We can do this analytically to get $\sin(1) = 0.841471$. If we do this by Monte Carlo method, we generate $X_1, \ldots, X_m$ as independent uniform[0, 1] random variables and compute $\overline{g(X)}_m$. The results we get for different values of m are given in the following table.

m	$\overline{g(X)}_m$
1	0.774
10	0.872
100	0.834
1000	0.833
10^4	0.840
10^5	0.841

Note that for the hypotheses of LLN and CLT to hold, we need a sequence of independent random realizations from the distribution of interest. Thus the central block of the method described above is generation of random variables X_i. We shall discuss the method of generation of uniform random variables in Sect. 8.1. In Sect. 8.2 we shall describe some methods of generating random variables from other distributions and the need to do so.

8.1 Generating Sequence of Uniform[0, 1] Random Numbers

In this section we shall discuss some algorithms for generating a sequence $U_1, \ldots, U_m$ of m independent uniform (0, 1) random numbers. An important thing to note is that since these numbers are generated by an algorithm, or rule, they are not really random. Such observations are called pseudo-random. The aim is to generate a sequence of numbers between zero and one that has statistical properties of a random sequence. In particular we need to check that

1. Each observation is uniformly distributed in [0, 1]
2. The observations are mutually independent.

8.1.1 Linear Congruential Generator

A linear congruential generator is a recurrence of the following form:

$$x_{i+1} = ax_i \bmod m$$
$$u_{i+1} = x_{i+1}/m.$$

Here, the multiplier a and the modulus m are integer constants that determine the values generated, given an initial value (seed) x_0. The seed is an integer between 1 and $m-1$ and is generally specified by the user. Because the result of the mod m operation is always an integer between 0 and $m-1$, the output values u_i are always between 0 and $(m-1)/m$. Value of x_0 should not be zero, as in that case, all subsequent values of x will be zero. Here is an R code to run the generator.

```
m = 17
a = 6
x = 7
u = rep(0,20)
for (i in 1:20){
  x=(a*x)%%m
  u[i]=x/m
}
```

With this specification, the values of x are

$$8, 14, 16, 11, 15, 5, 13, 10, 9, 3, 1, 6, 2, 12, 4, 7, 8, 14, 16, 11$$

and the values of u_i are $x_i/17$. Note that the first 16 elements are distinct. The 17th element is same as the first and then the sequence repeats itself. Since the procedure is deterministic, such loops are inevitable. Choosing large values of m is recommended as that will create larger loops. With the same m and x_0, if we change a to 2, then the values of x are

$$14, 11, 5, 10, 3, 6, 12, 7, 14, 11, 5, 10, 3, 6, 12, 7, 14, 11, 5, 10$$

Here the length of the loop is 8, and all integers between 1 and 17 do not appear in the sequence. Depending on the starting value x_0, we get one of two possible disjoint sets. This is also true for $a = 8, 9$ or 15. With $a = 4$ or 13 the loop is of length 4. With $a = 16$, the loop is of length 2.

Thus we see that with the same value of m we can have loops of different length, depending on the corresponding value of a. We would, of course like to have loops as long as possible. This is because, once the loop ends, the series just repeats itself and is therefore deterministic. A long cycle length also ensures that the values of the random variable are densely distributed in the interval (0, 1). The first condition for that is to choose a large enough m. The second condition is to choose an appropriate

value of a. A linear congruential generator that produces all $m-1$ distinct values before repeating is said to have full period. When m is prime, full period is achieved from any seed $x_0 \neq 0$ if

1. $a^{m-1}-1$ is a multiple of m
2. $a^j - 1$ is not a multiple of m for $j = 1, \cdots, m-2$.

A number a satisfying these two properties is called a primitive root of m. In the example, with $m = 17$ and $a = 16$, we have $a^2 - 1 = 255$ is a multiple of m. So the second condition is not satisfied and we have a shorter loop. The length of the loop is the smallest integer j such that $a^j - 1$ is a multiple of m. Table 8.1 displays moduli and multipliers for three linear congruential generators that have been recommended in the literature.

The next concern is the relation between multiple generated observations. Ideally we would like them to be independent. Starting with pairs of consecutive realizations, $(U_i, U_{i+1}), i = 1, 2, \ldots$, should be uniformly distributed over the unit square. With $m = 17$ and $a = 6$ (note this gives a full cycle), the plot of (U_i, U_{i+1}) is shown in Fig. 8.1. Note that the points lie in a grid.

This phenomenon is characteristic of all linear congruential generators. Of course this is not desirable as this contradicts the independence condition. The lattice structure of linear congruential generators is often used to compare their outputs and

Table 8.1 Pairs of values of m and a recommended by L'ecuyer (1988)

Modulus (m)	Multiplier (a)
2147483647	39373
2147483399	40692
2147483563	40014

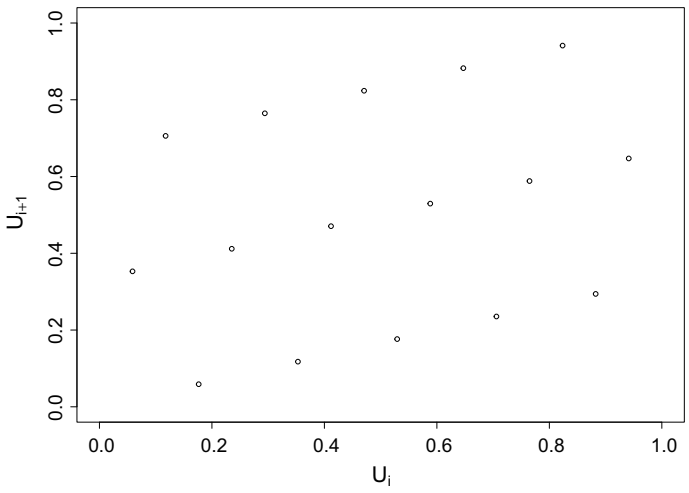


Fig. 8.1 Plot of consecutive realizations of random numbers generated by the linear congruent generator with $m = 17$ and $a = 6$

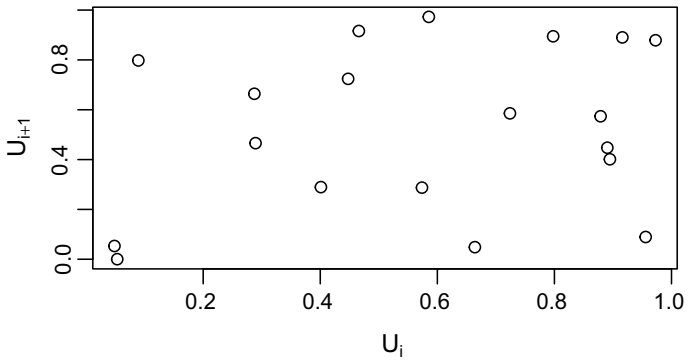


Fig. 8.2 Plot of consecutive realizations of random numbers generated by the linear congruent generator with $m = 2147483647$, $a = 39373$, and $x_0 = 7$

select parameters. The less pronounced the lattice structure, the more desirable is a particular generator. Obviously it is not possible to completely eliminate this lattice structure as there is an inherent dependence between successive observations due to the generating mechanism. We present a similar plot of (U_i, U_{i+1}) with 100 realizations starting with $x_0 = 7$ in Fig. 8.2. Note there is no discernable grid structure and recommended values of a and u.

8.1.2 Combining Generators

We have seen that the linear congruent generator, although quite simple in execution, suffers from two main drawbacks. Firstly the length of the period may not be large enough and if we need millions of random numbers there will be inevitable repetition of the full sequence. Secondly there is a clear lattice structure that conflicts with the assumption of independence. The lattice structure also means that the points that are supposed to be randomly distributed on a two-dimensional square really lie on a few straight lines, that is, a finite union of one-dimensional subspaces. So most of the area inside the square remains unexplored. One way to move beyond the basic linear congruential generator is to combine multiple generators. A combination of generators can have a much longer period than any of its components.

The following combining algorithm is proposed by Wichmann and Hill (1982).

Wichmann and Hill Algorithm

1. Start with J pairs of values (a_j, m_j), $j = 1, \ldots, J$
2. Define J sequences of linear congruential generators $u^{(1)}, \ldots, u^{(J)}$ as follows

$$x_{j,i+1} = a_j x_{j,i} \bmod m_j, \tag{8.2}$$
$$u_{i+1}^{(j)} = x_{j,i+1}/m_j,\ j = 1, \ldots, J.$$

3. Set u_{i+1} equal to the fractional part of $\sum_{j=1}^{J} u_{i+1}^{(j)}$.

Another way to combine generators is suggested by L'ecuyer (1988). They show that the combination does not exhibit lattice structure at a magnification where each of the individual sequences does. The procedure is as follows.

L'ecuyer Algorithm

1. Generate multiple sequences as in (8.2).
2. Generate an intermediate sequence as

$$z_{i+1} = \sum_{j=1}^{J} (-1)^{j-1} x_{j,i+1} \bmod (m-1),$$

where m is maximum of $(m_1, \ldots, m_J)$.

3. The output u is given by

$$u_{i+1} = \begin{cases} z_{i+1}/m, & z_{i+1} > 0 \\ (m-1)/m, & z_{i+1} = 0 \end{cases}$$

The most commonly used algorithm in statistical softwares is Mersenne Twister (Matsumoto and Nishimura, 1998).

8.2 General Sampling Methods

Let us suppose that we have a satisfactory method for generating a long independent sequence of uniform(0, 1) random numbers. The next question of interest is to generate a sequence of random numbers from a different specified distribution. This could be either a well-known or a new distribution, depending on the problem.

Using the R Generator R has a large number of functions that will generate random samples from standard distributions. If built-in function in R is available then we can use it directly. *However, if the built-in functions are NOT available then we have to use generic methods to draw samples from such distributions.*

In Chap. 10, we shall discuss some particular methods when the specified distribution is normal. Here we discuss some general methods that work for all distributions.

8.2.1 Inverse Transform

Suppose we want to sample from a cumulative distribution function F, that is, we want to generate a random variable X with the property that $P(X \leq x) = F(x) \forall x$. Define the function F^{-1} as

$$F^{-1}(u) = \inf\{t : F(t) \geq u\} \quad \text{for} \quad 0 < u < 1.$$

It follows that

$$F(x) \geq u \iff F^{-1}(u) \leq x. \tag{8.3}$$

Suppose $U \sim U(0, 1)$. Define $X = F^{-1}(U)$. Then $X \sim F$ since

$$P(X \leq x) = P(F^{-1}(U) \leq x) = P(F(x) \geq U) = F(x).$$

We can use this to generate random variables from the distribution F, by first generating a uniform(0,1) random variable and then applying the function F^{-1}.

Let us now consider two special cases, namely F is strictly continuous and F is purely discrete. When F is continuous, then given u, there is a unique value t such that $F(t) = u$. Thus the function F^{-1} is a mathematical inverse.

Example 8.2.1. Let us take the example of a geometric distribution. Here $F(x) = 1 - \exp(-\lambda x)$. We put $u = F(x)$ and solve for x in terms of u to get $x = -\lambda^{-1} \log(1 - u)$.

The following R code generates samples from the `Exponential(λ=1)` distribution. We simulate samples x generated by inverse transform method and samples `y` generated by the built-in function `rexp` in R. The histogram of the samples is presented in Fig. 8.3.

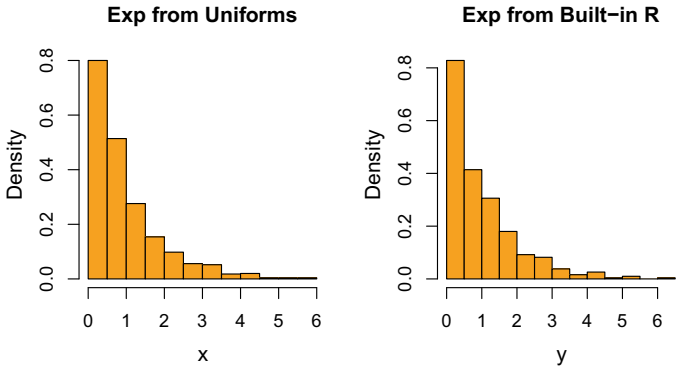


Fig. 8.3 Histogram of the samples from the `Exponential(λ=1)` distribution generated by inverse transform method (left) and by the built-in function `rexp` in `R` (right)

```
# Number of random samples
N=1000
# Simulate from uniforms
u=runif(N)
# transforms of uniforms
x= -log(1-u)
# exponential in R
y=rexp(N)
par(mfrow=c(1,2)) # plot parameters
hist(x,freq=F,main="Exp from Uniforms",col="orange")
hist(y,freq=F,main="Exp from Built-in R",col="orange")
```

The inverse transform method is useful for other probability distributions, as long as they can be obtained as a transformation of uniform random variables. We give examples of some distributions and the inverse of their CDFs in Table 8.2. We can generate random observations from these distributions using the inverse transform method. However, there is a limit to the usefulness of this method. Only when closed-form CDF is available the method is applicable. The method will not work for Gaussian distribution as the CDF is not in closed form. We shall discuss this in Chap. 10.

If $X_i \overset{iid}{\sim} \text{Exp}(1)$, three standard distributions can be derived as

- $Y = 2\sum_{i=1}^{n} X_i \sim \chi^2_{2n}$
- $Y = \beta \sum_{i=1}^{n} X_i \sim \text{Gamma}(n, \beta)$
- $Y = \frac{\sum_{i=1}^{n_1} X_i}{\sum_{i=1}^{n_1+n_2} X_i} \sim \text{Beta}(n_1, n_2)$

Table 8.2 CDF and inverse CDF of some distributions

Distribution	CDF	Inverse CDF
Logistic	$F(x) = \frac{1}{1+\exp\left(-\frac{x-\mu}{\lambda}\right)}$	$F^{-1}(u) = \mu - \lambda \log(\frac{1}{u} - 1)$
Cauchy	$F(x) = \frac{1}{2} + \frac{1}{\pi} \arctan(\frac{x-\mu}{\sigma})$	$F^{-1}(u) = \mu + \sigma \tan(\pi(u - \frac{1}{2}))$
Pareto	$F(x) = 1 - (\frac{x_m}{x})^{\alpha}$	$F^{-1}(u) = x_m(1-u)^{-\frac{1}{\alpha}}$
Weibull	$F(x) = 1 - \exp(-(\frac{x}{\lambda})^k)$	$F^{-1}(u) = \lambda(-\log(1-u))^{\frac{1}{k}}$

where $n \in \mathcal{N} = \{1, 2, ...\}$. These transformations are quite simple, and we will use them quite often.

When F is discrete, X takes values $x_1 < \cdots < x_k$ with probabilities $p_1, \ldots p_k$ such that $\sum_{i=1}^{k} p_i = 1$.

$$F(x) = \sum_{i=1}^{l} p_i \quad \text{where} \quad l = \sup\{m : x_m \leq x\}.$$

$$\text{Thus} \quad F^{-1}(u) = x_l \iff \sum_{i=1}^{l-1} p_i < u \leq \sum_{i=1}^{l} p_i.$$

The following code generates a sample of size 1000 from a discrete distribution with support $\{1, 2, 3, 4\}$ with probability vector p and draws a side-by-side barplot of the actual and observed probabilities in Fig. 8.4.

```
inv.transform <- function( p ) {
  U = runif(1)
  s = 0
  i = 0
  while(s < U ) {
    i = i+1
    s = s+p[i]
  }
  return(i)
}
n = 1000
p = c(0.2, 0.4, 0.3, 0.1)
names(p) = 1:4
samples = rep(0,n)
for(i in 1:n) {
  samples[i] = inv.transform(p)
}
pp = table(samples)/n
```

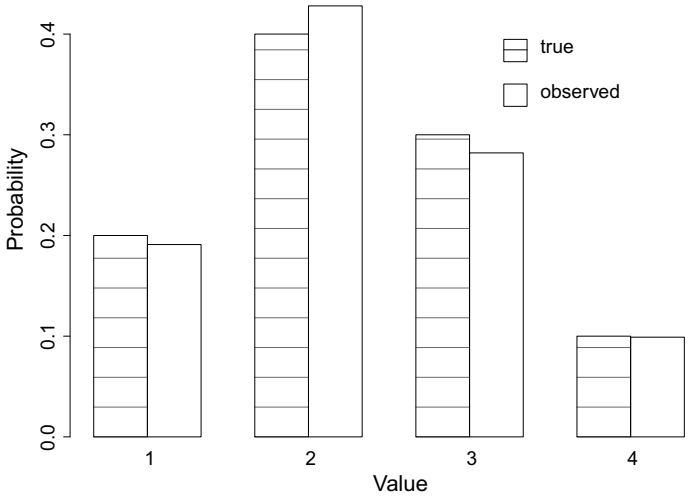


Fig. 8.4 Barplot of actual and observed probabilities for a sample of size 1000 from a discrete distribution

```
barplot(rbind(p,pp), beside=T,
        angle=c(0,0), density=c(10,0),
        ylab="Probability",xlab="Value")
legend("topright", c("true","observed"),
angle=c(0,0), density=c(10,0),
       bty="n", cex=0.75)
```

8.2.2 Accept–Reject Method

There are many distributions where the inverse transform method cannot be applied. For such cases, we shall now discuss some indirect methods. The 'Accept–Reject algorithm' is an excellent indirect method when known transformation methods are not available. The general idea is to generate a candidate random variable from some other distribution and accept it if it passes a test. This class of 'Accept–Reject algorithm' is remarkably robust. It allows us to simulate from virtually any distribution.

Suppose f is the **target density**, from which we want to simulate; but we don't know how. We take g as the **candidate density**. It should be chosen in such a way that it is simple to simulate random variables from g. The set of values that a random variable from a distribution can take is called the support of the distribution. We require that the support of f be contained in the support of g. We need to put another constraint on g. The set of values that a random variable from a distribution can take is called the support of the distribution. We require that the support of f be contained

in the support of g. In mathematical terms, the probability measure associated with f is absolutely continuous with respect to the probability measure associated with g. The algorithm to simulate a random observation $X \sim f$ goes as follows.

Accept–Reject Algorithm
Let $M = \sup_x \frac{f(x)}{g(x)}$.

1. Generate $Y \sim g$ and $U \sim \text{unif}(0, 1)$ independently.
2. If $U \leq \frac{1}{M}\frac{f(Y)}{g(Y)}$ set $X = Y$, that is accept the candidate random variable Y. Else, discard Y and U and start again.

We do lose a lot of candidate random variables in the process. The probability of accepting a particular candidate variable is $\frac{1}{M}$, and the expected waiting time between successive acceptances is M. Thus to design an efficient algorithm, we need to choose g so that M is small.

Why does this method work?

A straightforward probability calculation shows

$$P(Y \leq x|\text{Accept}) = P\left(Y \leq x|U \leq \frac{f(Y)}{Mg(Y)}\right) = P(X \leq x),$$

where $Y \sim g$ and $X \sim f$. Simulating from g, the output of this algorithm is exactly distributed as an observation from f.

The Accept–Reject method is applicable in any dimension. As long as g is a density over the same space as f. All we need is to know f/g up to a constant and the existence of an upper bound on M.

Next we present the R code to generate random variables from the target distribution Beta($a = 2.3, b = 7.9$) using the Accept–Rejection algorithm. Note that we use the candidate distribution g as Unif(0, 1). The histograms of the observations from the two distributions are presented in Fig. 8.5. The acceptance rate is 24.7%. That is from 5000 observations from the candidate distribution g, we obtain 1235 observations from the target distribution f.

```
set.seed(1010)
N=5000
sample.x = runif(N,0,1)
accept = c()
for(i in 1:length(sample.x)){
 U = runif(1, 0, 1)
 if(dunif(sample.x[i], 0, 1)*4*U<= dbeta(sample.x[i], 2.3, 7.9)){
   accept[i] = 'Yes'
 }else if(dunif(sample.x[i],0,1)*4*U > dbeta(sample.x[i], 2.3, 7.9)){
  accept[i] = 'No'
 }
 }
T = data.frame(sample.x
               , accept = factor(accept
                     , levels= c('Yes','No')))
```

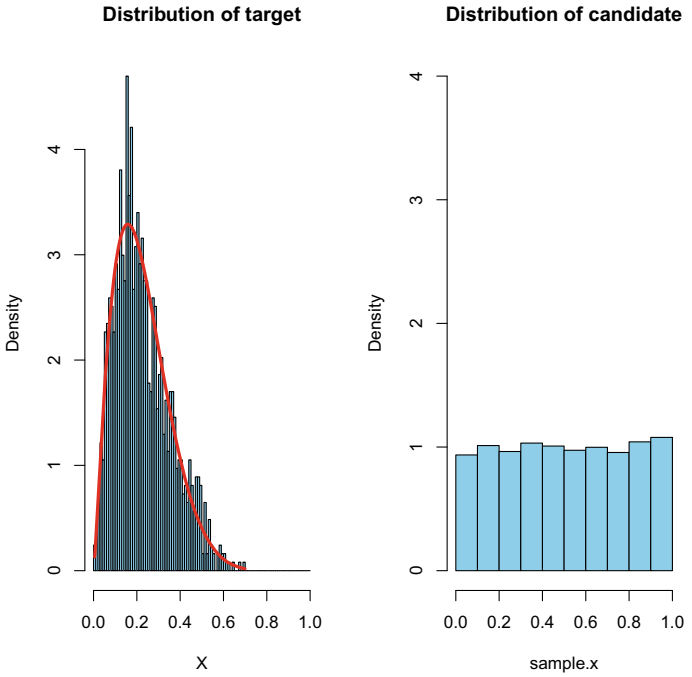


Fig. 8.5 Histograms of the observations from the two distributions. Left: 1241 observations from target distribution Beta($a = 2.3, b = 7.9$) superimposed with the density. Right: 5000 observations from candidate distribution Unif(0, 1)

```
x = sort(T[,1][T$accept=='Yes'])
par(mfrow=c(1,2))
hist(x, breaks = seq(0,1,0.01), freq = FALSE
     , main = 'Distribution of target'
     , xlab = 'X',col='skyblue')
lines(x, dbeta(x,2.3,7.9),col="red",lwd=3)
hist(sample.x
     ,main= 'Distribution of candidate'
     ,freq=FALSE,col='skyblue'
     ,ylim=c(0,4))
cat("Acceptance Rate :","\n")

## Acceptance Rate :

## Acceptance Rate :
(length(x)/length(sample.x))*100

## [1] 26.18
```

8.3 Exercises

1. Write the R code to get Fig. 8.2.
2. Prove Eq. 8.3.
3. Simulate 1000 observations from a Gamma($a = 2, b = 8$) distribution by the method described in Sect. 8.2.1 using exponential random variables. Draw a histogram and superimpose with the actual density as in Fig. 8.5.
4. Use the built-in function in R to simulate 1000 observations from a Gamma($a = 2, b = 8$) distribution. Draw a histogram and superimpose with the actual density as in Fig. 8.5. Compare the result with the result of the previous problem.
5. Write an R code to generate observation from a Binomial(10, 0.2) distribution using the Accept–Reject algorithm with candidate distribution discrete uniform on the set $\{0, 1, 2, \ldots, 12\}$.

Chapter 9
Lattice Models



It is common to model the stock price movement with a stochastic process that has continuous path, for example a geometric Brownian motion. Computationally one can only evaluate the path at discrete points of time, however dense. The general approach to simulating paths from such stochastic processes will be discussed in Chap. 10. It is often useful to have a probability model that is a computationally tractable discrete approximation to the continuous models. The model should explain no-arbitrage pricing and its relation to risk-neutral pricing. Also, the model should incorporate the theory of conditional expectation and martingale theory which is the core of risk-neutral pricing. To fulfill all these purposes the lattice models are often used that have discrete support. A lattice model is a discrete time stochastic process with discrete support. The Binomial Asset Pricing Model (BAPM) serves as a good starting point. Note that even with this discretization, option prices cannot be computed analytically in many cases and one needs to resort to simulation methods that we shall discuss in this chapter. This is particularly relevant for the American Options, or other exotic path-dependent options. We shall also present trinomial models as extensions of binomial. In general one can propose multinomial models with suitable restrictions on parameters.

9.1 Option Pricing with Binomial Lattices

Suppose at time t, the price of an asset is $S(t) = S$. Then the BAPM assumes that at time $t + \Delta t$, the price $S(t + \Delta t)$ can be either uS with probability q or dS with probability $1 - q$. Here u and d denote 'up' and 'down,' respectively. We assume

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/978-981-19-2008-0_9.

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

R. Sen and S. Das, *Computational Finance with R*, Indian Statistical Institute Series,
https://doi.org/10.1007/978-981-19-2008-0_9

that there exists a money market instrument like bond with continuously compounding interest rate r. If one invests ₹1 in the money market at time t, one receives ₹$R = \exp(r\Delta t)$ at time $t + \Delta t$. For simplicity, we assume r to be the interest rate for both borrowing and lending. This assumption is fair. For example, if you buy bond at yield r then it means you are lending money. If you sell bond that means you are borrowing at interest rate r.

Claim: To avoid arbitrage opportunities, we must have $u > R > d$.

If $R \geq u$, then the return on the bond is always at least as large as and sometimes bigger than the return of the stock, and no one would invest in the stock. One can sell the stock for S and lend the money in the money market. In the next period the value of the investment would be $RS \geq uS$. So one can buy back the stock and have non-negative profit, with positive profit if the price of the stock indeed goes down. This is an arbitrage opportunity. Hence R must be less than u. Similarly, $R \leq d$ cannot happen as it leads to arbitrage in the other direction. In that case the return of the stock is always at least as large as, and sometimes bigger than, the return of the bond.

In order to satisfy the *no-arbitrage* requirement, we need the existence of a risk neutral measure such that the discounted price of the stock is a martingale. If the probability of an upward movement is p, then the expectation of the price of the stock is $pSu + (1 - p)Sd$. This should equal RS and hence we have,

$$RS = pSu + (1 - p)Sd \quad \Longleftrightarrow \quad p = \frac{R - d}{u - d}. \tag{9.1}$$

Note that p is a probability ($0 < p < 1$) since $d < R < u$. This p is known as the risk-neutral probability and arises purely out of no-arbitrage conditions. This is different from the probability q under the physical measure, that is the observed probability of prices going up in the market.

The Binomial Option Pricing Model (BOPM) is the first step to understand, how pricing of an option contract of an asset works. Here we will focus mainly on the European options. Consider the BAPM described above. We want to find the price $V(x, s)$ of an option when the stock price is x at current time s. Under the risk neutral measure, discounted price process of any traded asset is a martingale. The option is a traded asset. Price of the option at time t is $V(S, t)$, the unknown quantity of interest. The discount is R and p is given by (9.1). Hence

$$V(S, t) = \frac{1}{R}\left(pV(uS, t + \Delta t) + (1 - p)V(dS, t + \Delta t)\right). \tag{9.2}$$

The RHS of Eq. (9.2) is the expected discounted price of the option at time $t + \Delta t$ under the risk neutral measure. Note that the probability q corresponding to the physical measure does not appear in the option pricing formula.

Let us assume a *call option* for strike price K exists such that $d < K/S < u$ and it is scheduled to expire at time $t + \Delta t$. For definition of call option, see Sect. 3.1. Now if the option holder decided to exercise the options at the end of the time-period, the payoff of the option will be following:

1. If the stock price increases, then payoff is:

$$V(uS, t + \Delta t) = \max\{Su - K, 0\} = Su - K.$$

2. If the stock price decreases, then payoff is:

$$V(dS, t + \Delta t) = \max\{Sd - K, 0\} = 0.$$

In this case, from Eq. (9.2) the price of the option is given by

$$V = \frac{1}{R}\frac{R-d}{u-d}(Su - K). \tag{9.3}$$

Example 9.1.1. **Option pricing with binomial lattice** We are interested in the value of a call option where,

$$\begin{aligned} S &= ₹100 \quad \text{current price,} \\ K &= ₹125 \quad \text{strike price,} \\ r &= 8\% \quad \text{risk-free rate of interest,} \\ \Delta t &= 1 \text{ year,} \\ uS &= ₹200 \quad \text{possible up price,} \\ dS &= ₹50 \quad \text{possible down price.} \end{aligned}$$

The situation is illustrated in Fig. 9.1. The price of the option is given by Eq. (9.3) where $R = \exp(r\Delta t)$ as

$$V = \frac{1}{R}\frac{R-d}{u-d}(Su - K) = ₹\frac{1}{\exp(.08)}\frac{\exp(.08) - .5}{2 - .5}(200 - 125) = ₹26.92.$$

Another way to arrive at the option price is through replicating portfolios. This is interesting in itself as it also gives the hedge ratios, that is, the amount of money to be invested in the underlying asset, say stock, and bond to replicate the option. Suppose at time t, we form a portfolio consisting of α units of stock and cash amount B in riskless bond. The total investment $\alpha S + B$ is the price of buying an option $V(S, t)$ at time t. Value of the portfolio at time $t + \Delta t$ is $\alpha uS + BR$ if the price goes up and $\alpha dS + BR$ if it goes down. To avoid arbitrage, this portfolio value has to match the option value at terminal time. Thus we have

$$V(uS, t + \Delta t) = \alpha uS + BR$$
$$V(dS, t + \Delta t) = \alpha dS + BR.$$

These are two linear equations in two unknowns α and B. So we can solve to get

$$\alpha = \frac{V(uS, t + \Delta t) - V(dS, t + \Delta t)}{(u - d)S}, \tag{9.4}$$
$$B = \frac{uV(dS, t + \Delta t) - dV(uS, t + \Delta t)}{(u - d)R}.$$

Now we can obtain the price of the option at the initial time t by substituting B and α in $V(S, t) = \alpha S + B$. One can check that this gives the same value for $V(S, t)$ as the one obtained in Eq. (9.3).

9.2 Binomial Option Price as Approximate Solution to Black–Scholes Equation

We are using the binomial model as an approximation to the geometric Brownian motion of the Black–Scholes model. Note that u and d are unspecified in the binomial model. In order for the approximation to work, we need to put some more conditions to specify these constants. One approach is to equate the second moment of the asset price in both continuous and discrete models. By doing this, we get

$$pu^2 + (1 - p)d^2 = R^2 \exp(\sigma^2 \Delta t), \tag{9.5}$$

where σ is the volatility in the Black–Scholes model. With this additional condition, we show in the following theorem that the binomial option price is an approximate solution to the Black–Scholes equation.

Theorem 4. *As the time interval Δt goes to zero, the one-period binomial option pricing formula (9.2) with the conditions (9.1) for risk neutrality and (9.5) for second moment solves the Black–Scholes equation.*

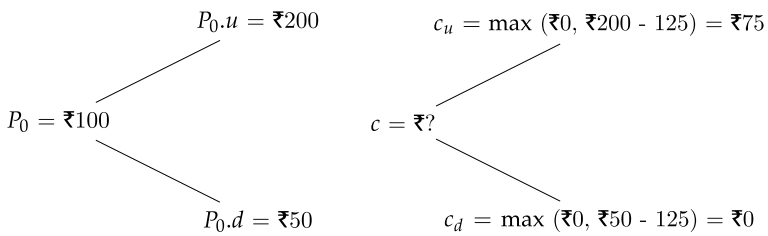


Fig. 9.1 One period binomial tree for Example 9.1.1. Left: Stocks, right: Options

$$\frac{\partial V}{\partial t} + rS\frac{\partial V}{\partial S} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} - rV = 0$$

Proof. Two variable Taylor expansion at (S, t) gives

$$\begin{aligned} V(xS, t + \Delta t) = V(S, t) &+ (xS - S)V_S + \Delta t V_t \\ &+ \frac{1}{2}\left((xS - S)^2 V_{SS} + 2(xS - S)\Delta t V_{St} + (\Delta t)^2 V_{tt}\right) \\ &+ \text{higher order terms} \quad \text{for} \quad x = u, d. \end{aligned}$$

Substituting in (9.2) gives

$$\begin{aligned} RV(S, t) = V(S, t) & \\ &+ SV_S(pu + (1 - p)d - 1) + \Delta t V_t \\ &+ \frac{1}{2}\left(S^2 V_{SS}(p(u - 1)^2 + (1 - p)(d - 1)^2)\right. \\ &\left. + 2S\Delta t V_{St}(pu + (1 - p)d - 1) + (\Delta t)^2 V_{tt}\right) \\ &+ \text{higher order terms.} \end{aligned}$$

From (9.5),

$$\begin{aligned} p(u - 1)^2 + (1 - p)(d - 1)^2 &= pu^2 + (1 - p)d^2 - 2(pu + (1 - p)d) + 1 \\ &= R^2 \exp(\sigma^2 \Delta t) - 2R + 1 \\ &= \sigma^2 \Delta t + O((\Delta t)^2) \quad \text{since} \quad R = \exp(r\Delta t). \end{aligned}$$

Also, from (9.1), $pu + (1 - p)d - 1 = R - 1 = r\Delta t + O((\Delta t)^2)$.

$$\begin{aligned} (R - 1)V(S, t) &= SV_S r\Delta t + \Delta t V_t \\ &+ \frac{1}{2}\left(S^2 V_{SS}\sigma^2 \Delta t + 2S\Delta t V_{St} r\Delta t + (\Delta t)^2 V_{tt}\right) \\ &+ O((\Delta t)^2). \end{aligned}$$

Dividing by Δt, we get $rV = rSV_S + V_t + \frac{1}{2}S^2 V_{SS}\sigma^2 + O(\Delta t)$. Ignoring the $O(\Delta t)$ term leads to the Black–Scholes equation. □

9.3 Specific Models

We have three unknown parameters in the binomial model, namely p, u, and d. Equations (9.1) and (9.5) relate them. But these are not enough to specify the parameters uniquely. So we need to impose some extra conditions. Below we present two such

extra conditions that lead to two sets of values for the parameters that are commonly used in practice.

The Cox–Ross–Rubinstein (CRR) model imposes the condition $ud = 1$. This leads to the solution

$$u = \frac{1}{2R}\left(\hat{\sigma}^2 + 1 + \sqrt{(\hat{\sigma}^2+1)^2 - 4R^2}\right)$$
$$d = \frac{1}{2R}\left(\hat{\sigma}^2 + 1 - \sqrt{(\hat{\sigma}^2+1)^2 - 4R^2}\right),$$

where $\hat{\sigma}^2 = \exp((2r + \sigma^2)\Delta t)$. If the terms of order $O((\Delta t)^{3/2})$ and higher are ignored then, this reduces to

$$u = \exp(\sigma\sqrt{\Delta t}), d = \exp(-\sigma\sqrt{\Delta t}), p = \frac{R-d}{u-d}$$

The Jarrow-Rudd model imposes the extra condition that $p = 1/2$. This leads to the solution

$$u = R\left(1 + \sqrt{\exp(\sigma^2\Delta t) - 1}\right)$$
$$d = R\left(1 - \sqrt{\exp(\sigma^2\Delta t) - 1}\right).$$

If the terms of order $O((\Delta t)^{3/2})$ and higher are ignored then, this reduces to

$$u = \exp((r - \sigma^2/2)\Delta t + \sigma\sqrt{\Delta t})$$
$$d = \exp((r - \sigma^2/2)\Delta t - \sigma\sqrt{\Delta t}).$$

9.4 *n*-Period Binomial Option Pricing Formula

One can model the price movement over time interval $[0, T]$ as a sequence of n binary steps. Let the price follow one of the up or down branches over each subinterval as presented in Fig. 9.2.

Assumptions:

1. Each step is independent of previous one.
2. Parameters p, u, and d are same for each step.

Suppose P_0 is the initial stock price. At each time step covering time interval of length $\Delta t = T/n$, the stock price either goes up by a factor of u or down by a factor of d. We can see from Fig. 9.2 that at time point 2, the price P_2 can be one of the three possible values $\{P_0.u^2, P_0.d.u, P_0.d^2\}$. Therefore for over n intervals, the possible prices at T are $P_0u^kd^{n-k}$, with $k = 0, 1, \ldots, n$. They follow the binomial distribution

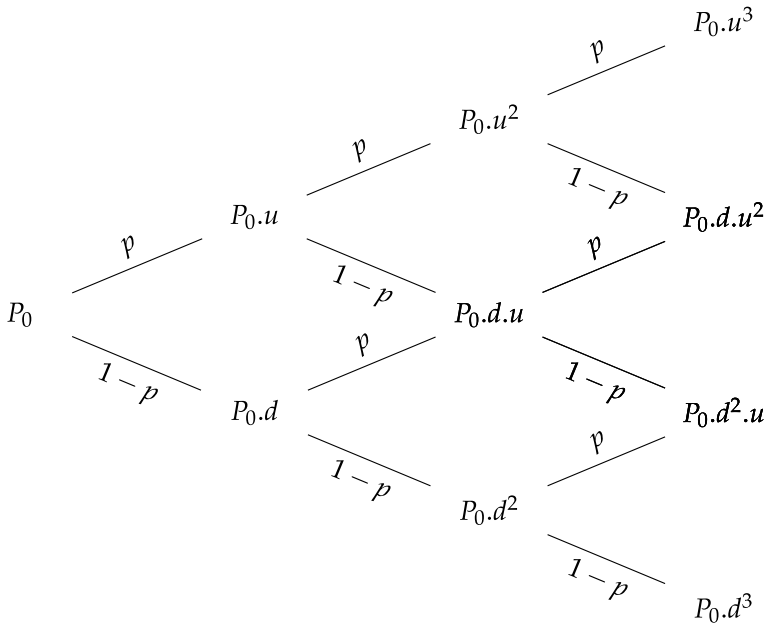


Fig. 9.2 Three period binomial model

$$\mathbb{P}[P_T = P_0 u^k d^{n-k}] = \binom{n}{k} p^k (1-p)^{n-k}, \quad k = 0, \cdots, n. \tag{9.6}$$

The fundamental asset pricing theorem states that there must be a probability distribution of these outcomes that satisfy the $\mathbb{Q}$-martingale measure conditions at each step. This will ensure that arbitrage opportunity does not exist in the market. This implies that the relation (9.1) is also true for n step binomial model. The following theorem provides a mathematical formula for the price of a call option under the n period BOPM.

Theorem 5. *The price of a call option with expiration T and strike K under an n step Binomial model is*

$$C(P_0, 0) = R^{-n} \sum_{k=0}^{n} \binom{n}{k} p^k (1-p)^{n-k} \max(u^k d^{n-k} P_0 - K, 0)$$

Proof. We shall prove this by induction.
Note that the result is true for $n = 1$ from the one-step binomial model, see Eq. (9.2).
Suppose the result is true for all $n \leq m - 1$.
For $n = m$, there are $m - 1$ steps between time Δt and T. So for $x = u, d$

$$C(xP_0, \Delta t) = R^{-m+1} \sum_{k=0}^{m-1} \binom{m-1}{k} p^k (1-p)^{m-1-k} \max(u^k d^{m-1-k} xP_0 - K, 0).$$

Thus,

$$pC(uP_0, \Delta t) = R^{-m+1} \sum_{k=1}^{m} \binom{m-1}{k-1} p^k (1-p)^{m-k} \max(u^k d^{m-k} P_0 - K, 0).$$

$$(1-p)C(dP_0, \Delta t) = R^{-m+1} \sum_{k=0}^{m-1} \binom{m-1}{k} p^k (1-p)^{m-k} \max(u^k d^{m-k} xP_0 - K, 0).$$

Also, $C(P_0, 0) = R^{-1}(pC(uP_0, \Delta t) + (1-p)C(dP_0, \Delta t))$. Combining, we get

$$\begin{aligned} C(P_0, 0) = R^{-m}((1-p)^m \max(d^m xP_0 - K, 0) + p^m \max(u^m P_0 - K, 0) \\ + \sum_{k=1}^{m-1} \left(\binom{m-1}{k} + \binom{m-1}{k-1} \right) p^k (1-p)^{m-k} \max(u^k d^{m-k} xP_0 - K, 0)). \end{aligned}$$

This proves the theorem since $\binom{m-1}{k} + \binom{m-1}{k-1} = \binom{m}{k}$. □

The following theorem further simplifies the result of the previous theorem for the price of a call option under the n-period binomial model.

Theorem 6. *Let m be the smallest integer such that $u^m d^{n-m} P_0 \geq K$. The call price formula simplifies to the following.*

$$C = P_0 \Psi(n, m, up/R) - KR^{-n} \Psi(n, m, p).$$

Here Ψ is the complementary binomial function defined by $\Psi(n, m, p) = P(X \geq m)$ where $X \sim Bin(n, p)$.

Proof. Since m be the smallest integer such that $u^m d^{n-m} P_0 \geq K$

$$\max(u^{kd^{n-k}} xP_0 - K, 0) = \begin{cases} 0 & \text{if} \quad k < m \\ u^{kd^{n-k}} xP_0 - K & \text{if} \quad k \geq m \end{cases}$$

$$C(P_0,0) = R^{-n}\sum_{k=m}^{n}\binom{n}{k}p^k(1-p)^{n-k}(u^k d^{n-k}P_0 - K)$$
$$= P_0\sum_{k=m}^{n}\binom{n}{k}\left(\frac{up}{R}\right)^k\left(\frac{d(1-p)}{R}\right)^{n-k} - KR^{-n}\sum_{k=m}^{n}\binom{n}{k}p^k(1-p)^{n-k}.$$

The result follows since $\frac{up}{R} + \frac{d(1-p)}{R} = 1$ from the no-arbitrage condition. □

9.5 Simulating from Binomial Asset Pricing Model

In this section, we shall simulate a path from the n-period binomial model. In Sect. 9.3, we have seen some specific versions of the binomial model. Here we shall use the CRR model that adds the assumption $ud = 1$ and ignores higher order terms to get the following values for the parameters

$$p_n = \frac{\exp(r/n) - d_n}{u_n - d_n}, \quad u_n = \exp\{\sigma/\sqrt{n}\} \quad \text{and} \quad d_n = \exp\{-\sigma/\sqrt{n}\}, \tag{9.7}$$

for $\Delta T = 1/n$.

The following R-code simulates a chain from Binomial Asset Pricing Model using the function `sim.BAPM`. The resulting price path is plotted in Fig. 9.3.

```
## simulate from Binomial Asset Pricing Model
sim.BAPM = function(P0,n,r,sigma){
  # n is the number of periods of the chain.
  # P0 is the value of the asset at the initial time.
  # r is the risk free rate.
  # sigma is the volatility.
  P = rep(NA,n) # Define a vector for price path
  P[1] = P0 # Initial price
  u = exp(sigma/sqrt(n)) # Up-factor
  d = 1/u # Down-factor
  p = (1+(r/n)-d)/(u-d) # Risk-neutral probability
  q = 1-p
  for(i in 2:n){
      # Drawing random toss from an unbiased coin
      toss = sample(c("Head","Tail"),1,replace = TRUE
      ,prob = c(p,q))
      # If head - price goes up by factor u
      if(toss=="Head")P[i]<-P[i-1]*u
      # If tail - price goes down by factor d
      if(toss=="Tail")P[i]<-P[i-1]*d
  }
  return(P)
```

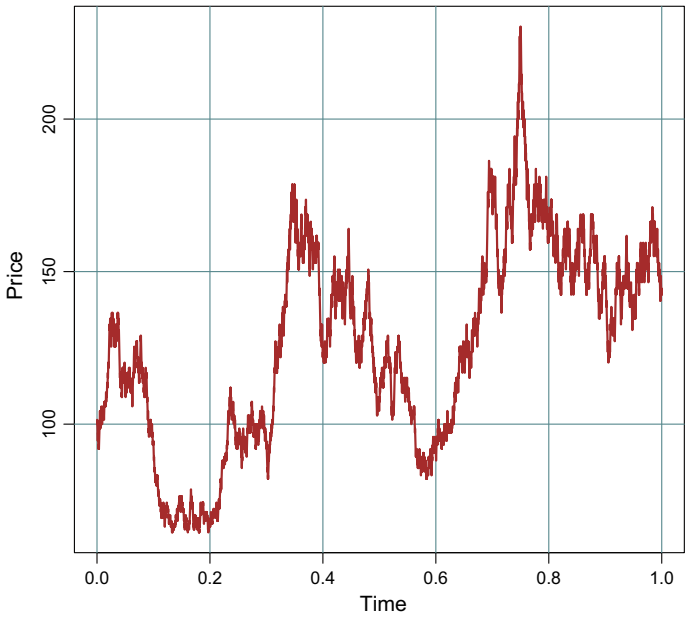


Fig. 9.3 Path of one realization of an n-period binomial model

```
}
set.seed(1234)
P = sim.BAPM(P0=100,n=5000,r=1,sigma=1)
plot(NULL,xlim=c(0,1),ylim=c(min(P),max(P))
     ,xlab="Time",ylab="Price")
grid(col='cadetblue4',lty=1)
points(seq(1:5000)/5000, P, type="l" ,lwd=2,col='brown',lty=1)
```

9.6 Log-Normal Distribution as Limit of BAPM

Let $\sigma > 0$ be a known constant and $r \geq 0$ be the interest rate. In this model the interest rate per period is $\frac{r}{n}$. As in the previous section, we assume the CRR model which gives the values of p_n, u_n, d_n as in Eq. (9.7). Furthermore, for large n, $\exp(r/n) \approx (1 + r/n)$. This leads to

$$\hat{p}_n = \frac{1 + \frac{r}{n} - \exp\{-\sigma/\sqrt{n}\}}{\exp\{\sigma/\sqrt{n}\} - \exp\{-\sigma/\sqrt{n}\}}.$$

Let t be an arbitrary positive rational number, and for each positive integer n, for which nt is an integer, define

$$R_{nt:n} = \sum_{i=1}^{nt} X_{k:n},$$

where $X_{1:n}, X_{2:n}, \ldots, X_{n:n}$ are independent, identically distributed random variables with

$$\tilde{\mathbb{P}}(X_{n:k} = 1) = \hat{p}_n, \quad \tilde{\mathbb{P}}(X_{n:k} = -1) = 1 - \hat{p}_n, \quad k = 1, 2, \ldots, n.$$

The stock price at time t in this model is

$$\begin{aligned} P_n(t) &= P_0 u_n^{\frac{1}{2}(nt + R_{nt:n})} d_n^{\frac{1}{2}(nt - R_{nt:n})} \\ &= P_0 \exp\left\{\frac{\sigma}{2\sqrt{n}}(nt + R_{nt:n})\right\} \exp\left\{-\frac{\sigma}{2\sqrt{n}}(nt - R_{nt:n})\right\} \\ &= P_0 \exp\left\{\frac{\sigma}{\sqrt{n}} R_{nt:n}\right\}. \end{aligned} \tag{9.8}$$

In Chap. 3 of (Shreve, 2004), it is presented that as $n \to \infty$,

$$\frac{\sigma}{\sqrt{n}} R_{nt:n} \xrightarrow{\mathcal{L}} N\big((r - \sigma^2/2)t, \sigma^2 t\big).$$

Hence we have the following theorem.

Theorem 7. *As $n \to \infty$, the distribution of $P_n(t)$ in (9.8) converges to the distribution of*

$$P_t = P_0 \exp\left\{\left(r - \frac{1}{2}\sigma^2\right)t + \sigma W_t\right\}, \tag{9.9}$$

where W_t is standard normal variable with 0 mean and variance t.

The distribution of P_t is often known as log-Normal distribution. Thus as the time interval Δt goes to zero, the n period binomial asset price formula tends to the Black–Scholes asset price formula.

9.7 Pricing American Options with R

So far we have used the European options as examples for pricing with binomial models. However, these can be priced directly using the Black–Scholes formula under the Black–Scholes model. That is not the case for American options. The payoff of these options depend on the entire path of the stock price and not only on the price at the terminal time. So it is not enough to look at the distribution of the

stock price at the terminal time. We need to simulate the whole path of the stock price process to price these options.

In this section we shall develop the R code to price an American call option with strike K and expiration T on stock with initial price S. The risk-free interest rate is r and the volatility of the stock is σ. We use the binomial approximation with N periods. First we present the code for the European put option as that helps us understand the setup. We start from the terminal time with the terminal payoff according to number of upward steps in the binomial model. We then move back one step in time and calculate the expected value of the option according to the binomial model. We continue until we reach the initial time.

```
EuroPut = function(S, T , K, r, sigma , N){
  dT=T/N  #length of each subinterval
  u=exp(sigma*sqrt(dT)) #using the CRR model
  d=1/u
  p=(exp(r*dT)-d)/(u-d)
  tree=matrix(NA,nrow=(N+1),ncol=(N+1))  #setting up the matrix
  for(i in 0:N){
    tree[i+1,N+1]=max(0,(K-S*u^i*d^(N-i)))
    #The pay-off at terminal time with i upward movements
  }
  for(j in (N-1):0){
    for(i in 0:j){
    tree[i+1,j+1]=exp(-r*dT)*(p*tree[i+2,j+2]
    +(1-p)*tree[i+1,j+2])
    #The expected value of the option at each previous step
    }
  }
  price=tree[1,1]
  return(price)
}
EuroPut(S=10,T=10,K=11,r=0.05,sigma=0.1,N=10)

## [1] 0.09864932
```

Now we are ready to build the program for the American option. The new feature of the program is the variable exercise. The option will be exercised at an intermediate time if the expected future payoff at that stage is less than the payoff from exercising the option at that stage. We mark this step in the program with ***.

```
AmericanCall = function(T, S, K, r,sigma, N) {
  dT =  T / N
  u  =  exp(sigma * sqrt(dT))
  d  =  1/u
  p=(exp(r*dT)-d)/(u-d)
  tree=matrix(NA,nrow=(N+1),ncol=(N+1))
  for(i in 0:N){
    tree[i+1,N+1] = max(0,(S*u^i*d^(N-i)-K))
  }
  for(j in (N-1):0){
    for(i in 0:j){
      # binomial value
```

```
      tree[i+1,j+1] = exp(-r*dT)*(p*tree[i+2,j+2]+(1-p)*tree[i+1,j+2])
      # exercise value
      exercise = S*u^i*d^(j-i)-K
      if(tree[i+1,j+1] < exercise) tree[i+1,j+1] = exercise  #***
    }
  }
  americanCall = tree[1,1]
  return(americanCall)
}
AmericanCall(S=12,T=10,K=12,r=0.05/250,sigma=0.01,N=100)

## [1] 0.1631424
```

9.8 Beyond Binomial

Binomial models, although most common, are not the only lattice models. We describe the trinomial model in detail here. In general, this can be extended to multinomial models, where at every time step the stock price can move to n positions with fixed probabilities $(p_1, \cdots, p_n)$. The positions are specified as fixed multiples of the current price.

Suppose the asset price is $S(t)$ at time t. At time $t + \Delta t$ the asset price can be uS (up-state) with probability p_u, mS (middle-state) with probability p_m, and dS (down-state) with probability p_d. This is a trinomial model. With proper choice of parameters, the trinomial model can be more efficient than the binomial model.

Let us look at the specification of the parameters. The probabilities need to add to 1. In addition, by equating the first and second moments of the asset price $S(t + \Delta t)$ in both continuous and discrete models, we obtain the following equations for the risk-neutral probabilities and step sizes:

$$\begin{aligned} p_u + p_m + p_d &= 1 \\ p_u u + p_m m + p_d d &= \exp(r\Delta t) \\ p_u u^2 + p_m m^2 + p_d d^2 &= \exp((2r + \sigma^2)\Delta t). \end{aligned}$$

There are three equations and six variables u, m, d and p_u, p_m, p_d. We need three more equations to uniquely determine these variables. Different sets of additional conditions have been proposed in the literature.

In the Boyle model, the three additional conditions are $m = 1, ud = 1, u = \exp(\lambda\sigma\sqrt{\Delta t})$, where $\lambda > 1$ is a free parameter. This leads to the risk-neutral probabilities

$$
\begin{aligned}
p_u &= \frac{(W-R)u-(R-1)}{(u-1)(u^2-1)} \\
p_m &= 1-p_u-p_d \\
p_d &= \frac{(W-R)u^2-(R-1)u^3}{(u-1)(u^2-1)},
\end{aligned}
$$

where $R = \exp(r\Delta t)$ and $W = \exp((2r+\sigma^2)\Delta t)$. Boyle claimed that if $p_u \approx p_m \approx p_d \approx 1/3$, then the trinomial scheme with 5 steps is comparable to the CRR binomial scheme with 20 steps.

The Hull–White model is the same as the Boyle model with $\lambda = \sqrt{3}$. If terms of order $O(\Delta t)$ are ignored, then the risk-neutral probabilities are

$$
\begin{aligned}
p_d &= \frac{1}{6} - \sqrt{\frac{\Delta t}{12\sigma^2}}(r-\sigma^2/2) \\
p_m &= \frac{2}{3} \\
p_u &= \frac{1}{6} + \sqrt{\frac{\Delta t}{12\sigma^2}}(r-\sigma^2/2).
\end{aligned}
$$

The Kamrad–Ritchken model is derived by using the first two moments of the log price process instead of those of the price process. They specify $u = \exp(\Delta x)$, $m = 1$, $d = \exp(-\Delta x)$ where $\Delta x = \lambda\sigma\sqrt{\Delta t}$ and $\lambda \geq 1$. By omitting the higher order term $O((\Delta t)^2)$, they showed that the risk-neutral probabilities are

$$
\begin{aligned}
p_u &= \frac{1}{2\lambda^2} + \frac{1}{2\lambda\sigma}(r-\sigma^2/2)\sqrt{\Delta t} \\
p_m &= 1 - \frac{1}{\lambda^2} \\
p_d &= \frac{1}{2\lambda^2} - \frac{1}{2\lambda\sigma}(r-\sigma^2/2)\sqrt{\Delta t}.
\end{aligned}
$$

Note that if $\lambda = 1$ then $p_m = 0$ and the trinomial scheme is reduced to the binomial scheme. KR claimed that if $p_u = p_m = p_d = 1/3$, then the trinomial scheme with 15 steps is comparable to the binomial scheme with 55 steps.

9.9 Exercises

1. Show that by substituting the values of α and B from Eq. (9.4) into $V(S,t) = \alpha S + B$, we get the same value for $V(S,t)$ as the one obtained in Eq. (9.3).
2. Starting with Eqs. (9.1), (9.5) and the condition $ud = 1$, show that if the terms of order $O((\Delta t)^{3/2})$ and higher are ignored, then we get $u = \exp(\sigma\sqrt{\Delta t})$, $d = \exp(-\sigma\sqrt{\Delta t})$.

3. Write an R code for evaluating the price of a European call option with the Binomial model.
4. Write an R code to price an American Put option with the parameters used in Sect. 9.6.
5. Show that with $T = 0.5, r = 0.05/250, \sigma = 0.01, N = 5$, if we take $\lambda = 1.25$ then the probabilities in Boyle's model are close to 1/3.
6. Write an R code for evaluating a European call option with the trinomial model. Using the parameters in the above problem, compare the accuracy to the CRR binomial model with 20 steps.

Chapter 10
Simulating Brownian Motion



Brownian motion and geometric Brownian motion are the most common models encountered in financial problems. In certain cases, it is possible to obtain analytical expressions for objects of interest from the model. Examples include pricing of vanilla options under the Black–Scholes model. When we want to price more complex options under other stock price mechanisms, it becomes impossible to find explicit expressions. In such cases, one has to resort to simulating the path of the underlying process multiple times and taking averages of the object of interest over these paths. The example of path-dependent options is discussed in this chapter. Time-dependent drift and volatility of the underlying process can also be handled using these methods. Other applications include the computation of Greeks that are useful in measuring sensitivity as well as in hedging. In another direction, many statistical properties of proposed methods cannot be proved explicitly, particularly in small samples. Simulation allows us to study such properties.

In this chapter we discuss methods of simulating paths of Brownian motion, in single and multiple dimensions in Sects. 10.2 and 10.5, respectively. In order to simulate observations from Brownian motion, it is necessary to simulate normal random variables and vectors as these are the building blocks. In Sect. 10.1 we present the simulation of these basic blocks. Since the most popular model for stock price dynamics is geometric Brownian motion, we devote Sect. 10.3 to this purpose. The most common reason for such simulation arises in the pricing of path-dependent options. Examples of such applications are presented in Sect. 10.4.

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/978-981-19-2008-0_10.

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

R. Sen and S. Das, *Computational Finance with R*, Indian Statistical Institute Series,
https://doi.org/10.1007/978-981-19-2008-0_10

10.1 Generating Normal Random Variables and Vectors

We begin this chapter with methods of simulating independent normal random variables. Note that if $Z \sim \mathcal{N}(0, 1)$, then $X = \sigma Z + \mu \sim \mathcal{N}(\mu, \sigma^2)$. Hence, it suffices to consider sampling from $\mathcal{N}(0, 1)$. In Chap. 8, we have described the methods for simulation of uniform random variables. Now we assume the availability of a sequence $U_1, U_2, \ldots$ of independent random variables uniformly distributed on the unit interval [0, 1] and consider methods for transforming these to normally distributed random variables.

10.1.1 Inverse Transform Method

The general inverse transform method of Chap. 8 can be used to generate samples from the normal distribution. Since the inverse of the normal cdf is not available in closed form, it is necessary to obtain numerical approximations. Even the cdf is not available in closed form and needs to be evaluated. Evaluation of the cumulative normal is in itself necessary for many financial applications (including evaluation of the Black–Scholes formula). One way is to use numerical integration methods discussed in Chap. 4. Here we describe an alternative method proposed by (Hastings, 1955). It takes the form

$$\Phi(x) \approx 1 - \phi(x)(b_1 t + b_2 t^2 + b_3 t^3 + b_4 t^4 + b_5 t^5), \quad t = \frac{1}{1 + px},$$

for constants b_i and p for positive x, where ϕ is the normal pdf and Φ is the normal cdf. The approximation extends to negative arguments through the identity $\Phi(-x) = 1 - \Phi(x)$. This method has a maximum absolute error less than 7.5×10^{-8}. The necessary constants for this approximation are as follows:

$$\begin{array}{lll} b_1 = 0.319381530 & b_2 = -0.356563782 & b_3 = 1.781477937 \\ b_4 = -1.821255978 & b_5 = 1.330274429 & p = 0.2316419 \end{array}$$

The problem of computing the inverse $\Phi^{-1}(u)$ can be posed as one of finding the root x of the equation $\Phi(x) = u$ and in principle addressed through any general root-finding algorithm. Newton's method, for example, produces the iterates

$$x_{n+1} = x_n - \frac{\Phi(x_n) - u}{\phi(x_n)}. \tag{10.1}$$

See (Marsaglia and Bray, 1964) for more details on starting values and alternative methods. For this, at every step of iteration, one needs to evaluate $\Phi(x_n)$ numerically as described above.

10.1.2 Box–Muller Method

It is clear from the above discussion that the inversion method involves numerous computations and approximations. This requires high computational time and leads to accumulation of errors. A simple alternative method is the Box–Muller (Box, 1958) algorithm. This algorithm generates a pair of independent univariate standard normal random variables at every step. The algorithm is based on the following two properties of the bivariate normal. If $Z \sim \mathcal{N}_2(0, I)$, then

1. $R = Z_1^2 + Z_2^2$ is exponentially distributed with mean 2, i.e., $P(R \leq x) = 1 - \exp(-x/2)$
2. Given R, the point (Z_1, Z_2) is uniformly distributed on the circle of radius $\sqrt{R}$ centered at the origin.

Thus, to generate (Z_1, Z_2), we may first generate R and then choose a point uniformly on the circle of radius $\sqrt{R}$. To sample from the exponential distribution we may set $R = -2\log(U_1)$, with $U_1 \sim Unif[0, 1]$. To generate a random point on a circle, we may generate a random angle uniformly between 0 and 2π and then map the angle to a point on the circle. The random angle may be generated as $V = 2\pi U_2, U_2 \sim Unif[0, 1]$. The corresponding point on the circle has coordinates $(\sqrt{R}\cos(V), \sqrt{R}\sin(V))$. The complete algorithm is given as follows.

Box–Muller Algorithm

1. Generate U_1, U_2 independent $Unif[0, 1]$
2. $R = -2\log(U_1)$
3. $V = 2\pi U_2$
4. $Z_1 = \sqrt{R}\cos(V)$
5. $Z_2 = \sqrt{R}\sin(V)$

Marsaglia and Bray (1964) developed a modification of the Box–Muller method that reduces computing time by avoiding evaluation of the sine and cosine functions. Instead, the Marsaglia-Bray method uses acceptance-rejection described in Sect. 8.2.2 to sample points uniformly in the unit disk and then transforms these points to normal variables.

The R command to generate normal random variable is `rnorm`. The method used can be `Kinderman-Ramage`, `Buggy Kinderman-Ramage`, `Ahrens-Dieter`, `Box-Muller`, `Inversion`, `user-supplied`. The default is `Inversion`. The command `RNGkind()[2]` can be used to find out which method is being used. The default method can be changed by resetting it as `RNGkind(normal.kind = "Box-Muller")`. For the actual inversion, the program `qnorm` is called which uses a C translation of Wichura (1988) and provides precise results up to about 16 digits.

The following code generates 1000 replications of normal random samples of size 500 and tests for normality for each of them using the Shapiro–Wilk test. Finally it reports what proportion of the p values are less than 5%. We repeat the process after

changing the method to Box–Muller. In both cases we get the observed proportion to be slightly over 5% indicating that the generated samples can be considered to be from normal distribution. The Q-Q plot of one such sample is presented in Fig. 10.1. The points lie roughly on the straight line with zero intercept and unit slope, as expected from a normal sample. Details on tests for normality are presented in Chap. 12.

```
t<-rep(0,1000)
for(i in 1:1000){
    x=rnorm(500,0,1)
    t[i]=shapiro.test(x)$p.value
  }
length(which(t<.05))/1000

## [1] 0.055

RNGkind(normal.kind = "Box-Muller")
for(i in 1:1000){
    x<-rnorm(500,0,1)
    t[i]<-shapiro.test(x)$p.value
  }
length(which(t<.05))/1000

## [1] 0.049

qqnorm(x,pch=20)
abline(0,1,lwd=2,col='purple')
```

10.1.3 Generating Multivariate Normal

A d dimensional random vector follows multivariate normal distribution if all linear combinations follow univariate normal. A multivariate normal distribution $\mathcal{N}_d(\mu, \Sigma)$ is specified by its mean vector μ and covariance matrix Σ. Note that if $Z \sim \mathcal{N}_d(0, I)$ and $X = \mu + AZ$, then $X \sim \mathcal{N}_d(\mu, AA^T)$. Using any of the methods discussed in the previous sections, we can generate independent standard normal random variables $Z_1, \ldots, Z_d$ and assemble them into a vector $Z \sim \mathcal{N}_d(0, I)$. Thus, the problem of sampling X from the multivariate normal $\mathcal{N}_d(\mu, \Sigma)$ reduces to finding a matrix A for which $AA^T = \Sigma$. Two common methods of achieving this are described below. Each has its own advantages. We shall return to these methods for the simulation of Brownian motion.

Among all such As, for which $AA^T = \Sigma$, a lower triangular one is particularly convenient because it reduces the calculation of $\mu + AZ$ to the following:

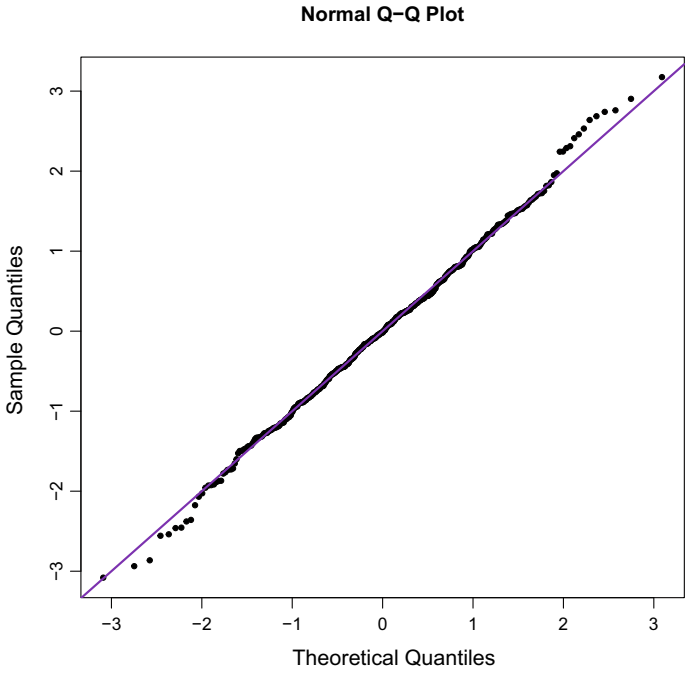


Fig. 10.1 Q-Q plot of a sample of size 500 from normal distribution

$$
\begin{aligned}
X_1 &= \mu_1 + A_{11}Z_1 \\
X_2 &= \mu_2 + A_{21}Z_1 + A_{22}Z_2 \\
&\cdots \\
X_d &= \mu_d + A_{d1}Z_1 + A_{d2}Z_2 + \cdots + A_{dd}Z_d.
\end{aligned}
$$

A full multiplication of the vector Z by the matrix A would require approximately twice as many multiplications and additions. Such a matrix A can be obtained by the Cholesky Factorization described in Chap. 2.

Alternatively, the equation $AA^T = \Sigma$ can also be solved by diagonalizing Σ. As a symmetric $d \times d$ matrix, Σ has d real eigenvalues $\lambda_1, \ldots, \lambda_d$, and because Σ must be positive definite or semi-definite the λ_i are non-negative. Furthermore, Σ has an associated orthonormal set of eigenvectors $v_1, \ldots, v_d$. It follows that $\Sigma = V\Lambda V^T$, where V is the orthogonal matrix with columns $v_1, \ldots, v_d$ and Λ is the diagonal matrix with diagonal entries $\lambda_1, \ldots, \lambda_d$. Hence, if we choose $A = V\Lambda^{1/2}$ then $AA^T = V\Lambda V^T = \Sigma$.

Unlike the Cholesky factor, the matrix A has no particular structure providing a computational advantage in evaluating AZ, nor is this matrix faster to compute than the Cholesky factorization. The eigenvectors and eigenvalues of a covariance matrix

do, however, have a statistical interpretation that is occasionally useful. We discuss this interpretation next.

If Σ has full rank, then A must have full rank and $X = AZ$ implies $Z = BX$ with $B = A^{-1}$. Thus, the factors Z_j are themselves linear combinations of the X_i. Since we have $A^{-1} = \Lambda^{-1/2} V^T$, it follows that Z_j is proportional to $v_j^T X$. The factors Z_j are optimal in a precise sense. Suppose the eigenvalues are ordered in decreasing order of magnitude and the Z_j's are ordered accordingly. The best single-factor approximation to X; i.e., the linear combination $w^T X$ that best captures the variability of the components of X is Z_1. The next best factor orthogonal to the first is Z_2 and so on. In the statistical literature, the linear combinations $v_j^T X$ are called the principal components of X. We may thus say that the principal components provide an optimal lower-dimensional approximation to a random vector. The variance explained by the first k principal components is the ratio

$$\frac{\lambda_1 + \lambda_2 + \cdots + \lambda_k}{\lambda_1 + \lambda_2 + \cdots + \lambda_k + \cdots + \lambda_d}.$$

The R function to generate multivariate normal is `mvrnorm` inside the `MASS` library. The matrix decomposition is done via principal components method. If $n = 1$ a vector of the same length as μ is returned. Otherwise an n by length(μ) matrix with one sample in each row is returned. The function `rmvnorm` in the library `mvtnorm` allows changing the method to Cholesky. Alternatively, one can directly compute the Cholesky factorization of Σ as in the code below that generates $N = 100$ replicates of bivariate normal with specified mean and variance.

```
N = 100
mu = c(1,2)
sigma = matrix(c(1,.5,.5,.3),2,2)
M = t(chol(sigma))
Z = matrix(rnorm(2*N),2,N)
bvn = t(M %*% Z) + matrix(rep(mu,N), byrow=TRUE,ncol=2)
```

10.2 Simulation of Brownian Motion

A stochastic process $B = \{B(t) : t \geq 0\}$ is called standard Brownian motion if

1. $B(0) = 0$ almost surely.
2. The mapping $t \to B(t)$ is a continuous function on $[0, T]$ with probability 1.
3. B has stationary and independent increments.
4. $B(t) - B(s) \sim N(0, (t - s))$, where $0 \leq s < t$.

A stochastic process $X = \{X(t) : t \geq 0\}$ is Brownian motion with initial value x, drift μ and volatility σ if

$$\frac{X(t) - x - \mu t}{\sigma}$$

is standard Brownian motion. Thus, we may construct X from a standard Brownian motion B by setting $X(t) = x + \mu t + \sigma B(t)$.

It follows that $X(t) \sim \mathcal{N}(x + \mu t, \sigma^2 t)$. Moreover, X solves the stochastic differential equation (SDE)

$$dX(t) = \mu dt + \sigma dB(t).$$

For time-varying but deterministic $\mu(t)$ and $\sigma(t) > 0$, we may define a Brownian motion with drift μ and diffusion coefficient σ^2 through the SDE

$$dX(t) = \mu(t)dt + \sigma(t)dB(t);$$

The process X has continuous sample paths and independent increments. Each increment $X(t) - X(s)$ is normally distributed with mean $\int_s^t \mu(u)du$ and variance $\int_s^t \sigma^2(u)du$.

10.2.1 Random Walk Construction

First we focus on simulating values $(B(t_1), \ldots, B(t_n))$ of standard Brownian motion at a fixed set of points $0 < t_1 < \ldots < t_n$. The vector $(B(t_1), \ldots, B(t_n))$ is a linear transformation of the vector of increments $(B(t_1), B(t_2) - B(t_1), \ldots, B(t_n) - B(t_{n-1}))$. Since these increments are independent and normally distributed, $(B(t_1), \ldots, B(t_n))$ has a multivariate normal distribution. Simulating $(B(t_1), \ldots, B(t_n))$ is thus a special case of the general problem of generating multivariate normal vectors as described in Sect. 10.1.3. In order to apply any of these methods, we first need to find the mean vector and covariance matrix of $(B(t_1), \ldots, B(t_n))$. It is easy to check that the mean vector is the zero vector and the covariance matrix $\Sigma = ((C_{i,j}))$ where $C_{i,j} = \min(t_i, t_j)$.

Consider the Cholesky factorization method described in Sect. 10.1.3. The matrix A in this case is

$$A = \begin{pmatrix} \sqrt{t_1} & 0 & \ldots & 0 \\ \sqrt{t_1} & \sqrt{t_2 - t_1} & \ldots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ \sqrt{t_1} & \sqrt{t_2 - t_1} & \ldots & \sqrt{t_n - t_{n-1}} \end{pmatrix} .. \tag{10.2}$$

Let $Z_1, \ldots, Z_n$ be independent standard normal random variables, generated using any of the methods in Sect. 10.1. $B(t_i)$ is the product of the i-th row of this A matrix and the random vector $Z = (Z_1, \ldots, Z_n)$, implying

$$\begin{aligned} B(t_i) &= \sqrt{t_1}Z_1 + \sqrt{t_2 - t_1}Z_2 + \cdots + \sqrt{t_{i-2} - t_{i-1}}Z_{i-1} + \sqrt{t_{i-1} - t_i}Z_i \\ &= B(t_{i-1}) + \sqrt{t_i - t_{i-1}}Z_i. \end{aligned}$$

This is due to the specific for A in this case, the nonzero entries of each column of A being identical.

This is called the random walk construction of Brownian motion.

For simulating Brownian motion X with drift μ and volatility σ at times $0 = t_0 < t_1 < t_2 < \cdots < t_n$, set $X(t_i) = X(0) + \mu t_i + \sigma B(t_i)$. We can also achieve this directly through the recursion

$$X(t_i) = X(t_{i-1}) + \mu(t_i - t_{i-1}) + \sigma\sqrt{t_i - t_{i-1}}Z_i, \quad i = 1, \ldots, n.$$

With time-dependent parameters $\mu(t)$ and $\sigma(t)$, the recursion becomes

$$X(t_i) = X(t_{i-1}) + \int_{t_{i-1}}^{t_i} \mu(s)ds + \sqrt{\int_{t_{i-1}}^{t_i} \sigma^2(u)du}Z_i, \quad i = 1, \ldots, n.$$

The Euler approximation given below is often convenient, although it will introduce a discretization error

$$X(t_i) = X(t_{i-1}) + \mu(t_{i-1})(t_i - t_{i-1}) + \sigma(t_{i-1})\sqrt{t_i - t_{i-1}}Z_i, \quad i = 1, \ldots, n.$$

The following R-code simulates paths from Brownian motion and plots it in the Fig. 10.2 for different values of the starting point, drift, and volatility.

```
sim.Brownian.motion = function(X0,t,mu,sigma){
   n = length(t)
   X = rep(NA,n)
   Z = rnorm(n,mean=0,sd=1)
   X[1] = X0+sigma*sqrt(t[1])*Z[1]+mu*t[1]
   for(i in 2:n){
     X[i] = X[i-1]+sigma*sqrt(t[i]-t[i-1])*Z[i]+mu*(t[i]-t[i-1])
   }
   return(X)
 }
t = seq(0,1,0.001)
X = sim.Brownian.motion(X0=0,t=t,mu=0,sigma=1)
plot(ts(P,start=0,frequency = 1000),ylab="",)
```

While the random walk construction suffices for most applications, it is interesting and sometimes useful to consider alternative sampling methods. In the following sections we describe two such approaches and their advantages.

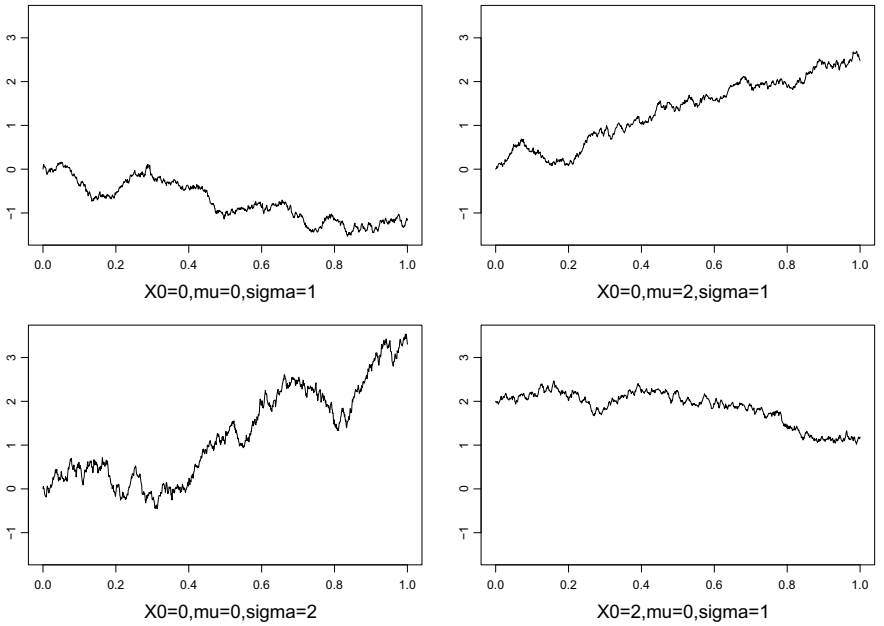


Fig. 10.2 Simulated Samples from Brownian Motion. Top Left: Starting at zero, drift 0, volatility 1. Top Right: Starting at zero, drift 2, volatility 1. Bottom Left: Starting at zero, drift 0, volatility 2. Bottom Right: Starting at 2, drift 0, volatility 1

10.2.2 Brownian Bridge Construction

The random walk method generates the vector $(B(t_1), \ldots, B(t_n))$ from left to right. We may however generate the $B(t_i)$ in any order we choose, provided that at each step we sample from the correct conditional distribution given the values already generated. For example, we may first generate the final value $B(t_n)$, then sample $B(t_{[n/2]})$ conditional on the value of $B(t_n)$, and proceed by progressively filling in intermediate values. This flexibility can be useful in implementing variance reduction techniques. Because of the joint normal distribution of finite dimensional marginals of Brownian motion, the conditional distribution needed at each step is itself normal and this makes conditional sampling feasible.

Conditioning a Brownian motion on its endpoints produces a Brownian bridge. As a first step in developing this method, suppose $0 < u < s < t$ and consider the problem of generating $B(s)$ conditional on $B(u) = x$ and $B(t) = y$. The joint distribution is given by

$$\begin{pmatrix} B(u) \\ B(s) \\ B(t) \end{pmatrix} \sim \mathcal{N}_3 \left(0, \begin{pmatrix} u & u & u \\ u & s & s \\ u & s & t \end{pmatrix} \right).$$

Calculating the conditional distribution gives

$$(B(s)|B(u) = x, B(t) = y) \sim \mathcal{N}\left(\frac{(t-s)x + (s-u)y}{t-u}, \frac{(s-u)(t-s)}{t-u}\right). \tag{10.3}$$

This is called Brownian bridge as the end points are specified and we are looking at the distribution of intermediate points. Suppose, more generally, that the values $B(s_1) = x_1, B(s_2) = x_2, \ldots, B(s_k) = x_k$ of the Brownian path have been determined at the times $s_1 < s_2 < \ldots < s_k$ and that we wish to sample $B(s)$ conditional on these values. Suppose that $s_i < s < s_{i+1}$. Then, due to Markov property of Brownian motion,

$$(B(s)|B(s_j) = x_j, j = 1, \ldots, k) = (B(s)|B(s_i) = x_i, B(s_{i+1}) = x_{i+1}),$$

in the sense that the two conditional distributions are the same. Thus, conditioning on all $B(s_j)$ is equivalent to conditioning on the values of the Brownian path at the two times closest to s. By repeatedly using these observations, we may indeed sample the components of the vector $(B(t_1), \ldots, B(t_n))$ in any order.

This construction can help in using variance reduction by stratification. It is difficult to use stratification in multiple dimensions, but we may use stratification just on the terminal value of asset price. Then we generate intermediate values using the bridge.

10.2.3 Principal Components Construction

As noted in Sect. 10.2.1,

$$B = (B(t_1), \ldots, B(t_n)) \sim \mathcal{N}_n(0, C), \tag{10.4}$$

where $C_{i,j} = \min(t_i, t_j)$. Thus we can use the method of principal components to simulate the multivariate normal vector as described in Sect. 10.1.3. If $C = A\Lambda A^T$ where $\Lambda = \text{diag}(\lambda_i)$, then

$$B = \sum_{i=1}^{n} a_i \sqrt{\lambda_i} Z_i,$$

where a_i are the columns of the matrix A and Z_i are independent standard normal random variables.

The advantage of this method is that we can obtain a pretty good approximation to the Brownian path by considering a smaller number of standard normals. Consider, for example, a 1000-step discrete Brownian path with equal time increments so that $t_{i+1} - t_i = 1/1000$. The magnitudes of the eigenvalues of the corresponding covariance matrix drop off rapidly. The five largest are 405.690204, 45.076763, 16.227688, 8.279474, and 5.008603. The variability explained by $Z_1, \ldots, Z_k$ is 81%, 90%, 93%, 95%, for $k = 1, \ldots, 4$. This indicates that although full construction of the 1000-step path requires 1000 normal random variables, most of the variability

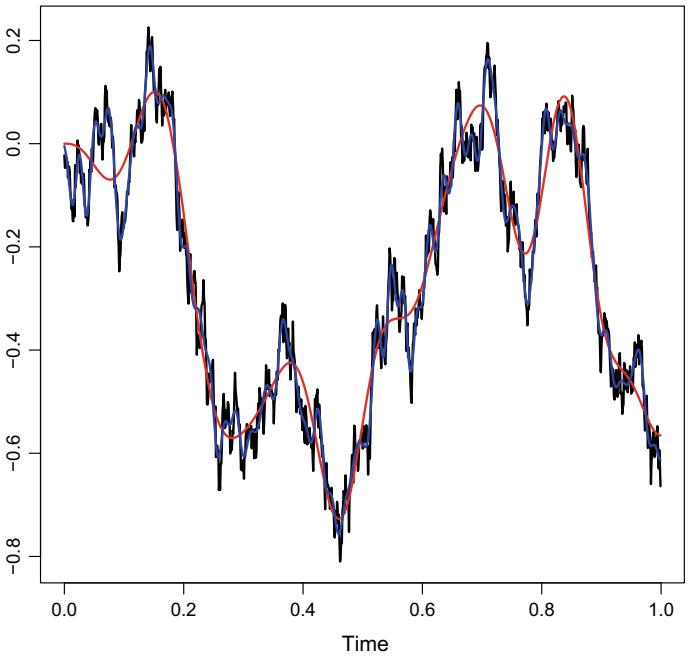


Fig. 10.3 Simulated sample from standard Brownian motion with step size 1/1000 (black) with approximations using 20 (red) and 100 (blue) eigenvalues

of the path can be determined using far fewer Z_is. The code below and Fig. 10.3 illustrates this with approximations using 20 and 100 normal random variables.

```
set.seed(896)
sig = matrix(0,1000,1000)
for(i in 1:1000){
  for(j in 1:1000){
    sig[i,j]=min(i,j)/1000
  }
}
u = eigen(sig)
v = cumsum(u$values)
v = v/v[1000]
z = rnorm(1000,0,1)
zz = z*sqrt(u$values)
ful = u$vectors%*%zz
app = u$vectors[,1:100]%*%zz[1:100]
app1 = u$vectors[,1:20]%*%zz[1:20]
plot(ts(ful,start=0,frequency = 1000),ylab="",lwd=2)
lines(ts(app1,start=0,frequency = 1000),col='red',lwd=2)
lines(ts(app,start=0,frequency = 1000),col='blue',lwd=2)
```

10.3 Geometric Brownian Motion

Geometric Brownian motion (GBM) is the most fundamental model of the value of a financial asset. Mathematically GBM is simply an exponentiated Brownian motion. The use of GBM as a model in finance is due primarily to work of Paul Samuelson in the 1960s. GBM is always positive because the exponential function takes only positive values. For GBM the percentage changes

$$\frac{S(t_2) - S(t_1)}{S(t_1)}, \frac{S(t_3) - S(t_2)}{S(t_2)}, \ldots, \frac{S(t_n) - S(t_{n-1})}{S(t_{n-1})}$$

are independent for $t_1 < t_2 < \cdots < t_n$. A GBM process with drift μ and volatility σ is denoted by $S \sim GBM(\mu, \sigma)$ and is specified through an SDE of the form

$$\frac{dS(t)}{S(t)} = \mu dt + \sigma dB(t)$$

an expression suggesting a Brownian model of the 'instantaneous returns' $dS(t)/S(t)$. If S has initial value $S(0)$, then, by using Ito's formula, we obtain the solution

$$S(t) = S(0) \exp[(\mu - 1/2\sigma^2)t + \sigma B(t)].$$

Note that $S(t) = \exp(X(t))$ where

$$X(t) = \left\{\left(\mu - \frac{\sigma^2}{2}\right)t + \sigma B_t\right\},$$

is a Brownian motion with drift $\left(\mu - \frac{\sigma^2}{2}\right)$ and volatility σ. We know how to simulate from Brownian motion using the methods of Sect. 10.2. We can now use X_t simulated from Brownian motion and exponentiate that to generate S_t.

The following R-code shows how to simulate from Geometric Brownian Motion using random walk starting at S_0 when the length of the interval is 1 and there are n equidistant intermediate points, that is $t_i - t_{i-1} = 1/n$. The plot is presented in Fig. 10.4.

```
set.seed(896)
mu = 5
sig = 3
step = 1/1000
length = 1
s0 = 100
N = length/step
m = (mu-sig^2/2)*step
s = sig^2*step
```

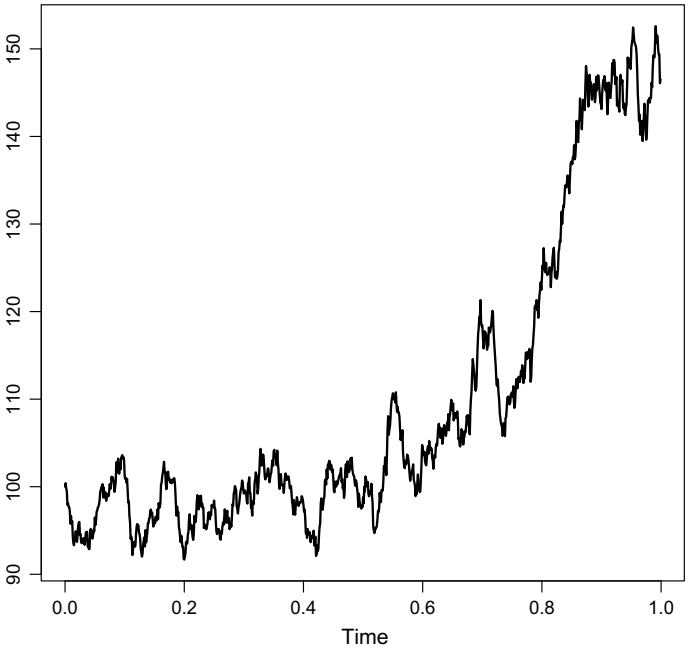


Fig. 10.4 Simulated Values of Geometric Brownian Motion starting at 100, with step size 1/1000, drift 2 and volatility 3

```
R = rnorm(N-1,m,s)
X = rep(0,N)
X[1] = log(s0)
for(i in 2:N){
  X[i] = X[i-1]+R[i-1]
}
GBM<-exp(X)
plot(ts(GBM,start=0,frequency=1/step),ylab="",lwd=2)
```

10.4 Example: Path-Dependent Options

Our interest in simulating paths of geometric Brownian motion lies primarily in pricing options, particularly those whose payoffs depend on the path of an underlying asset S and not simply its value $S(T)$ at a fixed exercise date T. Through the principles of option pricing, the price of an option may be represented as an expected discounted payoff. This price is estimated through simulation by generating paths of

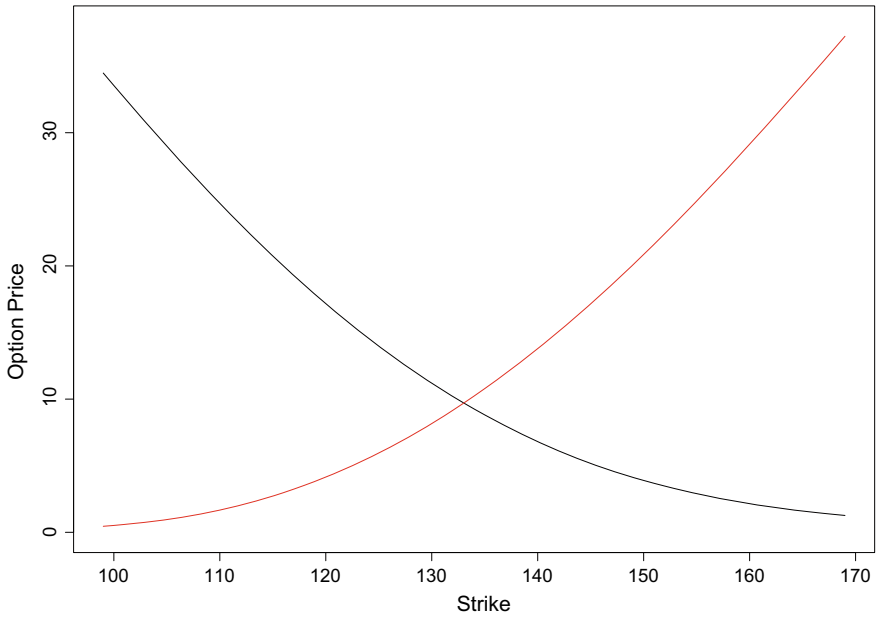


Fig. 10.5 Price of Asian Call (black) and Put (red) options for various strike prices and maturity 1 when the underlying stock follows Geometric Brownian Motion starting at 100, drift 2, and volatility 3

the underlying asset, evaluating the discounted payoff on each path, and averaging over paths.

Asian option An Asian option is an option on a time average of the underlying asset. Asian calls and puts have payoffs $(\bar{S} - K)_+$ and $(K - \bar{S})_+$, respectively, where the strike price K is a constant and $\bar{S} = \frac{1}{n}\sum_{i=1}^{n} S(t_i)$ is the average price of the underlying asset over the discrete set of monitoring dates $t_1, \ldots, t_n$. Other examples have payoffs $(S(T) - \bar{S})_+$ and $(\bar{S} - S(T))_+$. The discrete monitoring could be replaced by continuous monitoring, in which case, the sum is replaced by an integral. There are no exact formulas for the prices of these options, largely because the distribution of $\bar{S}$ is intractable. In Fig. 10.5 we present the price of several Asian options.

Barrier options Discretely monitored Barrier options are similar to those described in Chap. 6, except the set of time points is discrete. For instance, a down-and-out call with barrier b, strike K, and expiration T has payoff $1\{\tau(b) > T\}(S(T) - K)_+$, where $\tau(b) = \inf\{t_i : S(t_i) < b\}$ is the first time in $\{t_1, \ldots, t_n\}$ the price of the underlying asset drops below b (understood to be ∞ if $S(t_i) > b$ for all i) and $1\{\}$ denotes the indicator of the event in braces. Both discretely monitored and continuously monitored Barrier options are found in practice. Discretely monitored Barrier options generally do not admit pricing formulas but are easily priced by simulation through sampling of $S(t_1), \ldots, S(t_n), S(T)$.

Table 10.1 Prices of path-dependent options

Type of option	Price
Lookback call	12.43
Lookback put	78.15
Down-and-out call	38.19
Down-and-out put	5.28
Down-and-in call	9.12
Down-and-in put	24.74

Lookback options We have encountered continuous time Lookback options in Chap. 6. Here we describe the discrete time versions. Lookback puts and calls expiring at t_n have payoffs $(\max_{i=1,\ldots,n}(S(t_i) - S(t_n))$ and $(S(t_n) - \min_{i=1,\ldots,n} S(t_i))$, respectively. A Lookback call, for example, may be viewed as the profit from buying at the lowest price over $t_1, \ldots, t_n$ and selling at the final price $S(t_n)$.

The prices of the various options described in this section are given in Table 10.1 when the underlying price process is geometric Brownian motion with parameters $S_0 = 100$, drift 2, volatility 3, expiration 1, Strike 150, and Barrier at 90. The following code is an example to obtain the price of the Down-and-out Call when mingbm and lgbm are the vectors of minimum and ending value of the price process for several simulations.

```
ff = function(x){max(x,0)}
down = which(mingbm>90)
mean(sapply(lgbm[down]-150,ff))
```

10.5 Multiple Dimensions

We call a process $B(t) = (B_1(t), \ldots, B_d(t)), 0 \leq t \leq T$, a standard Brownian motion on $\mathbb{R}^d$ if it has $B(0) = 0$, continuous sample paths, independent increments, and $B(t) - B(s) \sim \mathcal{N}_d(0, (t-s)I), \forall 0 \leq s < t \leq T$, with I the $d \times d$ identity matrix. It follows that each of the coordinate processes $B_i(t), i = 1, \ldots, d$, is a standard one-dimensional Brownian motion and that B_i and B_j are independent for $i \neq j$.

Suppose μ is a vector in $\mathbb{R}^d$ and Σ is a $d \times d$ matrix, positive definite or semi-definite. We call a process X a Brownian motion with drift μ and covariance Σ (abbreviated $X \sim BM(\mu, \Sigma)$) if X has continuous sample paths and independent increments with $X(t) - X(s) \sim \mathcal{N}_d((t-s)\mu, (t-s)\Sigma)$.

If A is a $d \times k$ matrix satisfying $AA^T = \Sigma$ and if B is a standard Brownian motion on $\mathbb{R}^k$, then the process defined by $X(t) = \mu t + AB(t)$ is a $BM(\mu, \Sigma)$.

Let $Z_1, Z_2, \ldots$ be independent $N(0, I)$ random vectors in $\mathbb{R}^d$. We can construct a standard d-dimensional Brownian motion at times $0 = t_0 < t_1 < \ldots < t_n$ by setting $B(0) = 0$ and

$$B(t_{i+1}) = B(t_i) + \sqrt{t_{i+1} - t_i} Z_{i+1}, i = 0, \ldots, n-1.$$

This is equivalent to applying the one-dimensional random walk construction separately to each coordinate of B.

To simulate $X \sim BM(\mu, \Sigma)$, we first find a matrix A for which $AA^T = \Sigma$. If A is $d \times k$, let $Z_1, Z_2, \ldots$ be independent standard normal random vectors in $\mathbb{R}^k$. Set $X(0) = 0$ and

$$X(t_{i+1}) = X(t_i) + \mu(t_{i+1} - t_i) + \sqrt{t_{i+1} - t_i} A Z_i, i = 0, \ldots, n-1.$$

This is the random walk construction of d dimensional Brownian motion. Similar extensions can be worked out for the Brownian bridge and Principal components construction methods. An important component of these methods is the covariance matrix

$$Cov[X_i(s), X_j(t)] = \min(s, t)\Sigma_{ij}.$$

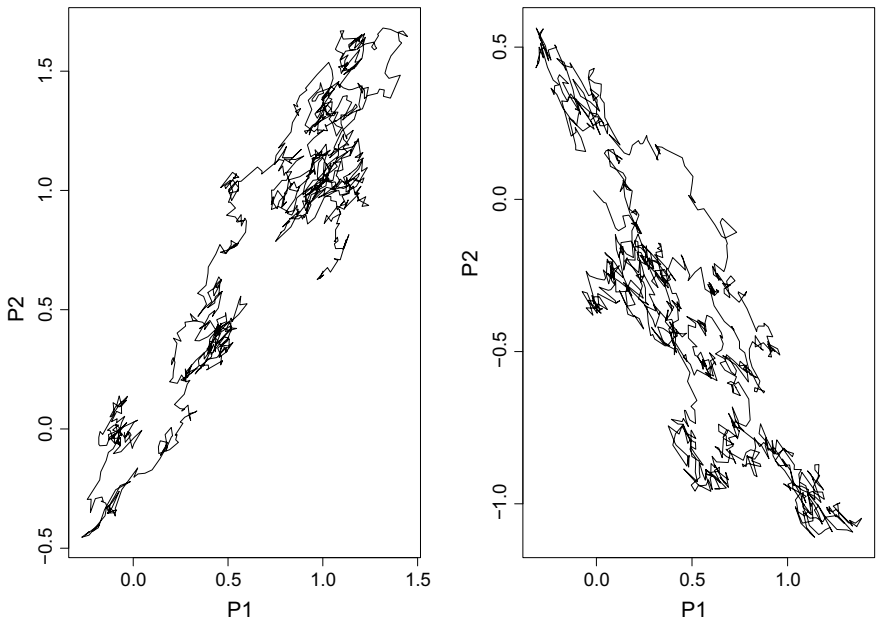


Fig. 10.6 Simulated sample from bivariate Brownian motion with both means 0, both variances 1, correlation 0.6 (left) and −0.6 (right)

In particular, given a set of times $0 < t_1 < t_2 < \cdots < t_n$, we can easily find the covariance matrix of $(X_1(t_1), \ldots, X_d(t_1), X_1(t_2), \ldots, X_d(t_2), \ldots, X_1(t_n), \ldots, X_d(t_n))$ as the Kronecker product of the matrix $C = ((\min(t_i, t_j))$ and Σ.

In Fig. 10.6 we present two simulated paths of bivariate Brownian motions with mean zero, variance 1 but different correlations.

10.6 Exercises

1. If $Z \sim \mathcal{N}_2(0, I)$, then show that $R = Z_1^2 + Z_2^2$ is exponentially distributed with mean 2 and given R, the point (Z_1, Z_2) is uniformly distributed on the circle of radius $\sqrt{R}$ centered at the origin.
2. For $\Sigma = ((C_{i,j}))$ where $C_{i,j} = \min(t_i, t_j)$, show that the matrix A in Eq. 10.2 gives the Cholesky decomposition of Σ as $\Sigma = AA^T$.
3. Prove Eq. 10.3.
4. Verify the numbers in Table 10.1 by writing your own R code.
5. Write an R code to simulate paths of a two-dimensional Brownian motion with mean zero, variance 1, and correlation 0.5.
6. Generate samples of size 100 from $\mathcal{N}(0,1)$ using inverse transform and Box-Muller methods and note the execution times. Repeat the above procedure 500 times and compare the median of the execution times for each procedure.

Chapter 11
Variance Reduction



The Monte Carlo methods described in the previous chapters are subject to variation due to the randomness inherent in sample generation. Ideally we would like this variation to be as small as possible so that in repeated implementations of the same procedure we obtain results that are close to each other. There are several methods to achieve this, and we discuss those in this chapter.

Let $Y_1, \ldots, Y_n$ be outputs from n replications of a simulation, each with distribution identical to a random variable Y. For example, Y_i could be the stock price at a future time point T for the i-th simulated path. The objective is to estimate $E(Y) = \mu_Y$. If these simulations are done repeatedly using the same procedure but with repeated random steps, then the Y_i are independent and identically distributed. The sample mean $\bar{Y} = (Y_1 + \cdots + Y_n)/n$ is an unbiased estimator for μ_y. By the law of large numbers, it converges to μ_Y with probability 1 as $n \to \infty$. The variance of $\bar{Y}$ is σ_Y^2/n where σ_Y^2 is the variance of Y. We are looking for alternative estimators that are also unbiased, but have a lower variance.

11.1 Control Variates

The first method we discuss will be that of control variates. This is very widely applicable as well as effective method of reducing the variance of a Monte Carlo method. It is similar to the concept of regression analysis in statistics where information on a predictor variable improves the estimation of a response variable. It uses the information about the errors in estimates of known quantities, like predictors, to reduce the error in an estimate of an unknown quantity, the response.

Suppose, we are interested in the mean of a variable Y. We identify another variable X, so that $E(X)$ is known. Then we generate i.i.d. observations from the joint

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/978-981-19-2008-0_11.

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

R. Sen and S. Das, *Computational Finance with R*, Indian Statistical Institute Series,
https://doi.org/10.1007/978-981-19-2008-0_11

distribution of (X, Y) as pairs $(X_i, Y_i), i = 1, \ldots, n$. The control variate estimator of $E(Y)$ for a fixed constant b is given by

$$\bar{Y}(b) = \frac{1}{n}\sum_{i=1}^{n} Y_i(b), \tag{11.1}$$
$$\text{where} \quad Y_i(b) = Y_i - b(X_i - E(X)).$$

Note that this is unbiased for μ_Y, since $\bar{X}$ is unbiased for $E(X)$. Since $Y_i(b)$ are i.i.d. random variables with mean μ_Y, the law of large numbers applies and $\bar{Y}(b)$ converges to μ_Y with probability 1 as $n \to \infty$. Now let us compare the variance of this new class of estimators with that of $\bar{Y}$.

$$\sigma^2(b) := Var(Y_i(b)) = \sigma_Y^2 - 2b\rho_{XY}\sigma_X\sigma_Y + b^2\sigma_X^2, \tag{11.2}$$

where ρ_{XY} is the correlation between X and Y and σ_X^2 is the variance of X. The variance of $\bar{Y}$ is $\sigma^2(b)/n$. This is smaller than the variance of $\bar{Y}$ if and only if

$$2b\rho_{XY}\sigma_Y > b^2\sigma_X. \tag{11.3}$$

The optimal coefficient b^* that minimizes the variance is given by

$$b^* = \frac{\sigma_Y}{\sigma_X}\rho_{XY}. \tag{11.4}$$

Note that this is same as the regression coefficient of Y on X in a simple linear regression setup.

Proposition 8 *The ratio of the variance of the optimally controlled estimator to that of the sample mean is* $\sigma^2(b^*)/\sigma_Y^2 = 1 - \rho_{XY}^2$.

The main challenge is, therefore, to find a variable X whose expectation is know and which is highly correlated, either positively or negatively, with the variable of interest Y.

In practice, the quantities σ_Y, σ_X, ρ_{XY} will be unknown and we replace them by the corresponding sample quantities s_X, s_Y, r_{XY}. For large sample size, these quantities converge to the population quantities by law of large numbers. The sample version of the optimally controlled estimator is

$$\bar{Y}(\hat{b}) = \bar{Y} - \hat{b}(\bar{X} - E(X)), \tag{11.5}$$
$$\text{where} \quad \hat{b} = \frac{s_Y}{s_X}r_{XY}.$$

Proposition 9 *The estimator* $\bar{Y}(\hat{b})$ *is a weighted Monte Carlo estimator of the form* $\sum_{i=1}^{n} w_i Y_i$ *where*

$$w_i = \frac{1}{n} - \frac{(X_i - \bar{X})(\bar{X} - E(X))}{\sum_{i=1}^{n}(X_i - \bar{X})^2}.$$

Example 11.1.1 Suppose we want to price an option by Monte Carlo method as described in Chap. 10. The starting choice of control variate is the underlying stock. The expectation is known from no arbitrage theory as $E(S_T) = \exp(rT)S_0$. The correlation is likely to be high, as the option price depends on the stock price. Let us now see what is the correlation between an option payoff and that of its underlying stock for various choices of parameters r, T and strike K, with $S_0 = 50$ and $\sigma = 0.3$. We restrict to the European Put option. Thus the variables are $Y = (K - S_T)_+$ and $X = S_T$, where

$$\log(S_T/S_0) \sim \mathcal{N}((r - \sigma^2/2)T, \sigma^2 T)$$

The following code computes the sample correlation between X and Y from Example 11.1.1 with 10000 random realizations. These correlations are reported in Table 11.1 for different values of interest rates, expirations and strikes. It is seen that the correlation is always negative with the strength getting higher for higher values of the strike. There is marginal decrease in strength with increase in interest rates. There is very little change with expiration except for small values of strike, where the strength increases with longer expiration.

```
r = 0.01
s = 0.3
s0 = 50
T = 1
u = rnorm(10000,0,1)
x = s0*exp(u*s*sqrt(T)+(r-s^2/2)*T)
f = function(z) max(z,0)
c = rep(0,8)
i = 0
for (K in seq(35,70,5)){
  i = i+1
  y = apply(matrix(K-x,1,10000),2,f)
  c[i] = cor(x,y)
}
```

Table 11.1 Correlation between stock price and payoff of a put option at expiration

r	T	K							
		35	40	45	50	55	60	65	70
0.01	1	−0.469	−0.597	−0.704	−0.788	−0.850	−0.896	−0.930	−0.953
0.05	1	−0.419	−0.552	−0.663	−0.753	−0.822	−0.873	−0.911	−0.938
0.05	0.25	−0.169	−0.388	−0.630	−0.808	−0.912	−0.965	−0.988	−0.996

Example 11.1.2 As described in Sect. 10.4, the Asian option put with strike has payoff $(K - \bar{S})_+$ where $\bar{S} = \frac{1}{n}\sum_{i=1}^{n} S(t_i)$. There are no exact formulae for the prices of these options, because the distribution of $\bar{S}$ is intractable. Thus the price has to be determined by Monte Carlo methods. That entails a variance that can be reduced using the underlying asset as control variate. However, in this case an even better control variate is the geometric mean. The distribution of the latter is tractable, because the geometric average of lognormal random variables is itself lognormal. In particular,

$$\left(\prod_{i=1}^{n} S(t_i)\right)^{1/n} = S(0)\exp([r - \sigma^2/2]\sum_{i=1}^{n} t_i + \frac{\sigma}{n}\sum_{i=1}^{n} W(t_i)).$$

Let us compute these with values in the previous example and $r = 0.05$, $T = 1$, $K = 55$ and $n = 12$. The results are presented in Fig. 11.1.

11.1.1 Multiple Controls

The methods of the previous section can be extended to multiple controls if there are d variables $X^{(1)}, \ldots, X^{(d)}$, correlated with the variable of interest Y whose expectations are known. For example, for option pricing, the expected discounted value of the underlying stock is known at all intermediate points and not only at expiration. We can, therefore, choose d time-points $t_1, \ldots, t_d$ in the interval $(0, T]$ and use the vector $(\exp(-rt_1)S(t_1), \exp(-rt_2)\ldots, \exp(-rt_d)S(t_d))$ with expectation $(S_0, \ldots, S_0)$.

Suppose the i-th replication of the simulation produces outputs Y_i and $X_i = (X_i^{(1)}, \ldots, X_i^{(d)})$, where $E(X)$ is known. Denote the covariance matrix of X by Σ_X, the variance of Y by σ_Y^2 and the covariance of X and Y by the vector Σ_{XY}. Then the control variate estimator for a fixed vector $b = (b_1, \ldots, b^d)^T$ is given by

$$\bar{Y}(b) = \bar{Y} - \sum_{j=1}^{d} b_j(\bar{X}^{(j)} - E(X^{(j)}). \tag{11.6}$$

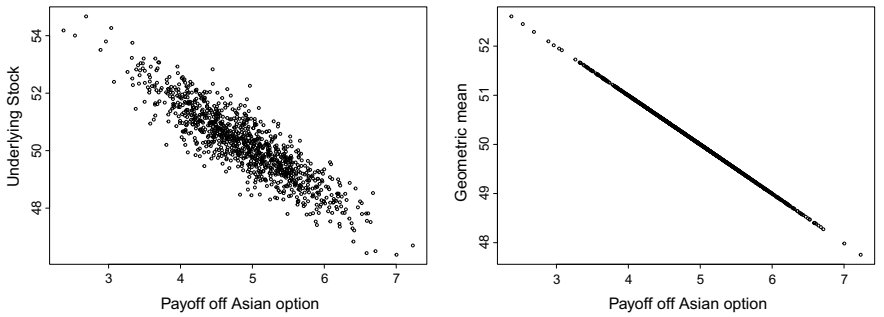


Fig. 11.1 Payoff of Asian Put option plotted against underlying stock (left) and geometric mean (right)

The variance of $\bar{Y}(b)$ is $\frac{1}{n}(\sigma_Y^2 - 2b^T\Sigma_{XY} + b^T\Sigma_X b)$. This is minimized at

$$b^* = \Sigma_X^{-1}\Sigma_{XY} \tag{11.7}$$

and the minimum value of the variance is

$$\frac{1}{n}(\sigma_Y^2 - \Sigma_{XY}^T\Sigma_X^{-1}\Sigma_{XY}). \tag{11.8}$$

Proposition 10 *The ratio of the variance of the optimally controlled estimator to that of the sample mean is* $\sigma^2(b^*)/\sigma_Y^2 = 1 - \frac{\Sigma_{XY}^T\Sigma_X^{-1}\Sigma_{XY}}{\sigma_Y^2} = 1 - R^2$.

R^2 measures the fraction of variance that is removed by optimally using the control variates. As with single control, the variance and covariance quantities involved in the optimal estimator are unknown and have to be replaced by the corresponding sample quantities. The sample version of the optimally controlled estimator is again a weighted Monte Carlo estimator of the form $\sum_{i=1}^n w_i Y_i$ with w_i now given by

$$\frac{1}{n} - \frac{1}{n-1}(X_i - \bar{X})^T S_{XX}^{-1}(\bar{X} - E(X)),$$

$$\text{where} \quad S_{XX} = \frac{1}{n-1}\sum_{i=1}^n (X_i - \bar{X})(X_i - \bar{X})^T. \tag{11.9}$$

11.1.2 Nonlinear Controls and Delta Method

In case the relation between the variables X and Y is non-linear, this can be incorporated into the control variate method through nonlinear controls. Some common examples are

$$\bar{Y}\left(\frac{E(X)}{\bar{X}}\right)^b, \bar{Y}\exp(b(\bar{X}-EX)), \bar{Y}-b\log\left(\frac{\bar{X}}{EX}\right)$$

for power, exponential and logarithmic relations respectively. Convergence of $\bar{X}$ to EX ensures that the adjustment vanishes as the sample size increases. The general form of nonlinear controls is $h(\bar{X}, \bar{Y})$ for any function h satisfying

$$h(EX, y) = y \quad \forall y. \tag{11.10}$$

The job is now to show that such an estimator $h(\bar{X}, \bar{Y})$ indeed converges to μ_Y as $n \to \infty$; that is, when we take a large enough number of Monte Carlo replicates and to find the variance of the new estimator. The tool used for this is known as Delta method in statistics. Let $\xi_1, \ldots, \xi_n$ be i.i.d. random variables in $\mathbb{R}^k$ with mean μ and covariance matrix Σ. Then from multivariate central limit theorem, we have

$$\sqrt{n}(\bar{\xi} - \mu) \Rightarrow \mathcal{N}(0, \Sigma).$$

Let $h : \mathbb{R}^k \to \mathbb{R}^m$ be a continuously differentiable function such that the partial derivatives of h are not all zero and $m \leq k$. Then, by the Delta method,

$$\sqrt{n}(h(\bar{\xi}) - h(\mu)) \Rightarrow \mathcal{N}(0, \nabla h(\mu)\Sigma\nabla h(\mu)^T),$$

where ∇h is the gradient, that is the matrix of partial derivatives of h.

In this case, we take $\xi = (XY)$. The proposed estimator is $h(\bar{\xi})$. By Eq. 11.10), $h(\mu) = \mu_Y$. Thus,

$$\sqrt{n}(h(\bar{X}, \bar{Y}) - \mu_Y) \Rightarrow \mathcal{N}(0, \tilde{\sigma}^2),$$

where $\tilde{\sigma}^2 = \nabla h(\mu)\Sigma\nabla h(\mu)^T$.

11.2 Antithetic Variates

The idea of antithetic variates is to generate random variables in pairs, such that they are negatively correlated. The computational effort in generating the pair should be very close to that of generating a single random variable. Thus for generating $2n$ variables (n pairs) the total computational effort is close to that of generating n variables. The variance may be reduced by a factor of more than $1/2$ when the variables are negatively correlated. Note that if Y and $\tilde{Y}$ are identically distributed with variance σ^2 and correlation $\rho < 0$, then

$$Var((Y + \tilde{Y})/2) = \sigma^2(1 + \rho)/2 < \sigma^2/2. \tag{11.11}$$

In summary, we are looking for pairs of random variables with same distribution and negative dependence, that do not take too much extra effort to generate.

The most common application of this method is illustrated below. Suppose we want to estimate the mean of X that has cdf F which is invertible. Then we can use the inverse transform method to generate observations from F by generating $U \sim U((0, 1)$ and taking $X = F^{-1}(U)$. With a sample of size n, the Monte Carlo estimator for $E(X)$ is $(\sum_{i=1}^{n} X_i)/n$. In the antithetic variates approach, in addition to $F^{-1}(U)$, we also consider $F^{-1}(1 - U)$. Since U and $1 - U$ have the same distributions, $F^{-1}(U)$ and $F^{-1}(1 - U)$ both have the same distribution as that of X. That is, we get a pair of random variables with the same realization of U. Thus the additional computational effort is very small. Since U and $1 - U$ are negatively correlated and F^{-1} is an increasing function, the pair $F^{-1}(U)$ and $F^{-1}(1 - U)$ are negatively correlated. Thus we have a pair of random variables from the same distribution which are negatively correlated with almost the same computational effort as getting one of them. If we generate n such pairs $(Y_i, \tilde{Y}_i), i = 1, \ldots, n$, then $Var(\bar{Y} + \bar{\tilde{Y}})/2 < \sigma^2/2n$. That is, the variance of the estimator based on the antithetic pairs is less than that obtained from $2n$ independent random variables.

11.3 Importance Sampling

Importance sampling is based on an alternative formulation of the law of large numbers (LLNs). For notational convenience we assume $p(\theta|y) = f(\theta)$, where

1. f is the target density,
2. g is the candidate density.

The objective is to estimate $\mathbb{E}_f(h(\theta))$, such that

$$\begin{aligned} E_f(h(\theta)) &= \int_{\Theta} h(\theta) f(\theta) d\theta \\ &= \int_{\Theta} h(\theta) \frac{f(\theta)}{g(\theta)} g(\theta) d\theta \\ &= E_g \left[\frac{h(\theta) f(\theta)}{g(\theta)} \right]. \end{aligned}$$

So in '**Importance Sampling**' generate samples from a candidate distribution g. Suppose $(\theta^1, \theta^2, ..., \theta^N)$ are the random samples generated from g. Estimate

$$\bar{h} = \frac{1}{N} \sum_{i=1}^{N} \frac{f(\theta^i)}{g(\theta^i)} h(\theta^i)$$

and by virtue of LLN we have

$$\bar{h} = \frac{1}{N}\sum_{i=1}^{N} \frac{f(\theta^i)}{g(\theta^i)} h(\theta^i) \longrightarrow E_g\left[\frac{h(\theta)f(\theta)}{g(\theta)}\right] = E_f(h(\theta)).$$

The logic underlying the 'importance sampling' lies in a simple rearrangement of terms in the target integral and multiplying by 1:

$$\int h(\theta)f(\theta)d\theta = \int h(\theta)\frac{f(\theta)}{g(\theta)}g(\theta)d\theta = \int h(\theta)\omega(\theta)g(\theta)d\theta$$

here $g()$ is another density whose support is same as $f()$ and we know how to simulate from $g()$.

The $\omega()$ is called 'importance function.' A good importance function will be large when the integrand is large and small otherwise.

Objectives of Importance sampling

1. The IS reduces the variance of an integral approximation.
2. Another objective of IS when you cannot generate from the density f.
3. A third objective of IS is to draw inference or generate sample when the target density is unnormalized

Example 11.3.1 **Importance Sampling to improve integral approximation**
Consider the function $h(x) = 10\exp(-2|x-5|)$. Suppose we want to calculate $E(h(X))$, where $X \sim$ `Uniform(0,10)`. That is we want to calculate

$$\int_0^{10} \exp(-2|x-5|)dx.$$

The true value for this integral is about 1. The simple way to do this is to generate x_i from the `Uniform(0,10)` and look at the sample mean of $h(x_i)$. Notice this is equivalent to importance sampling with importance function $\omega(x) = f(x)$, where $f(x) = \frac{1}{10}I(0 < x < 10)$. Note that $h(x) = 10\exp(-2|x-5|)$ where $x \sim$ `Uniform(0,10)`. The plot of $h(x)$ is presented in Fig. 11.2.

```
set.seed(7831)
sim.size = 50000
X = runif(sim.size,0,10)
Y = 10*exp(-2*abs(X-5))
cat('Estimate = ',round(mean(Y),3))

## Estimate =  0.999 nn
```

```
cat('Monte Carlo Error = ',round(var(Y),3))

## Monte Carlo Error =  3.993 nn

x = seq(0,10,0.1)
h = 10*exp(-2*abs(x-5))
plot(x,h,ylab="h(x)",type="l",lwd=2)
```

Note that $h(x) = 10\exp(-2|x-5|)$ where $x \sim$ `uniform(0,10)`. The function $h()$ in this case is peaked at 5 and decays quickly elsewhere; therefore, under the uniform distribution, many of the points are contributing very little to this expectation. Something more like a Gaussian function $c\exp(-x^2)$ with a peak at 5 and small variance, say, 1, would provide greater precision. We can rewrite the integral as

$$\int_0^{10} 10\exp(-2|x-5|)\frac{1/10}{\frac{1}{\sqrt{2\pi}}\exp(-(x-5)^2/2)}\frac{1}{\sqrt{2\pi}}\exp(-(x-5)^2/2)dx.$$

We want to estimate $\mathbb{E}[h(X)\omega(X)]$ where $X \sim N(5,1)$. So in this case $f(x) \sim Uniform(0,10)$, $g(x) \sim N(5,1)$. The weight function is

$$\omega(x) = \frac{\sqrt{2\pi}\exp((x-5)^2/2)}{10}.$$

```
w = function(x)dunif(x,0,10)/dnorm(x,mean=5,sd=1)
h = function(x)10*exp(-2*abs(x-5))
set.seed(7831)
sim.size = 50000
x = rnorm(sim.size,mean=5,sd=1)
y = h(x)*w(x)
cat('Estimate = ',round(mean(y),3))

## Estimate =  0.999 nn

cat('Monte Carlo Error = ',round(var(y),3))

## Monte Carlo Error =  0.358 nn
```

The example shows that the choice of correct weight function is important to reduce the Monte Carlo error.

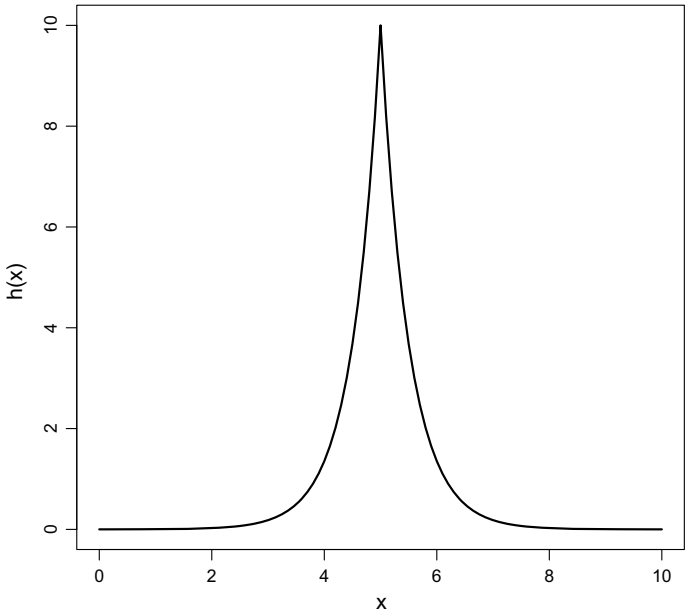


Fig. 11.2 Plot of the function $h(x)$ for Example 11.3.1

Example 11.3.2 Estimation of Rare Event Probability with the Importance Sampling
Suppose daily return of a stock is defined as

$$r_t = \frac{P_t - P_{t-1}}{P_{t-1}} \times 100$$

where P_t is the price of the stock on t^{th} day. It is believed that return of the stock follows $r_t \sim N(0, 1)$. **Objective**: We want to estimate $P(r_t < -5)$; i.e., we want to estimate the probability that the price of the stock will drop more than 5% in one day? As we are interested in $P(r_t < -5)$ where $r_t \sim N(0, 1)$, the probability in log scales, i.e., $\log(P(r_t < -5))$.

1. Simulating $Z^{(i)}$ from $N(0, 1)$ only produces a hit once in about 3 million iterations !
2. This is a very **rare event** for standard normal distribution
3. The following code estimates this log probability using Importance sampling

```
set.seed(5472)
a = -5
x = rnorm(10000,mean=a,sd=2)
y = rep(0,length(x))
y[x<a] = 1
#print(length(y))
g = dnorm(x,mean=a,sd=2)
f = dnorm(x)
w = f/g
print(log(mean(w*y)))

## [1] -15.03752
```

11.4 Stratified Sampling

Suppose the sample space consists of distinct subsets or strata. The idea of stratified sampling is to devise a sampling mechanism that constrains the fraction of observations drawn from each stratum. The main questions are

- How to construct the strata?
- How to choose the fractions?
- How to define the estimator so that the expectation remains same but variance is reduced.

We address these questions in this section and present two examples in finance in Sects. 11.4.2 and 11.4.3. To make this more precise, suppose Y is the variable of interest and the parameter that we are trying to estimate is EY. Let the support of Y be the set A and $A_1, \ldots A_k$ be a collection of disjoint exhaustive subsets. That is $\bigcup_{i=1}^{k} A_i = A$ and $A_i \bigcap A_j = \emptyset \forall i \neq j$. Then

$$\begin{aligned} EY &= \sum_{i=1}^{k} P(Y \in A_i) E(Y \mid Y \in A_i) \\ &= \sum_{i=1}^{k} p_i E(Y \mid Y \in A_i), \end{aligned}$$

where $p_i = P(Y \in A_i)$. In simple random sampling, the fraction of Y in A_i will not equal p_i. One way of choosing the fractions is to make them equal to the probabilities of the subsets and this is known as proportional allocation.

Thus the idea of proportional allocation is as follows. Suppose the sample size is n. Then for every i, from stratum A_i, we take a sample of size $n_i = np_i$, where $p_i = P(Y \in A_i)$. Let the sample drawn from A_i be denoted by $Y_{i1}, \ldots, Y_{in_i}$. The proposed estimator for EY is

$$\hat{Y} = \sum_{i=1}^{k} p_i \sum_{j=1}^{n_i} \frac{Y_{ij}}{n_i} = \frac{1}{n} \sum_{i=1}^{k} \sum_{j=1}^{n_i} Y_{ij}.$$

$\hat{Y}$ is unbiased for EY. In the following theorem we prove that stratification indeed reduces the variance.

Theorem 11 *The estimator $\hat{Y}$ obtained from a stratified sample with proportional allocation has lower variance than the sample mean $\bar{Y}$ obtained from simple random sample.*

Proof Let $E(Y \mid A_i) = \mu_i$, $EY = \mu$, $Var(Y \mid A_i) = \sigma_i^2$ and $Var(Y) = \sigma^2$. We know that $Var(\bar{Y}) = \sigma^2/n$.

$$Var(\hat{Y}) = \sum_{i=1}^{k} p_i^2 \frac{\sigma_i^2}{n_i} = \frac{1}{n} \sum_{i=1}^{k} p_i \sigma_i^2.$$

Note

$$\begin{aligned}
\sigma^2 &= Var(Y) \\
&= Var(E(Y \mid A_i)) + E(Var(Y \mid A_i)) \\
&= \left(\sum_{i=1}^{k} p_i \mu_i - \mu \right)^2 + \sum_{i=1}^{k} p_i \sigma_i^2 \\
&\geq \sum_{i=1}^{k} p_i \sigma_i^2.
\end{aligned}$$

Hence the result. □

Example 11.4.1 Let $Y = f(U)$ for some function f of uniform(0,1) random number U. In simple random sampling, $\bar{Y} = \frac{1}{n} \sum_{i=1}^{n} f(U_i)$. For proportional allocation with equal subsets, we take $A_i = [(i-1)/n, i/n]$, $i = 1, \ldots, n$ and $p_i = 1/n$. Let V_i to be uniform on A_i, which has the same distribution as the conditional distribution of $U \mid U \in A_i$. The stratified sample estimator is $\hat{Y} = \frac{1}{n} \sum_{i=1}^{n} E(f(V_i))$. Note that the histogram of V_i's is uniform with one observation in each cell of length $1/n$. This is different from the histogram of n iid U(0,1) random variables U_i's. Also, the V_i's are not independent. It is not necessary to take the intervals of equal size.

In the code below, we take Y to be exponential (2) random variables. By inversion method, these can be generated as functions of uniform random variables. With 1000 equal probability strata, in a simulation study below, we see that the mean of the sample means obtained by simple random sample and stratified sample are both close to 0.5 but the variance of the latter is much smaller.

```
bar = rep(0,100)
hat = bar
l = 2
set.seed(1010)
for(i in 1:100){
  U = runif(1000,0,1)
  Y = -log(1-U)/l
  bar[i] = mean(Y)
  V = seq(0,.999,.001)
  V = V+U*.001
  Y1 = -log(1-V)/l
  hat[i] = mean(Y1)
}
mean(bar)

## [1] 0.4979162

sd(bar)

## [1] 0.01487852

mean(hat)

## [1] 0.49998

sd(hat)

## [1] 0.0004921935
```

11.4.1 Optimal Allocation

The idea of allocation can now be generalized beyond proportional so that n_i are arbitrary nonnegative integers adding to n. In such case, the estimator is

$$\hat{Y} = \sum_{i=1}^{k} p_i \sum_{j=1}^{n_i} \frac{Y_{ij}}{n_i} = \frac{1}{n} \sum_{i=1}^{k} \frac{p_i}{q_i} \sum_{j=1}^{n_i} Y_{ij},$$

where $q_i = n_i/n$. As before this $\hat{Y}$ is unbiased for EY. Thus the job now is to find an optimal set of q_i's to minimize the variance of $\hat{Y}$. As in the proof of Theorem 11, let $E(Y \mid A_i) = \mu_i$, $EY = \mu$, $Var(Y \mid A_i) = \sigma_i^2$ and $Var(Y) = \sigma^2$.

$$Var(\hat{Y}) = \sum_{i=1}^{k} p_i^2 \frac{\sigma_i^2}{n_i} = \frac{1}{n} \sum_{i=1}^{k} \frac{p_i^2 \sigma_i^2}{q_i}.$$

So we have to minimize $\frac{1}{n}\sum_{i=1}^{k}\frac{p_i^2\sigma_i^2}{q_i}$ subject to $\sum_{i=1}^{k} q_i = 1$. This can be done using Lagrange multipliers and is left as an exercise. The resulting optimal proportions are given by

$$q_i^* = \frac{p_i\sigma_i}{\sum_{j=1}^{k} p_j\sigma_j}, \quad i = 1, \ldots, k.$$

The minimum variance is

$$\frac{1}{n}\sum_{i=1}^{k}\frac{p_i^2\sigma_i^2}{q_i^*} = \frac{1}{n}\left(\sum_{i=1}^{k} p_i\sigma_i\right)^2.$$

The practical problem with all these methods is that p_i and σ are not known, in general. In such cases we use approximate values of these parameters based on past studies or experience.

11.4.2 Stratifying a Poisson Process

An important variable in finance, apart from prices, is the arrival time of trades which is also related to the volume of trading in fixed intervals. This is particularly relevant when one is trying to model high-frequency data. The prices of securities are not observed continuously, but only when a trade takes place. It is, therefore, important to model this process of trading times and be able to generate observations from this model.

The simplest model for the arrival of trades is the homogeneous Poisson process (PP) with rate λ. It is a stochastic process $N(t)$ such that

1. $N(0) = 0$
2. The process has independent increments, that is, for $0 < s_1 < s_2 < t_1 < t_2$, $N(t_2) - N(t_1)$ is independent of $N(s_2) - N(s_1)$.
3. The distribution of the increment $N(t) - N(s)$ for any $0 < s < t$ is Poisson $(\lambda(t - s)$.

It is known that the waiting times, or interarrival times of a PP are independent exponential. So a straightforward way of generating observations from a PP would be to generate $X_1, \ldots$ as iid exponential λ. Then define the process $N(t)$ as

$$N(t) = \begin{cases} 0 & 0 < t < X_1 \\ 1 & X_1 \le t < X_1 + X_2 \\ \vdots & \end{cases}$$

Here we propose an alternative way using stratified sampling that will reduce the variance. First we restrict the total length of the interval to $[0, T]$. In practice we are

interested in performance of our models for finite time horizons, so this is not a very restrictive assumption. Then we use the following facts

1. The distribution of $N(T)$ is Poisson(λT). This follows directly from Condition 3 of the definition of Poisson process.
2. Given $N(T) = n$, the distribution of the n individual events in the interval $(0, T)$ is uniform. This can be proved from basic probability rules of conditional distributions.

First we generate $N(T)$ from the Poisson(λT) distribution. We can do this by the inverse transform method. Then we generate the arrival times $T_1, \ldots, T_n$ by stratifying based on the value n obtained for $N(T)$. Since these are uniform (0,T), we can generate n iid Unif(0,1) random variables, sort them and multiply them by T. The following are the R codes for functions to simulate from a Poisson process with and without stratification.

```
SPP = function(Tfinal){
  u = rep(0,100)
  k = rpois(1,10)
  u = sort(runif(k))
  return(u)
}
PP = function(Tfinal){
  X = rep(0,100)
  Y = X
  T = 0
  i = 1
  while(T<Tfinal){
    i=i+1
    X[i]=rexp(1,10)
    Y[i]=Y[i-1]+X[i]
    T=T+X[i]}
  Z<-(Y[1:i-1])
  return(Z)
}
```

A more realistic model for arrival of trades is an inhomogeneous PP with rate $\lambda(t)$ which is a non-constant function of t. In this case the first two conditions in the definition of PP remain same. The third is replaced by
The distribution of the increment $N(t) - N(s)$ for any $0 < s < t$ is Poisson with parameter $\int_s^t \lambda(u)du$. In this case, we have the following facts

1. The distribution of $N(T)$ is Poisson($\int_0^T \lambda(u)du$).
2. Given $N(T) = n$, the distribution of the n individual events in the interval $(0, T)$ is the same as that of order statistics of random variables with a density $f(t)$ proportional to $\lambda(t)$, that is

$$f(t) = \frac{\lambda(t)}{\int_0^T \lambda(u)du}.$$

In this case the inter-arrival times are not exponential any more. It is, therefore, not possible to use the straight forward method described above. However, the method of stratification can still be applied in this situation with a little modification. In the second step, instead of generating from uniform we need to generate from the density f.

11.4.3 Stratifying the Binomial Lattice

Another common application in finance is the Binomial lattice that was introduced in Chap. 6 and will be discussed in more detail in Chap. 9. The idea is to approximate the path of a Brownian motion by independent up and down steps over n equally spaced time intervals, where each up step happens with probability p. By changing n and p, we can approximate the evolution of a diffusion process in discrete space and discrete time. For valuing path dependent options, it is necessary to generate multiple realizations from the full path of the binomial lattice.

The direct method of generating a path through this lattice will go in n steps where at each step we decide to go up or down with probabilities p and $1-p$, respectively. The variance of this method can be reduced by stratification. For example, we can stratify based on the terminal value. We use the following two results from probability:

1. The distribution of the terminal value M of the number of up steps is Binomial(n, p).
2. Given M, each of the $\binom{n}{M}$ paths that have M up steps is equally likely.

So we can generate a value of M by inverse transform method from the Binomial(n, p) distribution. We then need to generate one random path from the set of k=$\binom{n}{M}$ paths. For this we can number all these paths from 1 to k. Then a single path can be chosen using a single uniform U and choosing the path numbered $\lceil kU \rceil$. Repeating this procedure multiple times will produce a sample of binomial paths.

11.5 Exercises

1. Prove Proposition 8.
2. Prove Proposition 9.
3. Show that the distribution of $[r - \sigma^2/2]\sum_{i=1}^n t_i + \frac{\sigma}{n}\sum_{i=1}^n W(t_i)$ is normal with mean $r - \frac{1}{2}\sigma^2 + \frac{1}{2}\bar{\sigma}^2$ and variance $\bar{\sigma}^2$ with $T = \frac{1}{n}\sum_{i=1}^n t_i$ and $\bar{\sigma}^2 = \frac{\sigma^2}{n^2 T}\sum_{i=1}^n (2i-1)t_{n+1-i}$
4. Show that the values of q_i that minimize $\frac{1}{n}\sum_{i=1}^k \frac{p_i^2\sigma_i^2}{q_i}$ subject to $\sum_{i=1}^k q_i = 1$ are given by

$$q_i^* = \frac{p_i \sigma_i}{\sum_{j=1}^{k} p_j \sigma_j}, \quad i = 1, \ldots, k.$$

5. Show that for a Poisson process $N(t)$, given $N(T) = n$, the distribution of the n individual events in the interval $(0, T)$ is uniform.

Part III
Statistical Methods

Chapter 12
Descriptive Statistics



So far we have studied mathematical models of market returns, volatility, interest rates, option prices, and other such financial quantities. These models involve unknown distributions and parameters which need to be calibrated to the prices observed in the market. We would also like to have a measure of reliability of the model and the parameters. Once the models are trained with data, we need to rate their performance. For all these purposes, statistical tools are necessary. We devote the rest of the book to these methods.

Statistics is the science of data. This involves collecting, summarizing, analyzing, and interpreting information. Statistical techniques are useful for describing and understanding variability. Variability arises because successive observations of a system or phenomenon do not produce exactly the same result. Statistics gives us a framework for describing this variability and for learning about potential sources of variability. Chapter 12 will be devoted to descriptive statistics which utilizes numerical and graphical methods to look for patterns in a dataset, to summarize the information revealed in a dataset and to present that information in a convenient form.

First we present some common concepts. An experimental unit is an individual object about which we collect data. A variable is a characteristic or property of an individual experimental unit. Data can be univariate, bivariate, or multivariate depending on how many variables we collect data on, for each experimental unit. Qualitative data is a measurement that cannot be recorded on a natural numerical scale, but is recorded in categories, like ratings of a company or sectors of an economy. Quantitative data is a measurement that is recorded on a naturally occurring numerical scale. These can be discrete like number of workers, units of sales or continuous like time to default, profit. The final concept is the method of data collection. This can

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/978-981-19-2008-0_12.

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

R. Sen and S. Das, *Computational Finance with R*, Indian Statistical Institute Series,
https://doi.org/10.1007/978-981-19-2008-0_12

be an observational study like a survey, a designed experiment to study the effect of different factors on experimental units, or a published source where already collected data is available.

12.1 Financial Data

We start with a sequence of prices $P_1, \ldots, P_n$ observed at time points $1, \ldots, n$ and are interested in calculating the return between two periods.

Definition 1 (simple return) The simple return R_t is defined as

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}}.$$

If the average return $R_t(k)$ need to be calculated over k periods, then the geometric mean is taken from the simple returns over the preceding k periods, i.e.,

$$R_t(k) = \left(\Pi_{j=0}^{k-1}(1 + R_{t-j})\right)^{1/k} - 1.$$

Definition 2 (log return) The log return r_t is defined as

$$r_t = \log \frac{P_t}{P_{t-1}} = \log(1 + R_t) = \log(P_t) - \log(P_{t-1}). \tag{12.1}$$

For the average log return over several periods we have

$$r_t(k) = \log(1 + R_t(k)) = \frac{1}{k} \sum_{j=0}^{k-1} r_{t-j},$$

i.e., for log returns the arithmetic average return is applied.

12.2 Graphical Representation of Qualitative Data

The most common representation of qualitative data is a barplot. The categories (classes) of the qualitative variable are represented by bars, while the height of each bar is the class frequency, which is the number of observations in that class. A related representation is a pie chart where the frequencies are proportional to the sizes of slices of a pie in a circle. These are illustrated in the following example.

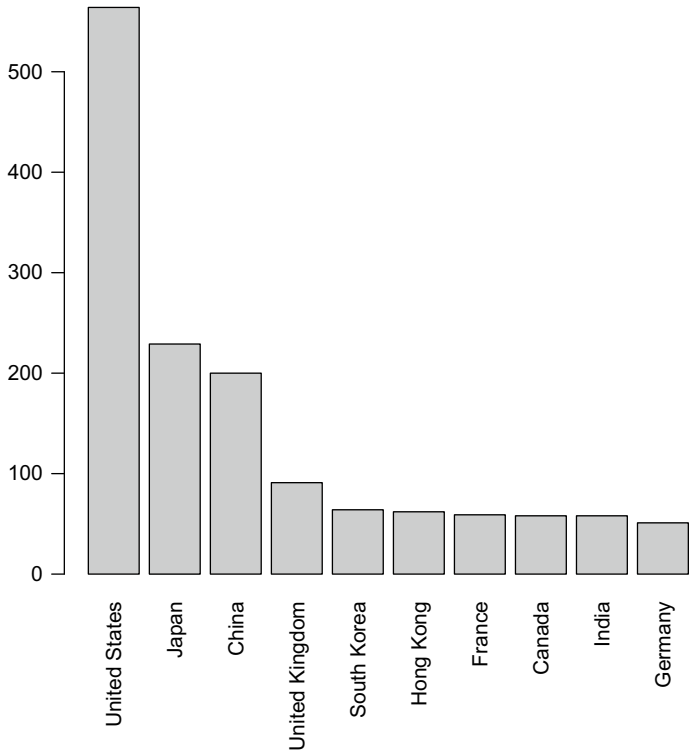


Fig. 12.1 Barplot of countries in Forbes data

Example 12.2.1 The Forbes Global 2000 is an annual ranking of the top 2,000 public companies in the world by Forbes magazine. The ranking is based on a mix of four metrics: sales, profit, assets, and market value. The list has been published since 2003. We consider the list of companies for the year 2017. We consider the 10 countries with highest representation. These account for 72% of the companies. We restrict to 10 countries because otherwise the plots are cumbersome and not very informative. Here country is a categorical or qualitative variable. We can represent the number of companies corresponding to each country by a bar graph or a pie chart as shown in Figs. 12.1 and 12.2. The R code to obtain these is given below.

```
x = Forbes2000
t = sort(table(x$Country), decreasing=TURE)
pie(t[1:10],las=2)
barplot(t[1:10],las=2)
```

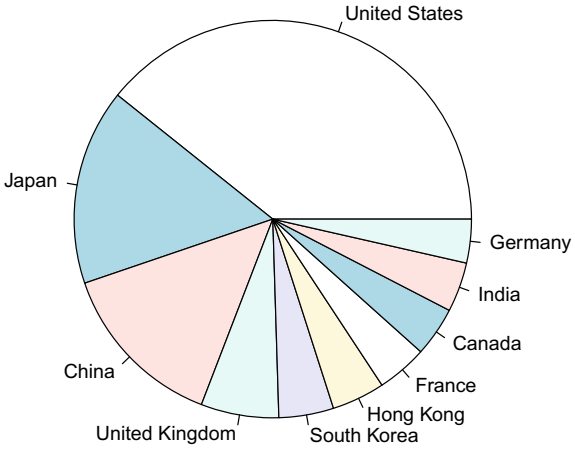


Fig. 12.2 Pie chart of countries Forbes data

12.3 Numerical Representation of Quantitative Data

For quantitative variables, it is possible to summarize the data through numerical measures. The most common numerical measure is the center, followed by the variability. Center can be measured by the arithmetic mean or average. For observations $x_1, \ldots, x_n$, the mean is denoted by $\bar{x}$ and equals the sum of the observed values divided by the number of values.

$$\bar{x} = \frac{1}{n} \sum_{i=1}^{n} x_i.$$

The mean is easily affected by few very large or very small values, called outliers. An alternative measure of center which is robust to outliers is the median. The median of a set of quantitative data is the value which is located in the middle of the data, arranged from lowest to highest values (or vice versa), with 50% of the observations above and 50% below.

The mean and median give us an idea of the central tendency, or where the 'middle' of the data is. Variability gives us an idea of how spread out the data are around that middle. We shall discuss the following measures of variability: range, variance, standard deviation, and interquartile range. Range is the difference between the largest and smallest observed values. It is easy to understand and compute, but not very informative as it considers only two observations, the smallest and the largest. The variance, s^2, of n measurements is equal to the average of the squared distances from the mean. For technical reasons, the denominator in the average is taken to be $n - 1$ instead of n.

$$s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2.$$

In order to have units of the measure of variability same as that of the original observations, it is more common to consider the standard deviation (SD) s, which is the square root of the variance. As in the case of mean, the standard deviation is also heavily affected by outliers. When this may be a concern, we use interquartile range as the measure of variability. The first quartile is defined as that value which is higher than 25% (a quarter) of the observations. Similarly, the third quartile is higher than 75% of the observations. The difference between these two is the interquartile range (IQR). Another common measure of variability is Mean Absolute Deviation (MAD) defined in Sect. 16.3.

In general, the quantity m_k defined below is known as the k-th central sample moment.

$$m_k = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^k.$$

The third moment is called the skewness and measures the symmetry in the data. The distribution has an asymmetrically long right (left) tail, and then it is said to be skewed to the right (left). For right(left)-skewed distributions, the mean is higher (lower) than the median.

The scaled fourth moment is called the kurtosis and measures the heaviness of tails of the distribution. Scaling involves dividing by square of the variance to make the quantity unit-free. A large kurtosis is associated with a high level of risk for an investment because it indicates that there are high probabilities of extremely large and extremely small returns. On the other hand, a small kurtosis signals a moderate level of risk because the probabilities of extreme returns are relatively low. The normal distribution has a kurtosis of 3.

The following R code shows the R commands for obtaining the numerical measures discussed above on the sales variable of the Forbes2000 data for the year 2004, as well as the corresponding results. The data is available in the R library `HSAUR`. Note that there is a high positive skewness, which can also be inferred from the observation that the mean is much higher than the median. The kurtosis is very high indicating a very heavy-tailed distribution.

```
data("Forbes2000",package="HSAUR")
s = Forbes2000$sales
summary(s)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   0.010   2.018   4.365   9.697   9.547 256.330
```

```
var(s)
```

```
## [1] 324.0933
```

```
sd(s)

## [1] 18.00259

#range function gives two values, max and min
range(s)[2]-range(s)[1]

## [1] 256.32

m=mean(s)
m3=sum((s-m)^3)/2000
# The skewness is positive and high
m4=sum((s-m)^4)/2000
m4/(var(s)^2)

## [1] 58.85951
```

12.4 Graphical Representation of Quantitative Data

For quantitative data the most common graphical representation is the histogram. The range of the data is divided into suitable classes that are plotted on the x-axis. The frequency of observations in each class is represented by drawing bars on these intervals. The area of each bar is proportional to the relative frequency of that class. In Fig. 12.3 we see the histogram of log(sales) of the Forbes data. For example, we can say from the figure that about 35% of the companies have log(sales) between 1 and 2. Translated to correct units, this is between exp(1) and exp(2), that is 2.72 and 7.39 billion dollars.

From a histogram we can say several things about the shape of the distribution. Most often, distributions have a single peak, called the mode. But sometimes there may be multiple modes and this is apparent from the histogram. These might indicate that the data is not homogeneous and arises from a mixture of several populations. Another aspect that is apparent from histograms is symmetry. If the distribution is asymmetric, it may have a few unusually large (small) measurements and be skewed to the right (left). Financial data like sales and income is mostly skewed to the right. In the preceding histogram the sales are extremely skewed. Transforming them to the log scale has made the histogram almost symmetric. It is easier to model symmetric distributions. That is one of the reasons we deal with log returns of stock prices.

Most financial variables like price, income, and profit are continuous. The histogram is more appropriate for discrete distributions as the probability within each class is constant and changes discontinuously from one class to another. Kernel density estimators are smooth and avoid this problem. Given n observations $X_1, \ldots, X_n$, a kernel density estimator is the function $\hat{f}_b$ defined by:

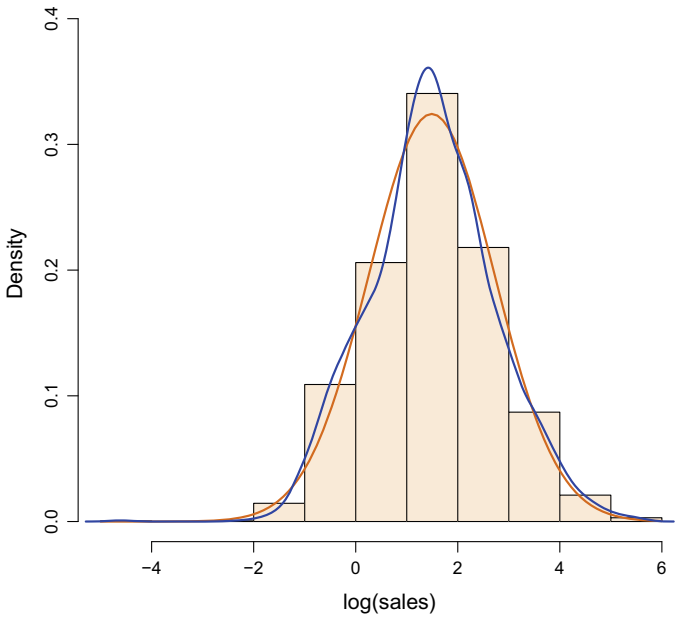


Fig. 12.3 Histogram of log(sales) in Forbes data superimposed with normal density (chocolate) and kernel density (blue)

$$\hat{f}_b(x) = \frac{1}{nb} \sum_{i=1}^{n} K\left(\frac{x - X_i}{b}\right), \tag{12.2}$$

where K is a fixed non-negative function which integrates to one (i.e., a probability density function) which we call the kernel, and $b > 0$ is a positive number which we call the bandwidth. The interpretation of formula (12.2) is as follows. Over each point X_i, we center a scaled copy of the kernel function K, and the final density estimate is the superposition of all these 'bumps.' The division by nb guarantees that the total mass is one, i.e., the integral of $\hat{f}_b(x)$ is one.

The following code plots the histogram and the kernel density of the log of the sales variable in the Forbes2000 data. These are superimposed with the normal density with mean and variance estimated from the data.

```
hist(log(Forbes2000$sales),freq=FALSE
     ,main = '',xlab='log(sales)'
     ,ylim = c(0,0.4),col='antiquewhite')
m = mean(log(Forbes2000$sales))
s = sd(log(Forbes2000$sales))
curve(dnorm(x,m,s),add=TRUE,col="chocolate",lwd=2)
lines(density(log(Forbes2000$sales)),lwd=2,col='blue')
```

12.5 Assessing Normality

The normal distribution is a very common and convenient model for log returns. Since log returns over a long time scale are the sum of many log returns over a smaller time scale, by the central limit theorem (in Chap. 13), the former is expected to have a normal distribution. This is exactly true under the assumption that the individual log returns are independent and identically distributed. The assumption can be somewhat relaxed. Many financial models, starting from the Black–Scholes, lead to normality of log returns.

In order to apply such a model, one first needs to assess whether for the particular dataset under consideration, the log returns are approximately normal or not. There are various ways of doing this. If the data is normal, we expect the following.

- The histogram looks like the normal curve.
- The mean $\pm$ s, 2s, and 3s approximate the empirical rule percentages. That is 68% of the points lie in the interval $\bar{x} \pm s$, 95% in the interval $\bar{x} \pm 2s$, and 99.7% in the interval $\bar{x} \pm 3s$.
- The ratio of the interquartile range to the standard deviation is about 1.3.
- The skewness is close to 0 and kurtosis is close to 3.
- A normal probability plot, a plot with the ranked data on one axis and the expected z-scores from a standard normal distribution on the other axis, is close to a straight line. The expected z-score for the i-th data point in a sample of size n is $\Phi^{-1}(i/n)$ where Φ is the cumulative distribution function (cdf) of the standard normal distribution.

Example 12.5.1 The Glassdoor Economic Research Blog has published the data on employee compensation for several companies for 2014. The data includes CEO identities, companies, CEO compensation, median worker compensation (compiled by Glassdoor), and the ratio of CEO to worker compensation. The data can be downloaded from here: https://dasl.datadescription.com/download/data/3105 . We look at the ratio of the median worker compensation to the CEO compensation for each company. This is our variable of interest, and we try to assess if the distribution is normal using the R code below. The normal probability plot is presented in Fig. 12.4. Based on the results we conclude that it is reasonable to assume a normal distribution for this variable.

```
attach(ceo.compensation.2014)
x = log(Ratio)
library(timeDate)
skewness(x)

# 0.08324829

kurtosis(x,method="moment")

# 3.873771

IQR(x)/sd(x)

# 1.201651

length(which(x<mean(x)+sd(x) & x>mean(x)-sd(x)))/length(x)

# 0.7211982

length(which(x<mean(x)+2*sd(x) & x>mean(x)-2*sd(x)))/length(x)

# 0.9562212

length(which(x<mean(x)+3*sd(x) & x>mean(x)-3*sd(x)))/length(x)

# 0.9907834

qqnorm(x)
abline(mean(x),sd(x))
```

12.6 Measures of Relative Position

So far we have dealt with measures that quantify all observations. Now we turn to the question of the position of an individual observation relative to the other values.

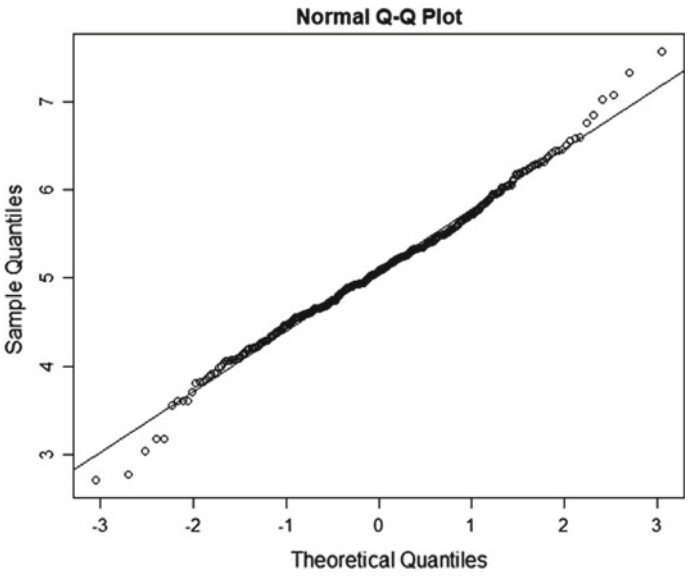


Fig. 12.4 Normal probability plot of the CEO compensation data

For any set of n measurements, the k-th percentile is a number such that $k\%$ of the measurements fall below that number and (100 - k)% fall above it. Here $0 < k < 100$. A modification of this is the quantile. The p-th quantile is the $100p$-th percentile, for $0 < p < 1$.

Another measure of relative position is the z-score. This tells us how many standard deviations above or below the mean a particular measurement is.

$$z = \frac{x - \bar{x}}{s}.$$

A related concept is that of order statistics. When the observations $X_1, \ldots, X_n$ are arranged in increasing order of magnitude, the k-th order statistic is the k-th value. This is denoted by $X_{(k):n}$. Thus, the order statistics form a permutation of the observations such that $X_{(1):n} \leq \cdots \leq X_{(n):n}$.

An outlier is a measurement that is unusually large or small relative to the other values. This can happen due to the following reasons.

1. Observation, recording, or data entry error.
2. The particular observation is from a different population.
3. A rare, chance event.

We designate a value as outlier if any of the following happens.

1. The corresponding z-score is more than 3 or less than -3.
2. It is less than the first quartile minus 1.5 times the IQR, or greater than the third quartile plus 1.5 times the IQR.



Fig. 12.5 Prices of Bosch

12.7 Stylized Facts

There are some properties of stocks which are common across markets, irrespective of the nature of the stock. They have been identified after observing that a large volume of data, across markets and time periods, follow these properties. These properties are known as the stylized facts. The paper (Cont, 2001), on the empirical properties of asset returns, lists such stylized facts. The current literature on the presence and identification of these facts is reviewed in (Thompson, 2013). We discuss some stylized facts here. Others will be introduced in the subsequent chapters. A detailed version of what follows is available in (Sen and Subramaniam, 2019).

For demonstration we use daily data for eleven years on fifty individual stocks traded in the National Stock Exchange (NSE). The historical data is downloaded from yahoo finance. The stocks are the constituents of the NIFTY index. For each stock the data contains seven attributes for each trading day, namely the date, closing price, opening price, the highest price, lowest price, volume traded, and the change percentage on that day. These were taken over the time range of January 2007 to November 2017. The plot of prices of the Bosch Ltd(BOSH) stock is shown in Fig. 12.5.

Since the prices are often non-stationary, it is more common to use log returns for statistical analysis. The log return of a stock has been defined in Equation (12.1). For the rest of the paper we deal with the log return and not the prices. For illustrative purpose, the log returns for the Bharat Petroleum Corporation Limited (BPCL) stock

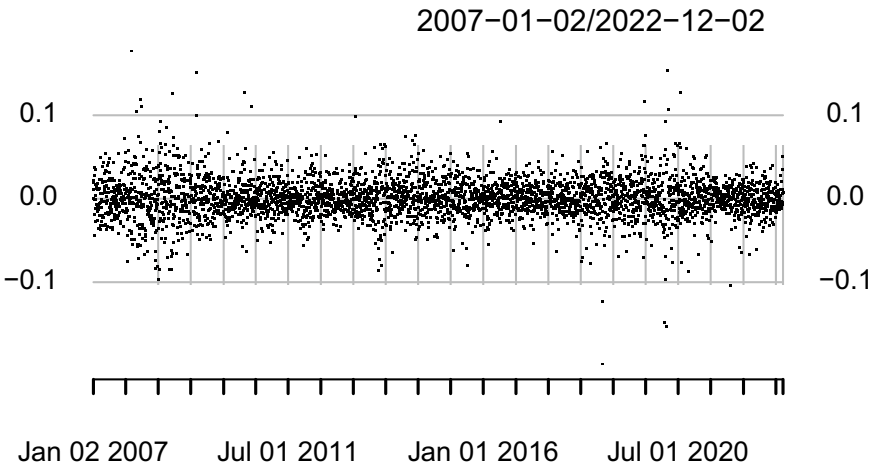


Fig. 12.6 Log returns of BPCL

are shown in Fig. 12.6. A visual comparison with Fig. 12.5 shows that the concern of non-stationarity has been addressed.

12.7.1 Gain-Loss Asymmetry

A common stylized fact is the gain-loss asymmetry. One observes large drawdowns in stock prices and stock index values but not equally large upward movements. Recall that we discussed in Sect. 12.3 how skewness measures the asymmetry of the probability distribution of a random variable. Large drawdowns compared to upward movements would correspond to a long left tail and hence, a negatively skewed curve.

For the dataset under consideration, skewness of the returns was calculated for each stock. The results are shown in Fig. 12.7. It is observed that most stocks have positive skewness and thus show larger upward movements than drawdowns. This is in contrast with the stylized facts reported in (Cont, 2001). In (Huang and Zhang, 2014), the authors compare asymmetry indices of historical prices from ten stock markets using market index data. They find that in most stock markets, price fall is faster than price rise, while in China and India, price rise is generally faster than price fall. Our results reconfirm this for the Indian market at the stock level.

12.7.2 Aggregational Gaussianity

The next stylized fact is aggregated Gaussianity, which is the following phenomenon. As one increases the time scale Δt over which returns are calculated, their distribu-

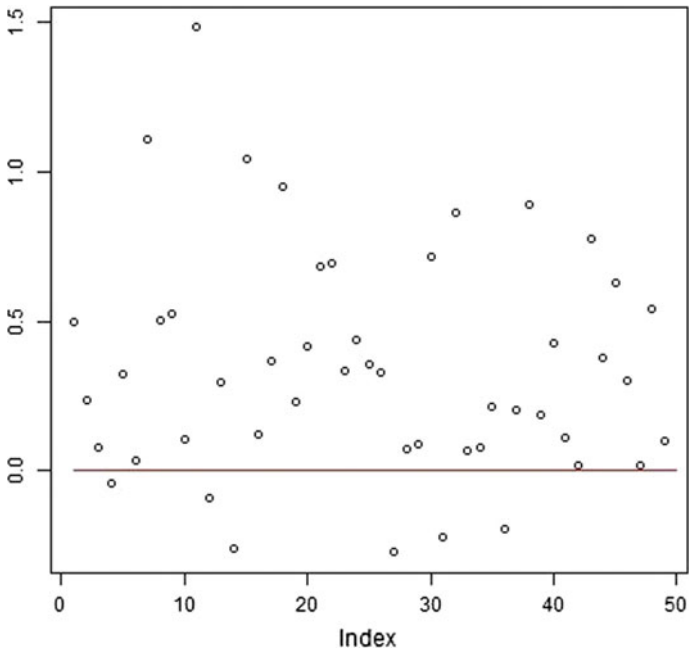


Fig. 12.7 A plot of the skewness values

tion looks more and more like a normal distribution. In particular, the shape of the distribution is not the same at different time scales.

Two normality tests in R are performed on the data, namely the Kolmogorov–Smirnov (KS) test and the Shapiro–Wilk (SW) test for daily, weekly, monthly, and quarterly returns. The p values increase as the time over which the returns are calculated increases. For background on hypothesis tests and p values see Sect. 13.3. It is observed that as the time scale increases, the p values are less and less concentrated around zero and the normality assumption is rejected for fewer stocks. Thus, the distribution of returns of many stocks becomes similar to the normal distribution as the time over which returns are calculated is lengthened. We conclude that aggregated Gaussianity is present in the data.

12.7.3 Heavy Tails

The stylized fact regarding tails states that the distribution of returns seems to display a power law or Pareto-like tail, with a tail index which is finite, higher than two and less than five.

A power law is a functional relationship between two variables, where the relative change in one variable is proportional to the relative change in the other. It is of the form

$$Y = kX^{\alpha}$$

where X and Y are variables of interest, α is the law's exponent and k is a constant. Pareto Law is a special power law, also known as distributional power law, where Y is the probability involving a random variable. For example,

$$P(S > x) = \frac{k}{x^{\zeta}}$$

is a Pareto Law. The exponent ζ is independent of the units in which the law is expressed.

Distributions of random variables are studied in comparison with the exponential distribution. The tail of a distribution is the part of the distribution where $|X|$ tends to infinity. The thickness of the tail is the tail index. Distributions can be classified as being heavy tailed or light tailed. A heavy-tailed distribution has a tail that is not bounded by the exponential tail, whereas the light-tailed distribution has a tail that falls below the exponential tail. Here we look only at the tail and not the part of the distribution before where the tail begins. The choice of the point in the distribution where the tail begins is also important in determining tail index. Consider any distribution $P(X)$ with cumulative distribution function $F(x) = 1 - \overline{F}(x)$ defined by $Pr(X > x) = \overline{F}(x)$, such that for some $\xi > 0$,

$$\overline{F}(x) = x^{\frac{-1}{\xi}} L(x)$$

where $L(x)$ is some slowly varying function for large x. The tail index of the fat-tailed distribution $P(X)$ is by definition ξ.

Using the `hill.adapt()` function in the `extremefit` library of `R`, the tail index is calculated. The values for all 50 stocks are shown in Fig. 12.8. It is observed that the returns indeed have heavy tails and the index is found to lie between two and five.

```
#The log returns of a stock are stored in the variable returns.
skewness(return) #gives the skewness
ks.test(return, pnorm, mean(return), sd(return) )
#conducts the Kolmogorov-Smirnov test of normality
# The following part is for testing heavy tailed-ness
t=rep(1,100)
for(i in 1:100) {t[i]=hillest(return, k=i)}
plot(t)
```

12.8 Exercises

1. For the CEO compensation data of Example 12.5.1 plot the histogram superimposed with the kernel density and normal density with parameters as the sample mean and variance.

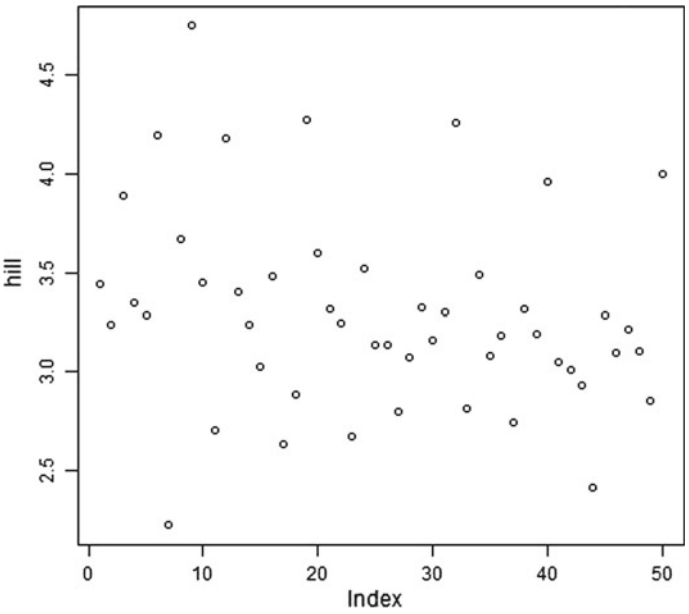


Fig. 12.8 Tail index of returns

2. For the data in the previous exercise test for normality and heavy tails as in Sect. 12.7.
3. Download the stock prices of any stock for one year from yahoo finance. Compute and plot the returns and log returns.
4. For the data in the previous example, find the mean, median, quartiles, variance, range, skewness, and kurtosis of the log returns.

Chapter 13
Inferential Statistics



The setup of inferential statistics is as follows. There is a population which is a set of all individuals or objects that we are interested in, but is too large to study in its entirety. Instead we obtain a random sample which is a subset of the population and use the information available in the subset to generalize to the population. Usually we specify a model for the probability distribution for the population. This is a probability density function (pdf) for continuous or probability mass function (pmf) for discrete distributions. Although the form of the distribution can be specified depending on the background information on the population, certain numerical characters may be unknown. These unknown but fixed numerical characteristics are associated with the model and are called parameters. A sample statistic is a numerical measure of a sample that can be calculated from the observations. The sample statistics are used to draw inference about population parameters.

Since we could draw many different samples from a population, the sample statistic used to estimate the population parameter is itself a random variable. The **sampling distribution** of a sample statistic calculated from a sample of n measurements is the probability distribution of the statistic. In repeated sampling, they tell us what values of the statistics can occur and how often each value occurs.

A typical analysis will go as follows. The sample consists of the random variables $X_1, \ldots, X_n$, which have joint distribution $f(x_1, \ldots, x_n|\theta)$. This distribution is known except for the parameter θ. Given realizations $x_1, \ldots, x_n$ of $X_1, \ldots, X_n$, the goal is to infer the unknown parameter θ. For example, the model could be binomial and the parameter could be the success probability p.

The set of all possible values of $\mathbf{x} = (x_1, \ldots, x_n)$ is called the sample space and is denoted by Ω.

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/978-981-19-2008-0_13.

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

R. Sen and S. Das, *Computational Finance with R*, Indian Statistical Institute Series,
https://doi.org/10.1007/978-981-19-2008-0_13

Example 13.0.1 Suppose we are interested in the number of customers who visit a store per day. This number is random and will change from day to day. A simple model for this would be a Poisson distribution with no dependence. If we take a sample of the number of customers $X_1, \ldots, X_n$ on n days, then the pmf for each X_i is given by $P(X_1 = k) = \exp(-\lambda)\lambda^k/k!, k = 0, 1, \ldots$ where λ is the unknown parameter.

The following theorems are useful in deriving properties of sample statistics and will be used later in the chapter. The first result, strong law of large numbers, tells us that if we take a large enough sample, then the sample mean will be close to the population mean. This is illustrated with the following R code and Fig. 13.1 where we generate sequences of size 1000 from a Bernoulli(1/2) distribution and plot the running mean. We repeat this 10 times and see that all the sequences converge toward the true mean of 1/2. The second result is the central limit theorem. This tells us that irrespective of the population distribution, for a large enough sample of independent elements, the sample mean has approximately a normal distribution.

Strong Law of Large Numbers Suppose $X_1, \ldots, X_n$ are independent and identically distributed random variables with mean μ and finite variance. Then $\mathrm{P}(\bar{X}_n \to \mu \quad \text{as} \quad n \to \infty) = 1$.

The law is illustrated by the R code below along with Fig. 13.1.

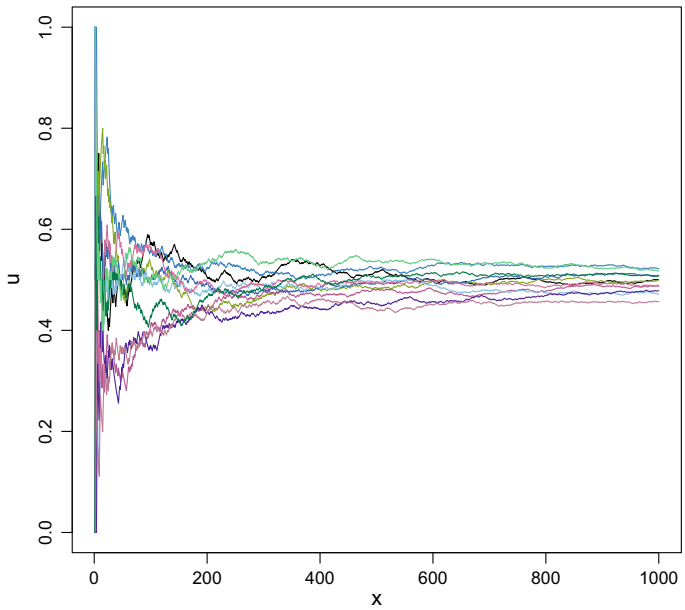


Fig. 13.1 Running average of x Bernoulli random variables

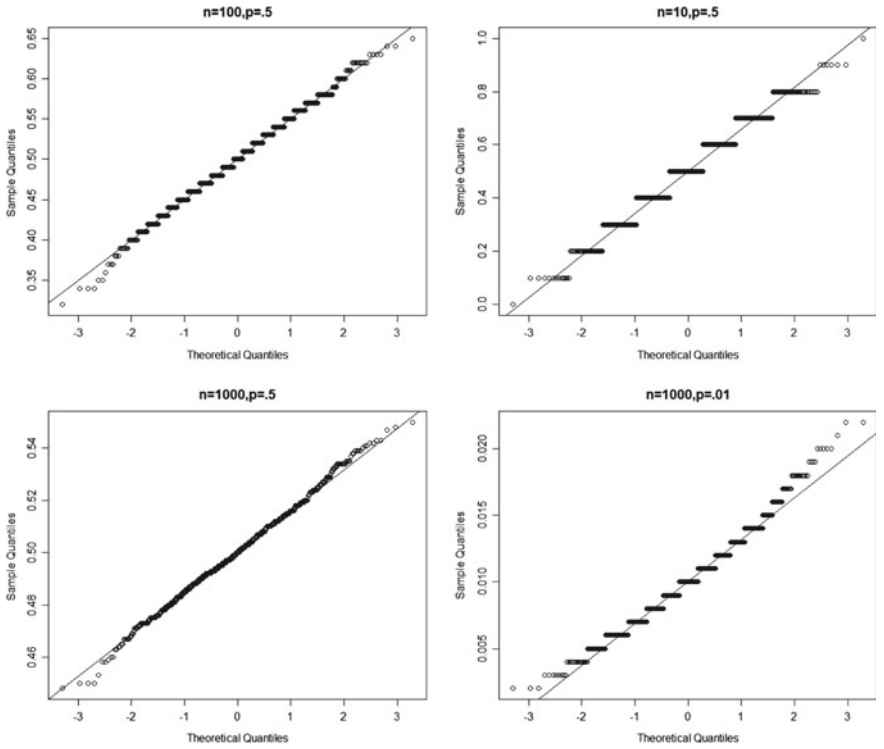


Fig. 13.2 Normal probability plot for mean of 1000 realizations from binomial distribution with different parameters

```
runningmean = function (x,N){
 y = sample(x,N, replace=TRUE)
 c = cumsum(y)
 n = 1:N
 c/n
}
set.seed(10101)
u = runningmean(c(0,1), 1000)
x=1:1000; plot(u~x, type="l",ylim=c(0,1));
replicate(10, lines(runningmean(c(0,1), 1000)~x,
type="l", col=rgb(runif(3),runif(3),runif(3))))
```

Central Limit Theorem Suppose $X_1, \ldots, X_n$ are independent and identically distributed random variables with mean μ and variance $\sigma^2 < \infty$. Then as $n \to \infty$, the limiting distribution of $\sqrt{n}(\bar{X}_n - \mu)/\sigma$ is standard normal.

The central limit theorem also implies that the sum of n observations is approximately normal with mean $n\mu$ and standard deviation $\sqrt{n}\sigma$. Many statistics that are

used for statistical inference are sums or averages of sample observations. This will allow us to describe their behavior and evaluate the reliability of our inferences using the normal distribution, even if we do not know the original distribution. The quality of the approximation depends on the distribution of the original random variables and the sample size. In the following R code, we generate 1000 random binomial observations and check how close the distribution is to a normal with QQ-plots. We present the results in Fig. 13.2 for several choices of n and p, the parameters of the binomial distribution. There are three plots with $p = 0.5$ and values of $n = 10, 100, 1000$. Note that for this value of p, higher sample size brings the points closer to the straight line, indicating a better quality of approximation. The last plot is with $n = 1000$, but $p = 0.01$. The original distribution is asymmetric, and the distribution of the mean also remains asymmetric even for a large sample size.

```
x = rbinom(1000,n,p)/n; qqnorm(x); abline(p,sqrt(p*(1-p)/n))
```

13.1 Estimation

A point estimator ($\hat{\theta}$) of a population parameter (θ) is a rule or formula that tells us how to use the sample data to calculate a single number that can be used as a value for the population parameter.

13.1.1 Methods of Estimation

One common and fairly simple method of finding an estimator is the method of moments (MoM). The idea is to compute the sample moments, equate them to the population moments, and then solve the resulting equations to obtain the estimators of the parameters of interest. It can be seen directly that for a population with mean μ, the sample mean $\bar{X}$ is a MoM estimator.

Example 13.1.1 Let $X_n, \ldots, X_n$ be a random sample from a population with mean μ and variance σ^2 and the parameter of interest is σ^2. The method of moments estimation procedure will go as follows:

- Equate μ to $\bar{X}$.
- Note that the population second moment is $E(X_i^2)=\mu^2 + \sigma^2$. Equate this $\bar{X^2}$.
- Solve the above to equations to eliminate μ and obtain the estimator $\hat{\sigma^2}_{MoM}$ of σ^2 as $\bar{X^2} - \bar{X}^2$.

Note that MoM estimators may not be unique. If there exist multiple choices for $\hat{\theta}_{MOM}$, one usually takes the estimate involving the lowest-order sample moment.

Example 13.1.2 (Different MoM estimators) $X_1, \ldots, X_n$ iid Poisson(λ). The first moment is λ. Thus, the method of moments estimator based on the first moment is $\bar{X}$. We could also consider using the second moment to form a method of moments estimator. The method of moments estimator based on the second moment solves $\bar{X^2} = \lambda + \lambda^2$ Solving this equation (by taking the positive root), we find that $\hat{\lambda} = -1/2 + (1/4 + \bar{X^2})^{1/2}$.

Definition 3 Suppose we are given a function $\Psi : \Omega \times \mathbb{R}^d \to \mathbb{R}^d$ and define

$$V(\theta_0, \theta) = E_{\theta_0}\Psi(X, \theta)$$

Suppose $V(\theta_0, \theta) = 0$ has θ_0 as its unique solution for all $\theta_0 \in \Theta$. Then we say $\hat{\theta}$ solving

$$\Psi(X, \hat{\theta}) = 0$$

is an estimating equation estimate.

Take $\Psi = \hat{\mu}_1 - \mu$ with $d = 1$ to get the method of moments estimator.

Definition 4 Consider a parameter that can be written as a function of F, i.e., $\theta = T(F)$. The plug-in estimator of θ is $T(\hat{F}_n)$ where F_n is the empirical cdf, that is $F_n(x) = \frac{1}{n}\sharp\{X_i \leq x\}$.

Plug-in estimators are not generally optimal. But they are good starting points for numerical algorithms. Sample median is a plug-in estimator of population median $\theta = F^{-1}(1/2)$, but not a MoM estimator.

Definition 5 Let $(X_1, \ldots, X_n)$ be a random vector with pdf (or pmf) $f(x_1, \ldots, x_n; \theta), \theta \in \Theta$. We call the function $L(\theta; x_1, \ldots, x_n) = f(x_1, \ldots, x_n; \theta)$ of θ the likelihood function.

The following code plots the likelihood function of the binomial parameter p when $n = 4$ for three different realizations. The output is given in Fig. 13.3.

```
f1<-function(p) p^4
f2<-function(p) (1-p)^4
f3<-function(p) 6*p^2*(1-p)^2
curve(f1,xlim=c(0,1),xlab="p",ylab="L(p)",lwd=2)
curve(f2,add=TRUE,col="blue",lwd=2)
curve(f3,add=TRUE,col="red",lwd=2)
```

Definition 6 A maximum likelihood estimate (MLE) is an estimate $\hat{\theta}_{ML}$ such that

$$L(\hat{\theta}_{ML}; x_1, \ldots, x_n) = sup_{\theta \in \Theta} L(\theta; x_1, \ldots, x_n).$$

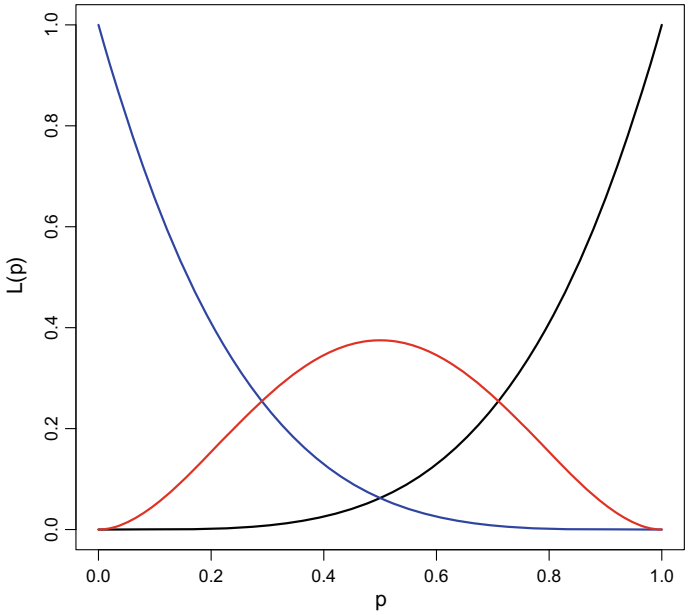


Fig. 13.3 Likelihood function for realizations 0 in black, 2 in red and 4 in blue from binomial(4,p)

The MLE is that point in the parameter space for which the observed sample is most likely. It is often convenient to work with $l = \log L$ when determining the maximum likelihood estimate. Since log is a monotone function, the maximum is attained at the same point. For finding the MLE, we compute the derivative of the log-likelihood function. This is known as the score function and we equate it to zero, giving us the score equation. We then use the double derivative to confirm that it is a local maximum. Finally we check the boundary and confirm the global maximum. When the dimension of θ is greater than one, second derivative test for maxima entails checking that the Hessian matrix (matrix of second derivatives) is negative definite. One check for this is that the sign of the determinant of any principal minor is $(-1)^r$, where r is the order.

If the score equation cannot be solved analytically, numerical methods become necessary. We have discussed some such methods for solving nonlinear equations in Chap. 3. Here we discuss the method of coordinate descent which is useful when the dimension of the parameter space is greater than one. The coordinate descent method works by using the bisection method iteratively. Suppose we have a k-dimensional parameter $(\theta_1, \ldots, \theta_k)$. The coordinate descent method goes as follows.

Coordinate Descent Algorithm Choose an initial estimate $(\hat{\theta}_1, \ldots, \hat{\theta}_k)$.

1. Set $(\hat{\theta}_1, \ldots, \hat{\theta}_k)_{old}$=$(\hat{\theta}_1, \ldots, \hat{\theta}_k)$
2. Solve the score equation $\frac{\partial}{\partial \theta_1}(\theta_1, \hat{\theta}_2, \ldots, \hat{\theta}_k) = 0$ for θ_1 using the bisection method. Reset θ_1 to the value $\hat{\theta}_1$ that solves this equation.
3. Solve the score equation $\frac{\partial}{\partial \theta_2}(\hat{\theta}_1, \theta_2, \ldots, \hat{\theta}_k) = 0$ for θ_2 using the bisection method. Reset θ_2 to the value $\hat{\theta}_2$ that solves this equation.
4. Continue to θ_k
5. Stop if the distance between $(\hat{\theta}_1, \ldots, \hat{\theta}_k)_{old}$ and $(\hat{\theta}_1, \ldots, \hat{\theta}_k)$ is less than some tolerance ϵ. Otherwise return to step 1.

The coordinate descent method converges to the maximum likelihood estimate when the log-likelihood function is strictly concave on the parameter space.

Next we present an example where the coordinate descent method is used. The following code generates a sample of size `20` from the `gamma(2,3)` distribution and uses the method of coordinate descent to find the MLE of the parameters. The log-likelihood is strictly concave, so the method converges. We found the method of moments estimates and use them as initial estimates.

$$\hat{a} = \bar{x}^2/\overline{x^2} \quad \text{and} \quad \hat{b} = \bar{x}/\overline{x^2}$$

for $\bar{x}^2$, $\overline{x^2}$ and $\bar{x}$ In this case, the computed MLE is (`2.46, 3.47`).

```
#R code for finding the MLE:
# Code for gamma distribution MLE
# x stores the data
# a1, b1 store current estimates of alpha and beta
# Generate data from Gamma(alpha=2,beta=3) distribution)
set.seed(10101)
x=rgamma(20,2,3)
# Set low and high starting values for the searches
aL=.001;
aH=20;
bL=.001;
bH=20;
# Use method of moments for starting values
a1=mean(x)^2/(mean(x^2));
b1=mean(x)/(mean(x^2));
a1

## [1] 0.6215671

b1

## [1] 0.8770733

A=a1;
B=b1;
# Define the derivative of the log likelihood
deriva=function(a,b,x){
  n=length(x);
  n *log(b)+sum(log(x))-n*digamma(a);
}
derivb=function(a,b,x){
```

```
  n=length(x);
  n*a/b-sum(x);
}
dist=1;
cc=1;
toler=.0001;
while(dist>toler){
  a2=uniroot(deriva,c(aL,aH),b=b1,x)$root;
  b2=uniroot(derivb,c(bL,bH),a=a2,x)$root;
  dist=sqrt((a2-a1)^2+(b2-b1)^2);
  a1=a2;
  b1=b2;
  A=c(A,a1);
  B=c(B,b1);
  cc=cc+1
}
a1

## [1] 2.455892

b1

## [1] 3.46543

cc

## [1] 45
```

Finally we discuss Bayesian method of estimation. So far, in what is known as the frequentist approach, we have taken θ to be a fixed unknown constant. In the Bayesian approach, we put a prior probability distribution on θ, say $p(\theta)$. The model is then the conditional distribution of the data given a value of θ. The joint distribution is therefore the product of the prior and the model. We use Bayes rule to obtain the conditional distribution of θ given the data. This is called the posterior distribution and is given below:

$$p(\theta|X_1,\ldots,X_n) = \frac{p(\theta)f(X_1,\ldots,X_n|\theta)}{\int_{\eta\in\Theta} p(\eta)f(X_1,\ldots,X_n|\eta)d\eta}.$$

The Bayes estimator is the conditional expectation of θ given the data, that is, the expectation of the posterior distribution and is given by:

$$E(\theta|X_1,\ldots,X_n) = \int_{\eta\in\Theta} \eta p(\eta|X_1,\ldots,X_n)d\eta.$$

Example 13.1.3 Suppose that we use a device, which is known to be quite precise in terms of the random errors, but may have a significant constant error, which we would like to estimate (and compensate from the device measurements in the future). More precisely, we perform n measurements and obtain an i.i.d. sample from N(θ, 1), where $\theta \in \mathbb{R}$ is the unknown parameter. Suppose we believe that the constant error term is symmetric around zero and is itself a random variable sampled from N(0, σ^2), where σ^2 (called hyperparameter), is known to us and it expresses our belief regarding the dispersion of the error.

$$X_i \overset{iid}{\sim} \mathcal{N}(\theta, 1), \quad \theta \sim \mathcal{N}(0, \sigma^2).$$

13.1.2 Properties of Estimators

The **bias** of an estimator $\hat{\theta}$ of a parameter θ is E($\hat{\theta}$)-θ where the expectation is taken under the sampling distribution of $\hat{\theta}$. An estimator is unbiased if the bias is zero. The **variance** of an estimator $\hat{\theta}$ of a parameter θ is E$(\hat{\theta} - E\hat{\theta})^2$

Example 13.1.4 Let $X_n, \ldots, X_n$ be a random sample from a population with mean μ and variance σ^2.

$$E(\bar{X}) = E\left(\frac{1}{n}\sum_{i=1}^{n} X_i\right) = \frac{1}{n}\sum_{i=1}^{n} E(X_i) = \frac{1}{n}\sum_{i=1}^{n} \mu = \mu.$$

Thus $\bar{X}$ is unbiased for μ.
Let $Y_i = X_i - \mu$. Then $\bar{Y} = \bar{X} - \mu$, E(Y_i)=0 and E(Y_i^2)=σ^2.

$$\begin{aligned} E(\bar{X} - E\bar{X})^2 &= E(\bar{X} - \mu)^2 \\ &= E(\bar{Y}^2) \\ &= \frac{1}{n^2} E\left(\sum_{i=1}^{n} Y_i\right)^2 \\ &= \frac{1}{n^2}\left[\sum_{i=1}^{n} EY_i^2 + \sum_{i \neq j} E(Y_i Y_j)\right] \\ &= \frac{1}{n^2} n\sigma^2 = \frac{\sigma^2}{n}. \end{aligned}$$

Thus the variance of $\bar{X}$ is σ^2/n.

Among multiple unbiased estimators, the one with the smallest variance is preferred. The **Mean Squared Error (MSE)** of an estimator W of a parameter θ is the function of θ defined by $E_\theta(W-\theta)^2$. The advantage of MSE is easy tractability and the interpretation MSE = Variance+Bias^2. For an unbiased estimator MSE = variance. But a biased estimator might have lower MSE and will be preferred in most cases.

Example 13.1.5 Let $X_1, \ldots X_n$ be iid Bernoulli(p). The MLE of p is $\bar{X}$ with MSE=Var=$p(1-p)/n$. Consider the Bayes estimator with $Beta(\alpha, \beta)$ prior. The estimator equals $\hat{p}_B = (\sum X_i + \alpha)/(n+\alpha+\beta)$. Taking $\alpha = \beta = \sqrt{n}/2$ makes MSE($\hat{p}_B$) constant as a function of p. With this prior, for small n, $\bar{X}$ has lower MSE than $\hat{p}_B$ unless p is close to zero or one. For large n, $\hat{p}_B$ has lower MSE than $\bar{X}$ unless p is close to half.

An estimator is **consistent** for a parameter if it converges to the parameter as the sample size grows to infinity. The LLN implies that the sample mean is a consistent estimator of population mean. More generally it implies that all sample moments are consistent for the population moments. Since method of moments estimators are functions of sample moments, they are also consistent estimators.

Another quantity of interest is the **asymptotic distribution** of an estimator, that is the limiting distribution as the sample size grows to infinity. If this limiting distribution exists and we can obtain it, then for sufficiently large n, it can be used as an approximation to the actual sampling distribution of the statistic. This procedure often leads to a much simplified method for obtaining the distribution of the statistic. The distribution is used in the following sections for constructing interval estimates and hypothesis testing. By the central limit theorem, the asymptotic distribution of all sample moments is normal. This result can be used to derive the asymptotic distribution of various statistics that can be expressed as functions of sample moments, like the MoM estimators.

13.2 Interval Estimation

An interval estimator or confidence interval is a formula that tell us how to use sample data to calculate an interval that contains a population parameter with a pre-specified probability. The confidence coefficient is the probability that a randomly selected sample will produce a confidence interval that encloses the population parameter. The confidence level is the confidence coefficient expressed as a percentage. 90%, 95%, and 99% are commonly used.

When the sample size is large, the distribution of $\sqrt{n}(\bar{X}_n - \mu)/\sigma$ is approximately normal by the central limit theorem. Thus

$$P(-z_{\alpha/2} < \sqrt{n}(\bar{X}_n - \mu)/\sigma < z_{\alpha/2}) = 1 - \alpha$$
$$\Rightarrow P(\bar{X}_n - z_{\alpha/2}\frac{\sigma}{\sqrt{n}} < \mu < \bar{X}_n + z_{\alpha/2}\frac{\sigma}{\sqrt{n}}) = 1 - \alpha, \tag{13.1}$$

where z_a is the $1 - a$ quantile of a standard normal distribution, that is $P(Z < z_a) = 1 - a$, where $Z \sim \mathcal{N}(0, 1)$. Also, by the LLN, when the sample size is large, the sample standard deviation s_n is very close to σ. Substituting s_n for σ in (13.1), we obtain a $1 - \alpha$ level confidence interval for μ as $(\bar{X}_n - z_{\alpha/2}s_n/\sqrt{n}, \bar{X}_n + z_{\alpha/2}s_n/\sqrt{n})$.

The following R code generates 500 observations from the Exponential(5) distribution and computes the 95% confidence interval using the above process. The parameter of interest is the mean, and the actual value of the parameter is $1/5 = 0.2$. The process is repeated 100 times. All 100 confidence intervals are displayed as vertical lines in Fig. 13.4 along with a horizontal line for the actual value of the parameter. We see that 7 of the 100 intervals miss the actual parameter value. We expect around 5 intervals to miss the parameter value since this is a 95% confidence interval.

```
a = rep(0,100)
b = a
c = 1.96/sqrt(500)
set.seed(10101)
for(i in 1:100){
  x = rexp(500,5) #mu=0.2, n=500
  a[i] = mean(x)-c*sd(x) #xbar-z s/sqrt(n)
  b[i] = mean(x)+c*sd(x)
}
length(which(a>.2|b<.2))/100

## [1] 0.07

plot(a,ylim=c(.15,.25),type="n", ylab="Interval")
for(i in 1:100){lines(c(i,i),c(a[i],b[i]))}
abline(.2,0)
index<-which(a>.2|b<.2)
cbind(a[index],b[index])
```

The above procedure cannot be used when the sample size is small or when the parameter of interest is not a function of sample moments. We shall discuss some common cases here.

When the population is normal and variance is known, then for any sample size n, the distribution of $\sqrt{n}(\bar{X}_n - \mu)/\sigma$ is standard normal. Therefore $1 - \alpha$ level confidence interval for μ is $(\bar{X}_n - z_{\alpha/2}\sigma/\sqrt{n}, \bar{X}_n + z_{\alpha/2}\sigma/\sqrt{n})$.

When the population is normal and variance is unknown, then for sample size n the distribution of $\sqrt{n}(\bar{X}_n - \mu)/s_n$ is t with $n - 1$ degrees of freedom. Therefore $1 - \alpha$ level confidence interval for μ is $(\bar{X}_n - t_{n-1,\alpha/2}\sigma/\sqrt{n}, \bar{X}_n + t_{n-1,\alpha/2}\sigma/\sqrt{n})$, where $t_{k,a}$ is the $(1 - a)$th quantile of the t distribution with k degrees of freedom.

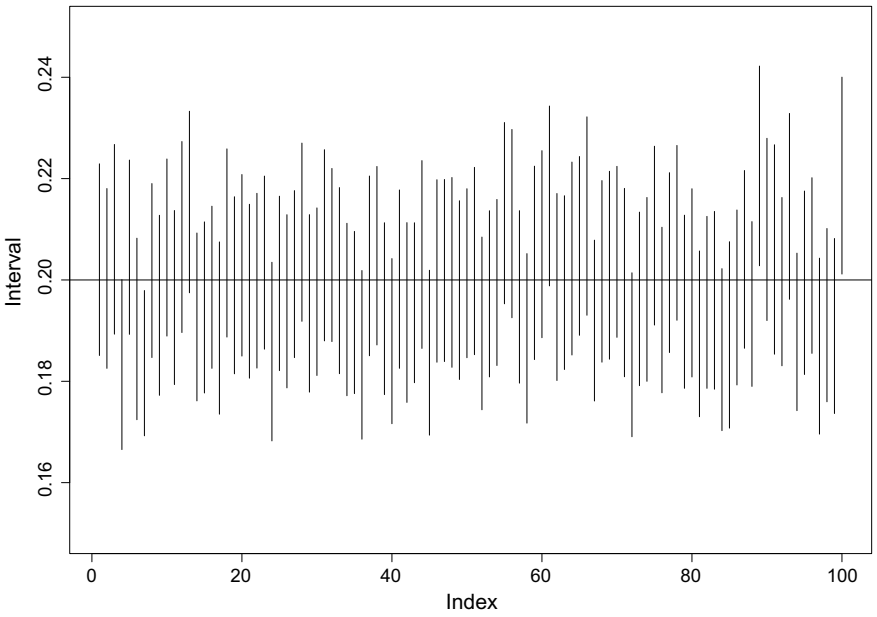


Fig. 13.4 100 confidence intervals for mean based on samples of size 500 from exponential distribution when $\mu = 0.2$

The function `ciz` computes the $(1 - \alpha)$ level confidence interval for μ from a sample x when the population is normal with standard deviation is σ. The function `cit` computes the $(1 - \alpha)$ level confidence interval for μ from a sample x when the population is normal with unknown standard deviation is unknown. The following code defines these functions, generates a sample of size 10 from the standard normal distribution, and computes the two intervals. Note that the z interval is shorter than the t interval since the latter incorporates the uncertainty about σ.

```
ciz = function(x, sigma, alpha=0.05){
  z = qnorm( 1-alpha/2)
  sdx = sigma/sqrt(length(x))
  c(mean(x) - z*sdx, mean(x) + z*sdx)
}
cit= function(x, alpha=0.05){
  n = length(x)
  t = qt( 1-alpha/2,n-1 )
  sdx = sd(x)/sqrt(n)
  c(mean(x) - t*sdx, mean(x) + t*sdx)
}
set.seed(10101)
x = rnorm(10)
```

```
ciz(x,1,0.05)

## [1] -0.7085761  0.5310140

cit(x,0.05)

## [1] -0.5982957  0.4207337
```

When the population is normal, then for sample size n the distribution of $(n-1)s_n^2/\sigma^2$ is χ^2 with $n-1$ degrees of freedom. Therefore $1-\alpha$ level confidence interval for σ^2 is $((n-1)s_n^2/\chi^2_{n-1,\alpha/2}, (n-1)s_n^2/\chi^2_{n-1,1-\alpha/2})$ where $\chi^2_{k,a}$ is the $(1-a)$th quantile of a χ^2 distribution with k degrees of freedom.

Although we have discussed only the case of population distribution being normal, exact distributions as above can be derived for several cases of parent distributions. When sample size is small and population distribution is unknown, then we generally resort to resampling techniques discussed in Chap. 15.

13.3 Hypothesis Testing

Another kind of statistical inference is hypothesis testing where we want to test a given statement regarding a population parameter. We start with a null hypothesis (H_0) and an alternative hypothesis (H_a). H_0 is the status quo and will be supported unless the data provides evidence that it is false. In the latter case, we reject H_0 and conclude that H_a is true.

For carrying out the test, we use a statistic that we call test statistic T. If the test statistic lies in a region of very low probability when H_0 is true, then H_0 is rejected. This region is called the critical region or rejection region. The pre-specified probability that the test statistic lies in the critical region under H_0 is called the level of the test and is denoted by α. If the null hypothesis is indeed true and the test statistic lies in the rejection region, then we reject H_0 and conclude erroneously that H_a is true. This is known as type I error. If, on the other hand, the alternative hypothesis is true and the test statistic does not lie in the rejection region, then we do not reject H_0. This error is known as type II error, and the probability of this kind of error is denoted by β. The power of the test is the probability of rejecting H_0 when it is indeed false and equals $1-\beta$.

Example 13.3.1 The log returns of a particular asset on a sample $n = 10$ days are given by

$$-0.76, 1.64, -0.64, -1.26, -0.61, -1.79, 3.42, 0.55, -0.59, 0.94$$

We are interested in testing if the mean log return of the population of all days for the same stock is zero or not. Thus

$$H_0 : \mu = 0, H_a : \mu \neq 0.$$

We assume that the distribution of log returns is normal. Then

$$T = \sqrt{n}(\bar{X}_n - \mu)/s_n \sim t_{n-1}.$$

Under H_0 the observed value of T is $\sqrt{10}\bar{X}_n/s_n = 0.0637$.
For $\alpha = 5\%$, the rejection region is $|T| > qt(0.975, n-1) = 2.262$ as the probability of this region under H_0 equals 5%. Since T does not lie in the rejection region, we do not reject H_0.

A related concept is the p-value. The **observed significance level, or p-value,** for a test is the probability of observing a test statistic as extreme or more than the one actually observed assuming the null hypothesis is true. In Example 13.3.1, the observed value of the test statistic is 0.0637. Since the alternative is two-sided, values away from the null on both sides are considered extreme. So the p-value is $P(|T| > 0.0637)$ where under the null hypothesis, the distribution of T is t with $n - 1 = 9$ degrees of freedom. This probability equals `2*(1-pt(0.0637,9))`=0.9506. Since the p-value is quite large, we do not reject the null hypothesis.

Definition 7 The likelihood ratio test statistic for testing $H : \theta \in \Theta_0$ versus $K : \theta \in \Theta_1$ is

$$\lambda(x) = \frac{\sup_{\theta \in \Theta_0} L(\theta \mid x)}{\sup_{\theta \in \Theta} L(\theta \mid x)}$$

where $\Theta = \Theta_0 \bigcup \Theta_1$ and L is the likelihood function.

A likelihood ratio test is of the form

$$\phi(x) = \begin{cases} 1 \text{ if } & \lambda(x) \leq c \\ 0 \text{ if } & \lambda(x) > c \end{cases} \tag{13.2}$$

The value of c is determined from the level of the test such that $P_H(\lambda \leq c) = \alpha$.

Example 13.3.2 Consider testing $H : \mu = \mu_0$ versus $K : \mu \neq \mu_0$ where $X_1, \ldots, X_n$ are iid $\mathcal{N}(\mu, 1)$.
For the numerator, $\sup_{\theta \in \Theta_0} L(\theta \mid x) = \frac{1}{(2\pi)^{n/2}} \exp(-\frac{1}{2} \sum_{i=1}^{n} (x_i - \mu_0)^2)$
The sup in the denominator is attained at $\theta = \bar{x}$ which is MLE.
Hence $\sup_{\theta \in \Theta} L(\theta \mid x) = \frac{1}{(2\pi)^{n/2}} \exp(-\frac{1}{2} \sum_{i=1}^{n} (x_i - \bar{x})^2)$

$$\lambda(x) = \exp(-\frac{1}{2}(\sum_{i=1}^{n} (x_i - \mu_0)^2 - \sum_{i=1}^{n} (x_i - \bar{x})^2))$$

$$\lambda(x) < c \iff \sum_{i=1}^{n} (x_i - \mu_0)^2 - \sum_{i=1}^{n} (x_i - \bar{x})^2 > c_1 \iff \mid \bar{x} - \mu_0 \mid > c_2. \tag{13.3}$$

Thus the likelihood ratio test rejects H when $\bar{x}$ differs from μ_0 by a large amount. The amount is determined from the constraint given by the level of the test, that is, $P_{\mu_0}(\mid \bar{x} - \mu_0 \mid > c_2) = \alpha$.

13.4 Exercises

1. Find the Bayes estimator for μ in Example 13.1.3.
2. Show that MSE = Bias^2+Variance.
3. Let $X_1, \ldots X_n$ be iid Bernoulli(p). Show that the Bayes estimator with $Beta(\alpha, \beta)$ prior equals $\hat{p}_B(\sum X_i + \alpha)/(n + \alpha + \beta)$.
4. In problem 3, find the MSE of the Bayes estimator when $\alpha = \beta = \sqrt{n}/2$. Compare this MSE with that of $\bar{X}$ for different values of n.
5. Find the likelihood ratio test for $H : \theta \leq \theta_0$ versus $K : \theta > \theta_0$ when $X_1, \ldots, X_n$ are iid from the exponential distribution with pdf $f(x \mid \theta) = \exp(-x + \theta) I(x > \theta)$.

Chapter 14
Bayesian Computation



Bayesian statistics analyses data and makes inference based on the Bayesian interpretation of probability. In this approach, we begin with prior probability models. Then as data arrives, we update the inference using Bayes theorem of probability and make inferences with the updated probability model, known as the posterior probability model. When data comes sequentially over time, the Bayesian update is natural. Suppose we have an idea about the possible value of the parameters of the interest. In that case, we can model the information as prior probability models. Then we can stitch the data and prior probability models through the Bayes theorem and develop the posterior probability models. We can infer the parameter of interest using the posterior probability models. Next, we formalise the Bayesian inference.

14.1 Introduction to Bayesian Inference

Suppose $Y \sim p(y|\theta)$ and $p(\theta)$ is the prior distribution over θ. Our objective is to make the Bayesian statistical inference about θ. The posterior distribution of θ is

$$p(\theta|y) = \frac{p(y|\theta)p(\theta)}{\int_{\Theta} p(y|\theta)p(\theta)d\theta}.$$

We want to make educated guess about θ under the light of the data Y. So we estimate the expected value of θ given the data Y as function of the data Y, i.e., the posterior mean of θ is

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/978-981-19-2008-0_14.

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

R. Sen and S. Das, *Computational Finance with R*, Indian Statistical Institute Series,
https://doi.org/10.1007/978-981-19-2008-0_14

$$\mathbb{E}(\theta|y) = \int_{\Theta} \theta p(\theta|y) d\theta = g(y).$$

We can see finding posterior mean of θ is an integration problem. We can obtain the posterior median as

$$\int_{-\infty}^{\mu_0} p(\theta|y) d\theta = \frac{1}{2},$$

where μ_0 is the posterior median of the θ. One can setup this problem as finding root, (see Sects. 3.2–3.6). Obtaining posterior median is also an integration problem like posterior mean. On the contrary, we can obtain the posterior mode as

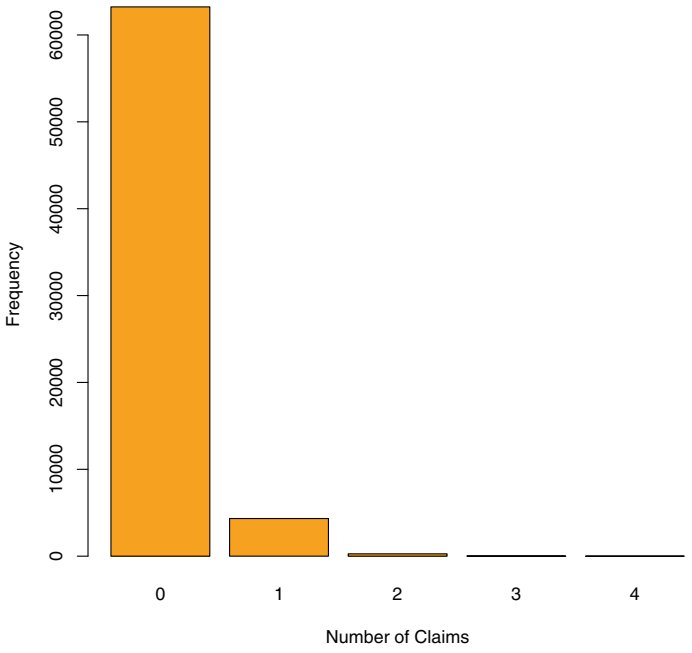
$$\begin{aligned} \hat{\theta} &= \text{argmax}_{\Theta}\ p(\theta|y) \\ &= \text{argmax}_{\Theta} p(y|\theta) p(\theta), \end{aligned}$$

where finding posterior mode of θ is a optimization problem.

14.2 Poisson Model for Count Data

The Poisson model is very popular for modeling count data. Here we consider the `dataCar` dataset from the `R` library called `insuranceData`. This dataset is based on one-year vehicle insurance policies taken out in `2004` or `2005`. There are `67856` policies, of which `4624 (6.8%)` claims are made.

```
library(insuranceData)
data(dataCar)
tbl = table(num_of_claims = dataCar$numclaims)
barplot(tbl,ylab = 'Frequency'
        ,xlab='Number of Claims'
        ,col='orange')
```




```
## Frequency Table of Number of Claims
tbl

num_of_claims      0     1     2     3     4
Frequency      63232  4333   271    18     2

c(mean = round(mean(dataCar$numclaims),3)
  , var=round(var(dataCar$numclaims),3))

##  mean   var
## 0.073 0.077
```

Since sample mean and sample variance of `numclaims` (i.e., number of claims) are same, therefore we can assume underlying data generating process is **Poisson distribution**.

Suppose X is number of claims, where $D = \{X_1, X_2, \ldots, X_n\} \overset{iid}{\sim} Poisson(\lambda)$. The statistic $T = \sum_{i=1}^{n} X_i \sim Poisson(n\lambda)$. Now what kind of prior probability model is suitable? Typically, if the conjugate prior is available, we can consider it a prior model, as it generally yields an analytical solution.

Conjugate Prior: If prior and posterior distributions belong to the same family, then they are called conjugate distribution, and the prior is known as conjugate prior. The **conjugate prior** for Poisson model Gamma distribution; i.e., $\lambda \sim Gamma(a, b)$ where $\mathbb{E}(\lambda) = \frac{a}{b}$, $\mathbb{V}ar(\lambda) = \frac{a}{b^2}$. Note that the parameters of prior distributions are known as hyper-parameters.

$$
\begin{aligned}
p(\lambda|t) &\propto f(t|\lambda)p(\lambda) \\
&= e^{-(n\lambda)}\frac{(n\lambda)^t}{t!} \times \frac{b^a}{\Gamma(a)}e^{-b\lambda}\lambda^{a-1} \\
&= e^{-(n+b)\lambda}\lambda^{t+a-1} \times \frac{n^t b^a}{t!\Gamma(a)} \\
&\propto e^{-(n+b)\lambda}\lambda^{t+a-1}.
\end{aligned}
$$

The **posterior distribution** of λ is $Gamma(a+t, b+n)$. The **posterior mean** is:

$$
\begin{aligned}
\mathbb{E}(\lambda|t) &= \frac{a+t}{b+n} \\
&= \frac{b}{b+n}\frac{a}{b} + \frac{n}{b+n}\frac{t}{n} \\
&= \frac{b}{b+n}\mathbb{E}(\lambda) + \frac{n}{b+n}\bar{X},
\end{aligned}
$$

Posterior mean is weighted average of prior mean and MLE, where

$$
\bar{X} = \frac{t}{n} = \frac{1}{n}\sum_{i=1}^{n} x_i \tag{14.1}
$$

is the MLE of λ. Note that as $n \to \infty$ the effect of prior mean will wash out and posterior mean will converge to MLE. Suppose we choose $Gamma(a=1, b=1)$ as prior distribution. We have the following analysis about λ.

```
t = sum(dataCar$numclaims)
t

## [1] 4937

n = nrow(dataCar)
n

## [1] 67856

### MLE of lambda
lambda_hat = t/n
lambda_hat

## [1] 0.07275701

## prior lambda ~ Gamma(a=1,b=1)
a=b=1

## Prior mean of lambda
cat('Prior mean of lambda = ',a/b,'\n')

## Prior mean of lambda =  1

## Posterior mean
cat('Posterior mean of lambda = ' ,(t+a)/(n+b),'\n')

## Posterior mean of lambda =  0.07277068
```

```
## 95% Frequentist Interval
se_mle = sqrt(lambda_hat/n)
Frequentist_CI = round(c(lambda_hat-1.96*se_mle,
                         lambda_hat+1.96*se_mle),3)
cat('Frequentist 95% CI = ', Frequentist_CI,'\n')

## Frequentist 95% CI =  0.071 0.075

## 95% prior Bayesian confidence interval
prior_CI = round(qgamma(c(0.025,0.975)
                        ,shape = a,rate=b),3)
cat('Prior 95% CI = ', prior_CI,'\n')

## Prior 95% CI =  0.025 3.689

## 95% Bayesian credible interval
posterior_CI = round(qgamma(c(0.025,0.975)
                            ,shape = t+a,rate=n+b),3)
cat('Posterior 95% CI = ', posterior_CI,'\n')

## Posterior 95% CI =  0.071 0.075
```

We see that 95% Bayesian interval and 95% frequentist interval are overlapping when the hyper-parameter choice is `a=1` and `b=1`. The effect of prior is effectively null, because the sample size `n = 67856` is large. If we decides to choose any other choice of hyper-parameters `a` and `b`, we can change the value in the above `R` code and recompile the code. We run the analysis for different choices of `a` and `b` and tabulated the results in Table (14.1).

The choice of `a=1000` and `b=1000` is a strong choice of `Gamma prior` where the 95% posterior credible interval completely non-overlapping with likelihood interval. However the choice for `a=b=1`; `a=b=10` , and `a=b=100` yield posterior intervals though overlap with likelihood interval but technically we can say each interval is distinctly different. Should we worry about it? The answer is 'Yes'. We should. Because our data analysis should be robust in a sense, it should not depend on the choice of hyper-parameters of the prior distribution.

Objective Prior: The objective Bayesian analysis tries to replace the subjective choice of prior with an algorithm that determines the prior. There are many differ-

Table 14.1 We present the Bayesian analysis for different choices of `a` and `b`; where `g(a,b)` is the gamma distribution with hyper-parameter `a` and `b`. Note that `a=0` and `b=0` yields MLE

Summary	`g(a=0,b=0)` ≡ MLE	`g(a=1,b=1)`	`g(a=100, b=100)`	`g(a=1000, b=1000)`
prior mean	-	1	1	1
prior sd.	-	1	0.1	0.032
prior 95% CI	-	(0.025 , 3.689)	(0.814 , 1.205)	(0.939 , 1.063)
posterior mean	0.073	0.073	0.074	0.086
posterior sd	0.001	0.001	0.001	0.001
posterior 95% CI	(0.071 , 0.075)	(0.071 , 0.075)	(0.072 , 0.076)	(0.084 , 0.088)

ent approaches, such as Jeffrey's prior, reference prior, empirical Bayes, penalized complexity, etc. The detailed discussion of all these choices of prior is beyond the scope of this book. Often these prior choices are improper. By improper prior, we mean the prior model is not a proper probability distribution. That is when,

$$\int_{\theta\in\Theta} p(\theta)d\theta = \infty,$$

we can say $p(\theta)$ is improper prior. Note that we can choose **improper prior** as our prior model; as long as the resulting posterior distribution is proper probability distribution, i.e.,

$$\int_{\theta\in\Theta} p(\theta|\mathbf{X})d\theta = 1.$$

For the Poisson model, the **objective priors** for the rate parameter λ is

$$p(\lambda) \propto \frac{1}{\lambda},$$

If you choose `a=1, b=0` then the pdf of `Gamma` prior yield the above prior for λ. Note that the prior is improper prior. However, the respective posterior distributions is proper probability distribution. Hence a valid Bayesian statistical inference can be drawn under the prior model.

```
### NIF prior
a=1; b=0
## Posterior mean under NIF prior
cat('posterior mean = ',(a+t)/(b+n),'\n')

## posterior mean =  0.07277175

## 95% Bayesian credible interval
## under NIF prior
Bayesian_CI = qgamma(c(0.025,0.975)
                     ,shape = t+a,rate=n+b)
Bayesian_CI = round(Bayesian_CI,3)
cat('Bayesian 95% CI = ', Bayesian_CI,'\n')

## Bayesian 95% CI =  0.071 0.075

### Jeffrey's prior
a=0.5; b=0
## Posterior mean under Jeffrey's prior
cat('posterior mean = ',(a+t)/(b+n),'\n')

## posterior mean =  0.07276438

## 95% Bayesian credible interval
## under Jeffrey's prior
Bayesian_CI = qgamma(c(0.025,0.975)
                     ,shape = t+a,rate=n+b)
Bayesian_CI = round(Bayesian_CI,3)
cat('Bayesian 95% CI = ', Bayesian_CI,'\n')

## Bayesian 95% CI =  0.071 0.075
```

Alternate approach: Cauchy and Half-Cauchy distribution as prior

The Cauchy distribution is an intriguing probability distribution. Though it is a proper probability distribution, the primary characteristics of distribution, such as mean, variance, higher-order moments, skewness, and kurtosis; nothing exists for the Cauchy distribution. Same is true for the half-Cauchy distribution. It means you cannot induce any regular distributional pieces of information, such as mean, variance, skewness, or kurtosis through Cauchy or half-Cauchy prior distribution. The pdf of half-Cauchy prior distribution is

$$p(\theta) = \frac{2}{\pi(1+\theta^2)} \quad \theta > 0.$$

```
dHalfCauchy= function(theta,log=FALSE){
  if(log==FALSE){
    return(2/(pi*(1+theta^2)))
  }else{
    return(log(2)-log(pi)-log(1+theta^2))
  }
}
```

As we assume $\lambda \sim$ Half-Cauchy(0, 1), about the 95% probability mass induces between 0 and 12.5; it means to begin with we are about 95% confident that the true value of λ is in the range between 0 and 12.5, i.e.,

$$\mathbb{P}(0 < \lambda < 12.5) \approx 0.95$$

As $T \sim Poisson(n\lambda)$ and $\lambda \sim$ Half-Cauchy(0, 1) the posterior will be proper, i.e.,

$$\begin{aligned} p(\lambda|t,n) &\propto l(\lambda|t,n)p(\lambda) \\ &\propto \underbrace{e^{-n\lambda}n\lambda^t}_{\text{likelihood of Poisson}} \times \underbrace{\frac{2}{\pi(1+\lambda^2)}}_{\text{pdf of half-Cauchy}} \end{aligned}$$

We estimate the **posterior mode** by maximising the following objective function,

$$\max_{\lambda} p(\lambda|t,n) = \max_{\lambda} k(\lambda|t,n) = \max_{\lambda} \log k(\lambda|t,n),$$

where maximising $p(\lambda|t,n)$ would yield the same results as maximising the $\log k(\lambda|t,n)$. The R code below presents the log-posterior (i.e., $\log p(\lambda|t,n)$) upto the kernel $\log k(\lambda|t,n)$

$$\max_{\lambda} p(\lambda|t,n)$$

```
logLike = function(lambda,t,n){
  l = dpois(t,lambda = n*lambda,log=TRUE)
  return(l)
}

logPosterior = function(lambda,t,n){
  lpost = logLike(lambda=lambda,t=t,n=n)
          +dHalfCauchy(theta=lambda,log=TRUE)
  return(lpost)
}

## Check log-posterior at an initial value
lambda.init = 1
logPosterior(lambda=lambda.init,t=t,n=n)

## [1] -49986.12
```

You can carry out the one-dimensional optimization using `optimize` function

```
optimize(f=logPosterior,lower = 0,upper=10
         ,t=t,n=n,maximum=TRUE)

## $maximum
## [1] 0.07276539
##
## $objective
## [1] -5.171245
```

So the posterior mode is 0.073. One question that we would like to address is what is the posterior mean of the rate parameter λ under Half-Cauchy$(0, 1)$ prior distribution? That is we have to solve the following integration problem

$$\mathbb{E}(\lambda|t, n) = \int \lambda p(\lambda|t, n) d\lambda.$$

Note that we do not know $p(\lambda|t, n)$ in a closed form. We know $p(\lambda|t, n)$ up to its kernel. Also as we are interested in the estimation of 95% Bayesian credible interval, i.e.,

$$\mathbb{P}(l \leq \lambda \leq u|t, n) = \int_l^u p(\lambda|t, n) = 1 - \alpha,$$

again another integration problem. However, we cannot solve it as we do not know $p(\lambda|t, n)$ fully, we know it only up to its kernel $k(\lambda)$. In the next Section, we present how to run Bayesian inference for any probability model with Monte Carlo method.

- So finding posterior mode is an optimization problem
- Estimation of posterior mean, posterior variance or 95% CI is an **integration problem**

14.3 Bayesian Inference with Monte Carlo for Any Model

In order to get the posterior mean we have to solve the following. Analytical solution does not exist for most of the sophisticated models. So we have to resort to simulation technique to solve this integration problem. Typically it is known as **Monte Carlo Integration** method.

As we want to estimate the **posterior mean**:

$$\mathbb{E}(\theta|y) = \int_{\Theta} \theta p(\theta|y) d\theta = g(y)$$

We can do so by simulating random samples from $p(\theta|y)$. Suppose $(\theta^1, ..., \theta^N)$ are random samples from $p(\theta|y)$, then we can approximate $g(y)$ by

$$\hat{g}(y) = \frac{1}{N} \sum_{s=1}^{N} \theta^s.$$

If we can ensure simple random sample then law of large number ensures

$$\hat{g}(y) = \frac{1}{N} \sum_{s=1}^{N} \theta^s \longrightarrow g(y) = \mathbb{E}(\theta|y)$$

as $N \to \infty$, where N is the simulation size.

Importance Sampling for Bayesian Inference We have discussed the general method of importance sampling in Chap. **??**. Here we apply that to the setting of Bayesian inference.

Bayesian inference is the primary example where you want to determine the properties of a probability distribution given up to its kernel function. Suppose $k(\theta)$ is the kernel function of the posterior distribution $p(\theta|y)$, i.e.,

$$f(\theta) = p(\theta|y) = \frac{k(\theta)}{C},$$

where $C = \int k(\theta)d\theta = \int p(y|\theta)p(\theta)d\theta$. Typically we do not know C and we know the posterior distribution in its unnormalized form, i.e.,

$$p(\theta|y) \propto p(y|\theta)p(\theta) = k(\theta)$$

If you want to calculate $\mathbb{E}[h(\theta)]$ where unnormalized density of $f(\theta)$ is $k(\theta)$. We can rewrite this as

$$\begin{aligned}\mathbb{E}[h(\theta)] &= \int h(\theta)\frac{k(\theta)}{C}d\theta = \frac{1}{C}\int h(\theta)\frac{k(\theta)}{g(\theta)}g(\theta)d\theta \\ &= \frac{1}{C}\int h(\theta)\tilde{w}(\theta)g(\theta)d\theta\end{aligned}$$

where $\omega(\theta) = \frac{f(\theta)}{g(\theta)} = \frac{1}{C}\frac{k(\theta)}{g(\theta)} = \frac{1}{C}\tilde{\omega}(\theta)$, $g(\theta)$ is the known probability density (typically known as proposal density), and $f(\theta) = p(\theta|y)$ is the normalized posterior density. By LLN if $\theta^1, \theta^2, ..., \theta^N$ are iid random samples from g, then

$$\frac{1}{N}\sum_{i=1}^{N} h(\theta^i)\tilde{\omega}(\theta^i) \longrightarrow CE[h(\theta)]$$

as $N \longrightarrow \infty$. Also by LLN,

$$\frac{1}{N}\sum_{i=1}^{N} \tilde{\omega}(\theta^i) \longrightarrow C$$

Therefore Monte Carlo estimator for $E[h(\theta)]$ using Importance Sampling Scheme is

$$\bar{h}_{IS} = \frac{\sum_{i=1}^{N} h(\theta^i)\tilde{\omega}(\theta^i)}{\sum_{i=1}^{N} \tilde{\omega}(\theta^i)}.$$

The method is only reliable when the weights are not too variable. As a rule of thumb, when

$$V = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left(\frac{\tilde{\omega}(\theta^i)}{\bar{\omega}} - 1\right)^2}$$

is less than 5, the IS method is reasonable. Note that $\bar{\omega} = \frac{1}{N}\sum_{i=1}^{N}\tilde{\omega}(\theta^i)$. You will know you chose a bad g when V is large.

Modeling Car Insurance Claim Following data presents number of monthly insurance claim of car accidents for last 12 months in a particular garage on national highway 34 (NH34),

$$D = \{6, 2, 2, 1, 2, 1, 1, 2, 3, 5, 2, 1\}.$$

We define y_i as the number of claim on i^{th} month, i.e., $y_1 = 6$, $y_2 = 2$,..., $y_{12} = 1$. We assume

$$y_i \overset{iid}{\sim} \text{Poisson}(\theta)$$

and

$$\theta \sim \text{log-Normal}(\mu = 2, \sigma = 1).$$

In this problem since a non-conjugate prior is being selected the posterior distribution is known up to normalizing constant. Simulating random samples from the unnormalized posterior distribution is difficult. So we simulate from a candidate distribution and using the importance sampling method we estimate the θ.

```
y = c(6, 2, 2, 1, 2, 1, 1, 2, 3, 5, 2,1)

IS=function(y,C,N){
  ## y : data
  ## C : C is the tuning parameter for
  ##     the g distribution
  ## N : the number of monte carlo samples

  # sample size
  n = length(y)

  # log-post
  log.posterior = function(t){
    lp = sum(dpois(y,t,log=TRUE))
             +dlnorm(t,2,1,log=TRUE)
    return(lp)
  }

  # parameters for the trial distribution
  a = C*mean(y); b = C;

  # log trial density, g
  log.g = function(t){
    dgamma(t,a,b,log=TRUE)
  }

  # log importance function
  log.w = function(t){
    log.posterior(t) - log.g(t)
  }

  # generate from the trial distribution
  U = rgamma(N, a, b)

  # calculate the list of log.w values
  LP = log.w(U)
```

```
  # factor out the largest value to prevent
  # numerical underflow
  w = max(LP)
  LP = LP - w

  # importance sampling estimate
  I = mean( exp(LP)*U )/mean(exp(LP))

  # calculate V
  V = mean(((exp(LP)/mean(exp(LP))) - 1)^2 )

  # calculate s.e. of the IS estimate
  Z = exp(LP)/mean(exp(LP))
  sig.sq = (1/N)*sum( (Z-I)^2 )
  se = sqrt( sig.sq/N )
  # return the 95 percent confidence interval
  # for the estimate
  return(c(I - 1.96*se, I + 1.96*se, V, I))
}
set.seed(5818)


## calculate V, E(theta|X), and var(E(theta|X))
## for a grid of values for c
const = seq(.05, 20, length=500)
A = matrix(0, 500, 2)

for(j in 1:500){
  OUT = IS(y, const[j], 1000)
  V = OUT[3]
  Var = (OUT[2]-OUT[1])/3.92
  A[j,] = c(V, Var)
}

# final Estimate
IS(y, const[which.min(A[,2])], 1000)[4]

## [1] 2.424335
```

Note that in Importance Sampling (IS), the simulated samples $(\theta^1, ..., \theta^N)$ are not from the target density, rather these are from the candidate density g.

14.4 Markov Chain Monte Carlo

The algorithm such as 'Acceptance-Rejection' described in Sect. 8.2.2 or 'Importance Sampling' does not perform well in solving the high-dimensional integration problem. The Markov chain Monte Carlo (MCMC) is a simulation technique that uses the Markov chain to solve the high-dimensional integration problem.

Suppose we want to solve the following integration problem

$$I = \int_{\Theta} h(\theta) f(\theta) d\theta,$$

where $h(\theta)$ is some function of θ and $f(\theta) = p(\theta|y)$ is the posterior distribution. Monte Carlo Integration approximates the integrals I via simulating M values from $f(\theta)$ and calculating

$$\hat{I} = \frac{1}{M}\sum_{i=1}^{M} h(\theta^i).$$

The Monte Carlo approximation $\hat{I}$ is a simulation estimator such that $\hat{I} \longrightarrow I$ as $M \longrightarrow \infty$ It follows from LLN. Let $\theta_1, \theta_2, \ldots$, be a sequence of **independent** and **identically** distributed random variables, each having a finite mean $\mu = \mathbb{E}(\theta_i)$. Then with probability 1,

$$\frac{\theta_1 + \theta_2 + ... + \theta_M}{M} \longrightarrow \mu \quad as \quad M \longrightarrow \infty.$$

Our objective is to solve integration by simulating independent samples from the posterior distribution $p(\theta|y)$. However, the MCMC cannot simulate the independent samples, then can we solve the integration?

14.4.1 Markov Chain

Consider the Θ as our parameter space. So the Markov chain is a collection of sample of $\theta \in \Theta$ that are each dependent on their previous draw. The chain wanders about the parameter space, remembering only where it was in the last time. The *transition kernel* governs the move in the parameter space.

Example: To understand how the Markov chain works, we consider the "discrete state-space" problem with k possible states. The corresponding transition kernel is a probability matrix of dimension $k \times k$. We consider an example of $k = 4$ states state-space model. The $\mathbf{P}$ is a 4×4 transition matrix, presented in Table (14.2), where θ can take four possible values that are marked with the superscripts index.

1. The rows define a conditional probability mass function, conditional on the current state. Each row sums to one.
2. The columns represent the marginal probabilities of being in a particular state next time.

We consider an initial distribution $\Pi_{(0)}$ (a probability vector of order $1 \times k$, that sum to one).

1. At iteration 1, our distribution $\Pi^{(1)}$ (from which $\theta^{(1)}$ is drawn) is

$$\Pi^{(1)}_{(1\times k)} = \Pi^{(0)}_{(1\times k)} \times \mathbf{P}$$

2. At iteration 2, our distribution $\Pi^{(2)}$ (from which $\theta^{(2)}$ is drawn) is

Table 14.2 Transition Matrix of the Markov Chain. The $p(\theta^j_{(t+1)}|\theta^i_{(t)})$ denotes the conditional probability that θ will be at θ^j in $(t+1)^{st}$ step, while θ is at θ^i during the current t^{th} iterations, where $i, j = 1, 2, 3, 4$

$p(\theta^1_{(t+1)}\|\theta^1_{(t)})$	$p(\theta^2_{(t+1)}\|\theta^1_{(t)})$	$p(\theta^3_{(t+1)}\|\theta^1_{(t)})$	$p(\theta^4_{(t+1)}\|\theta^1_{(t)})$
$p(\theta^1_{(t+1)}\|\theta^2_{(t)})$	$p(\theta^2_{(t+1)}\|\theta^2_{(t)})$	$p(\theta^3_{(t+1)}\|\theta^2_{(t)})$	$p(\theta^4_{(t+1)}\|\theta^2_{(t)})$
$p(\theta^1_{(t+1)}\|\theta^3_{(t)})$	$p(\theta^2_{(t+1)}\|\theta^3_{(t)})$	$p(\theta^3_{(t+1)}\|\theta^3_{(t)})$	$p(\theta^4_{(t+1)}\|\theta^3_{(t)})$
$p(\theta^1_{(t+1)}\|\theta^4_{(t)})$	$p(\theta^2_{(t+1)}\|\theta^4_{(t)})$	$p(\theta^3_{(t+1)}\|\theta^4_{(t)})$	$p(\theta^4_{(t+1)}\|\theta^4_{(t)})$

$$\Pi^{(2)}_{(1\times k)} = \Pi^{(1)}_{(1\times k)} \times \mathbf{P}$$

3. At iteration t, our distribution $\Pi^{(t)}$ (from which $\theta^{(t)}$ is drawn) is

$$\Pi^{(t)}_{(1\times k)} = \Pi^{(t-1)}_{(1\times k)} \times \mathbf{P} = \Pi^{(0)}_{(1\times k)} \times \mathbf{P}^t$$

Stationarity of Markov Chain The target distribution Π is a stationary distribution π, such that $\pi = \pi\mathbf{P}$. Here we view the Markov chain with its transition matrix $\mathbf{P}$. If the posterior distribution is proper then any Markov chain from the MCMC algorithm will converge to $p(\theta|y)$ regardless of the starting points. We develop a Markov chain with the posterior distribution $p(\theta|\mathbf{X})$ as the expected stationary distribution Π. We then execute the chain to generate samples from $p(\theta|\mathbf{X})$.

14.4.2 Markov Chain Monte Carlo Integration

Once we have a Markov chain that has converged to the stationary distribution, then the draws in our chain appear to be like draws from $p(\theta|y)$. Hence we use Monte Carlo Integration methods to learn the unknown parameter θ. However the MC draws are not independent. The independence property is required for Monte Carlo Integration to work, as a required condition for LLN to work. The solution to this problem is **Ergodic Theorem**.

Ergodic Theorem
Let $\theta^{(1)}, \theta^{(2)}, ..., \theta^{(T)}$ be T samples from Markov chain that is aperiodic, irreducible and positive recurrent (that is the chain is ergodic) and $\mathbb{E}[g(\theta)] < \infty$ Then with probability 1

$$\frac{1}{T}\sum_{t=1}^{T} g(\theta^{(t)}) \longrightarrow \int_{\Theta} g(\theta)\pi(\theta)d\theta$$

as $T \longrightarrow \infty$ where π is the stationary distribution.

This is the Markov chain analog to the LLN. But what does it mean for a chain to be aperiodic, irreducible, and positive recurrent?

Aperiodicity

Intuitively we can say the chain is aperiodic, as long as it does not replicate itself in an exact cycle. A robust definition of aperiodicity is as follows. A Markov chain P is aperiodic if for all $\theta^i \in \Theta$ we have

$$gcd\{t : P^t(\theta^i, \theta^i) > 0\} = 1,$$

$P^t(\theta^i, \theta^i)$ is the probability that θ^i returns in t steps.

Irreducibility

If a Markov chain can go from any state to any other state (in one or multiple steps) the chain is called *irreducible*. A Markov chain **P** is *irreducible* if for all x, y, there exists some t such that $P^t(x, y) > 0$. We can show if **P** is *irreducible*, then **P** is aperiodic $\iff$ there exists t such that $P^t(x, y) > 0$ for all $x, y \in \Omega$.

Positive Recurrence

A Markov chain is *recurrent* if for any given state i , if the chain starts at i, it will eventually return to i with probability 1. A Markov chain is *positive recurrent* if the expected return time to state i is finite; otherwise it is null recurrent.

So if our Markov chain is aperiodic, irreducible, and positive recurrent then it is ergodic and the ergodic theorem allows us to do Monte Carlo Integration by calculating $\mathbb{E}[g(\theta)]$ from draws, ignoring the dependence between draws.

What is MCMC Algorithm?

The MCMC is a class of Monte Carlo methods that simulate dependent samples, from a posterior probability distribution. These simulated samples are drawn from Markov chain. Hence full-form of MCMC is the Markov Chain Monte Carlo. In Bayesian statistics, there are generally two popular MCMC algorithms that we use: the Gibbs Sampler and the Metropolis–Hastings algorithm.

Gibbs Sampler

Suppose we want to sample from the joint posterior distribution $p(\theta_1, ..., \theta_k|y)$. The Gibbs sampler simulates from the posterior distribution if full conditional posterior distributions of each parameter are known. For each parameter, the **conditional posterior** distribution is denoted as $p(\theta_j|\theta_{-j}, y)$, where $\theta_{-j} = (\theta_1, \cdots, \theta_{j-1}, \theta_{j+1}, \cdots, \theta_k)$.

The Hammersley–Clifford Theorem:
Suppose we have a joint density $f(x, y)$. We can write out the joint density in terms of the conditional densities, i.e., $f(x|y)$ and $f(y|x)$

$$f(x, y) = \frac{f(y|x)}{\int \frac{f(y|x)}{f(x|y)} dy}. \tag{14.2}$$

We can write the denominator in Eq. (14.2) as

$$\begin{aligned}\int \frac{f(y|x)}{f(x|y)} dy &= \int \frac{\frac{f(x,y)}{f(x)}}{\frac{f(x,y)}{f(y)}} dy, \\ &= \int \frac{f(y)}{f(x)} dy, \\ &= \frac{1}{f(x)}.\end{aligned}$$

Thus, the right-hand side is

$$\frac{f(y|x)}{\frac{1}{f(x)}} = f(y|x)f(x) = f(x, y).$$

The theorem indicates that information from the conditional probability permits to obtain information regarding the joint distribution.

Gibbs Sampler Steps

- Suppose that we want to simulate from the posterior distribution $p(\theta|y)$, where $\theta = \{\theta_1, \theta_2, \theta_3\}$
- The steps of a Gibbs Samplers are

1. Choose a vector of initial values $\theta^{(0)}$ (*Defining a initial proposal distribution* $\Pi^{(0)}$ *and simulate* $\theta^{(0)}$ *from it.*)
2. Simulate a value $\theta_1^{(1)}$ from the $p(\theta_1|\theta_2^{(0)}, \theta_3^{(0)}, y)$.
3. Simulate a value $\theta_2^{(1)}$ from the $p(\theta_2|\theta_1^{(1)}, \theta_3^{(0)}, y)$. (*Note that we use the updated value* $\theta_1^{(1)}$ *for* θ_1.)
4. Simulate a value $\theta_3^{(1)}$ from the $p(\theta_3|\theta_1^{(1)}, \theta_2^{(1)}, y)$.

- Steps 2–4 are analogous to multiplying $\Pi^{(0)}$ and $\mathbf{P}$ to have $\Pi^{(1)}$ and then simulate $\theta^{(1)}$ from $\Pi^{(1)}$.

5 Simulate $\theta^{(2)}$ using $\theta^{(1)}$ and continuously apply the most updated value of θ.
6 Repeat until you have M many simulations with each samples being a vector $\theta^{(t)}$. values.

- 'Gibbs sampler' simulate samples of θ from the stationary posterior distribution. It is a special kind of Markov chain where the conditional posterior distribution of each parameter is known.

The Metropolis–Hastings Sampler

Suppose we have a posterior $p(\theta|y)$ that we want to sample from. However the posterior does not look like any distribution that we know. The grid approximations are not possible for high dimensional posterior distribution. Often the full conditional posterior does not follow any known distributions. Therefore Gibbs sampling is not possible for those whose full conditionals are not known. In such cases, we can use the **Metropolis–Hastings** algorithm.

The Metropolis–Hastings Algorithm

1. Choose initial value $\theta^{(0)}$
2. At iteration t, simulate a candidate θ^* from a proposal distribution $q_t(\theta^*|\theta^{t-1})$).
3. Compute the acceptance ratio

$$r = \frac{p(\theta^*|y)/q_t(\theta^*|\theta^{(t-1)})}{p(\theta^{(t-1)}|y)/q_t(\theta^{(t-1)}|\theta^*)}. \tag{14.3}$$

4. Accept $\theta^{(t)} = \theta^*$ if $U < \min(1, r)$ where $U \sim uniform(0, 1)$
   Otherwise $\theta^{(t)} = \theta^{(t-1)}$
5. Repeat steps 2–4 M times to get M draws from $p(\theta|y)$.

In step 1, choose an initial value $\theta^{(0)}$. Note that $\theta^{(0)}$ must have positive probability.

$$p(\theta^{(0)}|y) > 0.$$

In step 2, we can simulate $\theta^{(*)}$ from $q_t(\theta^*|\theta^{(t-1)})$. The proposal distribution $q_t(\theta^*|\theta^{(t-1)})$ decides where it goes in the following iteration of the Markov chain. The support of proposal density and the posterior distribution must overlap. The proposal distribution approximates the transition kernel density. If symmetric proposal

distribution is dependent on $\theta^{(t-1)}$, then it is known as **random walk Metropolis sampling**. If the proposal distribution does not depend on $\theta^{(t-1)}$

$$q_t(\theta^*|\theta^{(t-1)}) = q_t(\theta^*)$$

then it is known as **independent Metropolis–Hastings sampling**. In this case, all simulations, i.e., θ^* are drawn from the same distribution, regardless of where the previous draw was. It can be very efficient or very inefficient, depending on how close the proposal density is, to the posterior density. Typically, if proposal density has heavier tails than the posterior, then the chain will behave nicely.

In step 3, we estimate the acceptance ratio r using Eq. (14.3). When the proposal density is symmetric, the r is

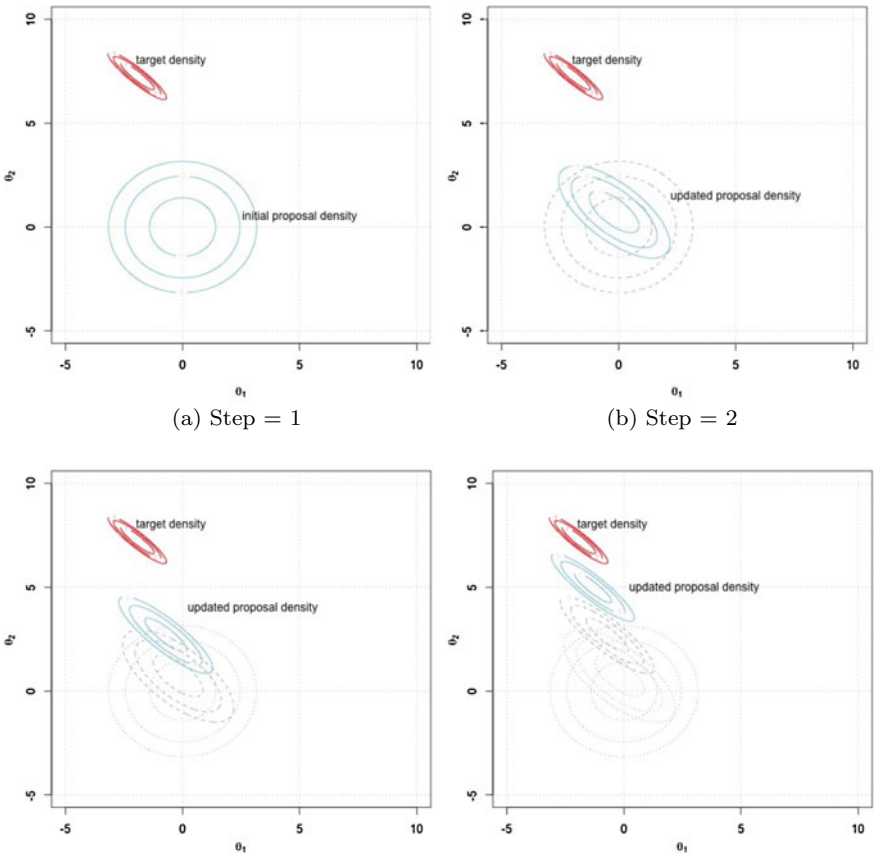
$$r = \frac{p(\theta^*|y)}{p(\theta^{(t-1)}|y)}.$$

If the probability of the proposal sample is higher than the current sample, then the proposal is better, so we accept it. Otherwise, we accept the proposal, corresponding to the ratio of the probabilities of the current and candidate samples. Since r is a ratio it will be sufficient if we know the posterior distribution up to a constant.

In step 4, we accept θ^* as $\theta^{(t)}$ with probability $min(r, 1)$. For each θ^*, draw a value u from the $Uniform(0, 1)$ distribution. If $u \leq r$ accept θ^* as $\theta^{(t)}$. Otherwise you accept last $\theta^{(t-1)}$ as $\theta^{(t)}$. Unlike in acceptance-rejection sampling, at each iteration t, the MCMC algorithm always yields a sample, either θ^* or $\theta^{(t-1)}$.

14.4.3 How MCMC Works?

In Fig. 14.1 we present an intuitive idea about how MCMC works. The **red color density** is our **target density**, which is **stationary density**. It is static in one place. We know the target posterior density only up to its kernel. In Fig. 14.1a, we choose a simple initial ('sky-blue' colored) **proposal density** which may be far from the **target density**. But we know how to simulate it. We simulate from proposal density and accept a value with a certain probability. As we move to the sub-Figure (14.1b), where we update our proposal density with newly accepted value. Then we repeat the process from the step 2, in the step 3 in the sub-Figure (14.1c), and in the step 4 in the sub-Figure (14.1d). Once accept a value, we update our proposal density and simulate from the new proposal density. Since the updation of proposal density was made using



(a) Step = 1 (b) Step = 2

(c) Step = 3 (d) Step = 4

Fig. 14.1 How MCMC works?

the transition kernel density; it creates a **Markov Chain**. The **fundamental theorem of Markov chain** ensures the convergence of the chain to its stationary distribution.

Example 14.4.1 Idiosyncratic Return Follows Laplace Distribution
We want to model the relationship between portfolio return and market return as follows

$$r_t^{\text{portf}} = \alpha + \beta r_t^{\text{S\&P500}} + \epsilon,$$

where

- r_t^{portf} is portfolio return,
- $r_t^{\text{S\&P500}}$ is market index return,
- $\epsilon \sim Laplace(0, \lambda)$.

That is the idiosyncratic return follows Laplace distribution. We consider the following priors on the parameters α, β, and λ;

$$\alpha \sim Cauchy(0, 1),$$

$$\beta \sim Cauchy(0, 1),$$

$$\lambda \sim \text{Half-Cauchy}(0, 1).$$

Our objective is to estimate $\mathbb{P}(\alpha > 0|Data)$. Note that if α is positive that means the portfolio manager is generating positive return over and above the expected return. We implement this model with Metropolis–Hastings. The R code is presented below. For this exercise we considered the stock treasury.csv dataset.

```
library('mvtnorm')

data = read.csv(file='stock_treasury.csv')
n = nrow(data)
cat('Sample size = ',n,'\n')

## Sample size =  250

# Risk Free Rate is in percentage and
# annualised.
# So the following conversion is required.
Rf = data$UST_Yr_1/(250)

## Compute log-return in percentage
ln_rt_snp500 = diff(log(data$SnP500))*100
ln_rt_snp500 = ln_rt_snp500 - Rf[2:n]
ln_rt_ibm = diff(log(data$IBM_AdjClose))*100
ln_rt_ibm = ln_rt_ibm - Rf[2:n]
ln_rt_apple = diff(log(data$Apple_AdjClose))*100
ln_rt_apple = ln_rt_apple- Rf[2:n]
ln_rt_msft = diff(log(data$MSFT_AdjClose))*100
ln_rt_msft = ln_rt_msft- Rf[2:n]
ln_rt_intel = diff(log(data$Intel_AdjClose))*100
ln_rt_intel = ln_rt_intel - Rf[2:n]

## log-return of the portfolio
```

```
ln_r = cbind(ln_rt_ibm,ln_rt_apple
             ,ln_rt_msft,ln_rt_intel)

### Portfolio
w = c(0.2,0.3,0.25,0.25)
ln_rt_portf = ln_r%*%w
```

```
log_likelihood = function(param,y,x){
  a = param[1]
  b = param[2]
  lambda = param[3]
  pred = a + b*x
  likelihoods = -log(2*lambda)-abs(y-pred)/lambda
  sumll = sum(likelihoods)
  return(sumll)
}

dHalfCauchy = function(theta,log=FALSE){
  if(log==FALSE){
    return(2/(pi*(1+theta^2)))
  }else{
    return(log(2)-log(pi)-log(1+theta^2))
  }
}

log_prior = function(param,x){
  a = param[1]
  b = param[2]
  lambda = param[3]
  a_prior = dcauchy(a,0,1,log = T)
  b_prior = dcauchy(b,0,1,log = T)
  scale_prior = dHalfCauchy(theta=lambda,log = T)

  return(a_prior+b_prior+scale_prior)
}

log_posterior = function(param,y,x){
  like = log_likelihood(param=param,y=y,x=x)
  prior = log_prior(param=param,x=x)
  post  = like + prior
  return ( post )
}

proposalfunction = function(param,x){
  X=cbind(rep(1,length(x)),x)
  S=param[3]*solve(t(X)%*%X)
  prop = c(rmvnorm(1
                   ,mean = param[1:2]
                   ,sigma = S)
           ,rgamma(1,param[3]*5,5))
  return(prop)
}

run_metropolis = function(startvalue, N.sim, burnin){
  iterations = N.sim + burnin
  chain = array(dim = c(iterations+1,3))
  chain[1,] = startvalue
  for (i in 1:iterations){
    proposal = proposalfunction(chain[i,],x=x)

    probab = exp(log_posterior(param=proposal
                               ,y=y,x=x)
                 - log_posterior(param=chain[i,]
                                 ,y=y,x=x))
```

```
    if (runif(1) < probab){
      chain[i+1,] = proposal
    }else{
      chain[i+1,] = chain[i,]
    }
  }
  return(chain)
}

options(warn=-1)
y=ln_rt_portf
x=ln_rt_snp500
startvalue = c(1,0,1)
N.sim=1000
burnin=0

set.seed(1)
chain = run_metropolis(startvalue=startvalue
                       ,N.sim=N.sim
                       ,burnin=burnin)
colnames(chain)=c("alpha","beta","lambda")
chain=chain[(burnin+1):nrow(chain),]
options(warn=0)
```

Purposefully we choose starting value far from MLE and keep burn-in at 0; so that we can study the convergence behavior. The trace-plot and kernel density plot are presented in Fig. 14.2. In the trace-plot, we plot the generated Markov-chain samples as time-series plot. The trace-plot of the Markov chain indicates the it takes about 200 iterations to converge. The kernel-density plot of the simulated values indicates how the probability density of posterior distribution looks like.

```
par(mfrow=c(3,2))
plot(ts(chain[,"alpha"]),lwd=2
     ,ylab = "alpha")
plot(density(chain[,"alpha"]),lwd=2
     ,main = "",col='red'
     ,xlab = "alpha")
plot(ts(chain[,"beta"]),lwd=2
     ,ylab = "beta")
plot(density(chain[,"beta"]),lwd=2
     ,main = "",col='red'
     ,xlab = "beta")
plot(ts(chain[,"lambda"]),lwd=2
     ,ylab = "lambda")
plot(density(chain[,"lambda"]),lwd=2
     ,main = "",col='red'
     ,xlab = "lambda")
```

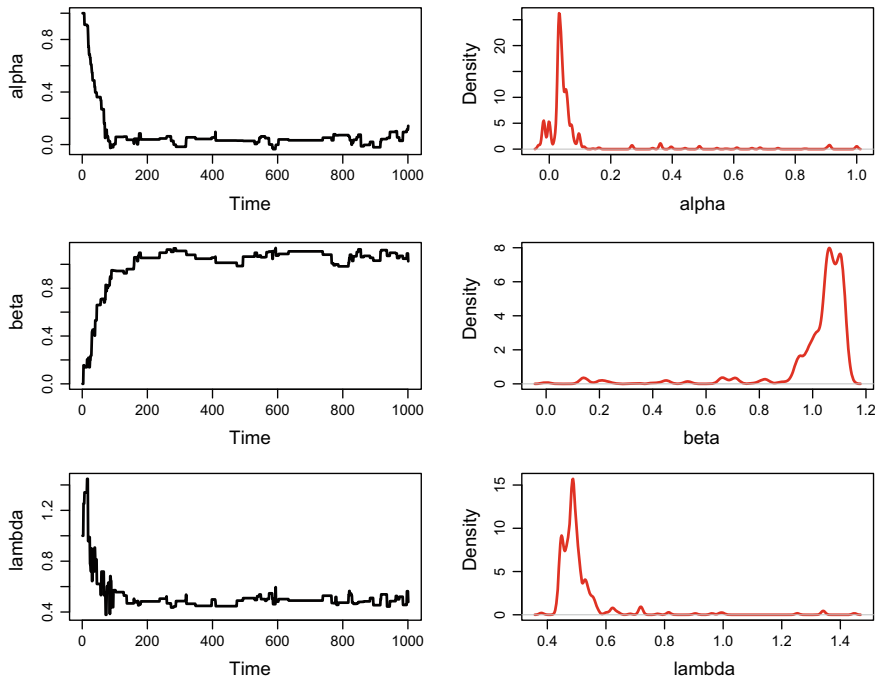


Fig. 14.2 Trace plot (left panel) and Kernel density plot (right panel) of Markov chain from Metropolis–Hastings Algorithm for regression model whose residual follow Laplace distribution

What is the $\mathbb{P}(\alpha > 0|Data)$?

```
alpha_star = chain[,'alpha']
no.of.alpha.gt.zero = length(alpha_star[alpha_star>0])
total.no.of.alpha = length(alpha_star)

prob_a_gt_zero = (no.of.alpha.gt.zero/total.no.of.alpha)

round(prob_a_gt_zero,4)

## [1] 0.9331
```

The posterior probability of positive is 0.9331 means that there is 93.31% probability that the portfolio manager generated positive return above the expected return.

Table 14.3 The total number of insurance products sold in two areas

Insurance	Area 1	Area 2
Life	33	32
General	39	24
Car	57	12
Health	55	10

14.5 Exercises

1. $X_1 \sim Poisson(\lambda_1)$ and $X_2 \sim Poisson(\lambda_2)$ are independent then show that $X_1|X_1 + X_2 = k \sim Binom\left(k, p = \frac{\lambda_1}{\lambda_1+\lambda_2}\right)$. If λ_1 and λ_2 follows $Gamma(a, b)$, then find the posterior distribution of p.
2. Table 14.3 presents the total number of insurance products sold by a sales team in two areas. We assume the number of insurance sold in Area 1 and Area 2 follow the Poisson distribution with rate λ_1 and λ_2 , respectively. Find the 95% CI of p for each insurance product separately. The p implies the proportion of sales in Area 1.
3. In Example 14.4.1, modify the `R` code presented in this section appropriately for the following prior distributions:

$$\alpha \sim N(0, 0.1),$$

$$\beta \sim N(1, 0.25),$$

$$\lambda \sim \text{Half-Cauchy}(0, 1).$$

Chapter 15
Resampling



In the previous chapters we have considered model-based statistical inference. In many cases, it is difficult to decide on an appropriate model. We are still interested in drawing inference from the data. In such cases, resampling techniques come in handy. Resampling methods require fewer assumptions regarding the underlying data generating mechanism and are, therefore, widely applicable.

One such situation is when we want to obtain the variance of an estimator, perhaps for constructing confidence intervals or carrying out a test of hypothesis. If the distribution of the statistic is unknown, then one way is to have a large sample and appeal to the central limit theorem. However, it is not always possible to obtain a large sample due to constraints of time and cost. In time series situations, a large sample might violate the stationarity assumptions as structural changes may happen.

Another situation is in mean–variance portfolio optimization using the efficient frontier. The estimation of mean vector and covariance matrix of the assets in the portfolio introduces variability in the estimated frontier curve. One way to address this is to carry out stochastic optimization that requires probabilistic modeling of the parameters. Another way is to look at robust optimization that considers the worst-case scenario. Again, in this case, resampling from the available data can give a fair idea about the variability of the frontier due to the estimation of the mean and covariance.

A third situation is in assessing risk of a portfolio, particularly when multiple assets are involved. Modeling joint distribution of assets is a difficult task in itself. When we have to restrict it to extreme values, then finding a suitable parametric model becomes even harder.

The final example is in the case where we have fitted a model to a dataset and we are interested in its performance on a new data point. Again, we can make assumptions of the distribution of the new data point and do the analysis. Alternatively, we can perform cross-validation that requires very few assumptions.

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/978-981-19-2008-0_15.

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

R. Sen and S. Das, *Computational Finance with R*, Indian Statistical Institute Series,
https://doi.org/10.1007/978-981-19-2008-0_15

15.1 Jackknife

The simplest resampling method is the Jackknife. Suppose we have a parameter of interest θ and a sample $X_1, \ldots, X_n$ from the population. We estimate θ using $\hat{\theta}_n$. The question, now, is regarding the distribution of $\hat{\theta}_n$. If we know the distribution of the sample observations, then we can derive the distribution of $\hat{\theta}_n$ theoretically. If the sample size is large enough, then even if we do not know the distribution of the observations, we can often approximate the distribution of $\hat{\theta}_n$ by a normal using the central limit theorem. We are now in a situation where the distribution of the observations in unknown and the normal approximation does not hold.

The Jackknife method works as follows. We leave out one observation and generate an estimator of θ from the remaining observations. If we leave out the first observation, then the sample is $X_2, \ldots, X_n$ and the estimator is denoted by $\hat{\theta}_{(-1)}$. We do this repeatedly by leaving out one observation at a time to obtain the n estimators

$$\hat{\theta}_{(-1)}, \hat{\theta}_{(-2)}, \ldots, \hat{\theta}_{(-n),}$$

and their average as

$$\hat{\theta}_{(.)} = \frac{1}{n} \sum_{i=1}^{n} \hat{\theta}_{(-i)}$$

Suppose we are interested in the bias of $\hat{\theta}_n$. In most situations, including MLE, the expectation of $\hat{\theta}_n$ can be written as

$$\mathrm{E}(\hat{\theta}_n) = \theta + \frac{a_1}{n} + \frac{a_2}{n^2} + o(n^{-2}).$$

Since each $\hat{\theta}_{(-i)}$ is based on $n-1$ observations, the expectation is same as above with n replaced by $n-1$. Consider a new estimator

$$\tilde{\theta}_n = n\hat{\theta} - (n-1)\hat{\theta}_{(.)}.$$

An easy calculation shows that

$$\mathrm{E}(\tilde{\theta}_n) = \theta + \frac{a_2}{n(n-1)} + o(n^{-2}). \tag{15.1}$$

Thus the order of the bias of $\tilde{\theta}_n$ is n^{-2} while that of $\hat{\theta}_n$ is n^{-1}. The Jackknife method proposes $\tilde{\theta}_n$ as a bias-corrected estimator of θ.

Inspired by the above calculations, Tukey (1958) defined the following pseudo observations.

$$\tilde{\theta}_{(i)} = \hat{\theta}_n + (n-1)(\hat{\theta}_n - \hat{\theta}_{(-i)}).$$

Note that the bias-corrected estimate $\tilde{\theta}_n$ is the average of the pseudo-observations. Tukey showed that these can also be used to compute a nonparametric estimate of the variance of $\hat{\theta}_n$ as

$$\widehat{\text{Var}}(\hat{\theta}_n) = \frac{1}{n(n-1)} \sum_{i=1}^{n} (\tilde{\theta}_{(i)} - \tilde{\theta}_n)^2.$$

Let us consider Example 13.3.1 where we were given $n = 10$ sample values of log-returns. Suppose we are interested in the skewness of the population giving rise to this dataset. The following code can be used to estimate the skewness as 0.801. The Jackknife estimate of the bias is -0.543 and of the variance is 0.292. Thus the bias-corrected estimate is 1.344.

```
x = c(-0.76,1.64,-0.64,-1.26,-0.61,-1.79,3.42,0.55,-0.59,0.94)
library(timeDate)


##      kurtosis, skewness

n = length(x)    #10
s = skewness(x)   #0.801
sj = rep(0,n)
for(i in 1:10){
  y = x[-i]
  sj[i] = s+(n-1)*(s-skewness(y))
}
#Generating the pseudo observations
bias = s-mean(sj)
bias

## [1] -0.543

corrected = mean(sj)
corrected

## [1] 1.344

variance = var(sj)/n
variance

## [1] 0.292
```

15.2 Bootstrap

The idea of bootstrap, as opposed to Jackknife where resampling is done systematically by leaving one out, is to sample randomly with replacement. Thus if $X_1, \ldots, X_n$ is the sample of observations, a bootstrap sample $X_1^*, \ldots, X_n^*$ is a sample of n values drawn randomly with replacement from the original observations. Here each X_i^* has equal probability of being one of the sample observations. For an estimator $\hat{\theta}$ of a parameter θ, we obtain a bootstrap version $\hat{\theta}^*$ for every bootstrap sample. This step is repeated multiple times to obtain $\hat{\theta}_1^*, \ldots, \hat{\theta}_B^*$ for a large B. The bootstrap estimate of θ is the average of $\hat{\theta}_i^*$s.

For the example considered above, that is, Example 13.3.1, the code to compute the bootstrap samples with $B = 1000$ is given below. The bootstrap estimate of skewness is 0.618. The histogram of the skewness in the bootstrap samples is presented in Fig. 15.1 with the sample skewness marked by the blue line.

In addition to obtaining estimates of the bias and variance of an estimator, the bootstrap procedure can be used to obtain confidence intervals for the parameter of interest. The bootstrap estimates generate the distribution of the estimator when samples are drawn repeatedly. The only catch is that the samples are not drawn from the population, but from the sample of observations. If the sample is fairly

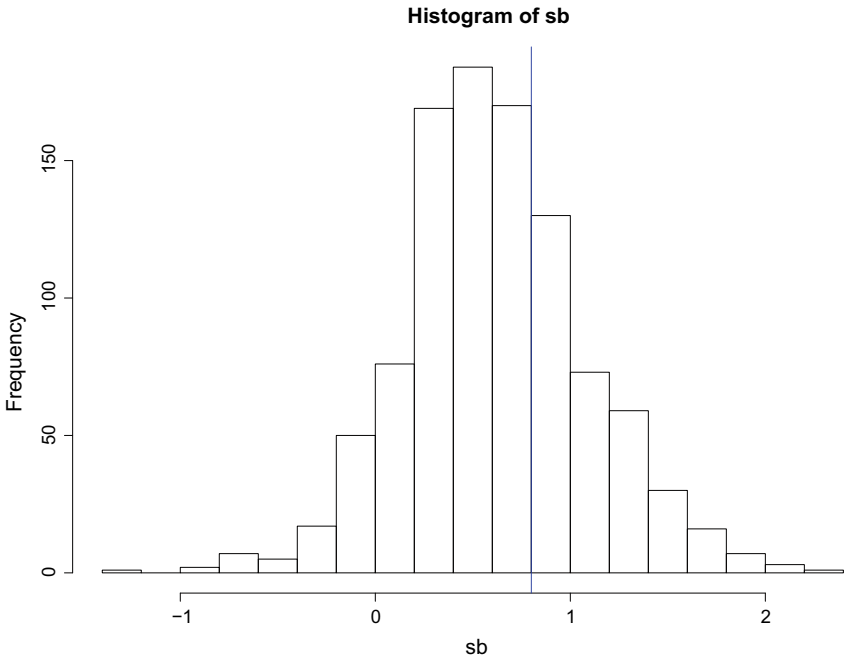


Fig. 15.1 Histogram of skewness in 1000 bootstrap samples

representative of the population, then the distribution of bootstrap estimates is fairly close to the sampling distribution of the estimator. Hence the quantiles of the former can be used to find a confidence interval for the parameter. For example, in the following code, we obtain a 95% confidence interval for the population skewness by taking the interval between the 2.5 and 97.5% quantiles of the bootstrap estimates. The interval, thus obtained, is $(-0.272, 1.660)$. Since the confidence interval contains the value zero, it is possible that the skewness of the population is indeed zero.

```
B = 1000
sb = rep(0,B)
set.seed(1)
for(i in 1:B){
  y = sample(x,n,replace=T)
  sb[i] = skewness(y)
}
hist(sb,breaks=20)
abline(v=0.8, col="blue")
mean(sb)

## [1] 0.6024635

quantile(sb,0.025)

## -0.2125221

quantile(sb,0.975)

## 1.508589
```

15.2.1 Parametric Bootstrap

In a parametric bootstrap, we assume a model for the population distribution with some unknown parameters. These parameters are estimated from the observed sample. Then the bootstrap samples are drawn from this fitted distribution, instead of the empirical distribution. For example, in the dataset of Example 13.3.1 that we have been looking at, we are dealing with log-returns. log-returns are often assumed to be normally distributed. Although the sample skewness for the dataset is positive, we have seen from the bootstrap confidence interval that it is possible that the population skewness is indeed zero, as in a normal distribution. In a parametric bootstrap setting, we start with the assumption that the dataset comes from a normal distribution. This is the assumed parametric model. We then estimate the mean and variance and that is the fitted distribution. The following R code generates $B = 1000$ bootstrap samples from this fitted distribution and obtains the histogram of skewness obtained from each bootstrap sample. The histogram of the skewness in the bootstrap samples

is presented in Fig. 15.2 with the sample skewness marked by the blue line. Note that the observed value of skewness lies within the central 95% of these values. Hence, we can use this to conclude that as far as skewness is concerned, there is not sufficient evidence to reject the hypothesis that the observed log-returns came from a normal distribution. Note that the 95% confidence interval obtained from the parametric bootstrap is more symmetric about the origin than the one obtained from the bootstrap interval obtained before. This is because we have assumed a symmetric distribution.

```
mu = mean(x)
sig = sd(x)
set.seed(1)
for(i in 1:B){
  y = rnorm(n,mu,sig)
  sb[i] = skewness(y)
}
hist(sb,breaks=20)
abline(v=0.8, col="blue")
quantile(sb,0.025)

## -1.043546

quantile(sb,0.975)

## 0.9517566
```

15.2.2 Bootstrap of Portfolio Returns

Here we present an example from finance where bootstrap is useful. Suppose we are interested in a portfolio of k stocks. The portfolio weights are adjusted once every day, and the quantity of interest is the terminal payoff. Of course, we can run this procedure over historical data and obtain the return of such a portfolio. The problem arises when we want to find the distribution of this return, in order to assess the performance of the portfolio in the future. If we want to do this parametrically, we shall need the joint distribution of the returns for the k assets. This can be done by assuming a multivariate normal, which is seen to be a very bad fit in empirical examples. Alternatively one can model using copulas. The first roadblock is to come up with an appropriate model. The second problem is estimation of the unknown parameters in the model, which becomes very challenging if the dimension k is large. Even for a four-dimensional problem, the number of parameters in the mean vector and the covariance matrix is 14.

As an alternative, we can generate bootstrap samples from the observed data, treating each time point as a unit, so as to preserve the relationship between the

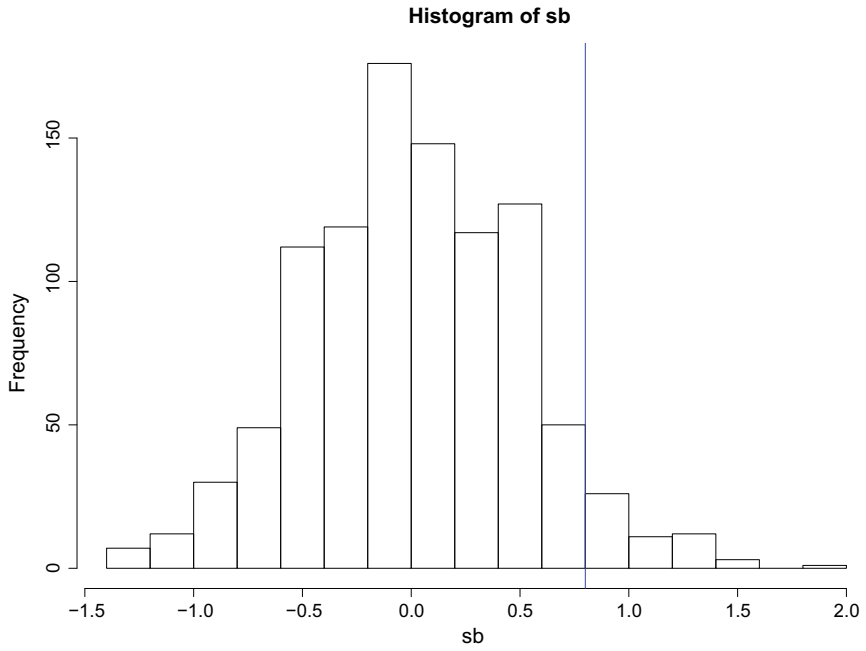


Fig. 15.2 Histogram of skewness in 1000 parametric bootstrap samples

assets. We demonstrate this using the example of 4 assets for the dataset introduced in Chap. 17.

Example 15.2.1 Consider the portfolio consisting of stocks of four companies IBM, Apple, Microsoft, and Intel. Using daily data we want to construct a portfolio with weights (0.1, 0.2, 0.3, 0.4). We rebalance the portfolio everyday to keep this weight and hold the portfolio from Feb 1st to Dec 31st, 2015. Using the R code below we find that the return of the portfolio is 1.984. Next we generate 1000 bootstrap samples and obtain the return for each. The histogram of the bootstrapped returns is presented in Fig. 15.3.

Since bulk of the bootstrap values lie to the right the observed value, it is very likely that the actual future return will be higher than 1.984.

```
x = stock_treasury[,c(3,5,7,9)]
x = as.ts(x)
xl = exp(diff(log(x),1))
weight = c(0.1,0.2,0.3,0.4)
for(i in 2:250){
  wealth = weight%*%(1+xl[i-1,])
  for(j in 1:4){
    weight[j]<-weight[j]*(1+xl[i-1,j])/wealth
  }
}
wealth    #1.984
set.seed(1)
for(k in 1:B){
  y = sample(seq(1:249),replace=T)
  xlb = xl[y,]
  weight = c(0.1,0.2,0.3,0.4)
  for(i in 2:250){
    wealth = weight%*%(1+xlb[i-1,])
    for(j in 1:4){
      weight[j] = weight[j]*(1+xlb[i-1,j])/wealth
    }
  }
  t[k] = wealth
}
hist(t)
```

15.3 Cross-Validation

In all the above, we have started with a model and tried to find distributions for parameter estimates. In contrast, sometimes we are interested in checking if the model itself is valid, or a good fit to the data. After fitting a model to a dataset, we can find the residual error. This could be a sum of squares or absolute values of the differences between the fitted and observed values for quantitative data. This could be the number of mismatches in a categorical scenario. In all these cases the measure of error is based on the same dataset that is used to fit the model. Hence it can be an underestimate of the actual error encountered for a new dataset. The idea of cross-validation is to leave out part of the dataset from the model fitting stage and then use that to evaluate the model.

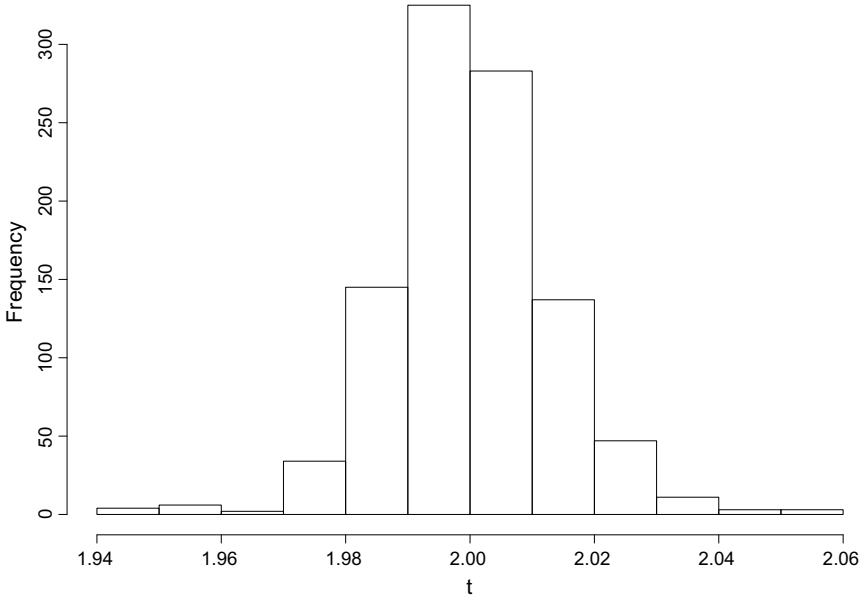


Fig. 15.3 Histogram of bootstrap samples of portfolio returns

There are several ways to do cross-validation. In leave-one-out approach, one leaves out a single observation and fits the model on all the others. The model is then evaluated on the left-out point and the discrepancy from the observed value is measured. This procedure is repeated for each of the observations and the average discrepancy is calculated. Another way is to divide the dataset into k subsets of roughly equal size, fit the model multiple times by leaving out one subset every time, and evaluate the performance on the left-out subset.

15.4 Exercises

1. Prove Equation (15.1).
2. Construct a portfolio with only two stocks, IBM and Apple with the data used in Example 15.2.1. Change the proportion(p) of wealth invested in IBM from zero to one in steps of size 0.05. For each choice of proportion, find the 95% bootstrap confidence interval for the return of the portfolio. Plot these intervals against p and see if there the length of the interval depends on the choice of p.
3. Generate a sample of size 10 from a χ^2_5 distribution. Suppose we are trying to estimate the mean with a confidence interval. Construct a 95% confidence interval for the mean based on the central limit theorem. Note that n is quite small. Check if the interval contains the true mean, 5. Now construct a bootstrap interval based on 100 bootstrap samples and perform same check. Carry out the above procedure 1000 times and verify which of the intervals have coverage closer to 95%.

Chapter 16
Statistical Risk Analysis



Risk is an inherent quantity in financial markets and is of utmost importance to investors and policymakers. Many methods have been proposed to measure, monitor, and control the risks. There are different kinds of risks. Market Risk is the risk to the portfolio from a decline in the market price of instruments in the portfolio. Liquidity Risk is the risk that the holder of an instrument will find that a position is illiquid, and will incur extra costs in unwinding the position resulting in a less favorable price for the instrument. Credit Risk involves the risk that promised payments on a loan or bond will not be made, or that a convertible instrument will not be converted in a timely manner or at all. Other types of risk include interest rate and foreign exchange risks that can also randomly change over time and are of relevance to an investment portfolio. Tools have evolved to measure all these different components of risk. Processes must be put into place to monitor the changing risks in a portfolio, and to control the magnitude of risks. For an extensive treatment of these topics, see Litterman et al. (1998). In this chapter we shall discuss only market risks, see Dowd (2007).

Risk and return play central and supplementary roles in financial theory. There is a need to measure risk in order to facilitate comparisons between investments with comparable returns. Markowitz (1952) formalized the trade-off between risk and return, by setting risk equal to the variance of the portfolio returns. The use of the variance as a risk measure has been debated and strongly questioned [see, e.g., Michaud (1989)]. To estimate the variance, all data are used. But in terms of risk, only the lower values of returns carry meaningful information. Another well-known criticism is that the variance is a proper measure of risk only for Gaussian distributions. The empirical evidence is that financial returns have heavier than Gaussian tails and are often skewed, making the normal assumptions unsuitable.

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/978-981-19-2008-0_16.

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

R. Sen and S. Das, *Computational Finance with R*, Indian Statistical Institute Series,
https://doi.org/10.1007/978-981-19-2008-0_16

The principle of safety-first was first propounded by Roy (1952). This led to the gradual development of downside risk measures that are common in today's financial industry. Downside risk is the risk associated with a decline in the value of an asset as a result of changes in market conditions.

16.1 Value at Risk

The most commonly reported measure of risk is value at risk (VaR). The VaR of a portfolio is the amount risked over some period of time with a fixed probability. It is of practical relevance to financial and insurance institutions. Furthermore, VaR forecasts are often used as a testing ground for fitting alternative models to financial returns. VaR-based risk management already emerged as common market practice in the early 1990s. In the 1996 Market Risk Amendment of the original Basel Accord, regulators chose VaR as the regulatory reporting tool. For a comprehensive review of value at risk see Philippe (2001).

The VaR of a portfolio measures the value which an investor would lose with some fixed small probability, usually between 1 and 10%, over a selected period of time. Because the VaR represents a hypothetical loss, it is usually a positive number. Specifically, the α-VaR of a portfolio is defined as the largest number such that the probability that the loss in portfolio value over some period of time is greater than the VaR is α.

We denote by P_t the value of the portfolio at time t. We fix a small probability α, the time t at which we want the measure and a time horizon Δt. Under these conditions, the associated required capital RC_t is defined as the extra capital needed to guarantee that the portfolio value will be negative at time $t + \Delta t$ with probability no greater than α. In other words, RC_t is defined by the equation:

$$\mathrm{P}(P_{t+\Delta t} + \mathrm{RC}_t < 0) = \alpha. \tag{16.1}$$

The VaR at time t is then defined as the sum of the current endowment and the required capital

$$\mathrm{VaR} = P_t + \mathrm{RC}_t. \tag{16.2}$$

Notice that $\mathrm{P}(P_{t+\Delta t} - P_t < -\mathrm{VaR}) = \alpha$, which implies that α-VaR is the negative of the α-th quantile of the distribution of the change $P_{t+\Delta t} - P_t$ in the value of the portfolio over the given horizon. As an example, if for a particular asset, the probability of a loss greater than \$ 1M in the next one year is 5%, then the 5% VaR is \$ 1M. Here Δt is 1 year and α is 5%.

It is often more convenient to express VaR in units of P_t, that is, to set $\mathrm{VaR} = \widetilde{\mathrm{VaR}} * P_t$. Expressed in units of P_t, the negative of the value at risk is the α-quantile of the distribution of the raw returns over the period in question. Note that raw returns and log-returns are very close to each other for small relative changes, and we have:

$$r_t = \frac{P_{t+\Delta t} - P_t}{P_t} \sim \log\left(\frac{P_{t+\Delta t}}{P_t}\right), \tag{16.3}$$

which justifies the fact that one often defines α-VaR at the horizon Δt, as the negative of the α-quantile of the distribution of the log-returns over a period of length Δt.

In the rest of this section we discuss the estimation of VaR. The 'edhec' dataset included with the R library `PerformanceAnalytics` will be used to illustrate the methods. The data is CSV conformed into an xts object with monthly observations starting from January 1997 and is periodically updated to include recent observations. The 13 indices included in this dataset can be found in Table 16.1.

16.1.1 Gaussian Estimation

The simplest form of VaR specifies a parametric model for the distribution of returns and derives the VaR from the α-quantile of this distribution. The most common distributional assumption is normal, that is, $r_t \sim \mathcal{N}(\mu, \sigma^2)$. The α-VaR is $-\mu - \sigma\Phi^{-1}(\alpha)$. The parameters can be directly estimated using maximum likelihood with the usual estimators: $\bar{R}$ for the mean and $\hat{\sigma^2}$ for the variance. The parametric VaR is thus calculated by

$$\text{VaR} = -\bar{R} - \hat{\sigma} * z_\alpha, \tag{16.4}$$

where z_α is the α-th quantile of the standard normal distribution.

The following code illustrates the use of the `VaR` function of the R library `PerformanceAnalytics` to find the VaR of the index *Convertible Arbitrage* at α=5% for one month horizon at August 2009 using all the available historical data (a). This is compared with the direct evaluation (b) using Eq. (16.4).

```
install.packages("PerformanceAnalytics")
library(PerformanceAnalytics)
data("edhec")
Rt = edhec$'Convertible Arbitrage'
a = VaR(Rt, p = 0.95, method = "gaussian", invert=FALSE)
b = -mean(Rt)-sd(Rt)*qnorm(.05)
```

The resulting values of a and b are 0.02125694 and 0.02130572, respectively. The minor difference is because the VaR program does not use the small sample correction (replacing n by $n-1$) in the estimation of the variance. The VaR of all the indices in the 'edhec' dataset is presented in Table 16.1.

Note that *Short Selling* is the riskiest index followed by *Emerging Markets*, while *Equity Market Neutral* carries the lowest risk. We now look at the time evolution of VaR for these three indices. For each month, we use a rolling window of the previous 5 years to estimate the VaR. The results are presented in Fig. 16.1. Notice that until 2008, for the first two the VaR goes down while for the last it stays steady. After

Table 16.1 Gaussian Value at risk for different indices at α=5% for one month horizon at August 2009 using monthly returns starting January 1997. The indices are arranged in increasing order of VaR

Type of index	VaR
Equity Market Neutral	0.0088
Merger Arbitrage	0.0115
Relative Value	0.0149
Fixed Income Arbitrage	0.0190
Global Macro	0.0202
Distressed Securities	0.0221
Event Driven	0.0225
Funds of Funds	0.0239
Convertible Arbitrage	0.0265
Long/Short Equity	0.0286
CTA Global	0.0347
Emerging Markets	0.0550
Short Selling	0.0862

2008, the VaR goes up for all the indices with the first having the least increase. The codes for the table and the figure are given below.

```
RRt = as.matrix(edhec)
cc = colnames(edhec)
c  =  VaR(RRt, p = 0.95, method = "gaussian", invert=FALSE)
d = matrix(0,93,13)
for(i in 1:93){
  j = i+59
  d[i,] = VaR(RRt[i:j,], p = 0.95, method = "gaussian", invert=FALSE)
}
e = index(edhec)[60:152]
plot(e,d[,1],ylab=cc[1],xlab='date',pch=20)
```

In a general parametric VaR model, some distribution for returns which depends on a set of unknown parameters θ is assumed; that is, $r_t \sim F_\theta$ and parameters are estimated by maximum likelihood. The VaR is then $-F_{\hat{\theta}}^{-1}(\alpha)$, the negative of the α-quantile of the estimated distribution. Non-normal parametric VaR estimation utilizes a different distribution for the losses to better account for the possible fat-tailed nature of downside risk. We shall discuss more on this in Sect. 16.1.4.

16.1.2 Modified and Robust Estimation

If the return series is skewed and/or has excess kurtosis, Cornish–Fisher estimates of VaR can be more appropriate. Zangari (1996) provides a modified VaR calculation

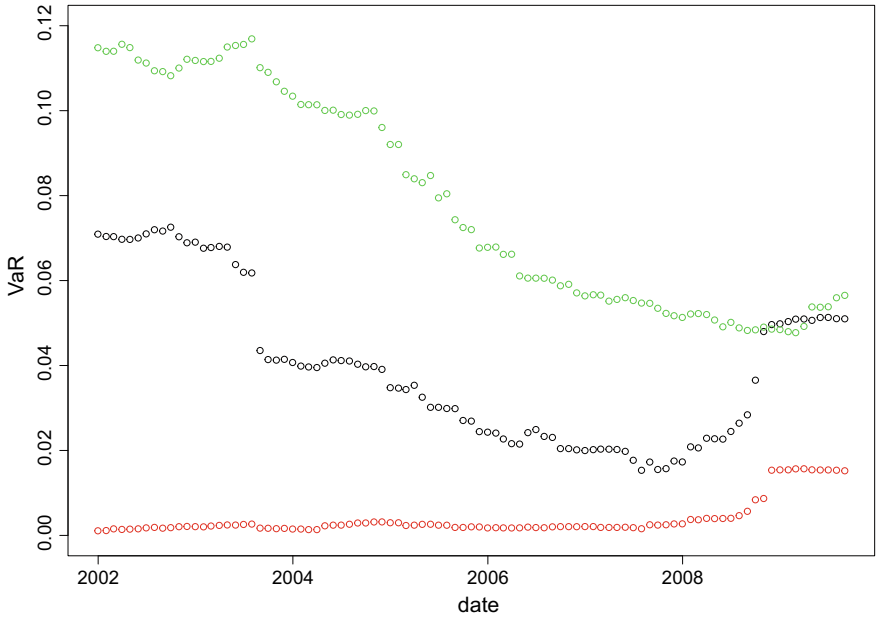


Fig. 16.1 Time evolution of Gaussian VaR of *Short Selling* (green), *Emerging Markets* (black) and *Equity Market Neutral* (white) at $\alpha = 5\%$ for one month horizon using monthly returns for previous five years

that takes the higher moments of non-normal distributions into account through the use of a Cornish–Fisher expansion. This coincides with the standard VaR if the returns follow a standard normal distribution. This measure is now widely cited and used in the literature, and is usually referred to as Modified VaR. The modified VaR is defined in the following manner:

$$\begin{aligned} z_{cf} &= z_c + [(z_c^2 - 1)S]/6 + [(z_c^3 - 3z_c)K]/24 - [(2z_c^3 - 5z_c)S^2]/36, \\ \text{VaR} &= -\bar{R} - \hat{\sigma} * z_{cf}, \end{aligned} \tag{16.5}$$

where S is the skewness and K is the excess kurtosis of R.

As discussed in Cont et al. (2010), it is important that the estimation of the VaR measure is robust to single outliers. This is especially the case for modified VaR and its decomposition, since they use higher-order moments. By default, the portfolio moments are estimated by their sample counterparts. The R library `robustbase` allows for data-cleaning in such situations. If `clean="boudt"`, then the $1 - p$ most extreme observations are winsorized if they are detected as being outliers, for a suitably chosen p. For more information, see Boudt et al. (2008). If the data consists of returns for highly illiquid assets, then `clean="geltner"` may be more appropriate to reduce distortion caused by autocorrelation. The Geltner autocorrelation for the adjusted return series may be calculated via:

```
Geltner.returns = [R(t) - R(t-1)*acf(R(t-1))]/1-acf(R(t-1))
```

where `acf(R(t-1))` is the first-order autocorrelation of the return series. The result is the return series adjusted for first-order autocorrelation.

16.1.3 Historical Simulation

The parametric model-based estimates are efficient if a correct assumption is made on the return distribution. Yet finding a specification which necessarily includes the true distribution is difficult or impossible. At the other end of the spectrum is a pure nonparametric estimate of the VaR using the α-quantile of the empirical distribution of the returns. This method is also popularly known as historical simulation (HS).

Consider that r_t , the return at time t, has been observed for $t = 1, \ldots, T$. The HS procedure for forecasting the VaR as is follows

$$\hat{\text{VaR}}_t^{\text{HS}} = -R_{(\omega)},$$

where $R(\omega)$ is the ω-th order statistic of returns, with $\omega = [T\alpha]$. HS is by far the most popular procedure for forecasting VaR among commercial banks. For instance, Pérignon and Smith (2010) document that almost three quarters of the banks that disclose their VaR method report using HS.

These estimates are rough, and a single new data point may produce very different VaR estimates. Smoothing the estimated quantiles generally reduces the variance of the estimate when compared to ones calculated directly based on the sorted returns. This is particularly true if the sample is small. Žiković and Aktan (2011) proposed forecasting the VaR using weighted historical simulation, by introducing an exponentially weighted moving average into the VaR forecast so that the most recent observations have larger weights, with a properly chosen smoothing parameter.

The advantage of nonparametric estimates of VaR is that they are generally consistent under very weak conditions and that they are trivial to compute. The disadvantage is that the VaR estimates can be poorly estimated or equivalently, that very large samples are needed for estimated quantiles to be accurate, particularly for small values of α.

Next we move on to a comparative study of the different estimation methods described so far using all 13 indices. Robustness issues discussed in Sect. 16.1.2 are incorporated through library `robustbase`. The three methods, namely Gaussian, modified, and historical, are compared. Thus there are 9 VaR estimates (3 methods and 3 kinds of robustness) for each index. The code is given below, and the results are presented in Fig. 16.2. Notice that the historical estimates are always highest, and the modified estimates are mostly lowest. Also, `"boudt"` correction for outliers

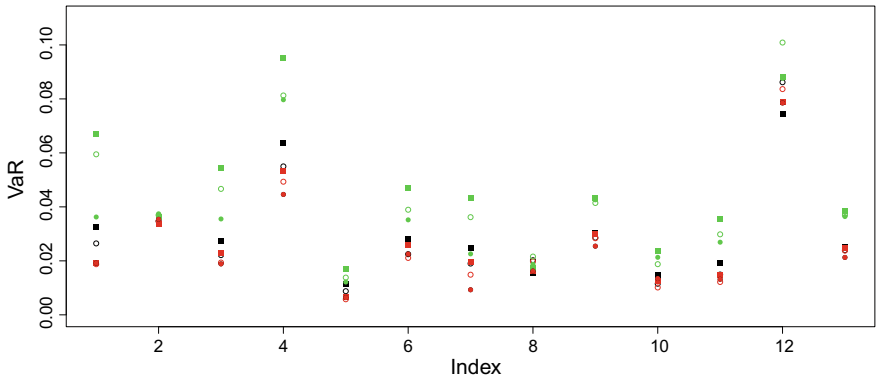


Fig. 16.2 Estimated VaR for 13 indices (on the x-axis) at $\alpha = 5\%$ for one month horizon at August 2009 using monthly returns starting January 1997. The colors are for different methods, namely Gaussian (black), modified (red), and historical (green). Symbols are for different cleaning procedures, namely "none" (circle), "boudt" (square), "geltner" (dot)

leads to higher estimates in most cases. More on the relative merits of the different methods will be discussed in Sect. 16.5.

```
install.packages("robustbase")
library(robustbase)
md = c("gaussian", "modified", "historical")
cn = c("none","boudt","geltner")
VaRk = matrix(0,3,3)
for(i in 1:3){
     for(j in 1:3){
       Rc = Return.clean(R, method = cn[j], alpha = .01)
       VaRk[i,j]<-VaR(Rc, p = 0.95, method = md[i], invert=FALSE)
     }
}
```

16.1.4 EVT-Based Estimation

Extreme value theory (EVT) is the branch of probability theory that deals with the extreme high and low values of a random variable. In particular, it gives results on the distribution of maxima (minima) of a sequence of random variables and the approximations to the distribution of a random variable beyond a given threshold. Details are given in Appendix C. From these results, it can be concluded that there exists a threshold u_0 below which the distribution of the returns can be well-approximated by a generalized Pareto distribution (GPD). The task is then to

- find the threshold
- estimate the parameters of the GPD
- use quantiles of this distribution to estimate the value at risk.

A common method of finding the threshold is to find relative stable regions of the Hill plot. We shall now describe this procedure. Note that the distribution of returns beyond a threshold is approximately a GPD. For $\xi > 0$, the cdf of a (GPD) is given by

$$G_{\xi,\beta}(x) = 1 - \left(1 + \frac{\xi x}{\beta}\right)^{-1/\xi}. \tag{16.6}$$

The Hill tail index estimator of ξ is given by

$$\hat{\xi}(k) = \left(\frac{1}{k}\sum_{i=n-k+1}^{n}\left(\log(X_{(i)}) - \log(X_{(n-i+1)})\right)\right)^{-1}, \tag{16.7}$$

where $X_{(i)}$ is the i-th order statistic of $X_1, \ldots, X_n$ and the estimator is based on the highest k order statistics. The plot of $\hat{\xi}(k)$ on k is called the Hill plot. A range of values of k for which the estimator does not fluctuate too much indicates a good fit to the GPD.

The R library `QRM` can be used to obtain the Hill plot, then estimate the parameters of the GPD by maximum likelihood and finally to obtain the quantile of the fitted distribution as the estimated VaR.

The following code takes a return series R, cleans it using the boudt method, and plots the Hill plot as shown in the left panel of Fig. 16.3. From the Hill plot $k = 25$ seems to be an appropriate threshold. Using this, the code fits a GPD. The goodness of the fit is assessed in the right panel of Fig. 16.3. Finally estimate of the VaR is obtained from the fitted GPD as r. Running this code with the index *Convertible Arbitrage* at α=5% for one month horizon at August 2009 using monthly returns starting January 1997, we get $r = 0.022710$. All estimates for this particular return series are presented in Table 16.2.

```
install.packages("QRM")
library(QRM)
Rc = Return.clean(R, method = cn[2], alpha = .01)
hillPlot(-Rc)
out = fit.GPD(-Rc,nextremes=25)
r = RiskMeasures(out,.95)[2]
```

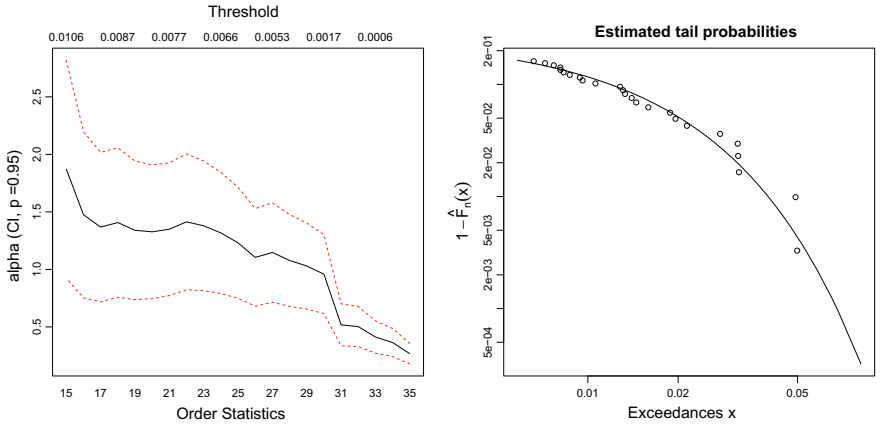


Fig. 16.3 Left: Hill plot of monthly returns index *Convertible Arbitrage* from January 1997 to August 2009. Right: the estimated tail probabilities according to the GPD fitted model (line) and the corresponding empirical probabilities (dots)

Table 16.2 Estimated value at risk for index *Convertible Arbitrage* at $\alpha = 5\%$ for one month horizon at August 2009 using monthly returns starting January 1997

	None	Boudt	Geltner
Gaussian	0.026458	0.032474	0.019160
Modified	0.018766	0.019282	0.019160
Historical	0.059468	0.067021	0.036242
EVT	0.022741	0.022710	0.037922
Threshold	25	25	33

Cleaning procedure is in the row, and the method is in the column

16.2 Expected Shortfall

The obvious benefit of VaR is that it is easy and intuitive. However, as a risk measure, the VaR has been criticized for not being coherent [see Artzner et al. (1999)]. In particular, the VaR is not subadditive, given that the VaR of a diversified portfolio could be greater than the sum of the VaRs of the individual portfolios.

Expected shortfall (ES) is a coherent risk measure. While keeping the spirit of VaR in considering only rare losses, it takes into account the actual sizes of these losses, something that VaR does not do. It is also known as conditional value at risk and is defined as the expected value of the portfolio loss given a value-at-risk exceedance has occurred. That is

$$\text{ES} = E(r_t | r_t < -\text{VaR}). \tag{16.8}$$

All the methods of estimation of VaR carry over to that of ES. The same code as that in Sect. 16.1.1 can be used to estimate the expected shortfall by replacing

`VaR` with `ES`. In this case also, *Short Selling* is the riskiest index (ES = 0.109118) followed by *Emerging Markets* (ES = 0.071054), while *Equity Market Neutral* carries the lowest risk (ES = 0.012513).

Advantages of ES:

- It is a coherent risk measure.
- It takes into account the actual sizes of the losses.

Disadvantages of ES:

- Computation involves integrals which, in most cases, need to be evaluated numerically.
- Estimation involves the entire tail of the distribution involving very rare events and is hence inefficient.

Although the ES is a sound risk measure and preferable to VaR from a theoretical viewpoint, the standard errors of ES estimates are by a multiple larger than standard errors of VaR estimates. Thus, although neglecting the severity of losses in the tail of the distribution for VaR results in a theoretically flawed risk measure, back-testing becomes easier because empirical quantiles are robust to extreme outliers, unlike typical estimators of ES. To get some idea about the impact of estimation error, we carry out a simulation exercise. VaR and ES are calculated from a sample of 1000 random observations following a generalized Pareto distribution with shape parameter 0.6. For simplicity, we set $\sigma = 1$ and $\mu = 0$. To obtain the error estimates, the VaR and ES calculations are performed 1000 times. The histograms of the estimates are presented in Fig. 16.4. These histograms represent the sampling distribution of the two measures. Estimation is done using both Gaussian and EVT methods to show that the effect is independent of the estimation method used. It is clear that the sampling distribution of VaR is far more concentrated than that of the ES. Hence, when the loss distribution is fat-tailed, ES estimates are more affected by whether large and infrequent losses are realized in the simulation, given that the ES considers the entire right tail of the loss distribution. Yet, large and infrequent losses have less effect on VaR because they do not take losses beyond the VaR level into account. Therefore, the ES estimate becomes more volatile when the probability of infrequent and large losses is high, resulting in an estimation error significantly larger than the error of the VaR estimate. An alternative measure that takes into account the actual size of losses as ES but is not affected too much by extreme losses is the median shortfall. This is the median of the portfolio loss given that a value-at-risk exceedance has occurred.

16.3 Other Measures of Risk

The **mean absolute deviation** (MAD) is the sum of the absolute value of the difference between the returns and the mean of the returns divided by the number of returns. This is a two-sided measure like variance, but more robust to extreme values.

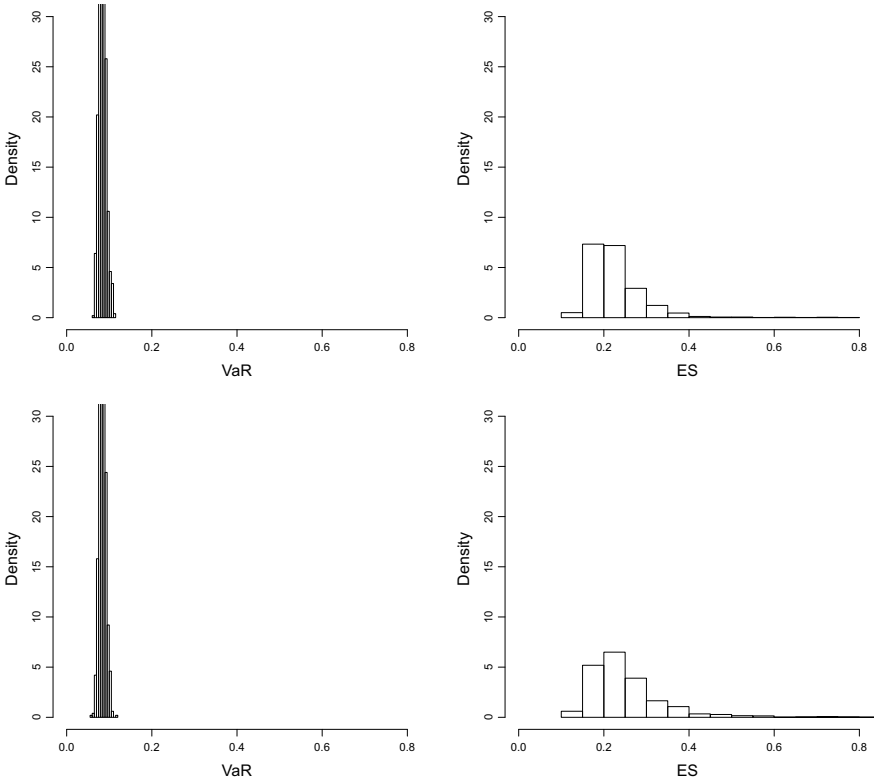


Fig. 16.4 Histogram of estimated VaR (left) and ES (right) using Gaussian (top) and EVT (bottom) methods

$$\frac{\sum_{i=1}^{n} \mid r_i - \bar{r} \mid}{n}. \tag{16.9}$$

The following measures arise from the Capital Asset Pricing Model (CAPM) and are hence related to a benchmark security. **Systematic risk** as defined by Bacon (2008) is the product of the CAPM beta by market risk. **Market risk** is the standard deviation of the benchmark. **Specific risk** is the standard deviation of the error term in the CAPM regression equation.

The following measures are based on drawdown. Any time the cumulative returns dip below the maximum cumulative returns, it is called a drawdown. Drawdowns are measured as percentage of the maximum cumulative return, in effect, measured from peak equity. The drawdown of the three indices *Short Selling*, *Emerging Markets*, and *Equity Market Neutral* are presented in Fig. 16.5. The **pain index** is the mean value of the drawdowns over the entire analysis period. Visually, the pain index is the area of the region that is enclosed by the horizontal line at zero percent and the drawdown line in the drawdown chart.



Fig. 16.5 Drawdown of the three indices *Short Selling*, *Emerging Markets* and *Equity Market Neutral*

The following measures are based on a minimum acceptable return (MAR). **Downside variance** is defined as

$$\frac{\sum_{i=1}^{n} \min(R_t - \text{MAR}, 0)^2}{n}. \tag{16.10}$$

The square root of this is the downside deviation, and the sum without the square is the downside potential.

The different risks for the indices in the *edhec* database at 5% are computed using the following code and reported in Table 16.3.

```
sd(edhec[,c(1)])
MeanAbsoluteDeviation(edhec[,c(1)])
SystematicRisk(edhec[,c(1)], edhec[,5])
SpecificRisk(edhec[,c(1)], edhec[,5])
DownsideDeviation(edhec[,c(1)])
PainIndex(edhec[,c(1)])}
```

For a more comprehensive overview of different risk measures see Bacon (2008).

16.4 Ratios to Compare Portfolios

Now that we have discussed the different measures of risk and how to estimate them, it is time to see how to put them together with the returns to arrive at an optimal portfolio. The **Sharpe ratio**, see Sharpe (1994), is simply the adjusted return per unit of risk, represented by standard deviation. The higher the Sharpe ratio, the

Table 16.3 Different risk measures for the indices

	sd	MAD	Systematic risk	Specific risk	Downside deviation	Pain index
Convertible arbitrage	0.020047	0.011909	0.034306	0.060182	0.014705	0.025157
CTA global	0.025131	0.020172	0.015335	0.085412	0.013718	0.023309
Distressed securities	0.018348	0.012699	0.037327	0.051274	0.011877	0.024272
Emerging markets	0.038571	0.027599	0.066253	0.115650	0.026932	0.080774
Event-driven	0.018350	0.012805	0.040154	0.049118	0.012110	0.022450
Fixed income arbitrage	0.014171	0.007394	0.020799	0.044321	0.011564	0.017837
Global macro	0.017020	0.012692	0.032762	0.048855	0.006839	0.010357
Long/short equity	0.022174	0.017033	0.048364	0.059478	0.012786	0.031118
Merger arbitrage	0.011168	0.007853	0.021156	0.032284	0.006675	0.006876
Relative value	0.013195	0.008820	0.028070	0.035954	0.008724	0.012539
Short Selling	0.055099	0.040748	0.058124	0.181205	0.034220	0.219271
Funds of funds	0.018212	0.012783	0.043181	0.045843	0.010888	0.023955

better the combined performance of risk and return,

$$\frac{\bar{R} - R_f}{\hat{\sigma}}, \tag{16.11}$$

where R_f is the risk-free return. Instead of $\hat{\sigma}$, VaR and ES are also used in alternative forms of Sharpe ratio.

The **Treynor ratio** is similar to the Sharpe ratio, except it uses the CAPM beta as the denominator.

Both the **Calmar** and the **Sterling ratio** are the ratios of annualized return over the absolute value of the maximum drawdown of an investment. The Sterling ratio adds an excess risk measure to the maximum drawdown, traditionally and defaulting to 10%. The **Burke ratio** is the ratio of the excess portfolio return, and the unscaled (no division by n) drawdowns deviation. The following code obtains the return risk ratios for the index *Convertible Arbitrage*. The ratios for the different indices are presented in Table 16.4.

Table 16.4 Ratios to compare portfolios for the different indices

	VaR sharpe	ES sharpe	Treynor	Calmer	Sterling	Burke
Convertible arbitrage	0.197344	0.064377	0.070039	0.263148	0.196136	0.244768
CTA global	0.191983	0.151475	0.156055	0.656951	0.353885	0.352683
Distressed securities	0.289290	0.130656	0.081497	0.425374	0.296172	0.377407
Emerging markets	0.129592	0.064479	0.044080	0.260187	0.203599	0.178912
Event-driven	0.271016	0.128982	0.072403	0.464056	0.309791	0.397582
Fixed income arbitrage	0.171437	0.071086	0.076011	0.283429	0.181766	0.226445
Global macro	0.495552	0.386847	0.089708	1.189059	0.525630	0.749010
Long/short equity	0.255469	0.164783	0.060644	0.430870	0.295461	0.368122
Merger arbitrage	0.456319	0.203972	0.123457	1.485945	0.535500	0.868805
Relative value	0.347861	0.137094	0.091486	0.516391	0.317325	0.461843
Short Selling	0.055994	0.046119	0.017527	0.065886	0.054824	0.050355
Funds of funds	0.236467	0.128144	0.051491	0.346116	0.232974	0.310710

```
SharpeRatio(edhec[,c(1)])[2:3]
TreynorRatio(edhec[,c(1)], edhec[,c(5)])
CalmarRatio(edhec[,c(1)])
SterlingRatio(edhec[,c(1)])
BurkeRatio(edhec[,c(1)])
```

16.5 Back-Testing VaR Models

Given a VaR model, we want to test if it is valid. Back-testing is a set of statistical procedures designed to check if the real losses are in line with VaR forecasts. Whatever the type of use of VaR, the VaR forecasts are generated based on a learning sample. The model is used to produce a sequence of out-of-sample VaR forecasts for a test sample. The back-testing procedure is based on the comparison of the observed returns to these VaR forecasts.

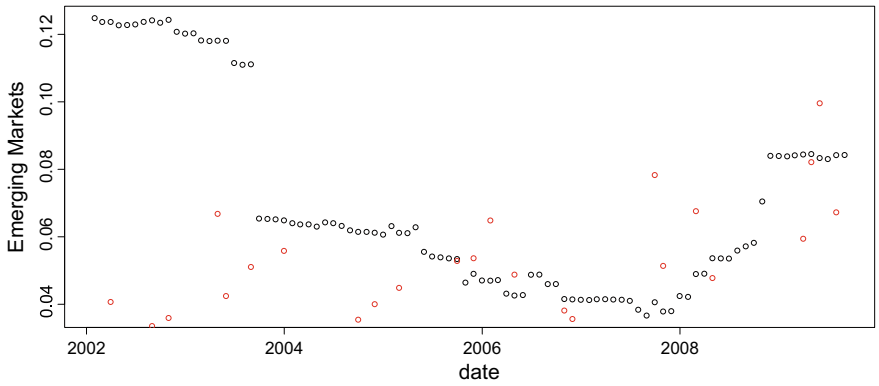


Fig. 16.6 Back-test: one-step ahead VaR using Geltner adjusted modified method (black) and actual observed returns (red) for the index *Emerging Markets*

For the index *Emerging Markets* the plot of the one-step ahead VaR and actual observed returns using Geltner adjusted modified method using last 5 years data for the 92 time points from Jan 2002 to Aug 2009 is shown in Fig. 16.6. Observe that there are 7 incidences where the red dot is above the black dot indicating a VaR exceedance.

In the above situation, the 9 VaR estimates (3 methods and 3 kinds of robustness) with one-step forward VaR lead to the following number of exceedances.

$$17\ 17\ 12\ 17\ 17\ 7\ 15\ 15\ 9$$

The expected number of exceedances is 5% of 92, that is 4.6. So we see that all the procedures exhibit more than expected number of exceedances. We now need to do a formal test to check if the exceedance probability is indeed 5%. The simplest such test is called the unconditional coverage (UC) test proposed by Kupiec (1995) and is a binomial likelihood ratio test. We denote $I_t(\alpha)$ the hit variable associated with the ex-post observation of an αVaR exception at time t:

$$I_t(\alpha) = \begin{cases} 1 \text{ if } \quad r_t < -\text{VaR}_{t|t-1}(\alpha), \\ 0 \text{ otherwise.} \end{cases} \tag{16.12}$$

Note that if we assume that the VaR exceptions on different days are independent, then $H = \sum_{t=1}^{n} I_t$ follows a binomial (n, α) distribution. We are interested in testing the null hypothesis that α =5% versus the alternative $\alpha \neq 5\%$. The likelihood ratio statistic can be derived as

$$\text{LR}_{\text{UC}} = \frac{(1-\alpha)^{n-H}\alpha^{H}}{(1-H/n)^{n-H}(H/n)^{H}}, \tag{16.13}$$

and from general results $-2\log \text{LR}_{\text{UC}}$ has an asymptotic χ_1^2 distribution.

For example, the one-sided p-value for the count 12 is 0.006 and that for the count 9 is 0.122. So for all the measures except Geltner adjusted modified and Geltner adjusted historical, we reject the null hypothesis and conclude that the VaR is not estimated correctly by the respective methods. In fact, in all such cases the VaR exceptions are higher than expected implying that the VaR is underestimated.

A crucial assumption in the above analysis is the independence of VaR exceedances, whereas empirically it is observed that these exceedances occur in clusters. An alternative is provided by the conditional coverage test proposed by Christoffersen (1998) which assumes that the violation process can be represented as a Markov chain with two states and the transition matrix

$$\Pi = \begin{pmatrix} 1-\pi_{01} & \pi_{01} \\ 1-\pi_{11} & \pi_{11} \end{pmatrix}, \tag{16.14}$$

where $\pi_{ij} = \mathrm{P}(I_t(\alpha) = j \mid I_{t-1}(\alpha) = i)$. The conditional likelihood ratio statistic, in this case is, can be derived as

$$\mathrm{LR}_{\mathrm{CC}} = \frac{(1-\alpha)^{n-H}\alpha^{H}}{(1-\hat{\pi}_{01})^{n_{00}}\hat{\pi}_{01}^{n_{01}}(1-\hat{\pi}_{11})^{n_{10}}\hat{\pi}_{11}^{n_{11}}}, \tag{16.15}$$

where n_{ij} are the observed transition counts and $\hat{\pi}_{ij} = n_{ij}/\sum_k n_{ik}$. In this case $-2\log \mathrm{LR}_{\mathrm{CC}}$ has an asymptotic χ^2_2 distribution.

16.6 Exercises

1. For the `Edhec` data, compute the expected shortfall for each index at $\alpha = 5\%$.
2. For the three indices Short Selling, Emerging Markets, and Equity Market Neutral of the `Edhec` data, plot the rolling window expected shortfall at $\alpha = 5\%$ for one month horizon using monthly returns for previous five years. Compare with Fig. 16.1.
3. Prove Eq. 16.13.
4. For the index Emerging Markets, compute the conditional likelihood ratio statistic for the one-step ahead VaR calculated at 5% based on previous 5-years data using the Historical method with no robustness check. Carry out the test using the χ^2 distribution and compare to the unconditional result presented in Sect. 16.5.

Chapter 17
Supervised Learning



Machine learning is going to play an ever-increasing role in the landscape of computational finance. The increasing market share of the 'Algorithmic Trading' is an example of the growing use of machine learning in finance. However, 'Algorithmic Trading' is one application of machine learning. There are several applications of machine learning in quantitative finance.

In this chapter, we discuss the central ideas of statistical machine learning (SML) in context of quantitative finance. The SML is an outcome of the natural intersection of both Computer Science and Statistics, see Mitchell (2006), UC Berkeley (2018).
Learning: What is known as *parameter estimation* or estimation of *unknown functions* in Statistics, is known as '**learning**' in Computer Science. That is, SML is concerned with learning about unknown from data.
Supervised Learning: A learning task that uses labels (aka. dependent variable, response variable, or target variable) to perform the learning activity is called *supervised learning*. The objective of a learning task associated with a label is to predict the response of a new test point. Supervised learning tasks where the response variable is a discrete quantity are called *classification* tasks. However, when the response is continuous, the learning task is called *regression*. For example, credit card companies use supervised learning to identify good customers. They use the potential customer's demographic profile and credit history as predictors or features in their classification task. In another example, an application that performs fraud detection based on transaction characteristics is an example of supervised learning.

17.1 Framework for Statistical Learning

Statistical learning theory presents the ideas and the analytical structure for the study of machine learning algorithms, see Sambasivan et al. (2020). Here, we review the fundamental conceptual approaches applied in statistical learning. The purpose of

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/978-981-19-2008-0_20.

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

R. Sen and S. Das, *Computational Finance with R*, Indian Statistical Institute Series,
https://doi.org/10.1007/978-981-19-2008-0_20

statistical learning theory is to study the characteristics of learning algorithms in a statistical framework, see Bousquet et al. (2004). The statistical learning theory is rooted in the Bayesian decision theoretic framework, see Berger (1993).

17.1.1 Bayesian Decision Theoretic Framework

The estimation of a function (which explains the relationship between responses and predictors) from a given dataset is the fundamental problem of statistical learning theory. The data for the learning problem consists of predictors x with labels y. The predictor space is $\mathcal{X}$, and the response space is $\mathcal{Y}$. When the learning problem is classification, the response space is finite, e.g., in a binary classification problem, $\mathcal{Y} = \{1, 0\}$. When the learning problem is regression, the response space is infinite, $\mathcal{Y} \subseteq \mathbb{R}$. The dataset D consists of (x, y) tuples from an *unknown* joint probability distribution $P(x, y)$. We consider a function $f(x)$ for predicting response y for given values of the predictor x. It is not reasonable to consider all possible functions. We require some specifications on the class of functions that we want to consider for the learning problem.

The class of functions analyzed for the learning problem is called the *hypothesis class or class of models*, $\mathcal{F}$. The hypothesis class $\mathcal{F}$ could be finite or infinite. Consider a function $f \in \mathcal{F}$. To predict response y, an algorithm $\mathcal{A}$ is applied to select the best candidate from $\mathcal{F}$. To obtain the best f, $\mathcal{A}$ measures the *loss* function $L(f(x), y)$. The *loss* function L is a performance indicator for $f(x)$. It measures the loss due to the error in the prediction from $f(x)$ against the true response y. We obtain the expected value of the *loss* to characterize the performance of f. Since the loss, $L(f(x), y)$, is a random variable. The expected value of the loss is called the *risk* and is defined as:

$$R(f) = \mathbb{E}[L] = \int L(f(x), y) P(\mathrm{d}x, \mathrm{d}y). \tag{17.1}$$

By conditioning on x, we can present Eq. (17.1) as

$$R(f) = \mathbb{E}[L] = \mathbb{E}_x \mathbb{E}_{y|x}(L(f(x), y)|x),$$

the $R(f)$ is also known as the posterior expected loss, see Berger (1993) [Chap. 4]. It is sufficient to minimize the risk $R(f)$ point wise as

$$f(x) = \operatorname{argmin} E_{y|x}(L(f(x), y)|x). \tag{17.2}$$

The solution $f(x)$ in (17.2) is known as the Bayes estimator under loss function L, see Berger (1993) [Chaps. 1 and 4] for more detail. When y assumes the set of K possible classes, i.e., the values in $\mathcal{Y} = \{1, 2, \ldots, K\}$. The loss function $\mathbf{L}$ would be defined as a $K \times K$ matrix. The (k, j)th elements of the loss matrix $\mathbf{L}$ is

$$L(k, j) = \begin{cases} 0 & k = j \\ l_{kj} & k \neq j, \end{cases}$$

where $l_{kj} \geq 0$ is the loss for classifying an observation y_j wrongly to y_k. A conventional choice of $\mathbf{L}$ is the zero-one loss function, where all misclassification is penalized as a single unit. We can write the risk as

$$R(f) = \mathbb{E}_x \sum_{k=1}^{K} L[y_k, \hat{f}] P(y_k|x),$$

and it is sufficient to minimize $R(f)$ point wise:

$$R(f) = \underset{f \in \mathcal{Y}}{\operatorname{argmin}} \sum_{k=1}^{K} L[y_k, \hat{f}] P(y_k|x).$$

With the $0-1$ loss function it reduces to

$$f(x) = \max_{y \in \mathcal{Y}} P(y|x).$$

The resulting solution is known as *Bayes classifier*, see Friedman et al. (2009) [Chap. 2] and Berger (1993). The method mentioned above classifies a point to the most probable class, using the posterior probability of $P(y|x)$. The decision boundary corresponding to the Bayes classifier is known as the Bayes-optimal decision boundary, and the error rate of the Bayes classifier is known as the *Bayes rate*. Suppose $p_k(x)$ is the class-conditional density of x in class $y = k$, and let $\pi_k = P(y = k)$ be the prior probability of class k, with $\sum_{k=1}^{K} \pi_k = 1$. A simple application of Bayes theorem gives us

$$P(y = k|x) = \frac{p_k(x)\pi_k}{\sum_{l=1}^{K} p_l(x)\pi_l}.$$

We recognize that in terms of ability to classify, having the $p_k(x)$ is almost comparable to having the posterior probability of $P(y = k|x)$. Methods based on the class densities are as follows: (1) linear and quadratic discriminant analysis use Gaussian densities; (2) Naive Bayes models assume that each of the class densities is products of marginal densities; i.e., it assumes that the inputs are conditionally independent in each class; and (3) general non-parametric density estimates for each class density allow the most flexibility.

In a regression problem the y is continuous. The approach discussed in Eqs. (17.1) and (17.2) works. However, we require a suitable loss function for the error. The most conventional loss function is squared-error loss: $L(f(x), y) = (y - f(x))^2$. The resulting solution under the squared-error loss function is

$$f(x) = E(y|x),$$

the conditional expectation, also known as the *regression* function. As an alternate loss function, we can consider the absolute error loss, i.e., $L(f(x), y) = |f(x) - y|$. The resulting solution is the conditional median,

$$f(x) = \text{median}(y|x).$$

The conditional median estimates are more robust than those for the conditional mean, in a sense that the conditional median does not get influenced by the significant outliers or influential points.

17.1.2 Learning with Empirical Risk Minimization

The finite dataset, often called the *training dataset* , is used to learn the unknown f. To assess the expected loss in Eq. (17.1), we require to evaluate the expectation over all potential datasets. In practice, the joint distribution of the data is unknown. Hence assessing this expectation is intractable. Instead, a part of the dataset $D_{\text{training}} \subset D$ is employed to perform the learning and the remainder of the dataset $D_{\text{test}} = D - D_{\text{training}}$ is used to assess the performance of f using the loss function L. The subset of the dataset D_{test}, known as the *test dataset*, is applied for the evaluation. The expected loss over the training dataset is the *empirical risk* and defined as:

$$R(\hat{f}) = \frac{1}{n} \sum_{i=1}^{n} L(f(x_i), y_i). \tag{17.3}$$

Here n is the number of samples in the training dataset, D_{training}. The learning algorithm uses the empirical risk, $R(\hat{f})$, as a proxy for the actual risk, $R(f)$, to evaluate the performance of f. The optimal function in $\mathcal{F}$ for the prediction task is the one associated with the lowest empirical risk. This principle is called *Empirical Risk Minimization*, and defined as

$$h = \inf_{f \in \mathcal{F}} R(\hat{f}). \tag{17.4}$$

The *learning task* apply algorithm $\mathcal{A}$ to determine h and algorithm $\mathcal{A}$ is known as the *learner*. For example, the maximum likelihood procedure is an example of the *learner*. Our interest is to obtain the lowest possible risk for the learning problem, i.e., the R^* associated with the function f^*. The lowest possible risk is the *Bayes risk*. The hypothesis class for f^* may or may not be the same as $\mathcal{F}$. Consider the response generated by the learner h. Comparing the risk for h with R^* (*excess risk*) provides another view of the quality of h, presented in Eq. (17.5):

$$R(\hat{h}) - R^* = \left(R(\hat{h}) - R(h)\right) + \left(R(h) - R^*\right). \tag{17.5}$$

In Eq. (17.5), the first term is the *estimation error* and the second term is the *approximation error*. We examine the hypothesis class that describes hyperplanes. The algorithm uses the training dataset to estimate the parameters of the hyperplane by applying the *empirical risk minimization* technique. The estimates for the hyperplane are different if we use a different training dataset, so error occurs from using a sample to learn h. Therefore, it is known as the estimation error. The approximation error shows how well a hypothesis class can approximate the best function. We consider an example where f^* (associated with R^*) is complex, say a high degree polynomial, and our hypothesis class $\mathcal{F}$ is the set of hyperplanes. We see a high approximation error. Note that we recognize the high approximation error if f^* is simple (like a hyperplane) and our hypothesis class $\mathcal{F}$ is complex (like a class of high degree polynomials). The choice of the hypothesis class $\mathcal{F}$ influences the performance of the learning task. In particular, the complexity associated with the hypothesis class $\mathcal{F}$ can affect the level of risk that the learning algorithm $\mathcal{A}$ can manage.

In essence, both the estimation error and the approximation error affect the algorithm's performance to determine the best hypothesis class for a problem. One must make a trade-off between the two, and it is called the *bias-variance* trade-off, see Friedman et al. (2009). Preferring more complex hypothesis classes than what is optimal can lead to a phenomenon called ***overfitting***. Usually, overfitting means the solid performance of the class on the training dataset but very bad performance on the test dataset. If an algorithm $\mathcal{A}$ can maintain the same level of precision on both the training and test datasets, such capacity is known as the **generalization**. If the algorithm $\mathcal{A}$ generalizes well, then the new insight learned from modeled data is likely to be **reproducible** in the latest test dataset, provided the training dataset is an accurate representation of the population.

17.2 Regression and Classification

The **regression** is a helpful technique for predicting a numeric response. For example,

1. We want to predict the price of a house,
2. We want to estimate the fair price of a stock in the stock market.

It has been around for a long time and is the topic of innumerable textbooks, see James et al. (2021), Friedman et al. (2009). On the other hand, the **classification** task considers the approaches for predicting qualitative responses. Predicting a qualitative response for a test case can be referred to as classifying that test case since assigning the case to a category or class. In this sense, classification also behave like regression methods. We face classification problems more often, perhaps even more so than regression problems. For example:

1. An online banking application wants to decide whether or not a transaction performed on the site is fraudulent based on the user's past transaction history, IP address, etc.

2. A bank executive wants to decide whether an applicant for a car loan will pay all the installments on time. In other words, the executive wants to figure out if the loan applicant is a potentially good customer or not.
3. An investment bank wants to assess whether a stressed company would file for bankruptcy or not.

In the regression/classification we start with training observations

$$(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$$

that we use to develop a classifier or a regression model. We want the classifier to perform well not only on the training data, but also on any test cases that were not used to learn the classifier.

We illustrate the idea of classification using the `credit card default dataset`, see Yeh (2009). We are interested in predicting whether individuals will default on their credit card payments based on bill statements in recent months. Based on the bill statement amount in April and September, can we predict if an individual will default their payments in October? The dataset displayed in Fig. 17.1. We plotted bill statements of April and September credit card balances for 30,000 individuals. Figure 17.1 displays individuals who defaulted in October in red and those who did not default in green. Typically, the default rate is less than 3%. However, in the data displayed in Fig. 17.1, default cases are oversampled up to 22%. It seems individuals who defaulted tended to accumulate higher credit card balances in September compared to April. In this chapter, we learn how to build a supervised learning model to predict `default` (Y) for any given value of `balance in September` (X_1) or `balance in April` (X_2) or other predictors such as `age` (X_3), `gender` (X_4), `education` (X_5), and `marital status` (X_6) of an individual.

Note that the relationship between `balance amount` and `default` is not so prominent. However, it is worth trying if supervised learning algorithms can develop a classifier that can predict a test case with higher accuracy.

17.3 Supervised Learning and Generalized Linear Models

The machine learning community considers supervised learning to solve two different problems, namely (i) Regression and (ii) Classification. However, the Statistics community considers supervised learning as a single solution class and models it applying the generalized linear models (GLM), see, e.g., McCullagh and Nelder (1989). The approach in Statistics is to model the dependent variable y using the natural exponential family

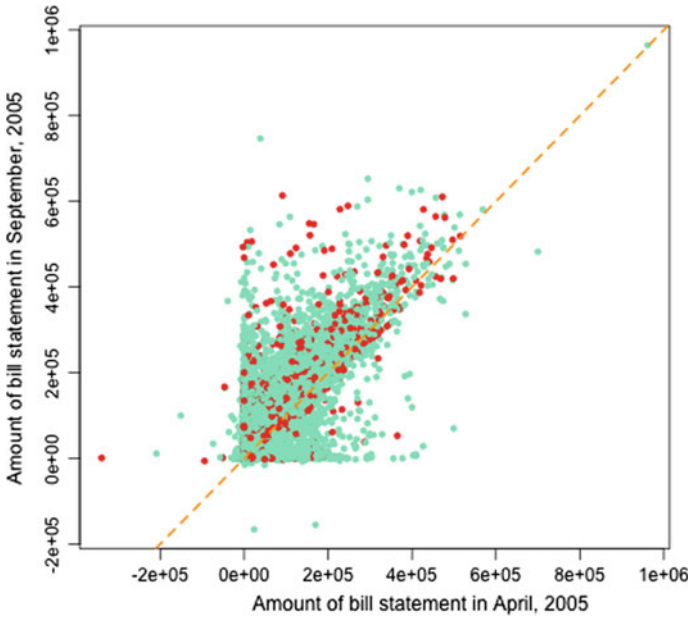


Fig. 17.1 The Amount of bill statement in April 2005 and September 2005 of a number of individuals. The individuals who defaulted on their credit card payments are shown in red, and those who did not are shown in green

$$p(y) = \exp\{\theta y - \psi(\theta)\}, \tag{17.6}$$

for different choices of θ, $p(\cdot)$ may represents a Gaussian, Binomial, Bernoulli, and Poisson distribution, see McCullagh and Nelder (1989) for particular. These distributions can model a continuous, binary, or count response variables, see Das and Dey (2006, 2013). Then features are modeled as the function of the conditional mean of y, through the link function $h(\cdot)$ as

$$\mathbb{E}(y|\mathbf{x}) = h(\beta_1 x_1 + \cdots + \beta_p x_p), \tag{17.7}$$

where $\mathbf{x} = (x_1, x_2, \ldots, x_p)$ is the vector of p predictors in the datasets. If we consider a binary response classification task, where the y follows Bernoulli distribution with probability p. The conditional mean of y for Bernoulli distribution is p, i.e.,

$$\mathbb{E}(y|\mathbf{x}) = p,$$

can be modeled though logit link function as

$$\log \frac{p}{1-p} = \beta_1 x_1 + \cdots + \beta_p x_p.$$

This is called the canonical link as it equals θ, the natural parameter, in (17.6).

17.4 Regression

We consider the `Boston Housing value` dataset in the `MASS` package in `R`. The dataset contains the median value of the 506 houses of owner-occupied homes (in $1000s). In addition there are 13 features of the houses, such as (i) average number of rooms per dwelling, (ii) distances from Boston employment centers, and (iii) pupil-teacher ratio by town. In Fig. 17.2, we present scatter plot between the average number of rooms per dwelling and median value of the house. Our objective is to predict the median value of the house (a numeric variable).

As we want to predict a numeric response, linear regression is perhaps the most popular model and is the topic of almost every statistics textbook. If we assume that the numeric response y follows the Gaussian distribution and in Eq. (17.7), if we consider the identity function, then the regression model is a particular case of GLM

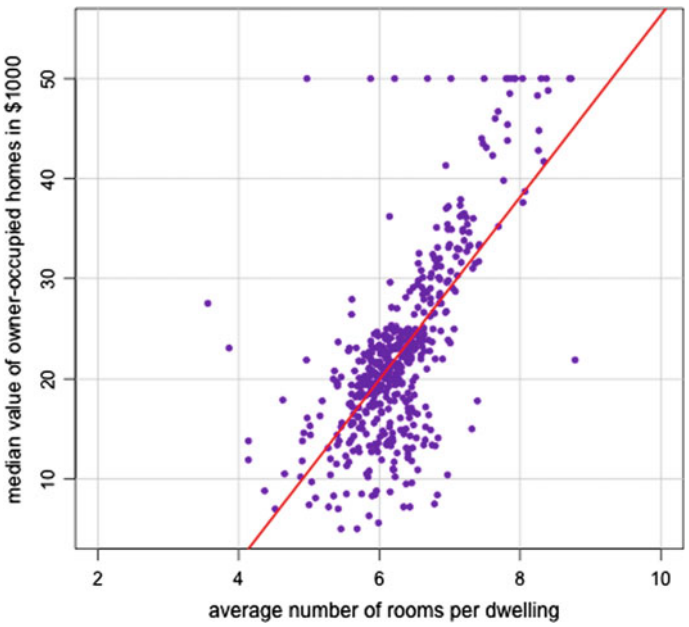


Fig. 17.2 A positive relationship between the average number of rooms per dwelling and median value of the houses

and presented as

$$\mathbb{E}(y_i) = \mu_i = \mathbf{x}_i^T \beta, \quad i = 1, 2, \ldots, n,$$

where $y_i \sim N(\mu_i, \sigma^2)$. The model can be alternatively presented as

$$y_i = \mathbf{x}_i^T \beta + \epsilon_i,$$

where $\epsilon_i \sim N(0, \sigma^2)$, $\mathbf{x}_i = (x_{i1}, x_{i2}, \ldots, x_{ip})^T$. To learn about the unknown parameters $\beta = (\beta_1, \ldots, \beta_p)^T$, we consider the *squared-error loss* as *empirical risk* in Eq. (17.3) and resulting *empirical risk minimization* procedure is also known as ordinary least-square (OLS) method. The OLS method is presented as

$$\frac{\partial}{\partial \beta}(y - X\beta)^T (y - X\beta) = 0,$$

where $y = (y_1, y_2, \ldots, y_n)$ is the response vector and $X = ((x_{ij}))_{n \times p}$ is feature matrix. The resulting OLS solution for β is

$$\hat{\beta} = (X^T X)^{-1} X^T y.$$

Since $y \sim N_n(X\beta, \sigma^2 \mathbf{I}_n)$, the sampling distribution of $\hat{\beta}$ is

$$\hat{\beta} \sim N(\beta, \sigma^2 (X^T X)^{-1}).$$

We use the sampling distribution of $\hat{\beta}$ to make the statistical inference on the regression coefficients. Suppose we are interested if the feature x_i has any statistically significant effect on y. We can check it via testing of hypothesis for regression coefficients β_i, i.e.,

$$H_0 : \beta_i = 0 \quad vs \quad H_a : \beta_i \neq 0.$$

Now we know $\hat{\beta}_i \sim N(\beta_i, \sigma^2 h_{ii})$, where h_{ii} is the ith diagonal elements of the $(X^T X)^{-1}$. We estimate the σ^2 by the following estimator, which is the mean square of the residuals.

$$s^2 = \frac{(y - X\hat{\beta})^T (y - X\hat{\beta})}{n - p}.$$

Under null hypothesis we can use the t statistics,

$$t = \frac{\hat{\beta}_i}{s\sqrt{h_{ii}}} \sim t_{n-p},$$

to test the above hypothesis. We can implement the analysis using the **lm** functions in the **stats** library of **R**. Following **R**-code implements the regression model to predict the median value of house (defined as `medv`) as linear function of average number of rooms per dwelling (defined as `rm`), i.e.,

$$\texttt{medv}_i = \alpha + \beta \texttt{rm}_i + \epsilon_i, \tag{17.8}$$

where $i = 1, 2, \ldots, n(= 506)$ and $\epsilon_i \sim N(0, \sigma^2)$. We want to check whether rm has effect on medv. So we conduct testing of hypothesis for $\beta = 0$. The estimated β is `9.102`, and standard error of β is `0.419`. The t-statistics is `9.102/0.419 = 21.72`. The `P value` presented in the fourth column is less than 0.001. Hence we can say that value of house significantly depends on the number of the rooms per dwelling. The `Adjusted R-square` for the `0.4825` implies that about `48.25%` of the total variability of the medv can be explained by the simple linear regression model with rm as only feature.

```
mod1 = lm(medv~rm,data = Boston)
summary(mod1)

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)  -34.671      2.650  -13.08   <2e-16 ***
rm             9.102      0.419   21.72   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1

Residual standard error: 6.616 on 504 degrees of freedom
Multiple R-squared:  0.4835,    Adjusted R-squared:  0.4825
```

Now we want to develop a regression model where we can predict the value of the house price as function of multiple features, such as

$$\texttt{medv}_i = \beta_0 + \beta_1 \texttt{rm}_i + \beta_2 \texttt{crim}_i + \cdots + \epsilon_i, \tag{17.9}$$

where $i = 1, 2, \ldots, n(= 506)$ and $\epsilon_i \sim N(0, \sigma^2)$. The following **R**-code implements the regression model to predict the median value of house (defined as medv) as linear function of all the features in the data.

```
mod2 = lm(medv~.,data = Boston)
summary(mod2)
```

The 'medv~.' is the formula that implements the linear model where medv is the target variable and all other columns of the dataset are considered the model's predictor.

```
Call:
lm(formula = medv ~ ., data = Boston)
```

Note that the `Adjuster R-square` increases to `73.38%`. Except for indus and age, all other features in the model have a statistically significant effect on the medv.

```
Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)  3.646e+01  5.103e+00   7.144 3.28e-12 ***
crim        -1.080e-01  3.286e-02  -3.287 0.001087 **
zn           4.642e-02  1.373e-02   3.382 0.000778 ***
indus        2.056e-02  6.150e-02   0.334 0.738288
chas         2.687e+00  8.616e-01   3.118 0.001925 **
nox         -1.777e+01  3.820e+00  -4.651 4.25e-06 ***
rm           3.810e+00  4.179e-01   9.116  < 2e-16 ***
age          6.922e-04  1.321e-02   0.052 0.958229
dis         -1.476e+00  1.995e-01  -7.398 6.01e-13 ***
rad          3.060e-01  6.635e-02   4.613 5.07e-06 ***
tax         -1.233e-02  3.760e-03  -3.280 0.001112 **
ptratio     -9.527e-01  1.308e-01  -7.283 1.31e-12 ***
black        9.312e-03  2.686e-03   3.467 0.000573 ***
lstat       -5.248e-01  5.072e-02 -10.347  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1

Residual standard error: 4.745 on 492 degrees of freedom
Multiple R-squared:  0.7406,    Adjusted R-squared:  0.7338
```

Note that both models were trained using all the observations. However, we should be careful about the fitted model to make sure whether it generalizes well. Hence we split the data into train and test. We consider 80% data for training and the rest of the data for testing. Following code split the data into train and test.

```
### split train and test
n = nrow(Boston) ## total sample in the data
m = floor(n*0.8) ## training sample size
set.seed(2021)
train_id = sample(1:n,size = m,replace = FALSE)
train_id = sort(train_id)
Boston_train=Boston[train_id,]
Boston_test=Boston[-train_id,]
```

Now we implement the model 17.8 and 17.9 for predicting the `medv` using the `Boston_train` dataset using the following `R` code. The out-of-sample RMSE for model 17.8 is `7.13`. However, the in-sample RMSE is `6.46`. It is expected that out-of-sample RMSE would be slightly higher than the in-sample RMSE, see Friedman et al. (2009). The out-of-sample RMSE for model 17.9 is `5.32` , and in-sample RMSE for the model is 17.9 is `4.55`.

```
mod3 = lm(medv~rm,data = Boston_train)
mod4 = lm(medv~.,data = Boston_train)
medv_hat3 = predict(mod3,newdata = Boston_test)
medv_hat4 = predict(mod4,newdata = Boston_test)
```

```
medv_actual = Boston_test$medv
rmse3 = sqrt(mean((medv_hat3 - medv_actual)^2))
rmse4 = sqrt(mean((medv_hat4 - medv_actual)^2))
## OutSample Root Mean Square Error (RMSE)
c(rmse3=rmse3,rmse4=rmse4)

   rmse3    rmse4
7.129945 5.322710

## In-sample RMSE
medv_bar3=mod3$fitted.values
medv_bar4=mod4$fitted.values
medv_actual_tr=Boston_train$medv
rmse3 = sqrt(mean(( medv_bar3- medv_actual_tr)^2))
rmse4 = sqrt(mean((medv_bar4 - medv_actual_tr)^2))
## In Sample Root Mean Square Error (RMSE)
c(rmse3=rmse3,rmse4=rmse4)

  rmse3    rmse4
6.464879 4.549169
```

P Value in Machine Learning Literature

Though computer scientists are very much aware of the theory of hypothesis testing in statistics, they often apply it in practice; it is seldom discussed or considered in mainstream machine learning literature. The reasons are the following:

- There is much development in ML by technology companies see, e.g., Shinal (2017). Therefore, the ML focuses more on predicting the target variable rather than finding an association between the target variable and features; and hypothesis testing whether a particular feature influences the target variable or not.
- Even if one is making inference, typically, ML deals with massive datasets, which requires handling of thousands of hypotheses. It results in multiple testing problems.
- A focuses on statistically significant, novel, and affirmative results, based on the *P value* approach leads to bias, known as *P-hacking*. When researchers keep collecting data until analyses yield statistically significant results, the studies become prone to *P-hacking*. Such results are generally not reproducible. If not correctly conducted, the testing of the hypothesis unknowingly might lead the scientists to *P-hacking*, a situation all scientists must avoid, see Head et al. (2015) for particular.
- The ML views each hypothesis as a model, and the Bayesian model selection strategy is a much more comprehensive strategy for Big data.

17.5 Logistic Regression

We consider again the credit card default dataset introduced in Sect. 17.2. The response `default` falls into one of two classes, `Yes` or `No`. Rather than modeling this Y directly, logistic regression models Y's *probability* of a particular class. For the `credit card default dataset`, the logistic regression models the probability of `default` given `balance` amount in September as

$$\mathbb{P}(\texttt{default = Yes}|\texttt{balance in Sep}).$$

We abbreviate the $\mathbb{P}(\texttt{default = Yes}|\texttt{balance in Sep})$ as $p(\texttt{balance in Sep})$ for space constraint. Now for any given value of `balance in Sep`, a prediction can be made for `default = Yes`. For example, we might predict `default = Yes` for any individual for whom $p(\texttt{balance in Sep}) > 0.5$. However, if a bank wishes to be conservative in predicting the individuals at risk for default, they may want to use a lower cut-off, such as $p(\texttt{balance in Sep}) > 0.2$.

The question is, how should we model the relationship between $p(X) = P(Y = 1|X)$? For ease, we use the generic 0/1 coding for the response. We can model $p(X)$ using a function that yields output between 0 and 1. Many mathematical functions satisfy this condition. The logistic regression model uses the logistic function,

$$P(Y = 1|X = x) = p(X) = \frac{\exp\{\beta_0 + \beta_1 X\}}{1 + \exp\{\beta_0 + \beta_1 X\}}. \tag{17.10}$$

The parameters (β_0, β_1) can be learned using the *maximum likelihood* method on training dataset. Algebraic manipulation of Eq. (17.10), we have

$$\frac{p(X)}{1 - p(X)} = e^{\beta_0 + \beta_1 X}. \tag{17.11}$$

The measure $\frac{p(X)}{1-p(X)}$ is called **odds** and can take the value on a positive real line. Values of the **odds** close to 0 indicate a low probability of `default`. For example, on average, 1 in 100 people with an odds of 1/99 will default, since $p(X) = 0.01$ means an odd $\frac{0.01}{1-0.01} = \frac{1}{99}$. Similarly, when six out of every ten people with an odds of 1.5 will `default` since $p(X) = 0.6$ implies an odds of $\frac{0.4}{1-0.4} = 1.5$. Odds are traditionally used in gambling since they narrate the correct betting strategy more naturally. By taking the logarithm of both sides of Eq. (17.11), we arrive at

$$\log\left(\frac{p(X)}{1 - p(X)}\right) = \beta_0 + \beta_1 X.$$

The left-hand side is called the **logit** or **log-odds**.

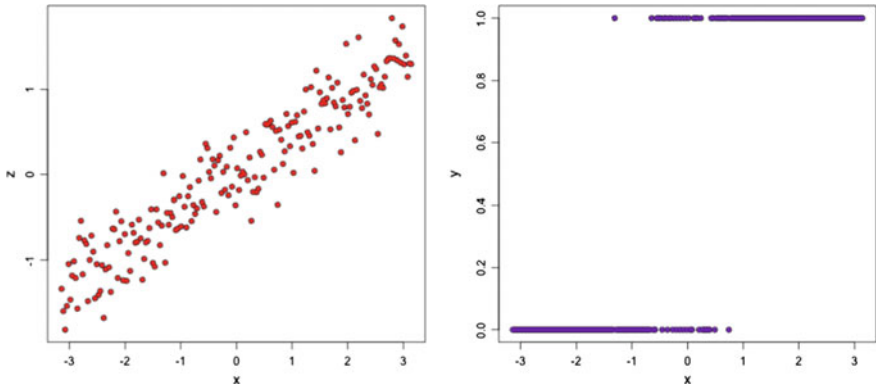


Fig. 17.3 Original relationship between x and z. We pretend we cannot observe z. However we do observe the x and y

17.5.1 Sigmoid Curve Behavior of Logistic Regression

The Logistic Regression does not behave like a simple linear regression. Instead, it behaves like a sigmoid curve presented in Fig. 17.4. We can visualize the nature of the sigmoid curve of using the following simulation study. We consider the predictor variable x between $(-\pi, \pi)$. We simulate the latent variable z using the relation as z=0.01+0.45*x+ e, where e $\sim$ **norm**(mean = 0, sd = 0.3)

```
set.seed(9879)
N = 200 ## number of points
x = seq(-pi,pi,length.out=N)
e = rnorm(N,mean = 0,sd=0.3)
z = 0.01+0.45*x+ e
## z is the latent variable.
```

Now we define the response variable y = 1, if z > 0, and y = 0, if z $\leq$ 0. Now we pretend actual response variable z is unobserved. The only data that we observe is x and y in D as data.frame. Simulated data presented in Fig 17.3 We model the relationship between the x and y using the **logistic regression** and we use the **glm** function in **stats** library of the R. The graph generated by this code presented in Fig. 17.5. The figure explains that the sigmoid curve looks more like elongated $\mathcal{S}$.

```
y=z
y[z>0]=1
y[z<=0]=0
## y is observed response
D = cbind.data.frame(x,y)
plot(x,y,pch=20,col='purple')

sigmoid_model = glm(y ~ x
```

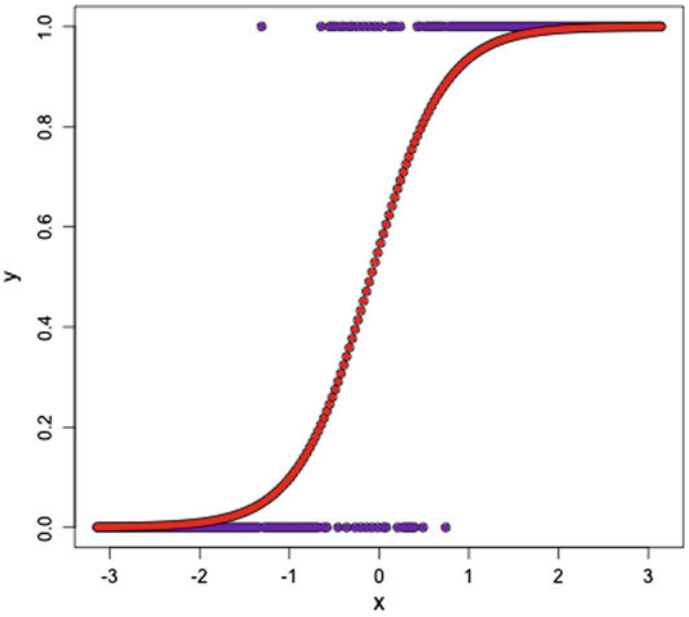


Fig. 17.4 The sigmoid curve from logistic regression looks more like elongated $\mathcal{S}$

```
                ,family=binomial(link = "logit")
                ,data=D)
summary(sigmoid_model)
y_hat = predict(sigmoid_model
               ,data=data
               ,type='response')

points(x,y)
points(x,y_hat,pch=20,col='red')
```

17.5.2 Non-monotonic Relation with Logistic Regression

The elongated $\mathcal{S}$ type behavior (aka., sigmoid curve) is well understood in literature, see Chap. 4, James et al. (2021). However, the elongated $\mathcal{S}$ only models the monotonic relationship between `x` and `y`. Here we discuss how we can model non-monotonic relation between `x` and `y` using the logistic regression. We can model the non-monotonic relation between `x` and `y` by using the higher-order polynomial, i.e.,

$$p(X) = \frac{\exp\{\beta_0 + \beta_1 X + \beta_2 X^2\}}{1 + \exp\{\beta_0 + \beta_1 X + \beta_2 X^2\}}. \tag{17.12}$$

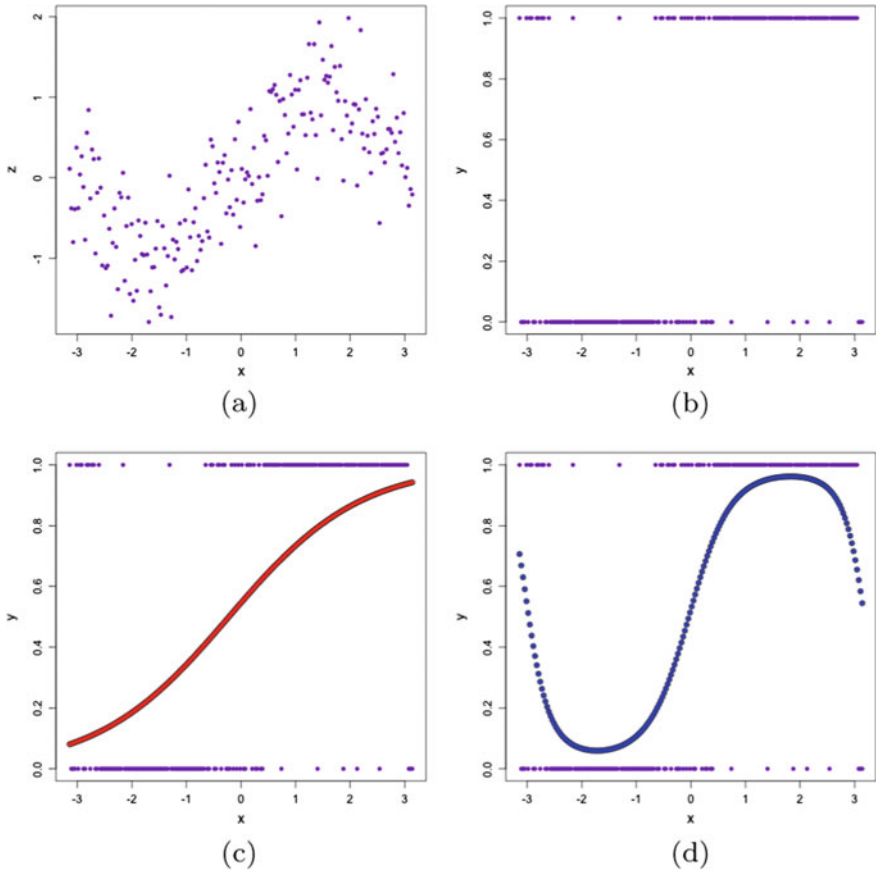


Fig. 17.5 **a** Scatter plot between `x` and `z` indicate the true relationship is sinusoidal. Though we pretend that actually we do not know the relation between `x` and `z`, and we do not observe `z`. **b** We plot `x` and `y` what we only observe. **c** The sigmoid curve from the logistic regression with Eq. (17.10) presented by the red curve. **d** The blue curve from the $logit(X) = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3$ can successfully model the underlying unknown sinusoidal behavior with standard `logit` regression

In Eq. (17.12), the quadratic equation is being presented to model the relation between X and Z. We can use higher-order polynomial model to capture the underlying relationship between X and Z. In the following `R` code, we demonstrate how non-monotonic relationship between X and Y can be easily captured by modeling the non-monotonic relationship between X and Z. First we assume the relationship between X and Z are sinusoidal. We consider the predictor variable `x` between $(-\pi, \pi)$. We simulate the latent variable `z` using the relation as `z=sin(x)+ e`, where `e` $\sim$ `N(mean = 0, sd = 0.5)`. We pretend that we do not know the true relationship between X and Z. So we fit a cubic relationship between X and Z, with

$$Z = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3.$$

The graphical output from the R code is presented in Fig. 17.5. The blue curve fitted with the $logit(X) = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3$ can successfully model the underlying unknown sinusoidal behavior between X and latent Z. However, the sigmoid curve from the logistic regression with Eq. (17.10) fail to model underlying the sinusoidal relation between X and latent Z.

```
## Beyond the sigmoid curve
set.seed(9879)
N=200
x = seq(-3.14,3.14,length.out=N)
e = rnorm(N,mean = 0,sd=0.5)
z = sin(x)+e
plot(x,z,pch=20,col='purple')
y=z
y[z>0]=1
y[z<=0]=0
plot(x,y,pch=20,col='purple')

data = cbind.data.frame(x,y)


## sigmoid function
sigmoid_model = glm(y~x,family=binomial(link = "logit")
                                        ,data=data)
summary(sigmoid_model)
y_hat = predict(sigmoid_model,data=data
                              ,type='response')

points(x,y_hat)
points(x,y_hat,pch=20,col='red')

## going beyond sigmoid function
beyond_sigmoid_model = glm(y~x+I(x^2)+I(x^3)
                           ,family=binomial(link = "logit")
                           ,data=data)

summary(beyond_sigmoid_model)
y_hat2 = predict(beyond_sigmoid_model,data=data
                               ,type='response')

points(x,y_hat2)
points(x,y_hat2,pch=20,col='blue')
```

17.6 Discriminant Analysis

In the logistic regression, we directly model $P(Y = k|X = x)$, using the Eq. (17.10). The regular logistic regression typically models the two response classes, where we model the conditional distribution of response Y for given features X. In the discriminant analysis, we model the features X separately in each response class as $f(x|Y = k)$ and apply the Bayes theorem to find the $P(Y = k|X = x)$. Suppose q_k

is the prior probability that a randomly chosen data point comes from the kth class. Let $f(x|Y=k) = f_k(x)$ is the *probability density function* of X for an observation that is generated from the kth class. Applying Bayes' theorem we have

$$P(Y=k|X=x) = \frac{q_k f_k(x)}{\sum_{i=1}^{K} p_i f_i(x)} = \frac{q_k f_k(x)}{m(x)}.$$

The $m(x)$ is known as *marginal distribution* of X. The main effort in the *discriminant analysis* is to model the $f_k(x)$. Many techniques are there to model $f_k(x)$. The linear and quadratic discriminant analysis assumes Gaussian densities. Other techniques such as mixture models, non-parametric density estimation, or copula methods are complicated and beyond the scope of this book. Here we focus only on the linear discriminant analysis (LDA) and quadratic discriminant analysis (QDA). In LDA and QDA, we model each class density as multivariate Gaussian density with *pdf* as

$$f_k(x) = \frac{1}{(2\pi)^{p/2}|\Sigma_k|^{1/2}} \exp\Big\{-\frac{1}{2}(x-\mu_k)^T \Sigma_k^{-1}(x-\mu_k)\Big\}. \tag{17.13}$$

If we assumes that all classes have a common covariance matrix, i.e., $\Sigma_k = \Sigma$ for all k, the resulting model would be the Linear Discriminant Analysis (LDA). To compare two classes, we look at the log-ratio as follows,

$$\begin{aligned}
\log \frac{P(Y=k|X=x)}{P(Y=m|X=x)} &= \log \frac{q_k f_k(x)}{q_m f_k(x)}, \\
&= \log \frac{q_k}{q_m} + \log \frac{f_k(x)}{f_m(x)}, \\
&= \log \frac{q_k}{q_m} - \frac{1}{2}\big[(x-\mu_k)^T \Sigma^{-1}(x-\mu_k) \\
&\qquad -(x-\mu_m)^T \Sigma^{-1}(x-\mu_m)\big].
\end{aligned} \tag{17.14}$$

After some manipulation of Eq. (17.14) we obtain

$$\begin{aligned}
\log \frac{P(Y=k|X=x)}{P(Y=m|X=x)} = \frac{q_k}{q_m} &- \frac{1}{2}(\mu_k+\mu_m)^T \Sigma^{-1}(\mu_k-\mu_m) \\
&+ x^T \Sigma^{-1}(\mu_k-\mu_m).
\end{aligned} \tag{17.15}$$

Now we can represent the Eq. (17.15) as

$$\log \frac{P(Y=k|X=x)}{P(Y=m|X=x)} = \alpha + x^T \beta,$$

where

$$\alpha = \frac{q_k}{q_m} - \frac{1}{2}(\mu_k + \mu_m)^T \Sigma^{-1}(\mu_k - \mu_m),$$
$$\beta = \Sigma^{-1}(\mu_k - \mu_m). \tag{17.16}$$

Note that α and β are functions of model parameters $\theta = (q_k, q_m, \mu_k, \mu_m, \Sigma)$. One can estimate the model parameters θ using *maximum likelihood estimation* (MLE) method and then we estimate the α and β by plug-in the MLE $\hat{\theta}$ in Eq. (17.16). Now a test point x_0 belongs to class k if

$$\log \frac{P(Y = k|X = x)}{P(Y = m|X = x)} = \hat{\alpha} + x_0^T \hat{\beta} \geq \xi,$$

otherwise it belongs to class m. From the Eq. (17.15), we obtain the *linear discriminant functions*,

$$d_k(x) = \log q_k - \frac{1}{2}\mu_k^T \Sigma^{-1}\mu_k + x^T \Sigma^{-1}\mu_k$$

with optimal decision rule as

$$\mathcal{G}(x) = \underset{k}{\operatorname{argmax}}\, d_k(x).$$

In practice we do not know the parameters of the Gaussian distributions. We estimate the parameters using the training data. The prior probability q_k can be estimated as

$$\hat{q}_k = \frac{f_k}{n},$$

where f_k is the number of observation that belongs to class-k, and n is the number of observations in the training dataset. The mean μ_k can be estimated by the sample mean of class conditional distribution, i.e.,

$$\hat{\mu}_k = \frac{1}{f_k} \sum_{i:y_i=k} x_i.$$

The covariance matrix can be estimated as

$$\hat{\Sigma} = \frac{1}{n - K} \sum_{k=1}^{K} \sum_{i:y_i=k} (x_i - \hat{\mu}_k)(x_i - \hat{\mu}_k)^T.$$

In the generic discrimination task 17.13, if Σ_k are not equal, then convenient cancellation in Eq. (17.15) would not work anymore. In particular, the *discriminant function* remains quadratic in x. Hence the *quadratic discriminant function* for QDA is

$$d_k(x) = -\frac{1}{2}\log|\Sigma_k| - \frac{1}{2}(x - \mu_k)^T \Sigma_k^{-1}(x - \mu_k) + \log q_k.$$

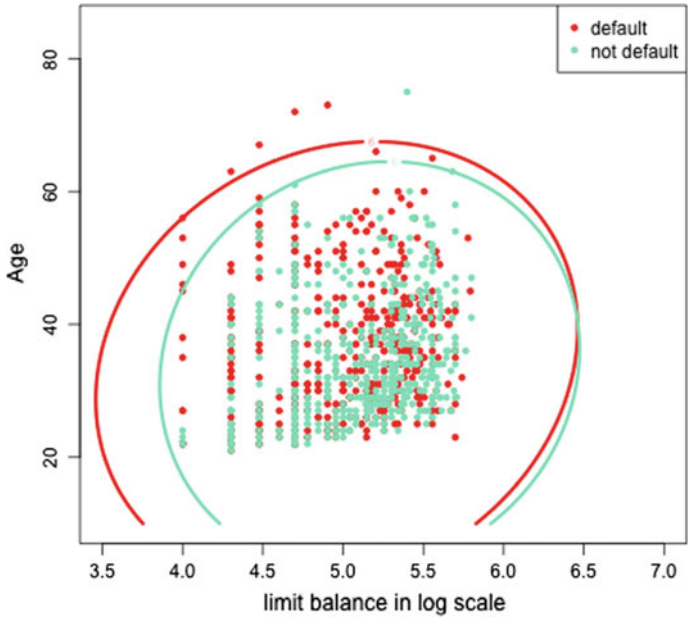


Fig. 17.6 Scatter plot of `Limit Balance` (in log scale) and `Age`. The defaulted data points are marked by red; otherwise, it has not defaulted. The Gaussian contour of defaulted cases marked by red is slightly different from that of not defaulted cases. Two contours overlap significantly. Nevertheless, we can say that the `Age` and `Limit Balance` have some weak signal that differentiates the default cases from not default cases

We implement the discriminant analysis on `Credit Card Default` data. Figure 17.6, presents the contour of the class conditional bivariate Gaussian distribution. The red contour represents the assumed Gaussian distribution over the features `Age` and `Limit Balance` (in log10 scale) for `defaulted` labels. Similarly, the green contour is the assumed Gaussian distribution over the same features for `not defaulted` labels.

In the left panel of the Fig. 17.7, we plot the decision boundary of LDA, and in right panel of the Fig. 17.7, we plot the decision boundary of QDA. We can implement the LDA and QDA using the `MASS` library in `R`. The following `R`-code implements the LDA. The out-of-sample accuracy for LDA model is `0.7701`. However, the out-of-sample accuracy for QDA model is `0.7668`.

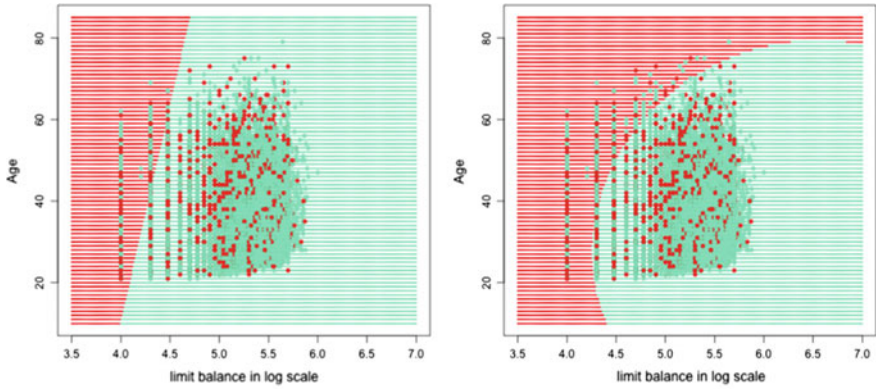


Fig. 17.7 Decision boundary on `Credit Card Default` data. In **a** we plot the decision boundary of LDA and **b** we plot the decision boundary of QDA

Regularized Discriminant Analysis

If there are p features, then the number of parameters to estimate in the covariance matrix Σ_k increases in quadratic rate as $\mathcal{O}(p^2)$. To solve the problem of too many parameters in QDA one can consider the approach of regularization to shrink the separate covariances of QDA toward a common covariance as in LDA. The regularized covariance matrices are

$$\tilde{\Sigma}_k = \omega\hat{\Sigma}_k + (1-\omega)\hat{\Sigma},$$

where $0 < \omega < 1$. One can show $\tilde{\Sigma}_k$ is the posterior mode of covariance matrix Σ, for particular choice of ω, such as

$$\omega = \frac{n_0 + p + 1}{n_0 + n + p},$$

if $p > n$ the one should choose $n_0 = (p - n) + 1$. The posterior mode of Σ is Bayesian shrinkage estimator which regularizes the solution, particularly when $p > n$, i.e., number of features are larger than the number of training sample, see Das and Dey (2010), Das et al. (2017).

```
set.seed(038411)
library(MASS)
n=nrow(data)

## split the data into train and test

m = floor(n*0.7)
train_id = sort(sample(1:n,size=m,replace = FALSE))
train_data = data[train_id,]
test_data = data[-train_id,]

## fitting LDA using MASS package
fit_lda = MASS::lda(default~AGE+log10(LIMIT_BAL)
                    ,data=train_data
                    ,prior=c(0.7,0.3))

pred = predict(fit_lda,newdata = test_data)
test_data$pred_default = as.character(pred$class)
test_data$pred_default = as.numeric(test_data$pred_default)
confusion_table = table(actual=test_data$default
                        ,predicted = test_data$pred_default)

accuracy=sum(diag(confusion_table))/sum(confusion_table)
accuracy
> 0.7701
```

```
### Fit QDA
fit_qda =MASS::qda(default~AGE+log10(LIMIT_BAL)
                    ,data=data
                    ,prior=c(0.7,0.3))

pred = predict(fit_qda,newdata = test_data)
test_data$pred_default = as.character(pred$class)
test_data$pred_default = as.numeric(test_data$pred_default)
confusion_table = table(actual=test_data$default
                        ,predicted = test_data$pred_default)

accuracy=sum(diag(confusion_table))/sum(confusion_table)
accuracy
> 0.7668
```

17.7 Tree Structured Model

The tree-structured methods for regression task are based on the partition of the input space into separate regions with a constant response for each region, Breiman et al. (1984). The tree models take the following form:

$$f(x) = \sum_{m=1}^{M} c_i \mathbf{I}(x \in R_m), \tag{17.17}$$

where:

- c_m represents the constant response for region R_m.
- $\mathbf{I}(x \in R_i)$ is the indicator function that is defined as $I(x) = \begin{cases} 1 & \text{if } x \in R_m \\ 0 & \text{otherwise} \end{cases}$
- M represents the number of terminal nodes and is an important parameter for the tree models.

If we consider the square error loss function, then the best option for c_m is the average response values y_i in the region R_m. The feature space is partitioned into regions $R_1, \ldots, R_M$ using a greedy algorithm, see Friedman et al. (2009) [Chap. 9, Sect. 9.2.2] for particulars. The number of regions (M), which partition the predictor space, is essential to the algorithm. The parameter M describes the length of the tree. It defines the complexity of the solution. An algorithm that works well is partitioning the feature space until there is a minimum (threshold) number of cases in each region. Then we shorten the tree using pruning. Pruning is aided by minimization of a loss function which is defined as follows.

- Let N_m be the number of cases that belong to region R_m and $c_m = \frac{1}{N_m} \sum_{X_i \in R_m} y_i$,
- Let T_0 denote the tree obtained without applying the pruning by developing the tree until a minimum number of cases in each leaf node is achieved.
- Let T be the tree that is subject to pruning. The pruning process involves collapsing nodes of T_0 to build an optimal tree. It has $\left|T\right|$ nodes.
- We define mean sum of squares of error (SSE) as,

$$Q_m(T) = \frac{1}{N_m} \sum_{y_i \in R_m} \left(y_i - c_m\right)^2. \tag{17.18}$$

The $Q_m(T)$ is also known as the node impurity measure. We can define the cost function that is minimized during ERM with the regression tree models as,

$$C_\alpha(T) = \sum_{m=1}^{\left|T\right|} N_m Q_m(T) + \alpha \left|T\right|, \tag{17.19}$$

where α is a parameter that controls the model complexity of T. Here $C_\alpha(T)$ is the penalized sum of square of errors. For each value of α, we obtain a model f_α by applying ERM where the Eq. (17.19) is minimized. Many variations of tree models are developed. The most popular are the **Decision Tree** and **Random Forest**, where the random forests is an ensemble learning of tree models, see Breiman (2001).

17.7.1 Decision Tree

Both regression and classification problems can be modeled using the Decision trees. If the target variable is a categorical variable with outcome taking values $1, 2, \ldots, K$, the only changes needed in the tree algorithm concern the rules for dividing nodes and pruning the tree. For regression, we consider the squared-error node impurity measure $Q_m(T)$ defined in Eq. (17.18). In a node m, representing a region R_m with N_m observations, suppose

$$p_{mk} = \frac{1}{N_m} \sum_{x_i \in R_m} I(y_i = k),$$

the proportion of observations that belong to class k, in a node m. We classify the observations in node m to class

$$k(m) = \underset{k}{\text{argmax}}\, p_{mk},$$

the majority class in node m. The are different measures $Q_m(T)$ of node impurity, such as (i) Misclassification error, (ii) Gini Index, or (iii) Cross-entropy or deviance. The classification error rate is the proportion of the training dataset that does not belong to the most common class,

$$\mathcal{E} = 1 - \max_k p_{mk}.$$

However, the classification error is not sensitive to tree-growing. Hence the Gini index is an alternative to classification error, defined as

$$G = \sum_{k=1}^{K} p_{mk}(1 - p_{mk})$$

a measure of total variance across the K classes. The Gini index takes a small value if all p_{mk}'s are close to zero or one. A small Gini value indicates that a node contains observations from a single class predominantly. An alternative to the Gini index is entropy, defined as

$$D = -\sum_{k=1}^{K} p_{mk} \log p_{mk}.$$

Note that $D \geq 0$. Like Gini index, entropy also takes small value if all p_{mk}s are close to zero or one. While pruning the tree, we can use any of the three methods. However, the classification error rate is preferable if the goal is to maximize the accuracy of the tree.

The following R code implements the decision tree regression model using the **rpart** function in **rpart**-package in R. We assume the underlying relationship between y

and x is

$$y = \mathbf{sin}(x) + \epsilon,$$

where $\epsilon \sim$ **nrom**(mean $= 0$, sd $= 0.3$). We pretend that we do not know the original relationship between the y and x. We model the relation using the **tree regression** model as discussed above. We present the estimated curve in Fig. 17.8. Clearly, the **tree regression** model can capture the nonlinear relationship between x and y.

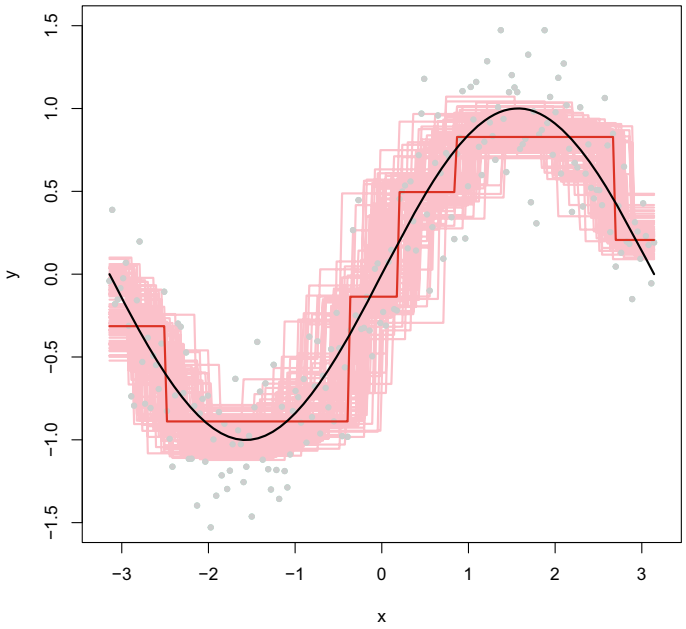


Fig. 17.8 The stepped red curve is the estimated tree regression curve of the underlying relationship between x and y. The 100 s of pink stepped curve are the Bootstrap regression tree curve and the smooth black curve is the true function. Clearly the tree regression model and random forest estimates of y approximate the nonlinear relationship well

In Fig. 17.8 the red curve, looks like the step function, is the estimated decision tree of the underlying relationship between x and y. The hundreds of pink stepped curve is the Bootstrapped decision tree.

```
## simulate data
set.seed(1)
N = 200
x = seq(-pi,pi,length.out=N)
z = sin(x)
y = z +rnorm(N,mean=0,sd=0.3)
D = cbind.data.frame(x=x,y=y)
plot(NULL,xlim=c(-3.2,3.2),ylim=c(-1.5,1.5),xlab='x',ylab='y')
points(D,col='aquamarine',pch=20)
## fit tree regression model and predict y
library(rpart)
tree_mod = rpart(y~x,data = D)
y_hat=predict(tree_mod,newdata = D)
lines(x,y_hat,col='red',lwd=2)
## fit random forest (Bootstrap on tree regression)
B=100
for(b in 1:B){
  id_star= sort(sample(1:N,N,replace = T))
  D_star = D[id_star,]
  tree_mod_b = rpart(y~x,data = D_star)
  y_hat_b=predict(tree_mod_b,newdata = D_star)
  lines(x,y_hat_b,col='pink',lwd=2)
}
points(D,col='grey',pch=20)
lines(x,y_hat,col='red',lwd=2)
lines(x,z,lwd=2)
```

17.7.2 Random Forest

Random forests present an improvement over the *decision tree*, by using the idea of the *bootstrap statistics* with a tweak that addresses *multicollinearity* and performs an intrinsic *feature selection*. Suppose

$$\mathcal{D} = \{(\mathbf{x}_i, y_i) | i = 1, 2 \ldots, n\}$$

is the training dataset, where $\mathbf{x}_i = (x_{i1}, x_{i2}, \ldots, x_{ip})$. We draw B many bootstrap dataset from the training dataset $\mathcal{D}$. Each bootstrap dataset contain m data points out of n data points and p' features out of p features from $\mathcal{D}$ by simple random sample scheme, where

$$\mathcal{D}_b^* = \{(\mathbf{x}_{i^*}, y_{i^*})|i^* = 1, 2 \ldots, m\}, \text{ such that } \mathbf{x}_{i^*} = (x_{i^*1}, x_{i^*2}, \ldots, x_{i^*p'}).$$

Suppose T_b is the decision tree trained on the bootstrap sample $\mathcal{D}_b^*$. Note that the tweak is we randomly sample p' many features out of p features. So if we have two trees, say T_b and $T_{b'}$ trained on two separate bootstrap dataset $\mathcal{D}_b^*$ and $\mathcal{D}_{b'}^*$, the two trees can be trained on very different set of features. Now final assessment will prefer the tree which will have better predictive power. In this way the two trees get decorrelated and avoid multicollinearity. Now out of B many bootstrap datasets, we train B many trees, i.e.,

$$\mathcal{T} = \{T_1, T_2, \ldots, T_B\},$$

where the class of $\mathcal{T}$ is called the *random forest*. If we have a test point $\mathbf{x}_0$, we make prediction from each tree and then choose the majority class predicted as the final prediction from $\mathcal{T}$.

17.8 Exercises

1. Suppose we r_p is the portfolio return calculated for 250 d and r_m is the market return calculated over exact same days. We consider the first 125 d of data as training dataset and latest 125 d of data as test data. We fit linear regression model to data, i.e.,

$$r_p = \alpha + \beta r_m + \epsilon.$$

   We also fit a quadratic regression, i.e.,

$$r_p = \alpha + \beta r_m + \gamma r_m^2 + \epsilon.$$

   (a) Suppose the true relationship between r_p and r_m is linear. Consider training mean square error (MSE) for linear and quadratic regression. Would you expect training MSE for linear regression to be lower than quadratic regression? Justify your answer.
   (b) Consider test MSE for linear and quadratic regression for (a). Would you expect test MSE for linear regression to be lower than quadratic regression? Justify your answer.
   (c) Suppose the true relationship between r_p and r_m is nonlinear; but we do not know what is the exact relationship between r_p and r_m? Consider training mean square error (MSE) for linear and quadratic regression. Would you expect training MSE for linear regression to be lower than quadratic regression? Justify your answer.
   (d) Consider test MSE for linear and quadratic regression for (c). Would you expect test MSE for linear regression to be lower than quadratic regression? Justify your answer.

2. If the Bayes decision boundary is linear, do we expect LDA to perform better than QDA on the training set? What about the test set? Run a simulation study.
3. If the Bayes decision boundary is nonlinear, do we expect LDA to perform better than QDA on the training set? What about the test set? Run a simulation study.

Chapter 18
Linear Systems



Many problems of computational finance, such as pricing of an asset, can be formulated as a system of linear equations. Hence, solving the system of linear equations is important in computational finance. An investor would like to assess if the price of a stock is less than its expected level. If the investor sees the stock is already overpriced, then the chance that it will appreciate further will be less and the investor would like to sell the stock. Other investors would also like to sell the stock as they also have the same public information. The stock will fall back to its expected level. On the contrary, if the stock is underpriced, then many investors would like to buy the stock with the assumption that the price will rise to its expected level. In finance, this is known as 'Asset Pricing,' and corresponding mathematical models are known as 'Capital Asset Pricing Models' (CAPM). Essentially, we can formulate the CAPM as a system of equations. It explains the expected risk and returns for a single asset or portfolio with respect to a benchmark. We can estimate the risk premium of assets using the CAPM, provided the following assumptions are correct!

1. As an investor, we face no transaction cost, and there are no taxes. This assumption is not valid. However, for large investors often transaction cost is very negligible.
2. The market is 'perfectly competitive.' That is all the market participants are price takers, and nobody can influence the price. This assumption is questionable in developing countries. However, markets in the developed nations are near 'perfectly competitive.'
3. All investors are 'rational.' This assumption is questionable. Because it means that investors' emotion is not going to affect their decision about the buying and selling, of the stock. 'Behavioral finance' tries to investigate this aspect of finance.
4. The market allows unlimited Short Selling. That means, as an investor, we can sell any number of shares that we want. This assumption is not realistic because no market in the world would allow unlimited Short Selling.

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/978-981-19-2008-0_17.

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

R. Sen and S. Das, *Computational Finance with R*, Indian Statistical Institute Series,
https://doi.org/10.1007/978-981-19-2008-0_17

5. All investors have homogenous expectations. Investors can borrow and lend unlimited amounts at the risk-free rate. This assumption is also impractical just like the previous one.
6. Assets are infinitely divisible. Holding fractional shares of a company is possible. This assumption is a theoretical assumption. It helps us to obtain the theoretically optimal solution. However, actual the solution may be suboptimal.

All the assumptions of the CAPM are questionable. This set of assumptions attracts criticism from the practitioners. Researchers attempted to relax the assumption and developed new models. Often these attempts resulted in very complicated models. In spite of unrealistic assumptions, CAPM is still popular and in practice. This is because CAPM is simple, easily interpretable and we can apply it in different situations. Also, it is easy to carry out for Big data.

Passive Investment Strategy with Market Portfolio

Suppose the market value of all traded assets is ₹500 crore and market value of stock *XYZ* is ₹3 crore. Then we can have a portfolio with weight allocated to stock *XYZ* as (3×100/500) 0.6%. Exchange-traded fund (ETF) follows such type of portfolio weighting scheme. Typically, ETFs which mimic the market index like Nifty 50 (benchmark for Indian stock market) can be considered as 'market portfolio.' This kind of investment strategy is known as a passive investment strategy.
If we believe that the market is efficient, then according to the 'efficient market hypothesis' (EMH) we cannot consistently outperform the market portfolio. Hence, it is not cost-effective to invest in stock, which may need an extra transaction cost. We will be better off to invest in the market portfolio through some ETF which mimics the market index. The capital market line (CML) creates a new, efficient frontier which combines the market portfolio with the risk-free asset.

18.1 Capital Market Line

We can make an ideal investment decision by allocating funds between *risk-free asset* and the *market portfolio* through the capital market line (CML). The CML assumes that all investors agree on the optimal risky portfolio, as they have similar expectations about returns, volatility, and correlations of all assets. This universally agreed upon optimal risky portfolio is called the '**market portfolio**,' denoted as M. *Note that the market index can be viewed as the market portfolio (aka benchmark portfolio).* 'Excess expected return' means the amount by which the expected return of the portfolio exceeds the risk-free return. It is also known as **risk premium**. Consider an efficient portfolio that allocates a proportion ω of its assets to the market portfolio (M) and $(1-\omega)$ to the risk-free asset. Then

$$R_P = \omega R_M + (1-\omega)R_F = R_F + \omega(R_M - R_F).$$

If we take expectation and variance on both sides of the equation, we have

$$\mathbb{E}(R_P) = R_F + \omega(\mathbb{E}(R_M) - R_F), \tag{18.1}$$

and

$$\sigma_P^2 = \omega^2 \sigma_M^2.$$

That is $\omega = \frac{\sigma_P}{\sigma_M}$ and replacing it in Eq. (18.1), we have the CML

$$\mathbb{E}(R_p) = R_F + \left[\frac{\mathbb{E}(R_M) - R_F}{\sigma_M}\right]\sigma_P,$$

where R_p is the return of the portfolio, R_F is the return of risk-free asset, R_M is the return of market portfolio or market index or benchmark portfolio, σ_M is the standard deviation of return on the market portfolio, and σ_p is the standard deviation of return of holding a portfolio. The *risk premium* of holding portfolio is $\mathbb{E}(R_p) - R_F$, and the risk premium of the market portfolio is $\mathbb{E}(R_M) - R_F$. The slope of CML is known as the market price of risk. The slope is more popularly known as the Sharpe ratio (SR), where

$$SR = \frac{\mathbb{E}(R_M) - R_F}{\sigma_M}.$$

In order to estimate the expected return for an efficiently diversified portfolio, the CML is very useful. Optimal way to invest using CAPM strategy is as follows:

1. Decide on σ_P such that $0 \leq \sigma_P \leq \sigma_M$
2. Calculate $\omega = \frac{\sigma_P}{\sigma_M}$.
3. Invest $\omega \times 100\%$ in an index fund, which mimics the market return. The rest of the $(1 - \omega) \times 100\%$ invest in risk-free sovereign bond or money market fund.

Example 18.1.1 Rekha is an investment advisor. Her client Sekhar wants to invest ₹10,000 for one year. One-year rate of return from risk-free sovereign infra bonds is 6% and the expected return from benchmark Nifty 50 index ETF is 11%. The annual volatility (or standard deviation) of the Nifty 50 index ETF is 20%.

According to the notation above, $R_F = 6\%$ and $\mathbb{E}(R_M) = 11\%$. Rekha suggests two strategies to Sekhar. One is an aggressive strategy with 70% weight in the market portfolio, i.e., in Nifty 50 index ETF and the other is a defensive strategy with 25% weight in Nifty 50 index ETF.

Rekha presents the following analysis of expected return and volatility to Sekhar.

Aggressive Strategy
Expected Return: $\mathbb{E}(R_P) = R_F + \omega(\mathbb{E}(R_M) - R_F) = 6 + 0.70 * (11 - 6) = 9.5\%$ and
Volatility: $\sigma_P = \sqrt{\omega^2\sigma_M^2} = \sqrt{0.7^2 \times 20^2} = 14\%$
Defensive Strategy
Expected Return: $\mathbb{E}(R_P) = R_F + \omega(\mathbb{E}(R_M) - R_F) = 6 + 0.25 * (11 - 6) = 7.25\%$ and
Volatility: $\sigma_P = \sqrt{\omega^2\sigma_M^2} = \sqrt{0.25^2 \times 20^2} = 5\%$

As Sekhar finds the volatility risk of the aggressive strategy is high, he asked Rekha what is the probability that portfolio return will be less than the risk-free asset, sovereign infra bonds? That is $P(R_p < R_F) =$? Sekhar finds the probability for defensive and aggressive strategies are as follows:

Aggressive $P(R_P < R_F)$=`pnorm(6,mean=9.5,sd=14)=0.401`
Defensive $P(R_P < R_F)$=`pnorm(6,mean=7.25,sd=5)=0.401`

Sekhar argues, if the probabilities are same for both the strategies, then why should he prefer one strategy over another! According to Sekhar, this could be viewed as an alternative measure of risk and it says risk for both strategies are same, which does not make sense to him.

Rekha replies that it is not the right approach to test the strategies. He should look for the probability that return is negative, i.e., $P(R_p < 0)$ and the probabilities are:

Aggressive $P(R_P < 0)$=`pnorm(0,mean=9.5,sd=14)=0.2487`
Defensive $P(R_P < 0)$=`pnorm(0,mean7.252,sd=5)=0.0735`

Here the defensive strategy has less risk of making a negative return compared to the aggressive strategies. Sekhar is not fully convinced.

In order to understand why $P(R_p < R_F)$ is the same for both strategies, we need to prove the following result.

Result For all $0 \leq \omega \leq 1$, $P(R_p < R_F) = P(R_M < R_F) = p$ where p is a constant.

The result can be easily proved, and therefore, we leave it as an exercise.

18.2 Systematic Risk: Beta

We have developed CML by combining the market portfolio and risk-free asset. Now we consider, combining the ith risky asset with weight ω_i and market portfolio with weight $(1-\omega_i)$. The return of this portfolio is

$$R_P = \omega_i R_i + (1-\omega_i) R_M.$$

The expected return is

$$\mathbb{E}(R_P) = \omega_i \mathbb{E}(R_i) + (1-\omega_i)\mathbb{E}(R_M),$$

and the volatility risk is

$$\sigma_P = \sqrt{\omega_i^2 \sigma_i^2 + (1-\omega_i)^2 \sigma_M^2 + 2\omega_i(1-\omega_i)\sigma_{i,M}}.$$

Note the $\sigma_{i,M}$ is the covariance between ith risky asset and the market portfolio. If we differentiate the expected return and risk with respect to weight, we have $\frac{\partial \mathbb{E}(R_P)}{\partial \omega_i} = \mathbb{E}(R_i) - \mathbb{E}(R_M)$ and

$$\frac{\partial \sigma_P}{\partial \omega_i} = \frac{1}{2\sigma_P}\left\{2\omega_i\sigma_i^2 - 2(1-\omega_i)\sigma_M^2 + 2(1-2\omega_i)\sigma_{i,M}\right\}.$$

Therefore

$$\frac{\partial \mathbb{E}(R_P)}{\partial \sigma_P} = \frac{\frac{\partial \mathbb{E}(R_P)}{\partial \omega_i}}{\frac{\partial \sigma_P}{\partial \omega_i}} = \frac{\sigma_P\left(\mathbb{E}(R_i) - \mathbb{E}(R_M)\right)}{\omega_i\sigma_i^2 - (1-\omega_i)\sigma_M^2 + (1-2\omega_i)\sigma_{i,M}}.$$

Choose $\sigma_P = \sigma_M$ and $\omega_i = 0$, then

$$\left.\frac{\partial \mathbb{E}(R_P)}{\partial \sigma_P}\right|_{\omega_i=0} = \frac{\sigma_P\left(\mathbb{E}(R_i) - \mathbb{E}(R_M)\right)}{\sigma_{i,M} - \sigma_M^2}.$$

Also $\omega_i = 0$, imply tangent portfolio therefore $\left.\frac{\partial \mathbb{E}(R_P)}{\partial \sigma_P}\right|_{\omega_i=0}$ must equal to the slope of CML, i.e.,

$$\frac{\sigma_M\left(\mathbb{E}(R_i) - \mathbb{E}(R_M)\right)}{\sigma_{i,M} - \sigma_M^2} = \frac{\mathbb{E}(R_M) - R_F}{\sigma_M},$$

which can be expressed as

$$\frac{\mathbb{E}(R_i) - \mathbb{E}(R_M)}{\mathbb{E}(R_M) - R_F} = \frac{\sigma_{i,M} - \sigma_M^2}{\sigma_M^2},$$

i.e.,

$$\mathbb{E}(R_i) = R_F + \frac{\sigma_{i,M}}{\sigma_M^2}(\mathbb{E}(R_M) - R_F),$$

i.e.,

$$\mathbb{E}(R_i) = R_F + \beta_i(\mathbb{E}(R_M) - R_F),$$

is the Security Market Line (SML) and defines beta as $\beta_i = \frac{\sigma_{i,M}}{\sigma_M^2}$. The SML is the representation of the CAPM, where $\mathbb{E}(R_i) - R_F$ is the risk premium of the ith risky asset, which is a product of its beta and risk premium of the market portfolio $(\mathbb{E}(R_M) - R_F)$. Therefore, beta measures the riskiness of the asset and reward for taking that risk. The beta is also a measure of how aggressive the risky asset is. That is

$$\begin{aligned} \beta_i > 1 &\implies \text{aggressive}, \\ \beta_i = 1 &\implies \text{fair}, \\ \beta_i < 1 &\implies \text{defensive}. \end{aligned}$$

Example 18.2.1 Christina wants to invest $ 2000 for one year. She decides to invest money in the stock market. Bob suggested the stock of two companies (i) IBM and (ii) Apple. Bob suggests the S&P 500 to be the market portfolio or benchmark portfolio. On the benchmark portfolio, the corresponding betas are 0.78 and 1.28, respectively.
Bob suggests if Christina wants to be conservative than she should invest in IBM stock. Because on a given day, if the benchmark portfolio S&P 500 index drops by 1%, then she can expect that IBM stock will fall only by 0.78%. Similarly, if the S&P 500 rise by 1%, then IBM stock rise by 0.78%. On the contrary, if Christina wants to be aggressive, then she should buy Apple stock. Because on a given day, if the benchmark portfolio S&P 500 index drops by 1%, then she can expect that Apple stock will fall by 1.28%. Similarly, if the S&P 500 rise by 1%, then Apple stock rise by 1.28%. However, Christina believes that IBM is too conservative. She wants an asset whose beta is about 0.9. Bob finds an easy solution for her. He suggests investing some money (say ω) in IBM and rest in Apple. Bob solves this issue by solving the following equation: $\omega \times \beta_{\text{IBM}} + (1 - \omega) \times \beta_{\text{Apple}} = 0.9$ i.e.,

$$\omega = \frac{0.9 - \beta_{\text{Apple}}}{\beta_{\text{IBM}} - \beta_{\text{Apple}}} = 0.76.$$

So Bob suggests that Christina could invest ($ 2000×0.76) = $ 1520 in IBM and rest in Apple.

Table 18.1 Portfolio allocation

	IBM (%)	Apple (%)	Microsoft (%)	Intel (%)
Weights	20	30	25	25

18.2.1 CAPM and Beta Using R

In `stock_treasury` dataset, we find the column `UST_Yr_1`. This variable is the one-year spot rate of US Treasury yield curve. We can consider this as the risk-free rate of return. We divide it by 100 to convert it from a percentage and then by 250 to a daily rate. Note that we do not take a log and difference of yield-curve rate. We should not treat it as price because these are itself rate reported as an annual percentage.

```
data<-read.csv("stock_treasury.csv")

# Risk Free Rate is in percentage and annualised.
# So the following conversion is required.
Rf<-data$UST_Yr_1/(100*250)

ln_rt_snp500<-diff(log(data$SnP500))-Rf
ln_rt_ibm<-diff(log(data$IBM_AdjClose))-Rf
ln_rt_apple<-diff(log(data$Apple_AdjClose))-Rf
ln_rt_msft<-diff(log(data$MSFT_AdjClose))-Rf
ln_rt_intel<-diff(log(data$Intel_AdjClose))-Rf

## log-return of the portfolio
ln_r = cbind(ln_rt_ibm,ln_rt_apple,ln_rt_msft,ln_rt_intel)
w = c(0.2,0.3,0.25,0.25)
ln_rt_portf = ln_r%*%w
```

The vector `ln_rt_apple, ln_rt_ibm, ln_rt_msft, ln_rt_intel` are the excess return of Apple, IBM, Microsoft, and Intel stocks over the risk-free rate of return. The vector `ln_rt_portf` is the portfolio return with the following portfolio weights, presented in Table 18.1.

The vector `ln_rt_snp500` is the excess return of market index the S&P 500 over the risk-free rate. We can fit the CAPM using `lm` function in `stats` library.

```
capm_ibm<-lm(ln_rt_ibm~ln_rt_snp500)
capm_ibm_analysis<-coefficients(summary(capm_ibm))
capm_ibm_analysis <- round(capm_ibm_analysis,digits = 5)
rownames(capm_ibm_analysis)<-c("alpha","beta")
## Result of capm using lm() for IBM
capm_ibm_analysis

##       Estimate Std. Error  t value Pr(>|t|)
## alpha -0.00049    0.00057 -0.84619  0.39826
## beta   1.01345    0.05812 17.43637  0.00000
```

Table 18.2 Adjusted *R*-square, alpha, and beta of portfolio

	IBM	Apple	Microsoft	Intel	Portfolio
Alpha	−0.00	−0.00	0.00	−0.00	0.00
P-value of alpha	0.40	0.92	0.28	0.96	0.84
Beta	1.01	1.15	1.23	1.00	1.11
P-value of beta	0.00	0.00	0.00	0.00	0.00
Adjusted *R*-square	0.55	0.45	0.47	0.41	0.72

For IBM, (see Table 18.2) we find that $\hat{\beta} = 1.01$. If the market returns increase by 1%, we can expect the IBM returns to increase by approximately 1.01%, and if the market return falls by 1%, we can expect the IBM returns to fall by approximately 1.01%. Hence, investors holding IBM stock can be considered conservative investors, compared to investors that hold the Microsoft. The beta for Microsoft is 1.23. That means **systematic risk** for Microsoft is higher than IBM.

We can see that the alpha for all four stocks is approximately equal to 0, and the corresponding *p*-values are all larger than 0.1. So we do not reject the null hypothesis (H_0: $\alpha = 0$) at the 0.05 level of significance. This test suggests that data is consistent with CAPM, and all four stocks are fairly priced. The adjusted *R*-square of the IBM, Apple, Microsoft, and Intel are 55%, 45%, 47% , and 41%, respectively. It means CAPM explains about 55% of the variability of IBM stocks. Similarly, **market risk** factor explains about 45, 47, and 41% of the variability of Apple, Microsoft, and Intel.

As we consider a portfolio with four stocks where the weights of IBM, Apple, Microsoft, and Intel are 0.2, 0.3, 0.25, and 0.25, respectively. The portfolio alpha and beta as presented in Table (18.2) 0.00 and 1.11, respectively, though we can estimate the portfolio alpha and beta as weighted average; however, we cannot calculate the adjusted *R*-square of the portfolio as a weighted average. To determine the adjusted *R*-square of the portfolio, we calculate the portfolio return (as `ln_rt_portf`) and implement the CAPM for the portfolio return separately. The result presented in Table 18.2. The adjusted *R*-square for the portfolio is 72%, which is distinctly higher than the adjusted *R*-square of IBM, Apple, Microsoft, and Intel. That is the market return can explain 72% variability of the portfolio (Fig. 18.1).

We can see that the portfolio's adjusted R-square is higher than the adjusted R-square of all the assets in the portfolio. This fact is the manifestation of diversification benefit. If we increase the number of assets in the portfolio, the adjusted R-square increases further. It means diversification eliminates the idiosyncratic risk.

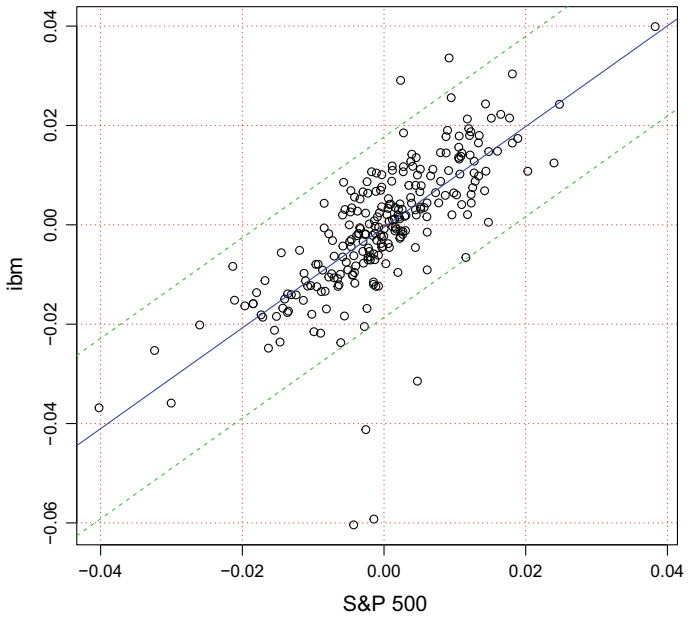


Fig. 18.1 CAPM Analysis of IBM. The adjusted R-square indicates that about 55% of the variability in the daily log-return of the IBM can be explained by the market movement; i.e., daily log-return of the S&P 500 index can explain the movement of the IBM stock price. So it can be attributed to systematic risk. The rest of the variability can be attributed to the idiosyncratic risk

18.2.2 Security Market Line

Suppose there are N risky assets with return $R_1, R_2, \ldots, R_N$ with weights $\omega_1, \omega_2, \ldots, \omega_N$. The return of the portfolio is

$$R_P = \omega_1 R_1 + \omega_2 R_2 + \cdots + \omega_N R_N.$$

Let R_M be the return of the market index. According to SML,

$$\mathbb{E}(R_i) = R_F + \beta_i \mathbb{E}(R_M - R_F),$$

which means,

$$\mathbb{E}(R_P) = R_F + \left(\sum_{i=1}^{N} \omega_i \beta_i\right) \mathbb{E}(R_M - R_F),$$

i.e.,

$$\mathbb{E}(R_P) = R_F + \beta_P \mathbb{E}(R_M - R_F),$$

where $\beta_P = \left(\sum_{i=1}^{N} \omega_i \beta_i\right)$. As we consider the portfolio with the 20% weight on IBM, 30% weight on Apple, 25% weight on Microsoft, and 25% on Intel, to calculate the portfolio beta, we first calculate the β for the assets of the portfolio, then we calculate the portfolio beta by summing up the individually weighted β_i. In this case, the portfolio beta is

$$\begin{aligned}\beta_P &= 0.2 \times \beta_{\text{IBM}} + 0.3 \times \beta_{\text{Apple}} + 0.25 \times \beta_{\text{Microsoft}} + 0.25 \times \beta_{\text{Intel}} \\ &= 0.2 \times 1.01 + 0.3 \times 1.16 + 0.25 \times 1.23 + 0.25 \times 1.01 = 1.11.\end{aligned}$$

The individual asset β_i is the slope parameters of regression model as estimated using `lm()`.

```
plot(ln_rt_snp500,ln_rt_ibm,xlab="S&P 500",ylab="ibm")
abline(capm_ibm,col="blue")
grid(col="red")
## Adjusted R-Squareed
summary(capm_ibm)$adj.r.squared

## [1] 0.548935

rse<-summary(capm_ibm)$sigma
al <-capm_ibm$coefficients[1]-2*rse
b <- capm_ibm$coefficients[2]
abline(a= al,b= b,col=3,lty=2)
au <-capm_ibm$coefficients[1]+2*rse
abline(a=au,b=b,col=3,lty=2)
```

18.2.3 Can Beta Be Negative?

It is possible for a beta to have negative value. For example, the prices of companies mining precious metals like gold tend to move opposite to the market. This type of association could yield negative beta for such stocks. The beta of a portfolio is

$$\beta_P = \sum_{i=1}^{N} \omega_i \beta_i,$$

so adding an asset with a negative beta to a portfolio can reduce the portfolio's risk. We can create a negative beta by taking a short position on an asset of the portfolio. By constructing a well-diversified portfolio of assets with positive and negative beta such that portfolio beta is close to 0, one can obtain a portfolio with effectively no-risk. But such portfolio only earns risk-free rate of return. This fact is an example of 'law of one price' or 'no-arbitrage.' We can consider a portfolio where we take a long position in IBM, Apple, and Intel with the equal weight of 50% each and take a short position on Microsoft with another 50% (Table 18.3).

Table 18.3 Portfolio allocation with short position at Microsoft

	IBM	Apple (%)	Intel (%)	Microsoft (%)
Weights	50	50	50	−50

$$\begin{aligned}\beta_P &= 0.5 \times \beta_{\text{IBM}} + 0.5 \times \beta_{\text{Apple}} - 0.5 \times \beta_{\text{Microsoft}} + 0.5 \times \beta_{\text{Intel}} \\ &= 0.5 \times 1.01 + 0.5 \times 1.16 - 0.5 \times 1.23 + 0.5 \times 1.01 = 0.975.\end{aligned}$$

We can see that a portfolio with a short position in one asset reduces the portfolio beta (β_P =`0.975`) to less than 1; even if all asset beta is greater than 1.

18.2.4 Beta Hedging

We can hedge or mitigate the systematic risk of the equity portfolio using the portfolio's beta (β_P). We can use the futures contract on a stock index (Index futures) for this purpose. The following formula provides the number of futures contracts required to completely hedge the systematic risk of the equity position, that is

$$\text{No. of Contracts} = \beta_P \times \left(\frac{\text{Portfolio Value}}{\text{Futures Price} \times \text{Contract Multiplier}} \right),$$

where contract multiplier is the number of shares one contract represent. That is for S&P 500 one contract multiplier is 250. Similarly the contract multiplier for Nifty 50 is 50.

Example 18.2.2 Bill is the manager of a portfolio of size $ 50 million that has a beta of 1.11 respect to the S&P 500. The S&P 500 is trading at 2050, and the multiplier is 250. Bill would like to hedge the exposure to market risk for the next month. The portfolio is long, so the Bill should construct a short hedge. That means to sell the future contact. The number of contracts to sell is equal to:

$$1.11 \times \frac{50{,}000{,}000}{2050 \times 250} \approx 108$$

18.2.5 Achieving Target Beta

If we use the beta of CAPM as a systematic risk measure, then hedging is nothing but reduction of portfolio beta. We can determine the number of futures contracts that are required to adjust the portfolio beta with the following formula:

$$\text{No. of Contracts} = (\beta_{\text{Target}} - \beta_P) \times \left(\frac{\text{Portfolio Value}}{\text{Futures Price} \times \text{Contract Multiplier}} \right)$$

The negative sign of the contract means we need to sell the contract and positive sign means we should buy the contract.

Example 18.2.3 We continue with the above Example 18.2.2 and suppose Bill wants to reduce the portfolio beta at 0.8 then

$$(0.8 - 1.11) \times \frac{50{,}000{,}000}{2050 \times 250} \approx -30$$

Bill needs to sell 30 future contracts (or take a short position at 30 future contracts) to reduce the portfolio beta at 0.8.

18.2.6 Effect of Outlier on Beta

Outlier return might affect the estimation of beta, which might lead to a wrong assessment of the systematic risk of the asset or portfolio. To hedge or adjust the portfolio's market risk exposure, the beta determines how many future contracts we should buy or sell. The concept of outlier was introduced in sect. 12.6. An outlier might lead to some biased estimation of beta and wrong estimation of a number of contracts for hedging. If we consider the portfolio allocation presented in Table (18.1), the beta of the portfolio is `1.11`. However, we can fit the CAPM by robust regression using M-estimator technique in `rlm` function of `MASS` library.

```
## Fit roboust linear regression using rlm
library(MASS)
capm_portf_robust<-rlm(ln_rt_portf~ln_rt_snp500)
capm_portf_robust_est<-summary(capm_portf_robust)
capm_portf_robust_est<-coefficients(capm_portf_robust_est)
rownames(capm_portf_robust_est)<-c("alpha","beta")
round(capm_portf_robust_est,digits=5)


##          Value Std. Error  t value
```

```
## alpha 0.00016    0.00036  0.43339
## beta  1.10030    0.03663 30.03600
```

The robustly estimated beta of the portfolio is `1.1003`. If we use this as the portfolio beta, then Bill's calculation in Examples 18.2.2 and 18.2.3 changes.

Example 18.2.4 We continue with the above Example 18.2.3 and as Bill wants to reduce the portfolio beta at 0.8, if he uses portfolio beta to be 1.11, then

$$1.11 \times \frac{50{,}000{,}000}{2050 \times 250} \approx 108$$

Bill needs to sell 108 future contracts (or take a short position at 108 future contracts) to hedge the systematic risk. However, if Bill uses the robustly estimated beta (`1.1003`) from `rlm`, then

$$1.1003 \times \frac{50{,}000{,}000}{2050 \times 250} \approx 107,$$

Bill needs to sell 107 contracts.

18.3 Measuring Active Return: Alpha

To measure the performance of a particular security or mutual fund against that of a benchmark portfolio, the 'security characteristic lines' (SCL) are very useful. The SCL equation is

$$\mathbb{E}(R_i) - R_F = \alpha_i + \beta_i(\mathbb{E}(R_i) - R_F)$$

where α_i is the active return.

In the model, the $\alpha_i < 0$ implies that the security is earning less than its assumed expected risk. We can see, if the risk premium of the benchmark is 0, then the security has a negative return. The investors would consider such an asset as expensive because the risk premium is lower than the risk from its beta. The investor will sell the asset or short the security. As a result of this, the price of the asset decline until the risk premium of the asset increase, so that negative α_i will vanish.

In the model, the $\alpha_i > 0$ implies that the security is earning more than its assumed expected risk. That is, if the market risk premium is 0, then the security has a positive return. The investors would consider such assets as undervalued because the risk premium is higher than the risk from its beta. The investor will buy the asset or long the security. As a result of this, the price of the assets rises until the risk premium of the asset decreases, so that positive α_i will vanish.

The $\alpha_i = 0$ implies that the security is earning as much as its assumed expected risk premium. As the market is efficient, any nonzero α_i will lead the price of the

stock rapidly move to fair prices and consequently nonzero α_i will quickly move to zero. It will be difficult for anybody to capitalize on such nonzero α_i. Hence, it will not be possible for anybody to achieve better results than benchmark, consistently for an extended period.

Another application of α is to measure the performance of the fund manager. If the α is greater than 0, then that will be attributed to the skill and the performance of the manager. Many believe that the market is not perfectly efficient. The small inefficiencies might be important to a manager of a large mutual fund portfolio. Suppose the size of a mutual fund portfolio is $ 5 billion. If the manager can outperform the market by one basis point, (i.e., 0.01%) that would result in $ 5 million more in return. This objective is certainly worth achieving. An investment manager should avoid losing money and make a certain sum of money for the fund over and above the benchmark. The additional risk premium above the expected premium after the beta adjusted premium of the benchmark is known as the 'alpha.' The 'performance fee' of a manager of an actively managed fund is a function of alpha.

18.4 Pricing Theory and System of Linear Equations

The security characteristic line (SCL) is nothing but statistical regression model

$$R_i - R_F = \alpha_i + \beta_i(R_M - R_F) + \epsilon_i,$$

where the risk premium of the ith security is a linear function of market risk premium. In the CAPM, market risk premium is the only systematic risk factor. However, there are other risk factors that affect the risk premium of security. Factor models generalize the CAPM by allowing more factors than simply the market risk and the unexplained unique risk of each asset. The model is defined as,

$$R_i - R_F = \beta_{0i} + \beta_{1i}x_1 + \cdots + \beta_{pi}x_p + \epsilon_i,$$

where $x_1, x_2, ..., x_p$ are the p factors. Typically, there are two major objectives. If the objective is to predict the return; any factor which has a good predictive power could be used in the model. A model with good predictive power means a higher chance of a good return on investment. For example, experts in behavioral finance try to measure the market sentiment with good predictive power and they use the 'market sentiment' as a factor in the model. However, if the objective is to analyze the relationship between the return and factors, one should try to find the theoretical reason for including a factor in the model. Possible factors could be (i) growth rate of GDP; (ii) inflation rate; (iii) spot rate spread of the long-term treasury bill and corporate bond. The portfolio return is

$$R_p = \omega_1 R_1 + \omega_2 R_2 + \cdots + \omega_N R_N$$

and the risk premium of the portfolio is

$$R_p - R_F = \omega_1(R_1 - R_F) + \omega_2(R_2 - R_F) + \cdots + \omega_N(R_N - R_F).$$

We can substitute $(R_i - R_F)$ by the factor model as,

$$R_i - R_F = \beta_{0i} + \beta_{1i}x_1 + \cdots + \beta_{pi}x_p + \epsilon_i,$$

where portfolio factor model can be presented as,

$$\begin{aligned} R_P - R_F = \beta_0 &+ \left(\sum_{i=1}^{N} \omega_i \beta_{1i}\right) x_1 \\ &+ \left(\sum_{i=1}^{N} \omega_i \beta_{2i}\right) x_2 + \cdots + \left(\sum_{i=1}^{N} \omega_i \beta_{pi}\right) x_p + \sum_{i=1}^{N} \omega_i \epsilon_i, \end{aligned}$$

i.e.,

$$R_P - R_F = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p + \epsilon, \tag{18.2}$$

where $\beta_j = \sum_{i=1}^{N} \omega_i \beta_{ji}$ is the beta of the jth risk factor to the portfolio.

18.4.1 Factor Models and Risk Matrix

We express the factor model in Eq. (18.2) in the matrix notation as follows:

$$r_t = Bx_t + \epsilon_t, \tag{18.3}$$

where

$$r_t = \begin{bmatrix} r_{t1} \\ \vdots \\ r_{tN} \end{bmatrix} = \begin{bmatrix} R_{t1} - R_F \\ \vdots \\ R_{tN} - R_F \end{bmatrix},$$

is the vector of excess return over risk-free rate on day t for N many assets,

$$B = \begin{bmatrix} \beta_{11} & \dots & \beta_{1p} \\ \vdots & \dots & \vdots \\ \beta_{N1} & \dots & \beta_{Np} \end{bmatrix}_{N \times p},$$

is the asset's sensitivity matrix of factors,

$$x_t = \begin{bmatrix} x_{t1} \\ \vdots \\ x_{tp} \end{bmatrix}$$

is the vector of risk factors. The portfolio return of day t is

$$r_{Pt} = \omega^T r_t = \sum_{i=1}^{N} \omega_i r_{ti},$$

where $\omega^T = \{\omega_1, \ldots, \omega_N\}$ is the weight vector, such that $\sum_{i=1}^{N} \omega_i = 1$ and ω_i is the weight of the ith asset in the portfolio. This can be expressed as

$$r_{Pt} = \omega^T r_t = \sum_{i=1}^{N} \omega_i r_{ti}.$$

This can be expressed as

$$r_{Pt} = \omega^T B x_t + \omega^T \epsilon_t.$$

The expected portfolio return is

$$\mathbb{E}(r_P) = \omega^T B x = \beta x,$$

where $\omega^T B = \beta$ is the portfolio's sensitivity to risk factor x. The portfolio's covariance matrix is

$$\begin{aligned} D(r_P) &= \omega^T B \Sigma_x B^T \omega + \omega^T \Sigma_\epsilon \omega, \\ &= \omega^T \left[B \Sigma_x B^T + \Sigma_\epsilon \right] \omega, \\ &= \omega^T \Sigma_P \omega, \end{aligned}$$

where Σ_ϵ is the diagonal matrix whose each diagonal elements are the unexplained volatility of each asset, Σ_x is the covariance matrix of risk factor, $B \Sigma_x B^T$ is the volatility that due to risk factor.

18.4.2 Benefit of Diversification

We can express the unexplained part of the portfolio volatility $\omega^T \Sigma_\epsilon \omega$ as

$$\omega^T \Sigma_\epsilon \omega = (\omega_1, \ldots, \omega_N)^T \begin{bmatrix} \sigma_1^2 & \ldots & 0 \\ \vdots & \ldots & \vdots \\ 0 & \ldots & \sigma_N^2 \end{bmatrix} \begin{pmatrix} \omega_1 \\ \ldots \\ \omega_N \end{pmatrix} = \sum_{i=1}^{N} \omega_i^2 \sigma_i^2.$$

If $\omega_i = \frac{1}{N}, \ \forall\, i$, then

$$\omega^T \Sigma_\epsilon \omega = \frac{1}{N^2} \sum_{i=1}^{N} \sigma_i^2 \leq \frac{\sigma_M^2}{N},$$

where $\sigma_M^2 = \max(\sigma_1^2, \ldots, \sigma_N^2) < \infty$ is the maximum of all unexplained variances, which is finite. Therefore

$$\omega^T \Sigma_\epsilon \omega \to 0, \text{ as } N \to \infty.$$

Here N is the total number of assets in the portfolio and as N increases the unexplained volatility approaches to zero. Therefore the unexplained volatility risk of a large portfolio with many assets will be small. We can see for 'diversified portfolio' most of the volatility can be explained by risk factor(s) and as a result the Adjusted R-square of portfolio increases as number of asset increases. For any portfolio we can split the total volatility into two parts:

$$D(r_P) = \omega^T B \Sigma_x B^T \omega + \omega^T \Sigma_\epsilon \omega.$$

First part is the volatility due to systematic risk factor, and the second part is the unexplained volatility risk.

18.4.3 Solving Factor Model as System of Equation

The factor model of the pricing theory as presented in Eq. (18.3) is nothing but the **multivariate regression model**. Equation (18.3) represents tth day or tth row of the database. The full matrix representation of the Asset Pricing model for N assets, with p factor, observed over T days, presented as follows:

$$\mathbf{r}_{T,N} = \mathbf{X}_{T,p} B_{p,N} + \epsilon_{T,N}, \tag{18.4}$$

where $\mathbf{r}_{T,N} = ((r_{t,j}))_{T,N} = ((R_{t,j} - R_f))_{T,N}$ is the risk-premium matrix of N stocks, over T many days, where

$$\mathbf{r}_{T,N} = \begin{pmatrix} r_{1,1} & r_{1,2} & \cdots & r_{1,N} \\ r_{2,1} & r_{2,2} & \cdots & r_{2,N} \\ \vdots & \vdots & \ddots & \vdots \\ r_{T,1} & r_{T,2} & \cdots & r_{T,N} \end{pmatrix};$$

is the risk-premium matrix of N assets observed over T-days;

$$\mathbf{X}_{T,p} = \begin{pmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,p} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{T,1} & x_{T,2} & \cdots & x_{T,p} \end{pmatrix};$$

is the matrix of p risk factor (risk premium of market indices and other risk factors) observed over T days;

$$B_{p,N} = \begin{pmatrix} \beta_{1,1} & \beta_{1,2} & \cdots & \beta_{1,N} \\ \beta_{2,1} & \beta_{2,2} & \cdots & \beta_{2,N} \\ \vdots & \vdots & \ddots & \vdots \\ \beta_{T,1} & \beta_{T,2} & \cdots & \beta_{T,N} \end{pmatrix};$$

is the N asset's sensitivity (or elasticity) matrix of factors (aka beta matrix); and

$$_{T,N} = \begin{pmatrix} \epsilon_{1,1} & \epsilon_{1,2} & \cdots & \epsilon_{1,N} \\ \epsilon_{2,1} & \epsilon_{2,2} & \cdots & \epsilon_{2,N} \\ \vdots & \vdots & \ddots & \vdots \\ \epsilon_{T,1} & \epsilon_{T,2} & \cdots & \epsilon_{T,N} \end{pmatrix};$$

is the white noise matrix. Our main objective is to estimate B. We can resort to the least-square method. The objective function can be defined as

$$\Delta = (\mathbf{r} - \mathbf{X}B)^T(\mathbf{r} - \mathbf{X}B)$$

is the sum of squares of errors of N assets. We should come up with an estimate of $B_{p,N}$, such that Δ should be minimum and we problem the method as minimization problem. That is

$$\frac{\partial \Delta}{\partial B} = 0$$
$$\frac{\partial}{\partial B}(\mathbf{r} - \mathbf{X}B)^T(\mathbf{r} - \mathbf{X}B) = 0$$

yields the following system of equations (aka., normal equations),

$$(\mathbf{X}^T\mathbf{X})B = \mathbf{X}^T\mathbf{r}.$$

We obtain the estimator for B by solving the normal equations as,

$$\hat{B} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{r}.$$

The R-packages mostly implement this least-square solution $\hat{B}$ for coefficient matrix B.

18.4.4 Stress Testing

Stress testing is a common technique to check how agile or sensitive the portfolio is to extreme market crash. The question like, if the market drops by 3%, what is the probability that the portfolio will drop by more than 3.5%? That is,

$$\mathbb{P}(r_p \leq -3.5 | r_M \leq -3) = ?$$

It requires conditional probability model on portfolio return r_p. We can consider CAPM to model the portfolio return. Also, we have to make an assumption about the probability distribution of market index return. Generally, the probability distribution of market index return is never Gaussian. Typically it follows a distribution that has much heavy tail distribution. We may assume Laplace distribution over the Market return.

$$\begin{aligned} r_p &\sim N(\alpha + \beta r_M, \sigma^2), \\ r_M &\sim \text{Laplace}(\mu, \lambda). \end{aligned}$$

The α, β, and σ^2 can be estimated using the least-square methods in `lm()`. The maximum likelihood estimate (MLE) of μ and λ is median and mean absolute deviation about median, respectively.

```
alpha_hat<-capm_portf_tabl[1,1]
beta_hat<-capm_portf_tabl[2,1]
sigma_hat<-sqrt(mean(capm_portf$residuals^2))

mu_hat<-median(ln_rt_snp500)
lambda_hat<-mad(ln_rt_snp500)
```

Now as the parameters of the model estimated above, we can use a Monte Carlo simulation technique to simulate portfolio return and following Monte Carlo estimates

$$\hat{p} = \frac{1}{N^*} \sum_{s=1}^{N^*} \mathbb{I}(r_p^s < -3.5 | r_M^s < -3),$$

can be used to estimate the $p = \mathbb{P}(r_p \leq -3.5 | r_M \leq -3)$. Note that N^* is the simulation size, r_p^s is the sth simulated portfolio return and r_M^s is the sth simulated market return. The estimated value of p is $\hat{p} \approx 0.673$.

```
    ## Monte Carlo Simulation to
    ## simulate return
    sim.size<-10000
    require(rmutil,quietly=TRUE)

    set.seed(321)
    ## simulate market return using
    ## Laplace distribution
    r_M<-rlaplace(n=sim.size
                  , m=mu_hat, s=lambda_hat)

    ## simulate portfolio return
    r_p<-rep(NA,sim.size)
    for(s in 1:sim.size){
      mu<-alpha_hat+beta_hat*r_M[s]
      r_p[s]<-rnorm(1,mean=mu
                    ,sd=sigma_hat)
    }

    sim.return<-cbind.data.frame(r_M,r_p)
    sim.return.sub<-subset(sim.return,r_M<=-3)
    n_star<-nrow(sim.return.sub)
    f_star<-nrow(sim.return.sub[sim.return.sub$r_p<=-3.5,])
    ## P(rp < -3.5 | rM < -3)
    f_star/n_star
```

```
## [1] 0.6725664
```

```
    ## P(rp > -3.5 | rM < -3)
    1- (f_star/n_star)
```

```
## [1] 0.3274336
```

In Fig. 18.2, (a) Scatter plot of market and portfolio return. The green points indicate that market return is less than -3% and portfolio return is less than -3.5%; (b) the QQ-norm plot of the portfolio return indicates that the simulated portfolio return does not follow Gaussian distribution. In fact, it is much heavy tail than the Gaussian distribution. Note that in the model, the conditional probability distribution of portfolio return is Gaussian. But the marginal distribution of the portfolio return is not Gaussian.

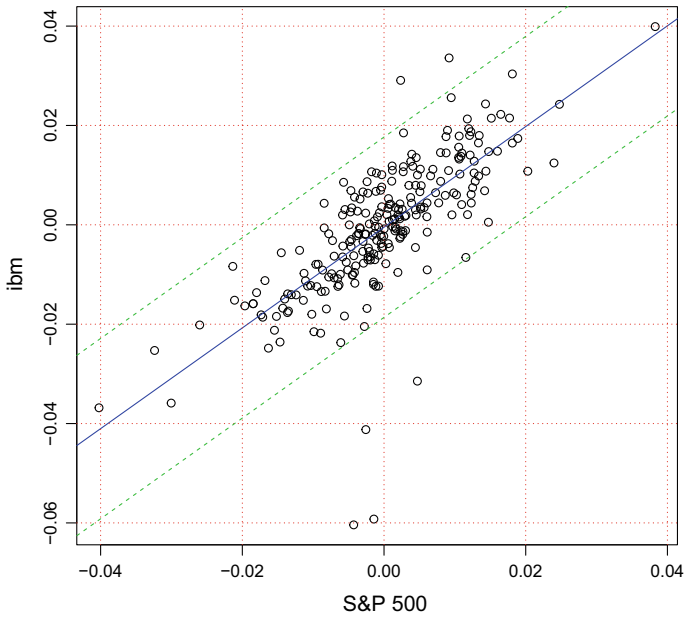


Fig. 18.2 **a** Scatter plot of market and portfolio return. The green points indicate that market return is less than -3%, and portfolio return is less than -3.5%; **b** QQ-norm plot of portfolio return. The plot clearly indicates that the simulated portfolio return does not follow Gaussian distribution. In fact it is much heavy tail than the Gaussian distribution. Note that in the model conditional probability distribution of portfolio return is Gaussian. But marginal distribution is not Gaussian; **c** Histogram of log-return of market index; **d** Histogram of log-return of Portfolio

18.5 Summary

- System of linear equations is an important tool in computational finance. The Capital Asset Pricing Model (CAPM) is an application of linear equations to evaluate if an asset is underpriced or overpriced.
- The 'beta' of CAPM is a measure of systematic risk, and root-mean-square error (RMSE) is a measure of idiosyncratic risk. The nonzero 'alpha' measure of a stock indicates that the stock is mispriced. Also, it indicates the existence of in-efficiency in the market. A diversified portfolio eliminates the idiosyncratic risk. However, the portfolio still faces systematic risk due to market movement.
- The Adjusted R-square of portfolio increases with the number of stocks in the portfolio. Increasing value of the Adjusted R-square of portfolio implies diminishing effect of idiosyncratic risk.
- Typically most of the software packages (including the `R`) uses the ordinary least-square methods to estimate the 'beta.' But the least-square method is sensitive to the outlier. Hence one should consider a robust estimation method.

- The CAPM or factor model-based linear equation method can be used to develop the stress testing system. The stress testing system helps us to identify how sensitive the portfolio is with respect to major market crash.

18.6 Exercises

1. Anuja wants to invest ₹10,000 for one year. She decides to invest the money in the stock market. Karina suggested the stock of two companies (i) Asian Paint and (ii) Berger Paint. She suggests Nifty 50 to be the 'market portfolio' or 'benchmark portfolio.' With the benchmark portfolio, the corresponding betas are 1.51 and 0.98, respectively. Anuja wants to invest in such a way so that the beta of her investment would be 1.2. How much should she invest Asian Paint and Berger Paint?
2. Suppose variance of Asian Paint return is 2.9%, variance of Berger Paint. Return is 3.23% and covariance is 2.03%. Calculate Anuja's portfolio volatility.
3. If Nifty 50 drops by 2%, then how much portfolio value is expected to drop?
4. What is the portfolio 3%-VaR if Nifty 50 drops by 2%?

Chapter 19
Time Series



Time series refers to any group of statistical information accumulated at regular intervals. It is a quantitative method used to determine patterns in data collected over time. When these patterns are projected to arrive at an estimate for the future, it is called forecasting. The quality of the forecasting is strongly related to the information that can be extracted and used from past data. In Fig. 19.1 we present a time series plot of the closing prices of IBM stock from daily data for the period 2001–2012.

19.1 Components

Variations involved in time series analysis can arise of the following sources.

1. **Secular trend**: This refers to the overall general tendency if the series is observed over a long period of time. There can be a persistent overall upward or downward pattern arising due to population, technology, etc. Generally trend is of several years duration. Trends can be linear or curvilinear. We discuss more on fitting the trend at the end of this section.
2. **Cyclical variation**: Some time series have cyclical movements that do not follow any regular pattern but move in a somewhat unpredictable manner. The period is more than one year, and we see repeating up and down movements. These are mostly seen in financial or economic data with duration of 2–10 years and are due to interactions of factors influencing the economy.
3. **Seasonal variation**: This refers to periodic patterns of change within a year that tend to be repeated from year to year. A regular pattern of up and down fluctuations can occur due to weather, customs, etc. In order to detect seasonal variation, time intervals must be measured in small units (days, weeks, months, or quarters). To measure seasonal variation, after the trend has been removed, we can use the average of the observations for each season/month. However this leads to a lot

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/978-981-19-2008-0_18.

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

R. Sen and S. Das, *Computational Finance with R*, Indian Statistical Institute Series,
https://doi.org/10.1007/978-981-19-2008-0_18



Fig. 19.1 Time series plot of closing prices of IBM stock

of fluctuations, and it is better to use the ratio-to-moving average method. Here we explain the method assuming monthly data, which can be generalized. The method entails finding the average ratio that a particular month contributes to the 12 month moving average and then taking the average of these ratios for each fixed month.

4. **Irregular variation**: This describes the variation which is completely unpredictable and changing in a random manner. It is unsystematic residual fluctuations occurring due to random variation or unforeseen events like union strike and war.

For the time series data of the closing prices of IBM stock, presented in FIG. 19.1, the following R code uses the function `stl` in the library `forecast` to separate the components and plot them in Fig. 19.2.

```
library(xts)
library(forecast) IBM.df =
read.csv(file="ibmdata.txt",header=TRUE,sep=",")
head(IBM.df)
IBM = xts(IBM.df$Close,as.Date(IBM.df$Date,format="%m/%d/%Y"))
plot(IBM)
plot(stl(ts(as.numeric(IBM), deltat=1/250), s.window=35))
```

Estimating Trend

When the form of the trend is known, say linear, quadratic, or exponential, then one can write down that form as a model. The parameters of the model can then be estimated by minimizing some error as the least squares. For example, linear trend can be estimated in the same fashion as linear regression with the predictor variable as time and the response as the given time series.

When the form is unknown and flexible, then smoothing techniques are used. The most common such technique is the moving average. This is a series of arithmetic means that provide an overall impression of data over time. The fitted value at any point of time is the arithmetic mean of the k preceding values. The value of k controls

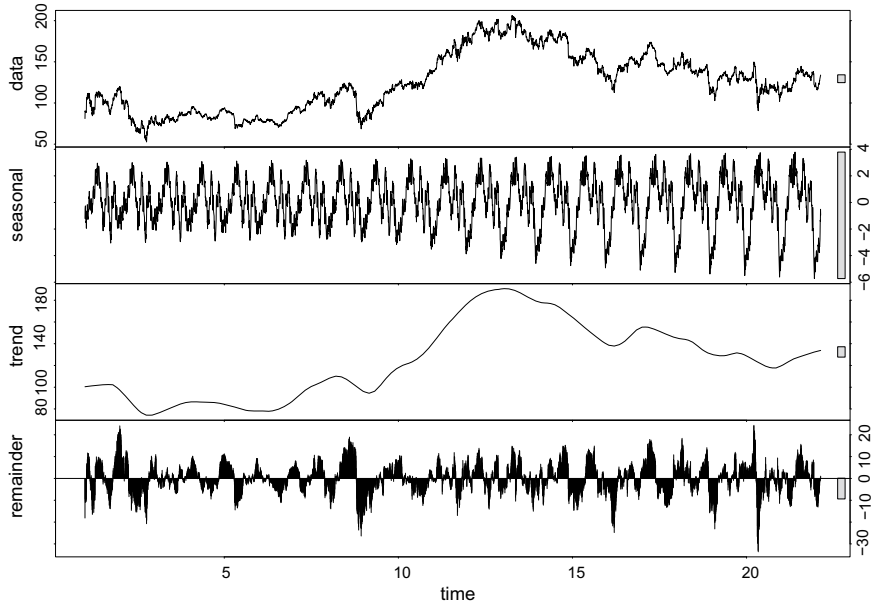


Fig. 19.2 Components of time series of closing prices of IBM stock

the extent of smoothing. If k is large, the fitted series is very smooth, but the difference from the original series is large.

Another technique is exponential smoothing where the fitted value is an exponentially weighted average of the past values, that is

$$\hat{Y}_t = \frac{\sum_{i=1}^{k} a^{t-i} Y_{t-i}}{\sum_{i=1}^{k} a^{t-i}},$$

where $0 < a < 1$. Here both k and a have to be chosen as before. Now there is a new parameter a that controls the relative weight of the past observations. The higher the value of a, the more weightage is given to observations in the far past. Methods of fitting higher-degree polynomials or splines can be used, as discussed in Sect. 1.2.1.

19.2 Stationary Time Series

A strictly stationary time series is one for which the probabilistic behavior of $\{X_t, X_{t+1}, \ldots, X_{t+k}\}$ is same as that of $\{X_s, X_{s+1}, \ldots, X_{s+k}\}$ for every s, k, t.

For a time series X_t, we define the mean as $\mu_X(t) = E(X_t)$, the variance as $\sigma_X^2(t) = \text{Var}(X_t)$ and the autocovariance as $\gamma_X(s,t) = \text{Cov}(X_s, X_t)$. Thus $\sigma_X^2(t) = \gamma_X(t,t)$. Autocorrelation is defined as

$$\rho_X(s,t) = \frac{\gamma_X(s,t)}{\sigma_X(t)\sigma_X(s)}.$$

A time series is weakly stationary if

1. $\mu_X(t)$ does not depend on t; that is, it is a constant as a function of t.
2. $\gamma_X(t+h,t)$ does not depend on t, but only on the lag h.

Another related concept is the partial autocorrelation function (PACF). For a weakly stationary series the PACF of order h is the correlation between X_t and X_{t+h} conditional on $\{X_{t+1},\ldots,X_{t+h-1}\}$, that is, the intermediate terms. We express it as follows.

$$\phi_X(h) = \text{corr}(X_t, X_{t+h} \mid X_{t+1},\ldots,X_{t+h-1}).$$

19.2.1 White Noise

The time series generated from uncorrelated variables with mean zero and common variance is used as a model for noise in engineering applications, where it is called white noise. Here $\mu_X(t) = 0$ and

$$\gamma_X(t+h,t) = \begin{cases} 0 & \text{if} \quad h \neq 0, \\ \sigma^2 & \text{if} \quad h = 0. \end{cases}$$

The white noise series is not smooth enough to serve as a plausible model for most applications. However it is a good starting point for understanding more complicated and realistic time series.

A related model, which is non-stationary, is random walk. This is the cumulative sum of white noise. Let X_t be a white noise series. Then the series Y_t given below is a random walk,

$$Y_t = X_t + Y_{t-1}, \quad Y_1 = X_1,$$

The random walk model is a good fit to time series of interest rates and it is difficult to beat this simple model with respect to forecast accuracy.

The following code simulates paths of a white noise and a random walk with Normal distribution. The plots are presented in Figs. 19.3 and 19.4 respectively.

```
x = rnorm(1000,0,1)
x = as.ts(x)
y[1] = x[1]
for (i in 2:1000){
  y[i]=x[i]+y[i-1]
}
ts.plot(x,ylab="White noise")
ts.plot(y,ylab="Random walk")
```

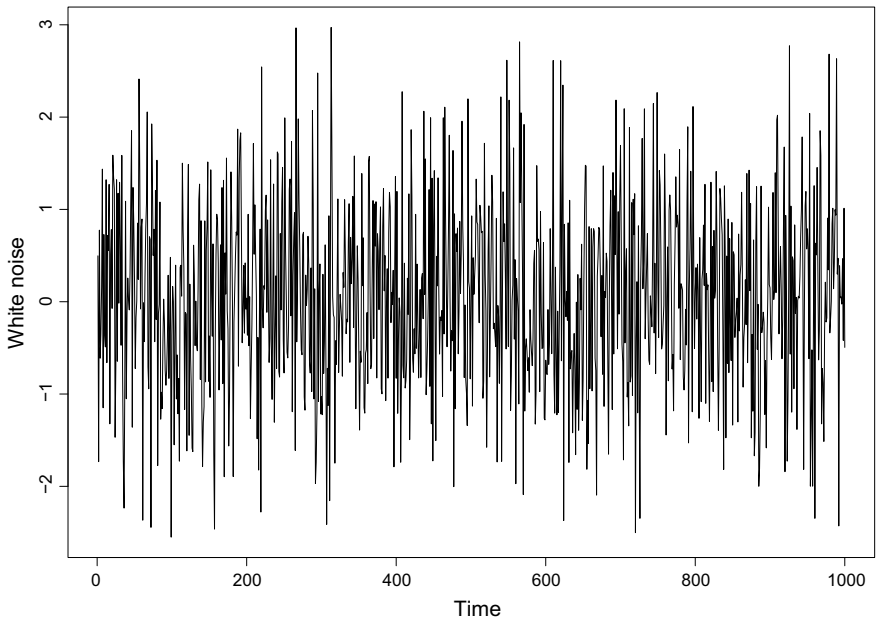


Fig. 19.3 Simulation of white noise with normal distribution

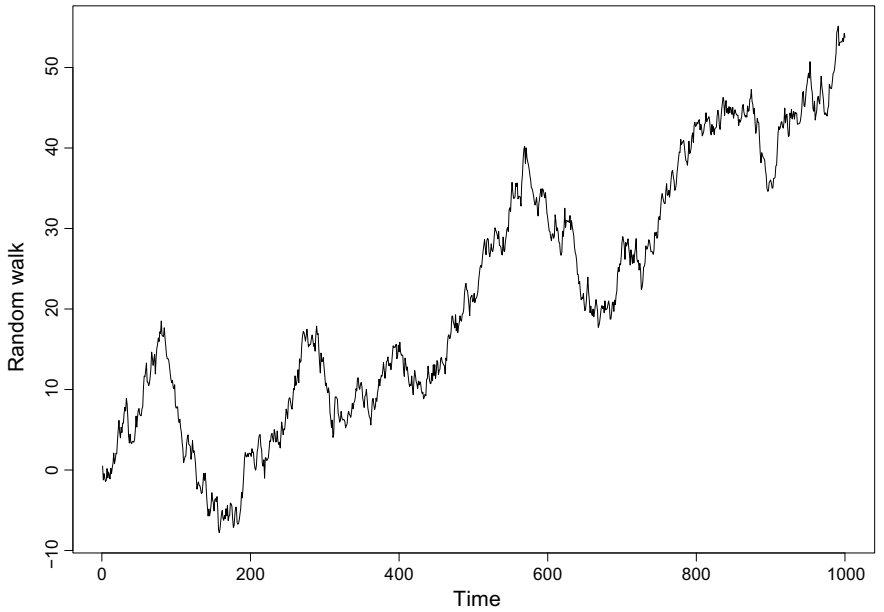


Fig. 19.4 Simulation of random walk with normal distribution

19.2.2 Moving Average Models

A moving average (MA) model smoothes a white noise series. For example, let X_t be a white noise series with variance σ^2. Then Y_t given below is a moving average model of order one,

$$Y_t = X_t + 0.2X_{t-1}.$$

Here $\mu_Y(t) = 0$ and

$$\gamma_Y(t+h,t) = \begin{cases} 0.2\sigma^2 & \text{if} \quad h = \pm 1, \\ (1+0.2^2)\sigma^2 & \text{if} \quad h = 0, \\ 0 & \text{if} \quad \mid h \mid \geq 2. \end{cases}$$

In general, a moving average model of order q has the form

$$Y_t = X_t + \theta_1 X_{t-1} + \cdots + \theta_q X_{t-q},$$

for constants $\{\theta_1, \ldots, \theta_q\}$. The process is stationary and $\gamma_Y(t+h,t) = 0$ for $\mid h \mid \geq q+1$.

The function `ARMAacf` in R can be used to find the ACF and PACF of MA models, provided the order and parameters are known. In the following code we take an MA(2) model with parameters 1.5 and 0.75 and present the ACH ad PACF plots in Fig. 19.5.

```
par(mfrow=c(1,2))
ACF
ma2_acf_th=ARMAacf(ar=0, ma=c(1.5,0.75),lag.max=5,pacf=FALSE)
plot(ma2_acf_th,type="h",xlab="lag",ylab="Theor. ACF")
abline(h=0)
PACF
ma2_pacf_th=ARMAacf(ar=0,ma=c(1.5,0.75),lag.max=30,pacf=TRUE)
plot(ma2_pacf_th,type="h",xlab="lag",ylab="Theor. PACF")
abline(h=0)
```

19.2.3 Autoregressive Models

Autoregressive models are based on the idea that the current value of the series can be explained as a linear function of p past values and an additive white noise error. The following is an AR(p) model with mean μ with X_t denoting the white noise,

$$(Y_t - \mu) = \phi_1(Y_{t-1} - \mu) + \cdots + \phi_p(Y_{t-p} - \mu) + X_t.$$

Let us first consider the AR(1) model. For this series to be stationary, we need $\mid \phi_1 \mid < 1$. By iterating backwards repeatedly, we can write

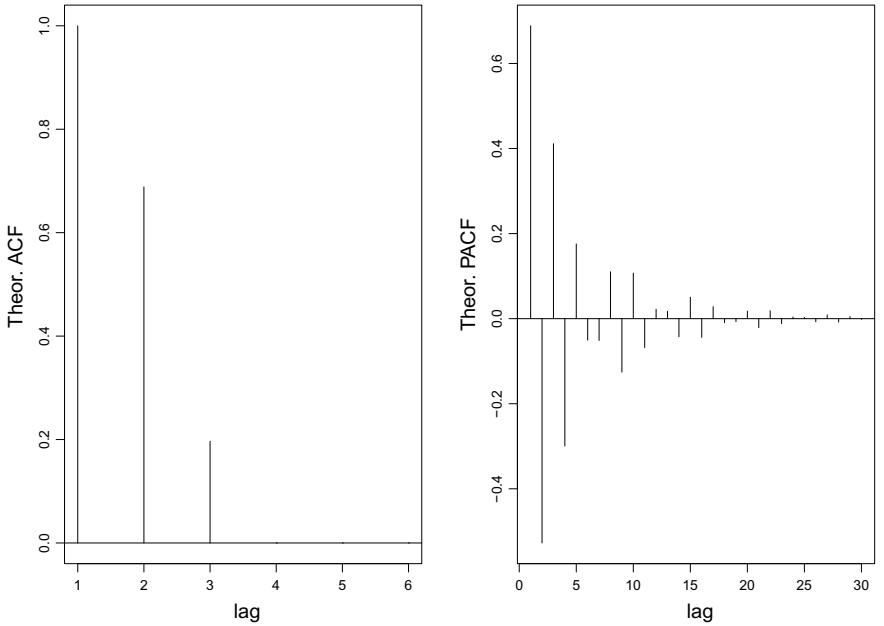


Fig. 19.5 Theoretical ACF and PACF for MA(2) model with $\theta_1 = 1.5$ and $\theta_2 = 0.75$

$$\begin{aligned} Y_t &= \phi_1 Y_{t-1} + X_t \\ &= \phi_1^2 Y_{t-2} + \phi_1 X_{t-1} + X_t \\ &\vdots \\ &= \phi_1^k Y_{t-k} + \sum_{i=0}^{k-1} \phi_1^j X_{t-j}. \end{aligned}$$

Note that Y_t is uncorrelated with $X_{t+1}, X_{t+2}, \ldots$.
Taking expectation on the first line we get, $\mu_Y(t) = \phi_1 \mu_Y(t-1)$.
By stationarity, $\mu_Y(t) = \mu_Y(t-1) = \mu$(say). Hence $\mu = 0$.
Taking variance on the first line we get, $\sigma_Y^2(t) = \phi_1^2 \sigma_Y^2(t-1) + \sigma_X^2$.
By stationarity, $\sigma_Y^2(t) = \sigma_Y^2(t-1) = \sigma_Y^2$(say). Hence $\sigma_Y^2 = \sigma_X^2/(1-\phi_1^2)$.
Using the last line, we get $\text{Cov}(Y_t, Y_{t-k}) = \phi_1^k \sigma_Y^2 = \phi_1^k \sigma_X^2/(1-\phi_1^2)$.
Thus, for the ACF, we have

$$\rho_Y(t, t+h) = \phi_1^h \quad \text{for} \quad h > 0.$$

Thus the ACF decreases exponentially for AR(1) process. For any AR process, the ACF decreases to zero. Also, for an AR(p) process, the PACF equals zero for lags higher than p.

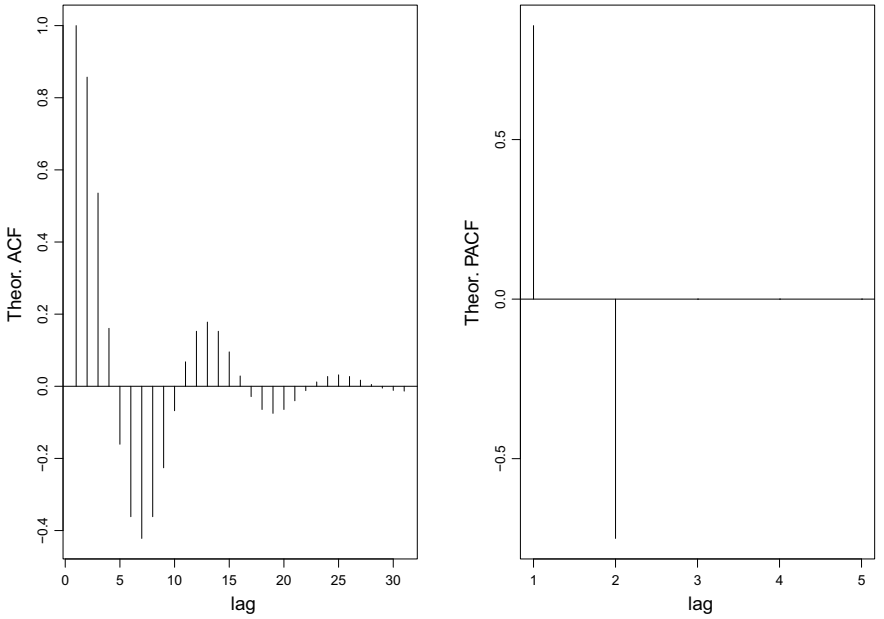


Fig. 19.6 Theoretical ACF and PACF for AR(2) model with $\phi_1 = 1.5$ and $\phi_2 = -0.75$

In the following code we take an AR(2) model with parameters 1.5 and -0.75 and present the ACH ad PACF plots in Fig. 19.6.

```
par(mfrow=c(1,2))
####
#ACF
ar2_acf_th=ARMAacf(ar=c(1.5,-0.75),ma=0,lag.max=30,pacf=FALSE)
plot(ar2_acf_th,type="h",xlab="lag",ylab="Theor. ACF")
abline(h=0)
####
# PACF
ar2_pacf_th=ARMAacf(ar=c(1.5,-0.75),ma=0,lag.max=5,pacf=TRUE)
plot(ar2_pacf_th,type="h",xlab="lag",ylab="Theor. PACF")
abline(h=0)
```

19.2.4 ARMA Models

The time series Y_t is an ARMA(1,1) process if, for X_t being white noise,

$$(Y_t - \mu) - \phi_1(Y_{t-1} - \mu) = X_t + \theta_1 X_{t-1}.$$

Notice that the right-hand side is an MA(1) process. If that is replaced by white noise, then the resulting Y_t would be an AR(1) process. Thus an ARMA(1,1) model is obtained by replacing the white noise in an AR(1) model with an MA(1) process. This is stationary if $\mid \phi_1 \mid < 1$. Similarly we can define an ARMA(p, q) model as follows,

$$Y_t - \phi_1 Y_{t-1} - \cdots - \phi_p Y_{t-p} - \alpha = X_t + \theta_1 X_{t-1} + \cdots + \theta_q X_{t-q}.$$

Here we have substituted $\alpha = \mu(1 - \phi_1 - \cdots - \phi_p)$. The ARMA class of models is quite large, and most stationary time series can be modeled well with them.

19.2.5 ARIMA Models

Sometimes observed series is not stationary, but become stationary with differencing. Given a time series Y_t, the first differenced series is given by $\bigtriangledown Y_t = Y_t - Y_{t-1}$. For example, we have seen that the random walk is non-stationary. But by differencing the random walk, we obtain white noise, which is stationary. In general, the d-th order differencing is obtained by applying the $\bigtriangledown$ operator d times. The integrated ARMA, or ARIMA model, is a broadening of the class of ARMA models to include differencing. A process Y_t is said to be ARIMA(p, d, q) if $\bigtriangledown^d Y_t$ is ARMA(p, q).

19.3 Fitting ARIMA Model to Data

The first step in model fitting is to identify the order. For this purpose we estimate the mean, ACF, and PACF.

The sample mean of an observed stationary time series $y_1, \ldots, y_n$ is calculated as follows and can be used as an estimate for the population mean,

$$\hat{\mu} = \frac{1}{n} \sum_{t=1}^{n} y_t.$$

Similarly, the sample autocovariance of y_t at lag h can be calculated as

$$\hat{\gamma}(h) = \frac{1}{n - \mid h \mid} \sum_{t=1}^{n-|h|} (y_{t+|h|} - \hat{\mu})(y_t - \hat{\mu}), \quad -n < h < n.$$

The sample autocorrelation is given by

$$\hat{\rho}(h) = \frac{\hat{\gamma}(h)}{\hat{\gamma}(0)}.$$

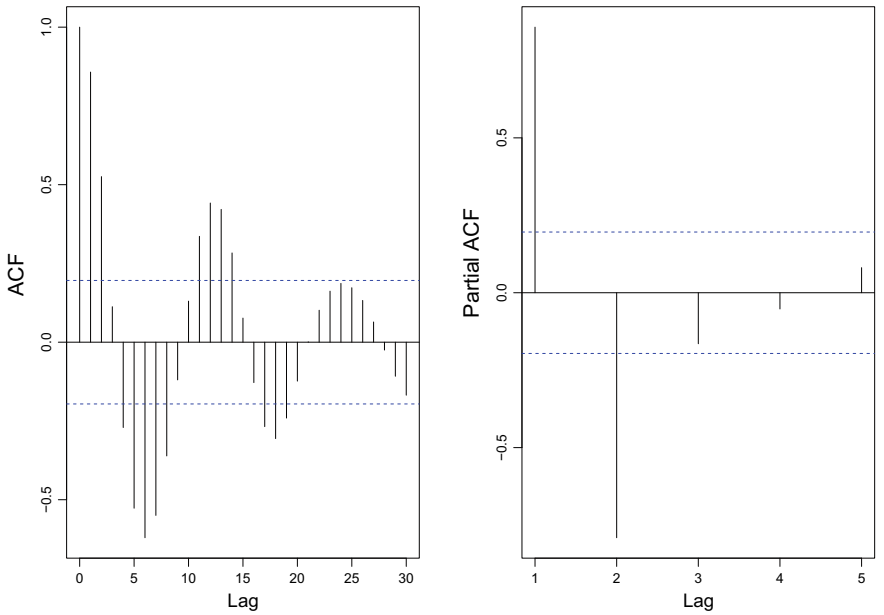


Fig. 19.7 Estimated ACF and PACF for time series of length 100 simulated from AR(2) model with $\phi_1 = 1.5$ and $\phi_2 = -0.75$

The estimate of PACF of order h is obtained by regressing y_t and y_{t+h} on $y_{t+1}, \ldots, y_{t+h-1}$ and then finding the sample correlation between the two series of residuals.

Suppose Y_t is iid with finite variance, then for large n, $\hat{\rho}$ is approximately iid $\mathcal{N}(0, 1/n)$. In the following code, we generate 100 observations from an AR(2) model and plot the estimated ACF and PACF along with the normal error bounds. The plot is presented in Fig. 19.7. Comparing to Fig. 19.6 of theoretical values, we see that the estimates are quite close. In particular the ACF and PACF plots can be used to choose the order of the ARMA model to fit to real data. In this case the ACF tapers off and the PACF is close to zero after lag 2, clearly pointing to an AR(2) model.

```
par(mfrow=c(1,2))
set.seed(123457)
ar2 = arima.sim(list(order=c(2,0,0), ar=c(1.5,-0.75)), n = 100)
acf_smp=acf(ar2,lag.max=30,plot=TRUE, main="")
acf_smp
pacf_smp=acf(ar2,lag.max=5,type=c("partial"),plot=TRUE, main="")
pacf_smp
```

If the series is nonstationary, the ACF will decay to zero very slowly. Suitable differencing will make the series stationary. If a given series is nonstationary, we look at its first difference. If this is stationary, we identify a suitable ARMA(p, q)

model. If the first difference is still nonstationary, we consider the second difference, and so on until we obtain a stationary process.

Once the order has been identified, the next step is to estimate the parameters. The common methods are Yule-Walker, maximum likelihood and least squares. We shall not go into the details of these methods here but use the `arima` function in R. For the simulated dataset of length 100 from AR(2) model, we get the following result. The estimated coefficients are 1.545 and -0.798 while the true parameter values are 1.5 and -0.75.

```
arima(ar2,order=c(2,0,0))
Coefficients:
         ar1      ar2  intercept
      1.5451  -0.7980    -0.0009
s.e.  0.0573   0.0568     0.3358
```

After fitting a model, we need to test if the residuals $\hat{W}_t$ are indeed white noise. For this, we need to test if all the correlations are zero. In practice we test the null hypothesis that $\rho(1) = \cdots = \rho(k) = 0$ for some fixed k vs the alternative that at least one of the correlations is nonzero. One procedure to carry this out is by using the Ljung–Box portmanteau test. The test statistic is

$$Q_{\text{LB}} = n(n+2)\sum_{h=1}^{k}\frac{\hat{\rho}(h)^2}{(n-h)}.$$

Under the null hypothesis Q_{LB} has χ^2 distribution with k degrees of freedom. The test can be performed by the R function `tsdiag` as in the following R code to obtain a plot as Fig. 19.8. In this case we observe that all the ACFs of the residuals are within the confidence bounds and the p-values of the Ljung–Box test for each k is away from zero. So we do not reject the null hypothesis and conclude that the residual series can be treated as white noise.

```
fit=arima(ar2,order=c(2,0,0))
tsdiag(fit)
```

19.4 Forecasting

The aim is to project past trend and seasonal variation into the future, while taking into consideration the inherent inaccuracies of this analysis. For stationary series with dependence, the irregular component can also be used for forecasting. Although the cyclical component does affect the future, it is erratic and difficult to use in forecasting. Note that accounting for most of the variation in a time series of past

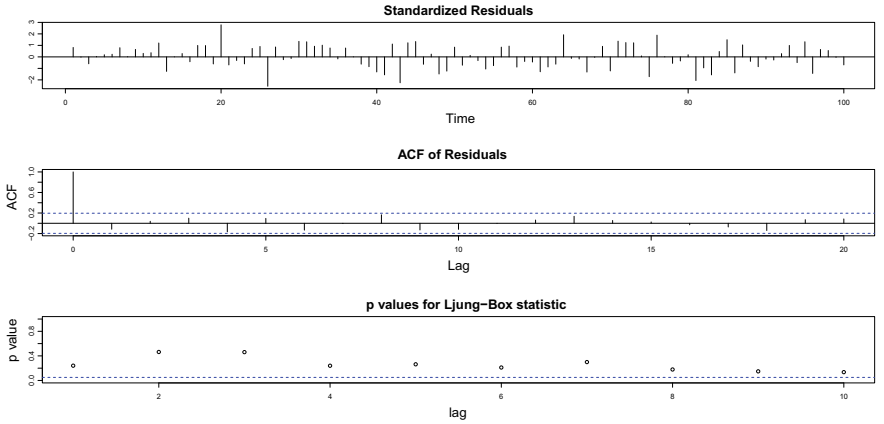


Fig. 19.8 Diagnostic plots and p-values of Ljung–Box test

data does not mean that this same pattern will continue in the future. Thus the best-fitted model may not be the same as the best forecasting model.

The procedure is, therefore, as follows. Estimate trend and generate forecast with estimate of forecast error. Remove trend and then estimate seasonality with estimate of error. Check if the residual is stationary and apply differencing if needed. Fit appropriate stationary model to residual and estimate the error. Generate forecast by putting together the forecasts and errors of the different components.

The following aims should be considered for forecasting. The model with the smallest forecast error should be chosen. Suppose two or more models provide good fit for data, then the simplest model should be selected. This is known as the principle of parsimony. In addition, there should be no deterministic pattern or trend in the forecast error.

We assert that we want the model with the smallest forecast error. However, when we are fitting a model to past data, we do not have future data at hand and do not know what the forecast error is going to be. One way around is to compute the expected log-likelihood of a future observation under each model and then to choose that model which maximizes this quantity. A related quantity is the Akaike information criterion (AIC), and we choose the model that maximizes the AIC. Given data $Y_1, \ldots, Y_n$ from a Gaussian ARMA(p, q) time series, suppose the MLEs of parameters are $\hat{\phi}$, $\hat{\theta}$ and $\hat{\sigma}^2$. Then the AIC is given by

$$AIC = -2 \log L(\hat{\phi}, \hat{\theta}, \hat{\sigma}^2) + 2(p + q + 1).$$

When we fit a model with higher number of parameters, the likelihood increases as the model fits the data better. But the second term of the AIC also goes up. Thus it strikes a balance and avoids overfitting.

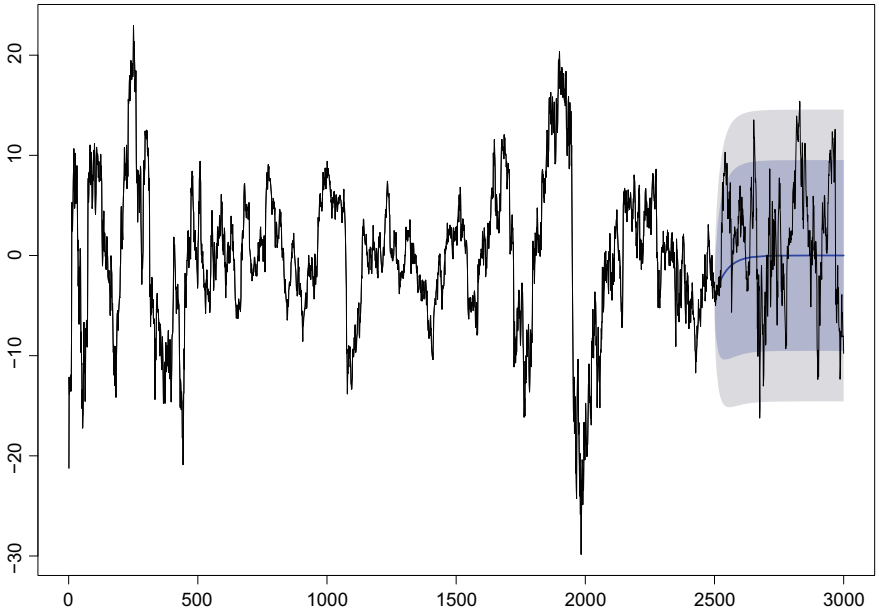


Fig. 19.9 Forecasting the irregular component of IBM data

```
s = stl(ts(as.numeric(IBM), deltat=1/250), s.window=35)
res = ts(as.numeric(s$time.series[,3]))
fit = auto.arima(res[1:2500])
summary(fit)
ARIMA(2,0,0) with zero mean

Coefficients:
         ar1      ar2
      0.9957  -0.0173
s.e.  0.0201   0.0202

sigma^2 estimated as 2.327:  log likelihood=-4603.73
AIC=9213.47   AICc=9213.48   BIC=9230.94
plot(forecast(fit,h=500), main="")
lines(seq(1:3000), res[1:3000])
```

The code above takes the residuals (irregular component) from the trend and seasonality-fitted IBM data. It then fits an ARIMA model to the residuals. It plots the irregular component of the original series along with the forecast for 500 days ahead along with error bounds. The best-fitted model is AR(2), and the output is presented in Fig. 19.9.

19.5 Exercises

1. Which of the components would we use to describe
   (a) the effect of Christmas on a retail department store?
   (b) a 7-year weather pattern?
   (c) the general growth and decline of the steel industry over the last two centuries?
2. Arrange the components of a time series in order of ease of predictability?
3. Find the mean–variance and autocovariance of random walk.
4. Generate a sample of size 100 from an MA(2) model with $\theta_1 = 1.5$ and $\theta_2 = 0.75$. Plot the sample ACF and PACF and compare with the theoretical ACF and PACF in Fig. 19.5.

Chapter 20
High-Frequency Data



Intra-day financial transactions and quotes data have become available since 1990's. These have sparked research on the micro-structure of markets. The two main financial markets that provide high-frequency data are the FOREX and the US equity markets. The latter is contained in the TAQ (Trade And Quote) database which contains all trades (including quantity traded) and quotes for all listed stocks since the early 1990s. Contrary to this centralized market, the FOREX market is decentralized and there are only observations for quote of many currency pairs but no transaction data. An interesting issue of the FOREX market is that it is always open and geographically disperse since the trading day starts in Tokyo, which is followed by the opening of the London market, and finally by New York. Table 20.1 presents the details of Facebook stock from dukascopy.com for the first three minutes of trading on July 6th, 2020. The prices are plotted in Fig. 20.1 and the log prices in Fig. 20.2.

```
data = read.csv('facebook0706.csv', header=TRUE)
datetime = strptime(data[,1], format="%d.%m.%Y %H:%M:%OS")
library(xts)
price = as.xts(data[,2:3], order.by=datetime)
colnames(price) = c("bid","ask")
library(highfrequency)
midprice  = (price[,1]+price[,2])/2
names(midprice) = "midpoint"
midprice1m = aggregatets(midprice, on="minutes", k=1)
plot(midprice1m)
logReturns = makeReturns(midprice1m)
plot(logReturns)
```

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/978-981-19-2008-0_19.

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

R. Sen and S. Das, *Computational Finance with R*, Indian Statistical Institute Series,
https://doi.org/10.1007/978-981-19-2008-0_19

Table 20.1 Bid and Ask prices and volumes for Facebook stock for the first three minutes of trading on July 6th, 2020

GMT time	Ask price	Bid price	Ask volume	Bid volume
13:30:00.726	234.122	233.578	2000	2000
13:30:00.776	234.422	234.168	750	750
13:30:00.943	234.423	234.037	2000	2000
13:30:01.021	234.423	234.347	750	750
13:30:01.071	234.583	234.347	2000	2000
13:30:01.181	234.583	234.267	2000	2000
13:30:01.865	234.582	234.318	2000	2000
13:30:01.990	234.583	234.317	2000	2000
13:30:02.040	234.582	234.268	2000	2000
13:30:02.158	234.582	234.368	750	750
13:30:02.424	234.582	234.397	750	750
13:30:02.528	234.583	234.417	2000	2000
13:30:02.578	234.583	234.407	2000	2000
13:30:02.630	234.582	234.368	750	750
13:30:02.707	234.583	234.427	2000	750
13:30:02.758	234.582	234.368	750	750
13:30:02.960	234.582	234.407	750	750



Fig. 20.1 Price of Facebook stock on July 6th, 2020

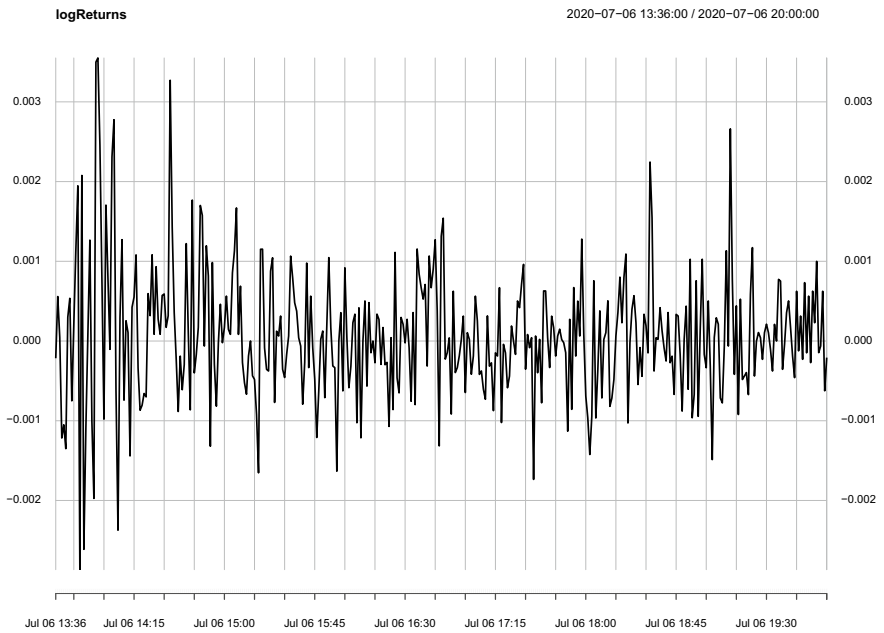


Fig. 20.2 Log price of Facebook stock on July 6th, 2020

20.1 Infill Asymptotics in Geometric Brownian Motion

We start with the standard Black–Scholes model for the stock price process and explain the issues related to estimation of parameters when intra-day data is observed from the trades in the market. Let $X_t = \log(S_t)$ denote the log price process. Then the Black–Scholes model with constant drift μ and volatility σ can be written as

$$X_t = X_0 + \mu t + \sigma W_t,$$

where W_t is standard a Brownian Motion (BM).

The beginning and end of the trading day are denoted by $t = 0$ and $t = T$, respectively. We have n observations, each corresponding to a trade, during the day. We shall assume, for the time being, that these observations are equally spaced. That is, the observation times are $t_{n,i} = i\,\Delta_n$ where $\Delta_n = T/n$. Although this assumption is unrealistic, it helps us understand the ideas and will then be relaxed to general random times of observations as long as they are dense enough. We retain the n in the notation since we are concerned with the case of high frequency, that is, as n tends to infinity.

The observations are therefore $X_{t_{n,i}}$. If we take differences, we get the log returns

$$\Delta X_{t_{n,i+1}} = X_{t_{n,i+1}} - X_{t_{n,i}}, i = 0, \ldots, n-1.$$

The log returns are independent and identically distributed (iid) with law $\mathcal{N}(\mu\Delta_n, \sigma^2\Delta_n)$. The parameters of interest are μ and σ. As described in Sect. 13.1.1, we can obtain the maximum likelihood estimates of these parameters as

$$\hat{\mu}_n = \frac{1}{n\Delta_n}\sum_{i=0}^{n-1}\Delta X_{t_{n,i+1}} = (X_T - X_0)/T,$$

$$\hat{\sigma}_n^2 = \frac{1}{n\Delta_n}\sum_{i=0}^{n-1}(\Delta X_{t_{n,i+1}} - \Delta\bar{X}_{t_n})^2,$$

Note that the estimator for μ does not depend on all the observations, or the sample size. It is a function of only the beginning and ending values of X. Hence increasing the sample size does not affect the estimator. So high-frequency data does not improve, in fact does not affect in any way, the quality of estimation of the drift.

We now proceed to show that $\hat{\sigma}_n^2$ is a consistent estimator (see Sect. 13.1.2 for definition and related results of consistency) of σ^2 for fixed T, as $n \to \infty$. Set $U_{n,i} = \Delta X_{t_{n,i}}/(\sigma\Delta_n^{1/2})$. Then the $U_{n,i}$ are iid with distribution $\mathrm{N}((\mu/\sigma)\Delta_n^{1/2}, 1)$. It follows from standard considerations for normal random variables that $\sum_{i=0}^{n-1}(U_{n,i} - \bar{U}_{n,\cdot})^2$ is χ^2 distributed with $n-1$ degrees of freedom. The latter variable has the same distribution as the sum of $n-1$ independent chi-square random variables, say $Z_1, \ldots, Z_{n-1}$, each with mean 1 and variance 2,

$$\hat{\sigma}_n^2 = \frac{\sigma^2}{n}\sum_{i=0}^{n-1}(U_{n,i} - \bar{U}_{n,\cdot})^2 \stackrel{d}{=} \sigma^2\frac{n-1}{n}\bar{Z}_{n-1}$$

Mean and variance of $\bar{Z}_{n-1}$ are 1 and $2/(n-1)$, respectively. So

$$E(\hat{\sigma_n^2}) = \sigma^2\frac{n-1}{n} \quad \text{and} \quad \mathrm{Var}(\hat{\sigma_n^2}) = \sigma^4\frac{(n-1)}{n^2}.$$

By strong law of large numbers, $\bar{Z}_{n-1}$ converges to the mean 1 of Z_1 almost surely. Note that the factor $(n-1)/n$ converges to 1. Hence $\hat{\sigma}_n^2$ converges to σ^2 almost surely.

Now we want to derive the asymptotic normality of $\hat{\sigma}_n^2$. From the Central Limit Theorem, $\sqrt{n-1}(\bar{Z}_{n-1} - 1)$ converges in distribution to $\mathcal{N}(0, 2)$.

$$\begin{aligned}\sqrt{n}(\hat{\sigma}_n^2 - \sigma^2) &\stackrel{d}{=} \sqrt{n}(\sigma^2 \frac{n-1}{n} \bar{Z}_{n-1} - \sigma^2) \\ &= \sqrt{n}(\sigma^2 \frac{n-1}{n} \bar{Z}_{n-1} - \sigma^2 \bar{Z}_{n-1} + \sigma^2 \bar{Z}_{n-1} - \sigma^2) \\ &= \sqrt{n}(\sigma^2 \frac{n-1}{n} \bar{Z}_{n-1} - \sigma^2 \bar{Z}_{n-1}) + \sqrt{\frac{n}{n-1}} \sqrt{n-1} \sigma^2 (\bar{Z}_{n-1} - 1) \\ &= -\frac{1}{\sqrt{n}} \sigma^2 \bar{Z}_{n-1} + \sqrt{\frac{n}{n-1}} \sqrt{n-1} \sigma^2 (\bar{Z}_{n-1} - 1).\end{aligned}$$

The first term converges to zero as $\bar{Z}_{n-1}$ converges to a constant 1 almost surely and $\frac{1}{\sqrt{n}}$ converges to zero. The fraction in the second term converges to 1. From the Central Limit Theorem, $\sqrt{n-1}(\bar{Z}_{n-1} - 1)$ converges in distribution to $\mathcal{N}(0, 2)$. Using Slutsky's theorem,

$$\sqrt{n}(\hat{\sigma}_n^2 - \sigma^2) \stackrel{d}{\Rightarrow} \mathcal{N}(0, 2\sigma^4).$$

The type of asymptotics as $n \to \infty$ that we are considering is different from the more common approach in time series where the length of the time intervals remains fixed and the number of intervals goes to infinity. For example, one considers daily data and lets the number of days increase. Here the total length of the interval is fixed, say at one day, and the number of sub-intervals goes to infinity. This kind of asymptotics is known as infill asymptotics.

The above result has been shown under the assumptions that the price process follows Black–Scholes model with constant drift and the interarrival times are on a uniform grid. Both these assumptions can be relaxed. Thm I.4.47 (p. 52) of Jacod and Shiryaev (2003) shows that the quadratic variation can be consistently estimated from the data if the underlying process is a semi-martingale. This is also true for quadratic covariation between two processes. Regarding the interarrivals, as long as the length of the longest interval goes to zero, the estimator is consistent.

Let us look at the estimator $\hat{\sigma_n^2}$ again.

$$\begin{aligned}\hat{\sigma}_n^2 &= \frac{1}{n\Delta t_n} \sum_{i=0}^{n-1} (\Delta X_{t_{n,i+1}} - \Delta \bar{X}_{t_n})^2 \\ &= \frac{1}{n\Delta t_n} \left(\sum_{i=0}^{n-1} (\Delta X_{t_{n,i+1}}^2) - n(\Delta \bar{X}_{t_n})^2 \right) \\ &= \frac{1}{T} \left(\sum_{i=0}^{n-1} (\Delta X_{t_{n,i+1}}^2) - n \frac{(X_T - X_0)^2}{n^2} \right) \\ &= \frac{1}{T} \sum_{i=0}^{n-1} (\Delta X_{t_{n,i+1}}^2) - \frac{(X_T - X_0)^2}{nT}.\end{aligned}$$

The second term goes to zero as n increases. The quantity $\sum_{i=0}^{n-1}(\Delta X^2_{t_{n,i+1}})$ is known as the realized volatility and is used to estimate $\sigma^2 T$.

20.2 Microstructure Noise

Transaction prices in markets are observed with noise. This does not pose any particular problem if we are dealing with low-frequency data, like daily or monthly data. However, if we are looking at high-frequency data, then the contribution of the noise to the estimated quadratic variation can be huge. The source of noise is market micro-structure like discrete tick size, lag in arrival and execution of trades, bid-ask spread, and asymmetric information to traders. Observed log prices Y_t are therefore modeled as

$$Y_{t_i} = X_{t_i} + \text{noise} \tag{20.1}$$

and X_t is a semi-martingale. In the special case of the Black–Scholes model, X_t is geometric Brownian motion.

An empirical illustration of the phenomenon of noise is given below. We have seen in the previous section that the realized volatility is a consistent estimator for $\sigma^2 T$. However in practice, this convergence does not hold. To see this, for fixed K, we use grids of the form

$$\mathcal{H}_k = \{Tk/n, T(K+k)/n, T(2K+k)/n, \ldots\}, \forall k \in \{0, \ldots, K-1\}$$

and compute the realized volatility $[Y, Y]_k$ for each grid. The length of each interval is TK/n, and there are K such grids. We then compute the average realized volatility (ARV)

$$\text{ARV}(Y, K) = \frac{1}{K}\sum_{k=0}^{K-1}[Y, Y]_k$$

The convergence of the realized volatility should be visible as K gets close to 1. Figure 1 looks at the ARV(Y, K) for Facebook for July 6, 2020. Note that as K goes down the ARV does not converge as expected, but steadily increases. This kind of behavior can be explained by the micro-structure model 20.1 since the ARV due to X_t converges but that due to the noise blows up if the noise is independent and identically distributed. The code for obtaining the plot is given below.

```
price1m = aggregatets(midprice, on="minutes", k=1)
rbp1m = rBPCov(price1m,makeReturns=TRUE)
price2m = aggregatets(midprice, on="minutes", k=2)
rbp2m = rBPCov(price2m,makeReturns=TRUE)
rbp = rBPCov(midprice,makeReturns=TRUE)
price6s = aggregatets(midprice, on="seconds", k=6)
```

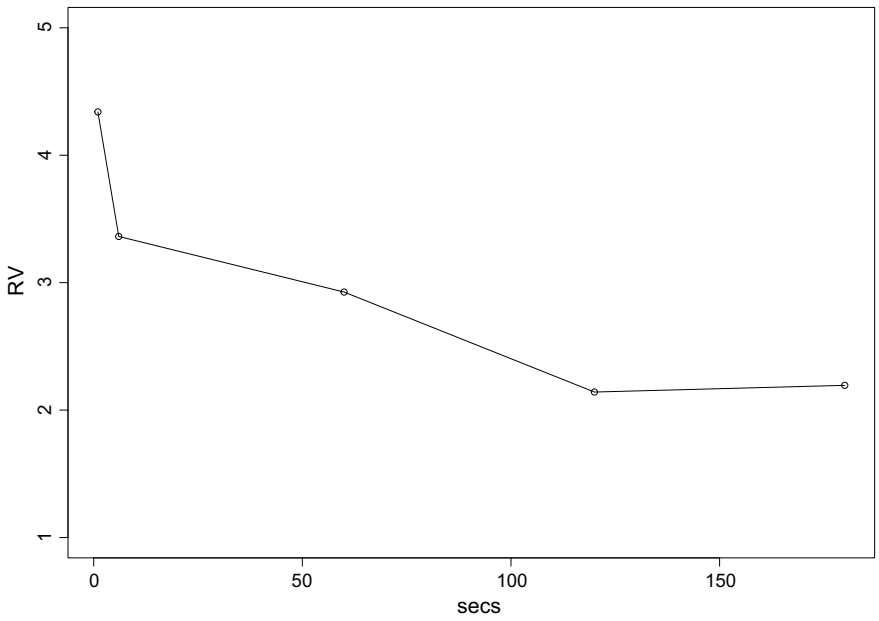


Fig. 20.3 Average realized volatility (ARV) for Facebook data for the day July 6th, 2020

```
rbp6s = rBPCov(price6s,makeReturns=TRUE)
price3m = aggregatets(midprice, on="minutes", k=3)
rbp3m = rBPCov(price3m,makeReturns=TRUE)
x = c(rbp,rbp6s,rbp1m,rbp2m,rbp3m)
x = x*10000
t = c(1,6,60,120,180)
plot(t,x,ylim=c(1,5),type="l",ylab="RV",xlab="secs")
points(t,x)
```

Plots of the type given in Fig. 20.3 were first considered by Andersen et al. (2000) and are called signature plots. The authors concluded that the most correct values for the volatility were the lower ones on the right-hand side of the plot, based mainly on the stabilization of the curve in this region. On the basis of this, the authors recommended to estimate volatility using a sparsely sampled subgrid. In this early literature, the standard approach was so subsample about every five minutes.

This approach is not quite satisfactory as it uses the data inefficiently and better methods can be found. There is quite a lot of work in this area. Following are some directions in which the research has progressed.

- Zhang et al. (2005) propose the two scales realized volatility (TSRV) as follows.

$$[X, X]_T^{(tsrv)} = a_n \text{ARV}(Y, K) - b_n \text{ARV}(Y, J)$$

This has been extended to a multi-scale realized volatility (MSRV) estimator that converges at the rate of $O_p(n^{-1/4})$. This rate is optimal. Also, the conditions on the noise structure in the TSRV have been weakened. Extensions exist for noise that is internally dependent and noise that can depend on the process X.

- An approach based on autocovariances, often called the 'kernel' approach, has been pioneered by Barndorff-Nielsen et al. (2008).
- Pre-averaging approach of Jacod et al. (2009) tries to reduce the noise by averaging observations before computing volatilities.
- Likelihood-based methods, like those of Xiu (2010) , use the likelihood function for constant σ and normal noise as a quasi-likelihood.

20.3 Asynchronicity

So far we have discussed the issues with intra-day data for a single asset. However, univariate modeling (that considers one stock at a time), although important for addressing certain kinds of problems, cannot be enough to unveil the nature and dynamics of financial markets because it does not take into account the interdependence of different financial instruments. Correlation dynamics models, therefore, have become an important aspect of theory and practice in finance. Correlation trading, which is a trading activity to exploit the changes in dependence structure of financial assets, and correlation risk that captures the exposure to losses due to changes in correlation have attracted the attention of many practitioners Krishnan et al. (2009).

Correlation is of paramount interest in a range of practical scenarios. For example, basket options are widely used although accurate pricing of the basket option is challenging. The primary reason is that they are cheaper to use for portfolio insurance. The cost-saving relies on the dependence structure between the assets. In the actuarial world, Monte Carlo-based approaches to joint modeling of risks, like dynamic financial analysis, rely heavily on the dependence structure. The choice of model and correlation has significant impact on the tail of the loss distribution and measures of extreme risks.

In order to perform multivariate analysis, we need multivariate data. This means that we need to have data for all $p(\geq 2)$ variables on n (sufficiently large) time points. However, for intra-day stock price data within a particular day we cannot expect to observe transactions in all the stocks at the exact same time points. In other words, it is not reasonable to assume that financial transactions take place synchronously. In Fig. 20.4, we have shown the trading times of two stocks within a small time interval. The figure illustrates the problem of asynchronicity. The effect of this asynchronicity, if not handled properly, can be quite serious. One such effect, reported in Epps (1979), is called Epps effect which shows that stock return correlations decrease as sampling frequency increases. It is also empirically shown that taking into account only the synchronous (or nearly synchronous) price ticks increases the estimated correlation Renò (2003). The Epps effect is a result of asynchronicity of price observations

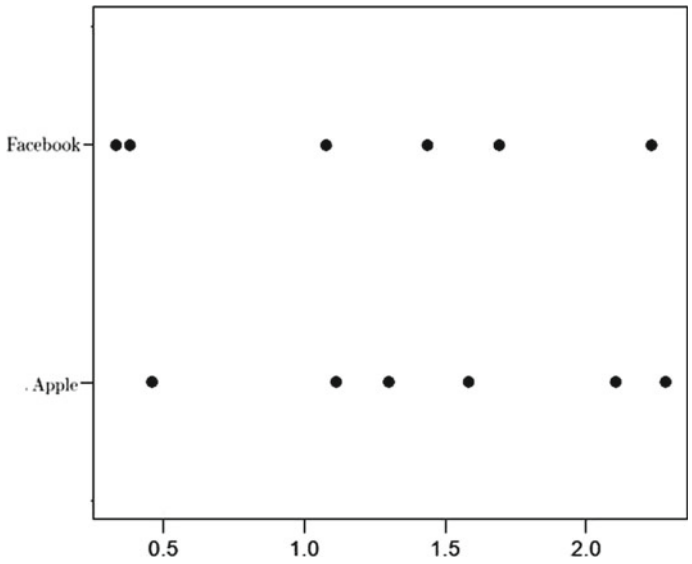


Fig. 20.4 Non-synchronous transaction times, calculated in seconds, for two stocks Facebook and Apple for first six ticks on a particular day 10/05/2017

and the lead-lag relation between asset prices. See Fig. 1 of Zhang (2011) for an illustration of this on stocks of Microsoft and Google during Oct 2005.

Suppose we have multiple stocks, say p stocks, whose price processes are denoted by $S_t^{(j)}\, j = 1, .., p$ and $X_t^{(j)} = \log S_t^{(j)}$. Alternatively, in vector notation, $\mathbf{X_t}$ is a p-dimensional diffusion process

$$\mathrm{d}\mathbf{X}_t = \mu_t \mathrm{d}t + \sigma_t \mathrm{d}\mathbf{W_t}, \tag{20.2}$$

where μ_t is p dimensional drift process and σ_t is a pxp matrix, called instantaneous covolatility process. $\mathbf{W_t}$ is a p dimensional standard Brownian motion. Our parameter of interest is

$$\Sigma_p = \int_0^1 \sigma_t \sigma_t^T \mathrm{d}t.$$

This quantity is called the Integrated Covariance matrix (ICV). Merton (1980) proposed an estimator of this quantity known as Realized Volatility Matrix. But this estimator is based on the assumption that all the observations are observed synchronously. Suppose that l-th observation of the i-th stock is observed at time point $\tau_l^i = \tau_l \; \forall i$. Then the Realized Covariance matrix is defined as follows:

$$\sum_{p}^{\text{RCV}} = \sum_{l=1}^{n} \Delta X_l \Delta X_l^T, \tag{20.3}$$

where $\Delta X_l = \begin{pmatrix} \Delta X_l^1 \\ \Delta X_l^2 \\ . \\ . \\ \Delta X_l^p \end{pmatrix} = \begin{pmatrix} X_{\tau_l}^1 - X_{\tau_{l-1}}^1 \\ . \\ . \\ . \\ X_{\tau_l}^p - X_{\tau_{l-1}}^p \end{pmatrix}$.

When trades happen at different times for different stocks, one solution is to convert the irregularly spaced observations to equal spaced intervals. This is done by fixing time points, say at one-minute or five-minute intervals and interpolating the available data. The common interpolation methods are *previous tick interpolation* and **linear interpolation**. In previous tick interpolation, the price at the last trade that happened before the time point under consideration is taken as proxy for the price at that time point. In linear interpolation, the prices at the last trade before and at the next trade after are considered and an average is taken as the proxy.

However, this forced synchronization can cause bias, as described under Epp's effect. In order to avoid force synchronization Hayashi and Yoshida (HY) proposed an alternative estimator for ICV matrix. The HY estimator is defined as follows:

$$\Sigma_p^{HY} = \sum_{k,l} \Delta X_k \Delta X_l^T \circ I(k,l), \tag{20.4}$$

where $\Delta X_l = \begin{pmatrix} \Delta X_l^1 \\ \Delta X_l^2 \\ . \\ . \\ \Delta X_l^p \end{pmatrix} = \begin{pmatrix} X_{\tau_l^1}^1 - X_{\tau_{l-1}^1}^1 \\ . \\ . \\ . \\ X_{\tau_l^p}^p - X_{\tau_{l-1}^p}^p \end{pmatrix}$.

Here '$\circ$' is Hadamard product and $I(k,l)$ is a $p \times p$ matrix whose (i, j)th element is the indicator function involving kth interarrival of ith stock and lth interarrival of jth stock: $I(I_k^i \cap I_l^j \neq \phi)$, where $I_k^i = (\tau_{k-1}^i, \tau_k^i)$. An illustration is provided in Fig. 20.5. This estimator is consistent and asymptotically normal. The function rHY-Cov inside the R library highfrequency can be used to obtain the Hayashi Yoshida estimator.

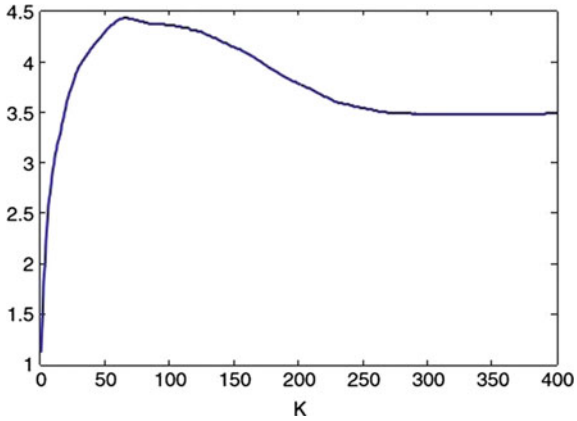


Fig. 20.5 Illustration of the Hayashi–Yoshida estimator

20.4 Exercises

1. Download high-frequency data on any asset for one day and create the ARV plot.
2. Download high-frequency data on any two assets for one day and compute the covariance by previous tick interpolation for 5- and 1-min intervals.
3. For the data in problem 2, find the Hayashi–Yoshida estimator.

Chapter 21
Unsupervised Learning



21.1 Introduction

Unsupervised learning is a class of algorithms; where a machine looks for patterns in a dataset with no labels (or target variables to predict). This is in distinction to supervised learning procedures, such as classification or regression, where a model is given training data of inputs and target variables; and learns to map from the inputs to the target. In unsupervised learning, only the inputs are available, and an algorithm looks for interesting patterns in the data. Unsupervised learning is aka., *knowledge discovery*.

Any good data analysis work starts with visualization. However, visualization techniques work well only for 2 or 3 dimensions. On the contrary, any good dataset will have many variables, often going up to hundreds. In such circumstances, dimension reduction methods like multidimensional scaling or principal component analysis are very important. These methods represent high dimension data in lower dimensions while preserving the key geometric features of the data. It helps us to visualize the key features of the dataset, and often it assists in identifying the number of inherent clusters or segments in the datasets.

21.2 Dimension Reduction

We would like to have more data, variables, and instances for accurate results; however, it is hard to visualize such datasets. The large datasets can also affect the performance of machine learning algorithms. When in a dataset, there are too many variables, the dimensionality reduction method is used. It lessens the number of data

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/978-981-19-2008-0_21.

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

R. Sen and S. Das, *Computational Finance with R*, Indian Statistical Institute Series,
https://doi.org/10.1007/978-981-19-2008-0_21

inputs to a flexible size while also conserving the critical features of the dataset as much as desirable. Here we discuss two dimensionality reduction methods (i) *Multidimensional Scaling* and (ii) *Principal Component Analysis*.

21.2.1 Multidimensional Scaling

Multidimensional scaling takes a set of dissimilarities between the objects. It returns a set of points on 2- or 3-dimensional Cartesian space. The Euclidean distances between the points are approximately equal to the dissimilarities. This job is performed by attaching observations to particular locations in the new lower-dimensional Euclidean space (usually two- or three-dimensional). The distances between points in the assigned domain match the given variations as nearly as possible. Often, the dimensions of this new space can be used to understand the data. If the variables are objectively measured, then we can use multidimensional scaling as a dimension reduction technique. The MDS is used to interpret knowledge about the pairwise 'distances' among a set of n samples or individuals into an arrangement of n points mapped into an abstract Cartesian space.

The MDS introduces a method to present the information contained in a distance matrix through data visualization. It is a class of nonlinear dimensionality reduction methods. Given the distances between each pair of instances in a set of n instances, an MDS algorithm assigns each case into lower-dimensional spaces. The between-instance distances are preserved as much as possible. For $p = 2$, or 3, the resulting points can be visualized on a scatter plot.

Algorithm 21.2.1 Input Matrix: $\mathbf{X}$, and proximity (or distance) matrix $\mathbf{D}^{(2)} = [d_{ij}^2]$

1. $\Sigma = \mathbf{X}\mathbf{X}'$
2. Compute $\Sigma = -\frac{1}{2}\mathbf{J}\mathbf{D}^{(2)}\mathbf{J}$ where $\mathbf{J} = \mathbf{I} - \frac{1}{n}\mathbf{1}\mathbf{1}'$, where n is the number of instances under consideration.
3. Consider k largest eigenvalues $\lambda_1, \lambda_2, \ldots, \lambda_k$ of Σ and corresponding k eigenvectors $\mathbf{e}_1, \mathbf{e}_2, \ldots, \mathbf{e}_k$.
4. A k dimensional spatial arrangement of the n objects is determined from the coordinate matrix $\mathbf{X} = \mathbf{E}_k \Lambda_k^{1/2}$, where $\mathbf{E}_k$ is the matrix of k eigenvectors and Λ_k diagonal matrix of k eigenvalues of Σ.
5. Typically $k = 2$ is used as default and $\mathbf{E}_k$ is returned.

The multidimensional scaling of a data matrix (aka., principal coordinates analysis) can be implemented by `cmdscale` in the `stats` library in `R`. In the `cmdscale` the only input require is distance matrix `d`. To illustrate we present the following example.

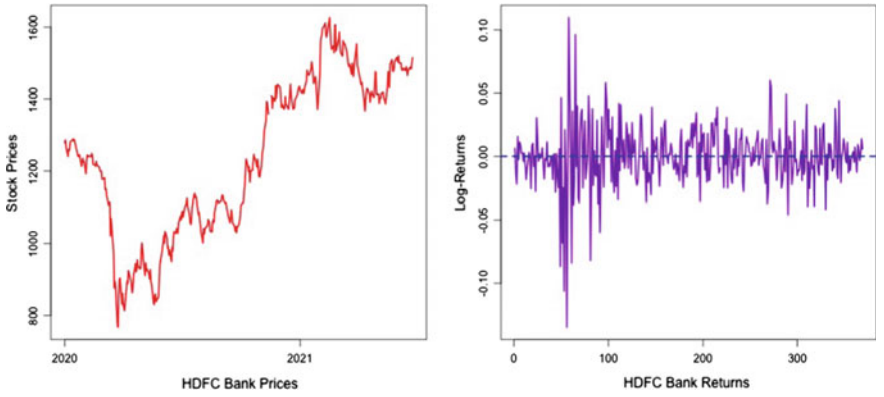


Fig. 21.1 On the left panel the adjusted close price of HDFC Bank over time is plotted. Clearly, the adjusted close prices are non-stationary process. On the right panel, the log-return of the HDFC Bank is presented. The log-return of the HDFC Bank is approximately mean zero stationary process

Example 21.2.1 We consider the adjusted close prices of the 110 companies listed in the Indian stock market from January 01, 2020, until June 25, 2021.

```
require(c('corrplot','cluster','igraph','ape'))
data = read.csv(file='NSE_100.csv',header = T
                                  ,stringsAsFactors=F)
data$date = as.Date(data$date,format='%Y-%m-%d')
bank_sector = data[,c('date',bank_and_fin_stocks)]
```

In this example, we consider only 15 banking stocks, and our objective is to understand the hidden pattern in these stocks.

Before we apply the MDS using the `cmdscale` function there are some pre-processing of data is required. Here we avoided all those details and can be found in the `R`-code provided. Note that '`bank_and_fin_stocks`' contain the name of all the stocks that belong to banking and financial industries.

The 15 banking and finance stocks that we are considering are as follows:

```
[1] "AXISBANK"    "BAJAJF"      "BAJFINANCE"  "BANKBARODA"
[5] "FEDERALBNK"  "HDFC"         "HDFCBANK"    "IBULHSGFIN"
[9] "ICICIBANK"   "INDUSINDBK" "KOTAKBANK"   "LICHSGFIN"
[13] "PNB"         "SBIN"        "YESBANK"
```

In Fig. 21.1, we present the adjusted close price, and the log-return of HDFC Bank over time, though the adjusted close prices are non-stationary process. However, the log-return of the HDFC Bank is mean zero stationary process. In Fig. 21.2, we present the pairwise scatterplot of log-returns. We can see that the distributions of log-returns

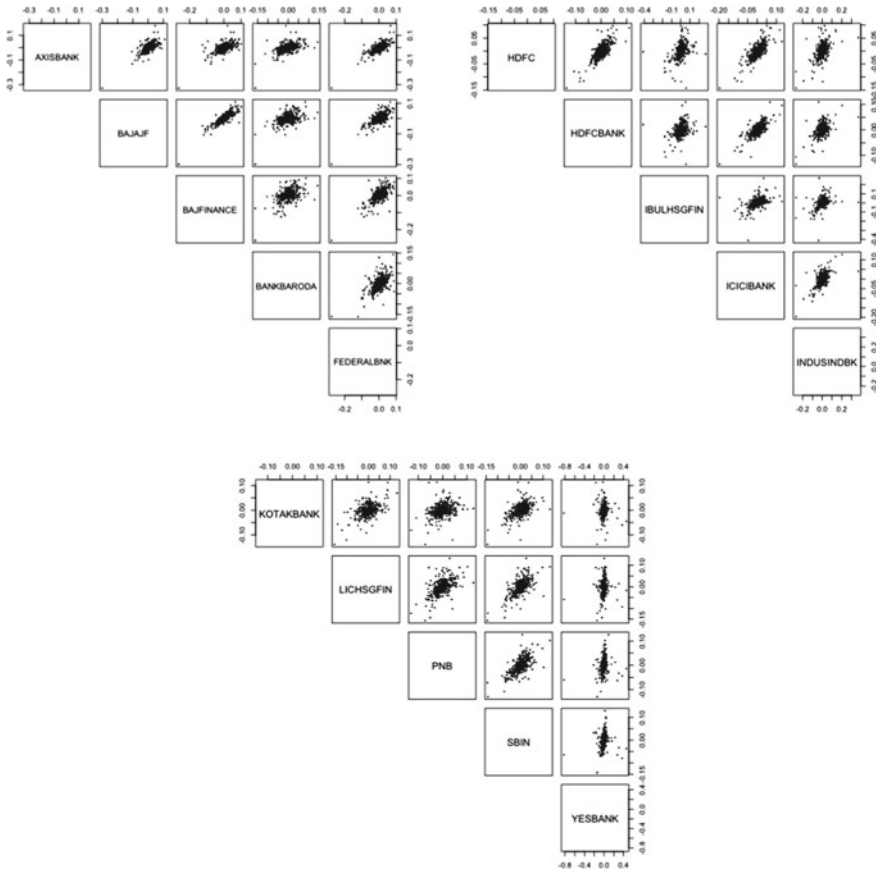


Fig. 21.2 The pairwise scatter plot log-return of plot of different banks

are sort of clubbed in one big cluster with few outliers. These outliers correspond to the days when the nationwide lockdown was announced.

We compute the correlation between the log-returns and make the correlation plot in Fig. 21.3. The R code is presented below. The column corresponding to *Yes Bank* is faded compared to all other columns; it means *Yes Bank* is not correlated with other financial and banking stocks.

```
## C is the correlation matrix
C = cor(Return,use='na.or.complete')
corrplot(C,method = 'color')
```

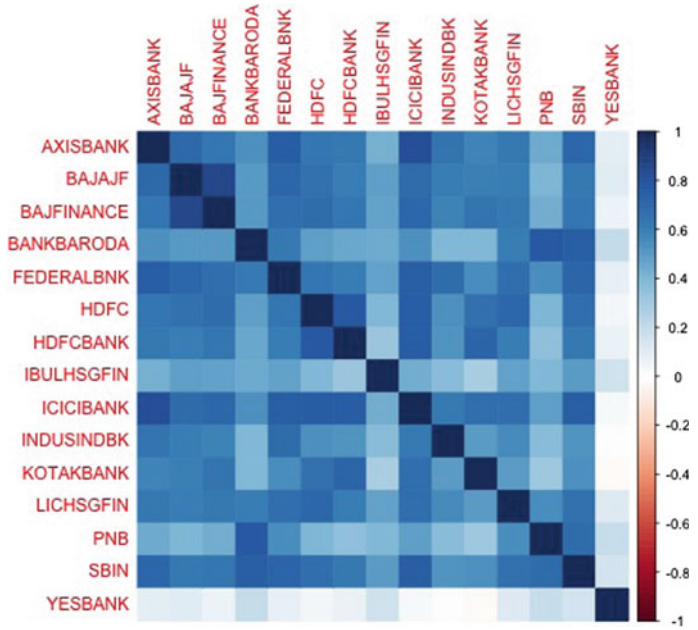


Fig. 21.3 Graphical display of a correlation matrix `C`. The column corresponding to *Yes Bank* is faded compared to all other columns, meaning *Yes Bank* is not correlated with other banking and financial stocks

The correlation matrix can be transformed into distance matrix as

$$D = \sqrt{2 * (1 - C)}.$$

We compute the distance matrix `D` and implement the *multidimensional scaling* (MDS). In the following `R`-code we provide the distance matrix `D` as input to the `cmdscale` function in `stats` package. The `cmdscale` function implement the multidimensional scaling which presents each bank or financial institution in a two-dimensional Cartesian space, while maintaining their relative proximity in the original 15-dimension, as their are 15 stocks in the consideration.

```
## Converting Correlation to a Distance Matrix
D = sqrt(2*(1-C))
D = daisy(D, metric = "gower")

##### Running and Plotting MDS ##########
mds = cmdscale(D)
plot(mds,pch=20)
```

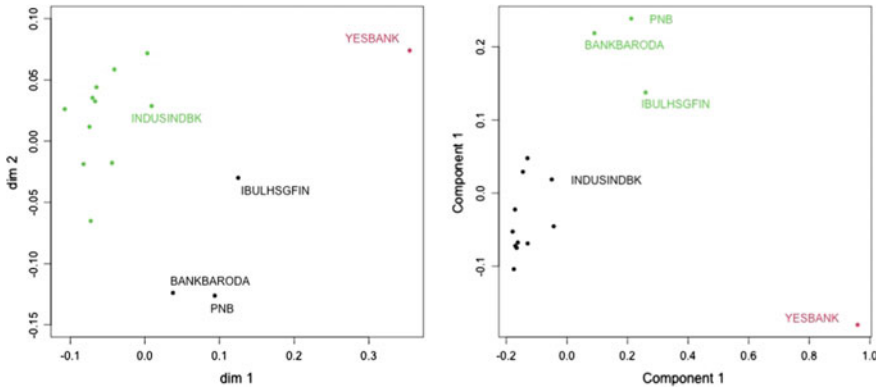


Fig. 21.4 The left panel displays the output of *multidimensional scaling*. The right panel displays the first two components of *principal component analysis*. The inference from both technique are very similar

In the left panel of Fig. 21.4, we present the output of MDS on 2-D scatter plot. Clearly, *Yes Bank* is an outlier. The three stocks, namely (1) *PNB*, (2)*IBULLHSGFIN*, and (iii) *BANKBARODA* , create a small cluster. All other 11 stock clusters together.

21.2.2 Principal Component Analysis

The primary objective of principal component analysis (PCA) is to decrease the dimension of a dataset consisting of many correlated variables. While doing so, it must retain the covariation in the dataset, i.e., present in the dataset as much as possible. This is done by remodeling to a new set of variables, the principal components (PCs), uncorrelated and ordered. Thus only a few new variables learn most of the variation present in all of the original variables.

Suppose $\mathbf{X}$ is a vector of p random variables, such that $Cov(\mathbf{X}) = \Sigma$. Typically Σ is unknown and we estimate the covariance Σ by sample covariance matrix S. Here for notational simplicity we assume that Σ is known. Suppose, α_k is a is a vector of p constants, and we want to obtain

$$\alpha_k^T \mathbf{X} = \sum_{j=1}^{p} \alpha_{kj} x_j,$$

that maximizes

$$\text{Var}(\alpha_k^T \mathbf{X}) = \alpha_k^T \Sigma \alpha_k,$$

where $\alpha_k^T \alpha_k = 1$. To maximize $\alpha_k^T \Sigma \alpha_k$, subject to $\alpha_k^T \alpha_k = 1$, we rewrite it with Lagrange multipliers. We maximize the function

$$\alpha_k^T \Sigma \alpha_k - \lambda(\alpha_k^T \alpha_k - 1),$$

by differentiating with respect to α_k. Hence we have

$$\frac{\partial}{\partial \alpha_k}\Big(\alpha_k^T \Sigma \alpha_k - \lambda_k(\alpha_k^T \alpha_k - 1)\Big) = 0.$$

By solving the derivative we have,

$$\begin{aligned} \Sigma\alpha_k - \lambda_k \alpha_k &= 0, \\ \Sigma\alpha_k &= \lambda_k \alpha_k. \end{aligned} \tag{21.1}$$

Now we can see α_k is the eigenvector and λ_k is the corresponding eigenvalue of the Σ. Which eigenvalue should we choose? Note that if we pre-multiply both sides of the Eq. (21.1) with α_k, then we have

$$\alpha_k^T \Sigma \alpha_k = \alpha_k^T \lambda_k \alpha_k = \lambda_k \alpha_k^T \alpha_k = \lambda_k.$$

So the eigenvalue λ_k represents the variability of the new variable $z_k = \alpha_k^T \mathbf{X}$ that we are trying to obtain. Hence we choose λ_k as large as possible. We call λ_1 be the largest eigenvector of Σ and α_1 is the corresponding eigenvector then the solution to

$$\Sigma\alpha_1 = \lambda_1 \alpha_1$$

is known as the first Principal Component (PC) of $\mathbf{X}$. The second PC, $\alpha_2\mathbf{X}$ maximizes $\text{Var}(\alpha_2^T \mathbf{X}) = \alpha_2^T \Sigma \alpha_2$, subject to two conditions: (i) $\alpha_2^T \alpha_2 = 1$ and (ii) $\alpha_2^T \mathbf{X}$ is uncorrelated with $\alpha_1^T \mathbf{X}$, i.e.,

$$\text{Cov}(\alpha_1^T \mathbf{X}, \alpha_2^T \mathbf{X}) = \alpha_1^T \Sigma \alpha_2 = \lambda_1 \alpha_1^T \alpha_2 = 0,$$

i.e., $\alpha_1^T \alpha_2 = 0$. So we present the objective function to maximize under the two conditions,

$$\alpha_2^T \Sigma \alpha_2 - \lambda_2(\alpha_2^T \alpha_2 - 1) - \phi \alpha_2^T \alpha_1.$$

Differentiating with respect to α_2, we have

$$\frac{\partial}{\partial \alpha_2}\Big(\alpha_2^T \Sigma \alpha_2 - \lambda_2(\alpha_2^T \alpha_2 - 1) - \phi \alpha_2^T \alpha_1\Big) = 0.$$

By solving the derivative we have,

$$\Sigma\alpha_2 - \lambda_2 \alpha_2 - \phi \alpha_1 = 0. \tag{21.2}$$

If we left multiply the Eq. (21.2) with α_1, we have

$$\alpha_1^T \Sigma \alpha_2 - \lambda_2 \alpha_1^T \alpha_2 - \phi \alpha_1^T \alpha_1 = 0,$$

then we can see ϕ must be 0. Hence we have

$$\Sigma \alpha_2 - \lambda_2 \alpha_2 = 0.$$

Clearly, it is the second equation corresponding to second eigenvalue. Hence α_2 is the eigenvector corresponding to the second largest eigenvalue results the second PC of $\mathbf{X}$, namely $z_2 = \alpha_2^T \mathbf{X}$. We repeat the the process for $k = 1, 2, \ldots, p$, yielding up to p different eigenvectors of Σ along with corresponding eigenvalues $\lambda_1, \lambda_2, \ldots, \lambda_p$. In general α_k is the kth PC of $\mathbf{X}$ and $Var(\alpha_k^T \mathbf{X}) = \lambda_k$. Note that $\sum_{k=1}^{p} \lambda_k$ can be considered as the measure of total variability in $\mathbf{X}$, captured through the Σ.

It means if we want to have a lower-dimensional projection of $\mathbf{X}$, we should choose first q PCs, as the target lower-dimensional representation of $\mathbf{X}$. Since the PCs are uncorrelated, the percentage of variance accounted for by retaining the first q PCs is

$$\frac{\sum_{k=1}^{q} \lambda_k}{\sum_{k=1}^{p} \lambda_k} \times 100.$$

If we choose q to be 2 or 3, then we would be able to visualize point cloud. The following `R`-code uses the `princomp` function in the `stats` package to implement the PCA for the Banking stocks data and plot the point clouds. The resulting graphics are presented in Fig. 21.4. The inferences from both *multidimensional scaling* (MDS) and PCA are very similar.

```
## Principal Component Analysis
pca = princomp(D)
scores = pca$scores
scores = data.frame(scores)
plot(x= scores$Comp.1,y=scores$Comp.2,pch=20)
```

21.3 Clustering

Clustering method groups a set of instances so that similar cases fall in the same group. Of course, we must establish what it implies for two or more cases to be similar or different. A domain-specific consideration must be made to define the similarity between the objects.

Suppose that we have a set of n samples, each with p features. The n observations could correspond to the price for apartments in Bangalore, India. The p features could correspond to measurements collected for each apartment; these could be

basic parameters of the apartment, for example, the number of bedrooms or size of the apartment in square meters. It is reasonable to consider some heterogeneity among the apartments. For instance, a few unknown subtypes of clustering were based on the builder type, which was not recorded in the data.

Now, as we can consider our banking and finance stocks data, it is natural to consider that there will be some groupings. These groupings may be based on simply profitability and other performance of the banks, which is not part of the recorded data. One approach might be dimension reduction via *MDS* or *PCA* algorithms. Other path is to identify these natural segments in the data via k-means clustering or hierarchical clustering algorithm.

21.3.1 K-means Clustering

K-means clustering is an elegant strategy for partitioning a dataset into K discrete, non-overlapping subdatasets. To perform K-means clustering, we first define the desired number of clusters K; then, the K-means algorithm assigns each sample to precisely one of the K clusters. Suppose $\mathcal{D} = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n\}$, where $\mathbf{x}_i = (x_{i1}, x_{i2}, \ldots, x_{ip})^T$ is the dataset under consideration. We partition the dataset $\mathcal{D}$ into K parts where each part is mutually exclusive, i.e., $\mathcal{D} = \mathcal{D}_1 \cup \mathcal{D}_2 \cup \ldots \cup \mathcal{D}_K$, such that $\mathcal{D}_i \cap \mathcal{D}_j = \phi$ foll all $i \neq j = 1, 2, \ldots, K$. For instance, if ith sample belongs to kth cluster then $\mathbf{x}_i \in \mathcal{D}_k$. The principle behind K-means clustering is to keep the within-cluster variation as small as possible. The within-cluster variation for cluster $\mathcal{D}_k$ is a measure $WV(\mathcal{D}_k)$ of the amount, the samples within a group deviate from each other. Hence we approach the problem as

$$\min_{\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_K} \left\{ \sum_{k=1}^{K} WV(\mathcal{D}_k) \right\}. \tag{21.3}$$

This formulation indicates that we partition the samples into K clusters. The total within-cluster variation, summed over all K clusters, is as small as possible. We define the within-cluster variation as $WV(\mathcal{D}_k)$ as

$$WV(\mathcal{D}_k) = \frac{1}{|\mathcal{D}_k|} \sum_{i,i' \in \mathcal{D}_k} \sum_{j=1}^{p} (x_{ij} - x_{i'j})^2 = 2 \sum_{i \in \mathcal{D}_k} \sum_{j=1}^{p} (x_{ij} - \bar{x}_{kj})^2, \tag{21.4}$$

where $|\mathcal{D}_k|$ denotes the number of observations in the kth cluster, and $\bar{x}_{kj} = \frac{1}{|\mathcal{D}_k|} \sum_{i \in \mathcal{D}_k} x_{ij}$ is the average of jth feature in kth cluster. Combining Eqs. (21.3) and (21.4), we have

$$\min_{\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_K} \left\{ \sum_{k=1}^{K} \left(2 \sum_{i \in \mathcal{D}_k} \sum_{j=1}^{p} (x_{ij} - \bar{x}_{kj})^2 \right) \right\}. \tag{21.5}$$

The K-means clustering tries to solve (21.5), that is, an algorithm to partition the dataset $\mathcal{D}$ into K clusters such that the objective (21.5) is minimized.

Algorithm 21.3.1 *K*-means clustering

1. Initiate an initial set of k means, i.e., $\{\bar{\mathbf{x}}_1^{(1)}, \bar{\mathbf{x}}_2^{(1)}, \ldots, \bar{\mathbf{x}}_k^{(1)}\}$
2. Iterate until the assignments no longer change:
   - **Assignment**: Assign each sample to the cluster with the nearest mean (with respect to Euclidean distance), where

$$\mathcal{D}_k^{(t)} = \left\{\mathbf{x}_p : ||\mathbf{x}_p - \bar{x}_k^{(t)}||^2 \leq ||\mathbf{x}_p - \bar{x}_i^{(t)}||^2; \;\; \forall i = 1, \ldots, K\right\}$$

   - **Update means**: Recalculate means with the samples assigned to each cluster, i.e.,

$$\bar{x}_k^{(t+1)} = \frac{1}{|\mathcal{D}_k^{(t)}|} \sum_{x_j \in \mathcal{D}_k^{(t)}} \mathbf{x}_j$$

Various changes of k-means such as k-medoids and spherical k-means are proposed to support using other distance measures.

Finding Optimal Number of Clusters: The strategy for identifying the optimal number of clusters is to train the k-means algorithm for $k = 1, 2, ..., K$ and calculate the within-cluster sum of squares. We choose the k for which we observe the most significant drop.

We can implement the k-means clustering using the `kmeans` functions in the `stats` package using the following code. In this case, we can apply the k-means clustering on the `mds` scores (Fig. 21.5).

```
## Identify the optimal number of cluster
wss = (nrow(mds)-1)*sum(apply(mds,2,var))
num_of_cluster_to_try =10
for (i in 2:num_of_cluster_to_try){
  wss[i] = sum(kmeans(mds, centers=i)$withinss)
}
plot(1:num_of_cluster_to_try, wss
     , type="b", xlab="Number of Clusters",
     ylab="Within groups sum of squares",pch=20)
```

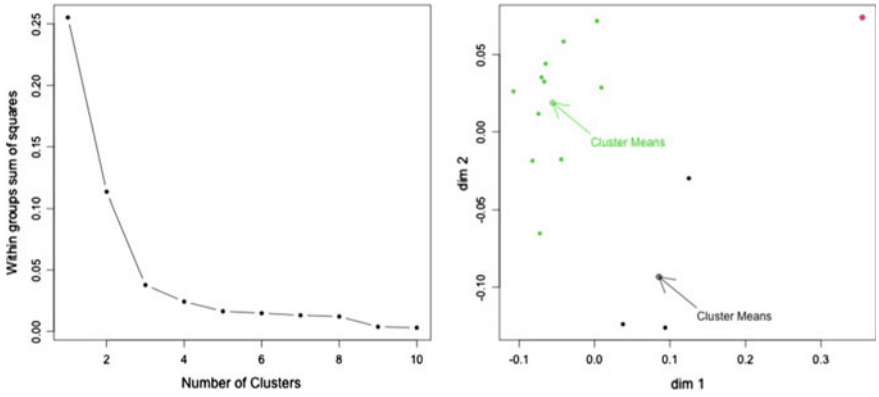


Fig. 21.5 In the left panel we present the number clusters versus within clusters sums of squares (WSS) . Clearly with *three clusters*, we observe the largest drop in WSS. So we fit the 3-means cluster algorithm to group the banking and finance sector into three clusters. *The Yes Bank* is a cluster by itself, as it is far away from all others

21.3.2 Hierarchical Clustering

One possible problem of K-means clustering is that it needs us to pre-specify the number of clusters K. Hierarchical clustering is an alternative method that does not need a particular choice of K. Hierarchical clustering has an added benefit that it results in an attractive tree-structured representation of the samples, known as a *dendrogram*. Approaches for hierarchical clustering usually fall into two types.

1. **Divisive** or **top-down** approach: In this method, all samples start in one cluster, and splits are done recursively as we run down the hierarchy. The time complexity for the divisive clustering with an exhaustive search is $\mathcal{O}(2^n)$. Therefore *top-down* method is less attractive in the *hierarchical clustering*.
2. **Agglomerative** or **bottom-up** approach: In *hierarchical agglomerative clustering* (HAC), each instance starts in its own cluster, and pairs of groups are merged as we lead up the hierarchy. The time complexity of HAC is $\mathcal{O}(n^3)$ and memory complexity is $\mathcal{O}(n^2)$. It makes HAC too slow for even moderate-size datasets. Hence HAC often relies on heuristic type algorithms.

Dendrograms: A dendrogram is a diagram that represents clusters in a tree. In hierarchical clustering, the dendrogram illustrates the pattern of the clusters generated by the HAC. In left panel of Fig. 21.6 each leaf of the dendrogram represents one of the 15 banking stocks in Fig. 21.4. As we go up the tree, some leaves start to join into branches. These correspond to instances (i.e., stocks) that are alike. As we go higher up the tree, branches themselves join, sometimes with leaves or other branches. Lower in the tree, when joins happen, the more similar groups of instances (i.e., stocks) are to each other. However, instances that join later (near the top of the tree) are quite different. The height of this union, as estimated on the vertical axis,

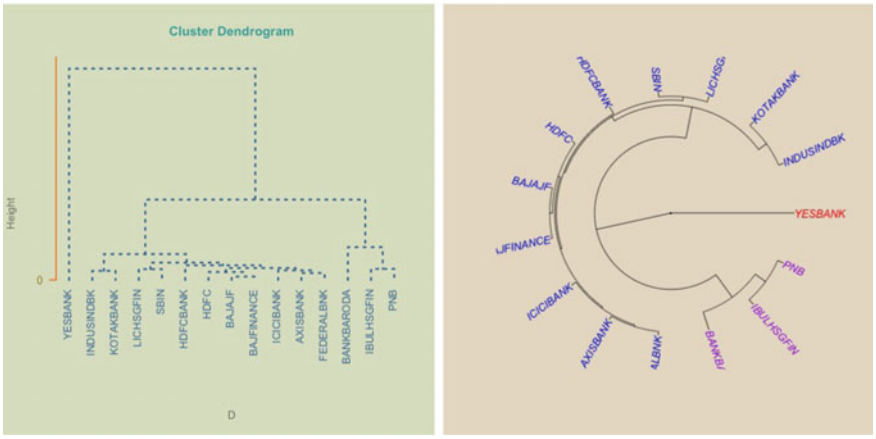


Fig. 21.6 Left: Dendrogram obtained from hierarchically clustering the data from Fig. 21.4 with the 'complete linkage' and Euclidean distance. Right: Alternate presentation of dendrogram known as fan dendrogram or phylogenetic tree

shows how different the two instances are. Thus, instances that join at the very foot of the tree are pretty similar to each other. However, instances that weld close to the head of the tree are considerably different. A very critical point in understanding dendrograms is that we decide the similarity between the instances based on the vertical axis of the dendrogram. We should not form decisions about the similarity of two instances based on their closeness along the horizontal axis.

We can see *Yes Bank* and *INDUINDBK* are pretty similar since they join at the lowest point on the dendrogram in Fig. 21.6. However, it is alluring; it would be wrong to conclude from the Fig. 21.6 that observations *Yes Bank* and *INDUINDBK* are similar to each other on the basis that they are located near each other on the dendrogram. In fact, *Yes Bank* is no more identical to observation *INDUINDBK* than it is to peer *PNB*, *BANKBARODA*, and *IBULHSGFIN*. We can observe it from Fig. 21.4.

The hierarchical clustering algorithm is presented below 21.3.2. We first define the dissimilarity metric between each pair of instances. Often, we use the Euclidean distance (or other distances suitable to the problem), and the algorithm proceeds iteratively. We start with n instances and treat each of them as a single clusters, i.e., at the bottom of the dendrogram. The two most similar clusters are then joined so that there now are $(n - 1)$ many clusters. Then the two most similar clusters are joined again so that there are now $(n - 2)$ clusters. We iterate the process until it forms a single cluster.

Table 21.1 Four popularly used *linkage* in the hierarchical clustering

Linkage type	Formula
Complete	$\max\{d(x, y): x \in X, y \in Y\}$
Single	$\min\{d(x, y): x \in X, y \in Y\}$
Average	$\frac{1}{\lvert X\rvert \cdot \lvert Y\rvert} \sum_{x \in X} \sum_{y \in Y} d(x, y)$
Centroid	$\lvert\lvert c_x - c_y \rvert\rvert$, where c_x and c_y are the centroids of clusters x and y

Algorithm 21.3.2 Hierarchical Clustering:

- **Initial**: Begin with all n instances and compute dissimilarity measure (like Euclidean distance) of all $\binom{n}{2} = \frac{n(n-1)}{2}$ pairwise dissimilarities. Consider each instance as its own cluster.
- for $i = n, n-1, \ldots, 2$:
  1. Compute all pairwise dissimilarities among the i clusters and identify the pair of most similar clusters. Join these two clusters. The dissimilarity between these two bunches (or clusters) indicates the height in the dendrogram at which the join should be pinned.
  2. Compute the new pairwise dissimilarities among the $(i-1)$ remaining clusters.

Linkage: One last problem is how to extend the concept of dissimilarity between a pair of observations to a pair of groups of observations. This extension is done by defining the notion of linkage, which measures the dissimilarity between two groups of observations. The four popular types of linkage are (i) *complete*, (ii) *average*, (iii) *single*, and (iv) *centroid*. Some popular linkage methods between two sets of observations are presented in Table 21.1.

21.4 Discussion

Essentially there are two major approaches in unsupervised learning (i) dimension reduction, which helps us visualize the data, and (ii) Clustering, the objective of which is to segment the data. However, some issues arise in clustering and dimension reduction. To perform clustering or dimension reduction, some parameters must be set beforehand. How many lower dimensions should we consider? Should we standardize or normalize the features? What dissimilarity measure should be considering? What type of linkage should we use? Where should we cut the dendrogram

in order to obtain clusters? How many clusters should we look for when we perform the k-means cluster? Each of these decisions does influence the obtained results. In practice, we try several different choices and look for the one with the most interpretable results. Hence there is no single correct solution—one should try as many choices as possible and any solution that exposes some interesting aspects of the data should be considered.

21.5 Exercises

1. Why is the agglomerative clustering approach more preferred than the divisive clustering approach?
2. Prove that

$$WV(\mathcal{D}_k) = \frac{1}{|\mathcal{D}_k|} \sum_{i,i' \in \mathcal{D}_k} \sum_{j=1}^{p} (x_{ij} - x_{i'j})^2 = 2 \sum_{i \in \mathcal{D}_k} \sum_{j=1}^{p} (x_{ij} - \bar{x}_{kj})^2,$$

   where $|\mathcal{D}_k|$ denotes the number of observations in the kth cluster, and $\bar{x}_{kj} = \frac{1}{|\mathcal{D}_k|} \sum_{i \in \mathcal{D}_k} x_{ij}$ is the average of jth feature in kth cluster.
3. Suppose the eigenvalue corresponding to the first principle component is ten times larger than the eigenvalues of all other components put together. How would you interpret the result of the PCA?

Appendix A
Basics of Mathematical Finance

A.1 Fundamental Theorems of Asset Pricing

Single Period Market: Consider a market in which K assets, labeled $A_1, A_2, \ldots, A_K$, are freely traded. For simplicity, we start with a single period, that is time t is 0 or 1. Assume that one of the, say A_1, is riskless, that is, its value at time $t = 1$ does not depend on the market scenario. We shall call this bond. The share price of asset A_j at time $t = 0$ is S_0^j. Without loss of generality, we may assume that $S_0^1 = 1$. Uncertainty about the behavior of the market is encapsulated in a finite set of N possible market scenarios, labeled $\omega_1, \omega_2, \ldots, \omega_N$. The share prices $S_1^2, S_1^3, \ldots, S_1^K$ of the $K - 1$ assets at time $t = 1$ are functions of the market scenario.

Observe that, since asset A_1 is riskless, there is a constant r, called the riskless rate of return, such that the share price S_1^1 of A_1 in any scenario ω_i is

$$S_1^1(\omega_i) = e^r \quad \forall \quad i = 1, 2, \ldots, N. \tag{A.1}$$

Definition A.1.1 A **portfolio** is a vector $\underline{\theta} = (\theta_1, \theta_2, \ldots, \theta_K) \in \mathbb{R}^K$ of K real numbers. The entry θ_j represents the number of shares of asset A_j that are owned; if $\theta_j < 0$ then the portfolio is said to be short $\mid \theta_j \mid$ shares of asset A_j .

The value of the portfolio $\underline{\theta}$ at time $t = 0$ is

$$V_0(\underline{\theta}) = \sum_{j=1}^{K} \theta_j S_0^j \tag{A.2}$$

and the value of the portfolio θ at time $t = 1$ in market scenario ω_i is

$$V_1(\underline{\theta}; \omega_i) = \sum_{j=1}^{K} \theta_j S_1^j(\omega_i). \tag{A.3}$$

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

R. Sen and S. Das, *Computational Finance with R*, Indian Statistical Institute Series,
https://doi.org/10.1007/978-981-19-2008-0

Definition A.1.2 An **arbitrage** is a portfolio $\underline{\theta}$ that 'makes money from nothing', formally, a portfolio $\underline{\theta}$ such that either

$$V_0(\underline{\theta}) \leq 0 \text{ and } V_1(\underline{\theta}; \omega_i) > 0 \quad \forall \quad i = 1, 2, \ldots, N \tag{A.4}$$

$$\text{OR} \quad V_0(\underline{\theta}) < 0 \text{ and } V_1(\underline{\theta}; \omega_i) \geq 0 \quad \forall \quad i = 1, 2, \ldots, N \tag{A.5}$$

Definition A.1.3 A probability distribution $\pi = (\pi(\omega_1), \ldots, \pi(\omega_n))$ on the set of possible market scenarios is said to be an **equilibrium measure** (or **risk-neutral measure**) if, for every asset $j = 1, \ldots, K$, the share price of asset A_j at time $t = 0$ is the discounted expectation, under π, of the share price at time $t = 1$, that is, if

$$S_0^j = e^{-r} \sum_{i=1}^{N} \pi(\omega_i) S_1^j(\omega_i) \quad \forall \quad j = 1, 2, \ldots, K. \tag{A.6}$$

Theorem A.1.1 (First Fundamental Theorem of Arbitrage Pricing) *There exists an equilibrium measure if and only if arbitrages do not exist.*

The first implication is easy to prove. Suppose that there is an equilibrium measure π. Then for any portfolio $\underline{\theta}$, the portfolio values at time $t = 0$ and $t = 1$ are related by discounted expectation:

$$V_0(\underline{\theta}) = \sum_{i=1}^{N} \pi(\omega_i) e^{-r} V(\underline{\theta}; \omega_i). \tag{A.7}$$

To see this, just multiply Eq. (A.6) by θ_j , sum on j, and use the definitions of portfolio value in (A.2)–(A.3) above. If $V(\underline{\theta}; \omega_i) > 0$ for every market scenario ω_i (as must be the case for an arbitrage portfolio), then Eq. (A.7) implies that $V_0(\underline{\theta}) > 0$, and so $\underline{\theta}$ cannot be an arbitrage. Thus, arbitrages do not exist.

The second implication, that absence of arbitrages implies the existence of an equilibrium measure, is harder to prove and we shall skip it. The main ingredient is the Separating Hyperplane Theorem.

Example A.1.1 Let us consider the pricing of the European call option on an asset Stock. The strike price is K, and so the terminal value of the option is given by

$$V_1 = (S_1 - K)_+ = S_1 - K \quad \text{if} \quad S_1 \geq K \tag{A.8}$$

$$= 0 \quad \text{if} \quad S_1 < K. \tag{A.9}$$

Thus in the absence of arbitrage, the price of the option at time $t = 0$ is

$$V_0 = e^{-r}(S_1 - K)\pi(\omega_1),$$

where $\pi(\omega_1) = P(S_1 \geq K)$ under the risk-neutral measure.

Two-Scenario Market: There are two possible market scenarios, ω_1, ω_2. The value of one share of Stock at time 1 is $S_1(\omega_i) = d_i$ in scenario ω_i, with $d_1 < d_2$. The riskless rate of return is r. By the fundamental theorem, in an arbitrage-free market, there is a probability distribution π on the two scenarios that determines prices by discounted expectation, and so, in particular,

$$S_0 = \pi(\omega_1)e^{-r}d_1 + \pi(\omega_2)e^{-r}d_2. \tag{A.10}$$

Because there are only two market scenarios, Eq. (A.10) uniquely determines the equilibrium measure π (note that π is a probability measure, so adds up to one):

$$\pi(\omega_1) = (d_2 - S_0e^r)/(d_2 - d_1), \tag{A.11}$$
$$\pi(\omega_2) = (S_0e^r - d_1)/(d_2 - d_1). \tag{A.12}$$

Finally, if the call option is to be freely traded, and if the market is to remain arbitrage-free, then its value at time $t = 0$ is also determined by discounted expectation. Since there is only one possible equilibrium measure, as in the last displayed equations, the value of the call at time $t = 0$ is

$$V_0 = \pi(\omega_2)(d_2 - K) = \frac{(S_0e^r - d_1)(d_2 - K)}{(d_2 - d_1)}. \tag{A.13}$$

Three-Scenario Market: Consider now the pricing of the call option with strike K in the three-scenario market discussed earlier. If the only freely traded assets in the market were Stock and MoneyMarket, then the pricing formulas (A.6) would not uniquely determine the equilibrium distribution π, because formulas (A.6) provide only two equations in three unknowns. Thus, any probability distribution (π_1, π_2, π_3) on the three scenarios such that $S_0e^r = d_1\pi_1 + d_2\pi_2 + d_3\pi_3$ would be allowable as an equilibrium measure. Call the set of all such probability distributions Π. Then any element $\pi \in \Pi$ such that

$$v = e^{-r}(p_2(d_2 - K)_+ + p_3(d_3 - K)) \tag{A.14}$$

holds, would be an equilibrium measure for the enlarged market in which the freely traded assets are Stock, Bond, and Call, where Call is the call option on Stock with strike K, provided the $t = 0$ price of Call is given by (A.14). By the Fundamental Theorem, any such market is arbitrage-free.

Definition A.1.4 Suppose the freely traded assets in a market are $A_1, \ldots, A_k$. A **derivative security** is a tradeable asset whose value V_1 at time $t = 1$ is a function $V(\omega_i)$ of the market scenario. In the language of probability, it is a random variable measurable on the sigma field generated by the asset prices.

Definition A.1.5 Consider a market with freely traded assets $A_1, \ldots, A_k$ and B. A portfolio $\underline{\theta}$ in $A_1, \ldots, A_k$ is **replicating portfolio** for B if

$$S_1^B(\omega_i) = \sum_{i=1}^{K} \theta_j S_1^j(\omega_i) \quad \forall i = 1, \ldots, N. \tag{A.15}$$

Replicating portfolios enable financial institutions that sell asset B to **hedge** by using the following strategy. For each share of B sold, buy θ_j shares of asset A_j and hold till time $t = 1$. Then at time $t = 1$, net gain = net loss =0. The institution selling B makes money by charging a fee or premium.

Definition A.1.6 A market is **complete** if every derivative security can be hedged.

Theorem A.1.2 (Second Fundamental Theorem of Asset Pricing): *Consider a market model that has a risk-neutral measure. The market is complete iff the risk-neutral measure is unique.*

Proof See Shreve page 232.

Remark Risk-neutral measure may be, and generally is, different from the physical measure. Under the physical measure, in general, expected returns of stocks are more than that of bonds. This is measured by the market price of risk. This is related to risk/return relations as in CAPM. But for pricing derivatives, one has to use the risk-neutral measure.

Remark Market is complete iff $A_1, \ldots, A_K$ **linearly** span the space of derivative securities. In general, the latter are random variables, that is functions, but not necessarily linear.

Remark We have done the whole treatment in a two-period setup. All the results can be generalized to a continuous-time setup. The mathematics get very involved, but the concepts are the same.

A.2 Brownian Motion

Why Brownian motion?

- In real markets, trading takes place in continuous time. Problems of pricing and hedging derivative securities in continuous-time markets require continuous-time models.
- In equilibrium, there exists a measure under which the discounted price process of any tradeable asset.
  A stochastic process, X_t is a martingale if $E(X_t|\mathcal{F}_s) = X_s$ for all $s < t$, where $\mathcal{F}_s$ is the σ field generated by $\{X_u, u \leq s\}$. In layman terms this means all the information that is available through the process at time s. In the single period model, this condition reduces to the existence of a risk-neutral measure under which A.7 holds.

- The prices of traded assets vary continuously with time and have finite quadratic variation. We shall define quadratic variation later in this section.

Brownian motion now rears its head for the following basic reason, a fundamental theorem of Paul Levy:

Theorem A.2.1 *Every continuous-time martingale with continuous paths and finite quadratic variation is a time-changed Brownian motion.*

Definition A.2.1 A standard Brownian (or a standard Wiener process) is a stochastic process $\{W_t\}_{t\geq 0}$ (that is, a family of random variables W_t , indexed by nonnegative real numbers t, defined on a common probability space (Ω, F, P)) with the following properties:

1. $W_0 = 0$
2. With probability 1, the function $t \to W_t$ is continuous in t.
3. The process has stationary, independent increments.
4. The increment $W_{t+s} - W_s$ has the Normal$(0, t)$ distribution.

The term independent increments means that for every choice of nonnegative real numbers $0 \leq s_1 < t_1 \leq s_2 < t_2 = \ldots = s_n < t_n < \infty$, the increment random variables $W_{t_1} - W_{s_1}, W_{t_2} - W_{s_2}, \ldots, W_{t_n} - W_{s_n}$, are jointly independent.

The term stationary increments means that for any $0 < s, t < \infty$ the increment $W_{t+s} - W_s$ has the same distribution as the increment $W_t - W_0 = W_t$.

It is not obvious that properties (1)–(4) in the definition of a standard Brownian motion are mutually consistent. So it is not a priori clear that a standard Brownian motion exists. The main issue is to show that properties (3)–(4) do not preclude the possibility of continuous paths. That it does exist was first proved by N. Wiener in about 1920.

A.2.1 Quadratic Variation of Brownian Paths

Fix $t > 0$, and let $\Pi = \{t_0, t_1, t_2, \ldots, t_n\}$ be a partition of the interval $[0, t]$. Let $X(s)$ be any process indexed by s. For any partition Π, define the quadratic variation of X relative to Π by

$$QV(X; \Pi) = \sum_{j=1}^{n} (X(t_j) - X(t_{j-1}))^2.$$

For each natural number n, define the $n-$th dyadic partition $D_n[0, t]$ to be the partition consisting of the dyadic rationals $k/2^n$ of depth n (here k is an integer) that are between 0 and t. So $D_n[0, t] = 0, 1/2^n, 2/2^n, \ldots, [t * 2^n]/2^n$ where $[x]$ is the floor function for the largest integer less than or equal to x. We have the following result.

Proposition A.2.1 *Let $\{W(t)\}_{t\geq 0}$ be a standard Brownian motion. For each $t > 0$, with probability one,*

$$\lim_{n\to\infty} QV(W; D_n[0,t]) = t.$$

The primary significance of this result is that it is the key to the Ito formula of stochastic calculus. It also implies that the Brownian path cannot be of bounded variation in any interval, and so in particular is not differentiable in any interval. Actually, even more is known: the Brownian path is nowhere differentiable.

Partial Proof. We will first prove the weaker statement that the convergence holds in probability. To simplify things, assume that $t = 1$. Then for each $n \geq 1$, the random variables $\xi_{n,k} := 2^n(W(k/2^n) - W((k-1)/2^n))^2$, $k = 1, 2, ..., 2^n$ are independent, identically distributed χ^2 with one degree of freedom (i.e., they are distributed as the square of a standard normal random variable).

Observe that $E\xi_{n,k} = 1$. Now $QV(W; D_n[0,1]) = 2^{-n}\sum_{k=1}^{2^n} \xi_{n,k}$. The right side of this equation is the average of $2n$ independent, identically distributed random variables, and so the Weak Law of Large Numbers implies convergence in probability to the mean of the χ^2 distribution with one degree of freedom, which equals 1. The stronger statement that the convergence holds with probability one can be deduced from the Chebyshev inequality and the Borel–Cantelli lemma.

A.3 Ito Integral

In ordinary calculus the Riemann integral is defined by a limiting procedure. One first defines integral of step functions in such a way that the integral represents the area under the curve. Then one extends this to a larger class of functions by approximating the original function by a series of step functions.

In Ito calculus we do a similar process. Step functions are replaced by simple processes. A random process $\theta(t)$ is called a **simple process** if there exist times $0 = t_0 << t_1 < \ldots < t_m < \infty$ and random variables $\xi_0, \ldots, \xi_m$ such that ξ_j is measurable relative to $\mathcal{F}_{t_j}$ and

$$\theta(s) = \sum_{j=0}^{m-1} \xi_j I(t_j < s \leq t_{j+1}).$$

We are avoiding details on measurability here and the interested reader is referred to Karatzas and Shreve (2012).

For a simple process θ, define the Ito integral with respect to Brownian motion W as

$$\int \theta(s)\mathrm{d}W_s = \sum_{j=0}^{m-1} \xi_j(W(t_j) - W(t_{j-1}).$$

This definition can now be extended to a wide class of stochastic processes as long as they can be approximated by simple processes. In fact any process H such that $P(\int_0^\infty H^2(s)\mathrm{d}s < \infty) = 1$ can be approximated by a sequence $\theta_1, \theta_2, \ldots$ of simple processes such that

$$\lim_{n\to\infty} \int_0^\infty E(\theta_n(s) - H(s))^2 \mathrm{d}s = 0.$$

For such a function H, the Ito integral is defined as the limit of the Ito integrals of the processes θ_n.

A.4 Ito's Formula

We start with Ito's lemma for a process that is a function of Brownian motion and time.

Lemma A.4.1 *Let $g(x, t)$ be a function that is twice continuously differentiable in x and once continuously differentiable in t. Let the partial derivatives be denoted by g_x, g_{xx} and g_t. Then*

$$g(W_t, t) = g(0, 0) + \int_0^t g_x(W_s, s)dW_s + \int_0^t g_t(W_s, s)ds + \frac{1}{2}\int_0^t g_{xx}(W_s, s)ds.$$

The first two terms are similar to what we see in ordinary calculus for a function of two variables. The addition of the third term comes from the quadratic variation of Brownian motion.

An Ito process is a stochastic process of the form $X_t = \int_0^t \mu_s \mathrm{d}s + \int_0^t \sigma_s \mathrm{d}W_s$. Ito's lemma can be extended to functions of Ito processes as

$$\begin{aligned} g(X_t, t) = g(0, 0) &+ \int_0^t g_x(X_s, s)\sigma_s \mathrm{d}W_s \\ &+ \int_0^t g_x(X_s, s)\mu_s \mathrm{d}s \\ &+ \int_0^t g_t(X_s, s)\mathrm{d}s + \frac{1}{2}\int_0^t g_{xx}(X_s, s)\sigma_s^2 \mathrm{d}s. \end{aligned}$$

A.5 The Black-Scholes Model

In 1973 Fisher Black and Myron Scholes ushered in the modern era of derivative securities with a seminal paper on the pricing and hedging of (European) call and put options. In its simplest form, the Black–Scholes–Merton model involves only two underlying assets, a riskless asset Bond and a risky asset Stock. The asset Bond appreciates at the short rate, or riskless rate of return r_t, which is assumed to be nonrandom, although possibly time-varying. Thus, the price B_t of the Bond at time t is assumed to satisfy the differential equation

$$\frac{\mathrm{d}B_t}{\mathrm{d}t} = r_t B_t, \tag{A.16}$$

whose unique solution for the value $B_0 = 1$ is

$$B_t = \exp\left(\int_0^t r_s \mathrm{d}s\right). \tag{A.17}$$

The share price S_t of the risky asset Stock at time t is assumed to follow a stochastic differential equation (SDE) of the form

$$\mathrm{d}S_t = \mu_t S_t \mathrm{d}t + \sigma S_t \mathrm{d}W_t, \tag{A.18}$$

where $\{W_t\}_{t\geq 0}$ is a standard Brownian motion, μ_t is a nonrandom (but not necessarily constant) function of t, and $\sigma > 0$ is a constant called the volatility of the Stock.

Theorem A.5.1 *If the drift coefficient function μ_t is bounded, then the SDE (A.18) has a unique solution with initial condition S_0, and it is given by*

$$S_t = S_0 \exp\left(\sigma W_t - \sigma^2(t/2) + \int_0^t \mu_s ds\right) \tag{A.19}$$

Moreover, under the risk-neutral measure, it must be the case that

$$r_t = \mu_t. \tag{A.20}$$

Proof The proof relies heavily on Ito's formula. Consider first the formula (A.19)for the share price of Stock. To see that this defines a solution to the SDE (A.18), apply Ito's formula to the function

$$u(x,t) = \exp\left(\sigma x - \sigma^2(t/2) + \int_0^t \mu_s \mathrm{d}s\right). \tag{A.21}$$

Next, to see that the drift coefficient μ_t must coincide with the riskless rate of return r_t in the risk-neutral world, recall that under the risk-neutral measure the discounted share price of Stock must be a martingale. The appropriate discount factor is B_t, since that makes the Bond price a martingale. So the discounted share price of Stock is

$$S_t^* = S_t/B_t = S_0 \exp\left(\sigma W_t - \sigma^2(t/2) + \int_0^t (\mu_s - r_s)\mathrm{d}s\right). \tag{A.22}$$

Applying Ito's formula once more, one finds that

$$\mathrm{d}S_t^* = \sigma S_t^* \mathrm{d}W_t + S_t^*(\mu_t - r_t)\mathrm{d}t. \tag{A.23}$$

In order that S_t^* be a martingale, it is necessary that the $\mathrm{d}t$ term be 0; this implies that $\mu_t = r_t$.

Corollary A.5.1 *Under the risk-neutral measure, the log of the discounted stock price at time t is normally distributed with mean* $\log S_0 - \sigma_t^2/2$ *and variance* $\sigma^2 t$.

A.6 The Black–Scholes Formula for the Price of a European Call Option

Recall that a European Call on the asset Stock with strike K and expiration date T is a contract that allows the owner to purchase one share of Stock at price K at time T. Thus, the value of the Call at time T is $(S_T - K)_+$. According to the first Fundamental Theorem of Arbitrage Pricing, the price of the asset Call at time $t = 0$ must be the discounted expectation, under the risk-neutral measure, of the value at time $t = T$, which, by Proposition 1, is

$$C(S_0, 0) = C(S_0, 0; K, T) = E(S_T^* - K/B_T)_+ \tag{A.24}$$

where S_T^* has the distribution specified in Corollary 1. A routine calculation, using integration by parts, shows that $C(x, T; K)$ may be rewritten as

$$C(x, 0; K, T) = x\Phi(z) - \frac{K}{B_T}\Phi(z - \sigma\sqrt{T}), \tag{A.25}$$

where

$$z = \frac{\log(xB_t/K) + \sigma^2 t/2}{\sigma\sqrt{T}} \tag{A.26}$$

and Φ is the cumulative normal distribution function.

Observe that the derivation works essentially unchanged for all European-style derivative securities. Let $F: \mathbb{R} \mapsto \mathbb{R}$ be a function and consider the derivative security whose value at $t = T$ is $F(S_T)$. Then the value at time $t = 0$ of this derivative security is

$$V_0 = E\, B_T^{-1} F(B_T S_T^*), \tag{A.27}$$

where S_T^* has the lognormal distribution specified in Corollary A.5.1.

Appendix B
Introduction to R

Here you will see an introduction to the R programming environment and its most essential features. If you are already familiar with R, then you may skip this part. The R is an open-source language and environment for statistical computing and graphics. A vast and active worldwide research community uses R. Most commercial statistical software cost thousands, if not tens of thousands of dollars. R is free! If you are a teacher, student, consultant, or a bootstrap startup, the benefits are obvious. It runs on wide variety of platforms including Windows, UNIX, and MacOS. It is a comprehensive statistical platform, offering all manner of state of the art analytical technique. You can use R to create beautiful graphics.

Starting R is straightforward. The process will depend on your computing platform. You can launch it from a system menu, by double-clicking the R icon, or by entering the command R at the system command line. This will either produce a console window like in Fig. B.1 or cause R to start up as an interactive program in the current terminal of Linux or Mac. R works fundamentally by the question-and-answer model. You enter a line with a command and press 'Enter'. Then the program does something and prints the result.

B.1 R Language Essentials

Assignments

To assign the value to the variable x, you can write `x <- 5` in the R-console and then enter. You can write non-executable comments after hash sign '#'.

```
x<-5 # Assign value 5 in variable x
```

The two characters <- should be read as a single symbol; an arrow pointing to the variable x to which the value 5 is assigned. This is called the *assignment operator*. You can try the following operations.

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

R. Sen and S. Das, *Computational Finance with R*, Indian Statistical Institute Series,
https://doi.org/10.1007/978-981-19-2008-0

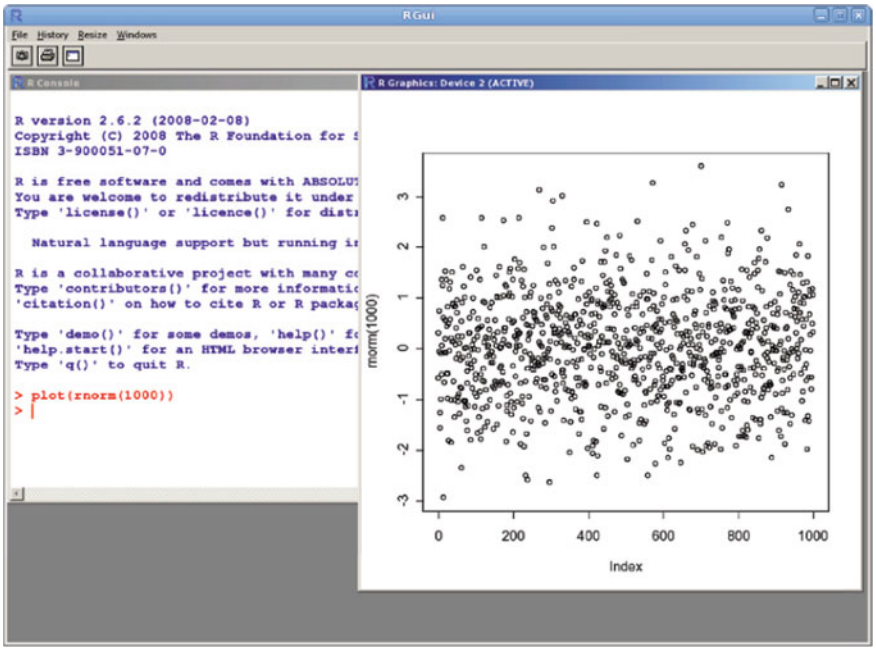


Fig. B.1 Screen image of R for Windows

```
y<-3 # Assign value 3 in variable y
x+y # Add x and y : output should be 8

## [1] 8

x-y # Subtract y from x : output should be 2

## [1] 2

x*y # Multiply x and y : output should be 40

## [1] 15

x/y # Divide x by y : output should be ~ 1.667

## [1] 1.666667
```

B.2 Vector

You cannot do much analysis with single numbers. Rather, you have to handle prices of group of stocks. R is an object-oriented language and one strength of R is that it can handle entire data vectors as single objects. A data vector is simply an array of numbers, and a vector variable can be constructed like this:

```
SnP500.ETF<-c(235.33,235.48,234.78,235.44,235.20)
SnP500.ETF

## [1] 235.33 235.48 234.78 235.44 235.20
```

The construct `c(...)` is used to define vectors. It combines values into a vector. This is not the best way to enter data; nor the preferred way; but short vectors are used for many other purposes, and the `c(...)` construct is used extensively. The calculation of the mean and standard deviation of the `SnP500.ETF` variable can be implemented easily in the following ways. First calculate means $\bar{x} = \sum_{i=1}^{n} x_i/n$:

```
sum(SnP500.ETF) ## Add all the values

## [1] 1176.23

length(SnP500.ETF) ## n or sample size

## [1] 5

xbar <- sum(SnP500.ETF)/length(SnP500.ETF) ## sample mean
xbar

## [1] 235.246
```

Now proceed with the calculation of SD $= \sqrt{\sum_i (x_i - \bar{x})^2/(n-1)}$. We do this in steps to see the individual components. The deviations from the mean are

```
SnP500.ETF-xbar

## [1]  0.084  0.234 -0.466  0.194 -0.046
```

Notice `xbar`, has length 1, is recycled and subtracted from each element of `SnP500.ETF`. The squared deviations will be

```
(SnP500.ETF-xbar)^2

## [1] 0.007056 0.054756 0.217156 0.037636 0.002116
```

The sum of squared deviations is similarly obtained with

```
sum((SnP500.ETF-xbar)^2)

## [1] 0.31872
```

and all the standard deviation becomes

```
sqrt(sum((SnP500.ETF - xbar)^2)/(length(SnP500.ETF) - 1))

## [1] 0.2822765
```

Of course, since R is a statistical program, these calculations are already built into the available functions, and you get the same results just by entering

```
mean(SnP500.ETF)

## [1] 235.246
sd(SnP500.ETF)

## [1] 0.2822765
```

You have already seen numeric vectors. Now you will see the character vectors and logical vectors. A *character vector* is a vector of text strings, whose elements are specified and printed in quotes:

```
c("IBM","MSFT","AAPL","INTC")

## [1] "IBM"  "MSFT" "AAPL" "INTC"
```

You can use either single or double-quote symbols. As long as the left quote is the same as the right quote, it works.

```
c('IBM','MSFT','AAPL','INTC')

## [1] "IBM"  "MSFT" "AAPL" "INTC"
```

Logical vectors can take the value TRUE or FALSE. In input, you may use the convenient abbreviations T and F (you should be careful not to redefine them). Logical vectors are constructed using the c function just like the other vector types:

```
c(T,T,F,T)

## [1]  TRUE  TRUE FALSE  TRUE
```

Seldom, you will specify logical vectors in the manner above. It is much more common to use single logical values to turn an option on or off in a function call. Vectors of more than one value most often result from *relational expressions*:

```
SnP500.ETF > 235

## [1]  TRUE  TRUE FALSE  TRUE  TRUE
```

Functions that create vectors
Here we introduce three functions, `c`, `seq`, and `rep`, that are used to create vectors in various situations. You already used `c`, it is short for 'concatenate', joining items end to end.

```
x <- c(1, 2, 3)
y <- c(13,17,19)
c(x,y,23)

## [1]  1  2  3 13 17 19 23
```

The second function, `seq` ('sequence'), is used for equidistant series of numbers. Writing

```
seq(15,25)

##  [1] 15 16 17 18 19 20 21 22 23 24 25
```

yields, the integers from 15 to 25. If you want the sequence in jumps of 2, write

```
seq(15,25,2)

## [1] 15 17 19 21 23 25
```

You can use the third function, `rep` ('replicate'), to generate repeated values. You can use either of the two variants, depending on whether the second argument is a vector or a single number.

```
prime<-c(7,11,13)
rep(prime,3)

## [1]  7 11 13  7 11 13  7 11 13

rep(prime,1:3)

## [1]  7 11 11 13 13 13

rep(prime,c(2,3,5))

##  [1]  7  7 11 11 11 13 13 13 13 13

## 1st number in prime is 7 which repeats 2 times
## 2nd number in prime is 11 which repeats 3 times
## 3rd number in prime is 13 which repeats 5 times
```

B.3 Matrices and Arrays

A matrix in mathematics is just a two-dimensional array of numbers. In R, matrices and arrays are represented as vectors with dimensions:

```
x <- 1:12
x

##  [1]  1  2  3  4  5  6  7  8  9 10 11 12

dim(x)<-c(4,3)
x

##      [,1] [,2] [,3]
## [1,]    1    5    9
## [2,]    2    6   10
## [3,]    3    7   11
## [4,]    4    8   12
```

The `dim` function changes the dimension attribute of `x`, causing R to treat the vector of 12 numbers as a 4×3 matrix. Notice that the storage is column-major; that is, the elements of the first column are followed by those of the second, etc. A convenient way for you to create matrices is to use the `matrix` function:

```
matrix(1:12,nrow=4,byrow=TRUE)
```

```
##      [,1] [,2] [,3]
## [1,]    1    2    3
## [2,]    4    5    6
## [3,]    7    8    9
## [4,]   10   11   12
```

Notice how the `byrow = TRUE` switch causes the matrix to be filled in a rowwise fashion rather than columnwise. You can use some very useful functions that operate on matrices include `rownames`, `colnames`, and the transposition function `t` (notice the lowercase `t` as opposed to uppercase `T` for `TRUE`) which turns rows into columns and vice versa.

```
x <- matrix(1:12,nrow=4,byrow=T)
rownames(x) <- LETTERS[1:4]
x
```

```
##   [,1] [,2] [,3]
## A    1    2    3
## B    4    5    6
## C    7    8    9
## D   10   11   12
```

```
t(x)
```

```
##      A B C  D
## [1,] 1 4 7 10
## [2,] 2 5 8 11
## [3,] 3 6 9 12
```

The character vector `LETTERS` is a built-in variable that contains the capital letters A-Z. Similarly you can use some other useful vectors like `letters`, `month.name`, and `month.abb` with lowercase letters, month names, and abbreviated month names. You can join vectors together, columnwise or rowwise, using the built-in `cbind` and `rbind` functions:

```
A <- 1:4
B <- 5:8
C <- 9:12
cbind(A,B,C)
```

```
##      A B  C
## [1,] 1 5  9
## [2,] 2 6 10
## [3,] 3 7 11
## [4,] 4 8 12
```

```
## An alternative way to write the same thing as
cbind(A=1:4,B=5:8,C=9:12)
```

```
##      A B  C
## [1,] 1 5  9
## [2,] 2 6 10
## [3,] 3 7 11
## [4,] 4 8 12
```

```
## You can also use rbind or row-wise binding
rbind(A=1:4,B=5:8,C=9:12)
```

```
##   [,1] [,2] [,3] [,4]
## A    1    2    3    4
## B    5    6    7    8
## C    9   10   11   12
```

B.4 Lists

A list is a tool that can be used to store different types of information in one object. It can also be defined as a variable containing different other variables.

```
IBM <- c(172.14,172.45,172.88,174.52,174.50)
AAPL<- c(143.34,143.66,144.02,144.77,143.70)

## Define the list as follows:

MyList<-list(SnP500.ETF.value = SnP500.ETF , IBM_price =
IBM , AAPL_price = AAPL) MyList


## $SnP500.ETF.value
## [1] 235.33 235.48 234.78 235.44 235.20
##
## $IBM_price
## [1] 172.14 172.45 172.88 174.52 174.50
##
## $AAPL_price
## [1] 143.34 143.66 144.02 144.77 143.70
```

The components of the list are named according to the argument names used in list. You can extract the named components like this:

```
MyList$IBM_price


## [1] 172.14 172.45 172.88 174.52 174.50
```

Following code shows that you can store very different objects like string and/or matrices of numbers in single object using `list`.

```
x<-c(LETTERS[1:4])
y<-matrix(1:12,nrow=3)
list(x=x,y=y)


## $x
## [1] "A" "B" "C" "D"
##
## $y
##      [,1] [,2] [,3] [,4]
## [1,]    1    4    7   10
## [2,]    2    5    8   11
## [3,]    3    6    9   12
```

B.5 Data Frame

A data frame corresponds to a 'data matrix' or a 'dataset.' It is a list of vectors of the same length. Also, it has a unique set of row names.

```
d <- data.frame(SnP500.ETF,IBM,AAPL)
d

##   SnP500.ETF    IBM   AAPL
## 1     235.33 172.14 143.34
## 2     235.48 172.45 143.66
## 3     234.78 172.88 144.02
## 4     235.44 174.52 144.77
## 5     235.20 174.50 143.70
```

As with lists, components (i.e., individual variables) can be accessed using the $ notation:

```
d$IBM

## [1] 172.14 172.45 172.88 174.52 174.50
```

B.6 Indexing

If you need a particular element in a vector, for instance the price of IBM on 3rd day, you can do

```
d$IBM[3]

## [1] 172.88
```

You can use the box brackets for the selection of data, also known as *indexing* or *subsetting*. You can use the vector to index vector in the following way. Suppose you want 1st, 3rd and 4th row of the *data frame*, you can do the same process using the following code:

```
row_Indx<-c(1,3,4)
d[row_Indx,]

##   SnP500.ETF    IBM   AAPL
## 1     235.33 172.14 143.34
## 3     234.78 172.88 144.02
## 4     235.44 174.52 144.77
```

If you want to access 2nd and 3rd column, then you can do it in the following way:

```
col_Indx<-c(2,3)
d[,col_Indx]

##      IBM   AAPL
## 1 172.14 143.34
## 2 172.45 143.66
## 3 172.88 144.02
## 4 174.52 144.77
## 5 174.50 143.70
```

An interesting feature of R is the possibility of negative indexing. You can get all rows except nos. 1, 3, and 4 by writing

```
d[-row_Indx,]

##   SnP500.ETF    IBM   AAPL
## 2     235.48 172.45 143.66
## 5     235.20 174.50 143.70
```

You should not try to mix the positive and negative indices. That would be confusing.

B.7 Looping

Like any standard programming language, R allows you conditional execution and looping constructs.

```
i <- 1
while(i < 6){
   print(i)
   i <- i+1
}
```

```
## [1] 1
## [1] 2
## [1] 3
## [1] 4
## [1] 5
```

Notice the construction of `while`; the expression evaluates as long as the condition is `TRUE`. You can implement a variation of the same algorithm with the test at the end of the loop with `repeat` construction.

```
i <-1
repeat{
  print(i)
  i <- i+1
  if(i >=6) break
}
```

```
## [1] 1
## [1] 2
## [1] 3
## [1] 4
## [1] 5
```

This code illustrates an interesting flow control: (a) several expressions held together between curly braces; (b) an `if` construction for conditional execution; and (c) a `break` expression, which causes the loop to exit.

The most frequently used looping construct is `for`, which loops over a fixed set of values as in the following example, which plots a set of power curves on the unit interval.

```
for(i in 1:5){
  print(i)
}


## [1] 1
## [1] 2
## [1] 3
## [1] 4
## [1] 5
```

B.8 Functions

One of the great strengths of R is the user's ability to add functions. In fact, many of the functions in R are functions of functions. The typical structure of a function is provided below.

```
name_of_the_function <- function(arg1,arg2,...){
  statements
  return(object)
}
```

First, you must assign a suitable name to a function. Then you provide an argument with open brackets (although you do not have to) and a body of code that does something. In the end, it usually should (although does not have to) return an object out of the function. The principal idea behind functions is that the objects that are inside the function are local to the environment of the function. But you can `return` the value of the object from the function, that means you can pass the value of a function into the global environment.

You can write a function, where the argument is the prices of the stock or the value of the index. Inside the function, it will compute the log-return of the prices, then compute the average return, standard deviation (aka. volatility) and return it as a `list`.

```
## Name of the function is "firts_stock_analysis"
firts_stock_analysis<-function(x){

  ## x is the price series
  ## calculate log-return
  ## diff() function calculate the consecutive
  ## differences of log(x) where
  ## log(x) is the log of price-series

  rt<-diff(log(x))*100

  ## calculate average return
  avg.return<-mean(rt)

  ## calculate standard deviation (aka. volatility)
  volatility<-sd(rt)

  ## combine result as list
  result<-list(avg.return=avg.return
               ,volatility=volatility)

  return(result)
}

firts_stock_analysis(x=SnP500.ETF)


## $avg.return
## [1] -0.01381421
##
## $volatility
## [1] 0.2457182

firts_stock_analysis(x=IBM)


## $avg.return
## [1] 0.3404161
##
## $volatility
## [1] 0.4173073

firts_stock_analysis(x=AAPL)


## $avg.return
## [1] 0.06270906
##
## $volatility
## [1] 0.5527996
```

The advantage of writing a `function` is apparent from the above code. You can write one single piece of code and call that code again and again if you know the task is going to be repetitive.

B.9 Download Stock Market Data Using R

One way to check the efficiency of your investment strategy is to check its performance on historical data. So get the publicly available data from the internet is an important task. There are quite a few R packages which have the built-in function to download the data. One such function in `quantmod` package is `getSymbols`, which download the data from Google/Yahoo server.

```
library(quantmod)
SPY<-getSymbols(Symbols='SPY'
            ,src='google'
            ,from = '2017-04-01'
            ,to = '2017-04-07'
            ,auto.assign=FALSE)

## Previously we entered these values manually
## in SnP500.ETF variable.
SnP500.ETF


## [1] 235.33 235.48 234.78 235.44 235.20

## Check it out with SPY.Close column
SPY
```

Notice that `Symbols = 'SPY'`, typically it is a character string giving the name of the stock or index, for which you would like to download the data. It is a unique ID. For SPDR S&P 500 Trust ETF, the name is 'SPY'. For IBM it is 'IBM,' and for Apple Inc., it is 'AAPL'. Following a piece of code download price of Apple Inc. from Jan 1, 2017, and draw a bar chart in Fig. B.2.

```
AAPL<-getSymbols('AAPL'
            ,src='google'
            ,from = '2017-01-01'
            ,to = '2017-01-25')

candleChart(AAPL) ## Draw candle chart
addBBands() ## Draw Bollinger Bands
```



Fig. B.2 Candle chart of Apple stocks with Bollinger Bands

Appendix C
Extreme Value Theory (EVT) in Finance

In this chapter we present the main results in extreme value theory (EVT) that are required for estimation of VaR and ES. For a complete treatment of EVT, see (Embrechts et al., 2013).

Extreme Value Theory has two significant results. First, the asymptotic distribution of a series of maxima (minima) is modeled and under certain conditions, the distribution of the standardized maximum of the series is shown to converge to the Gumbel, Frechet, or Weibull distributions. A standard form of these three distributions is called the generalized extreme value (GEV) distribution. The second significant result concerns the distribution of excess over a given threshold, where one is interested in modeling the behavior of the excess loss once a high threshold (loss) is reached. This result is used to estimate the very high quantiles (0.999 and higher). EVT shows that the limiting distribution is a generalized Pareto distribution (GPD).

Fisher Tippet Theorem: Let $X_1, \ldots, X_n$ be a sequence of independent and identically distributed (iid) random variables and $M_n = \max(X_1, \ldots, X_n)$. If there exist constants $c_n > 0, d_n \in R$, and some non-degenerate distribution H such that $c_n^{-1}(M_n - d_n)$ converges in law (weakly) to H, then H belongs to one of the following families of distributions.

$$\Phi_\alpha(x) = \exp[-x^{-\alpha}]I(x > 0) \quad \text{for} \quad \alpha > 0 \quad \text{Frechet}, \tag{C.1}$$

$$\Xi_\alpha(x) = \exp[-(-x)^{\alpha}]I(x \leq 0) \quad \text{for} \quad \alpha > 0 \quad \text{Weibull}, \tag{C.2}$$

$$\Lambda(x) = \exp[-\exp(-x)] \quad \text{Gumbel}. \tag{C.3}$$

In this case we say that F belongs to the maximum domain of attraction of H and write $F \in \text{MDA}(H)$. This result is very significant, since the asymptotic distribution of the maximum, if it exists, always belongs to one of these three distributions, whatever the original distribution may be. The asymptotic distribution of the maximum can be estimated without making any assumptions about the nature of the original distribution of the observations (unlike with parametric VaR methods), that distribution being generally unknown.

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

R. Sen and S. Das, *Computational Finance with R*, Indian Statistical Institute Series,
https://doi.org/10.1007/978-981-19-2008-0

The Frechet distribution corresponds to fat-tailed distributions and has been found to be the most appropriate for fat-tailed financial data.

For a cumulative distribution function (cdf) F, let us introduce the following notations:

1. $\overline{F}(x) = 1 - F(x)$
2. $F_u(x) = Pr[X - u \leq x | X > u] = (F(x+u) - F(u))/(1 - F(u))$

A positive function L is slowly varying if, for any $t > 0$,

$$\lim_{x \to \infty} \frac{L(tx)}{L(x)} = 1. \tag{C.4}$$

Next we define regularly varying (RV) cdfs. For $\xi > 0$, $F \in \mathrm{RV}(\xi)$ if for some slowly varying function L,

$$\overline{F}(x) = x^{-1/\xi} L(x)$$

Theorem (3.3.7 Embrechts et al.) The df F belongs to the maximum domain of attraction of Φ_α if and only if $F \in \mathrm{RV}(\alpha)$.

For $\xi > 0$, the cdf of a generalized Pareto distribution (GPD) is given by

$$G_{\xi,\beta}(x) = 1 - \left(1 + \frac{\xi x}{\beta}\right)^{-1/\xi}. \tag{C.5}$$

Theorem (Pickands–Balkema–de Haan) $F \in \mathrm{RV}(\xi)$ if and only if there exists a function $\beta(u)$ such that

$$\lim_{u \to \infty} \sup_{0 \leq x < \infty} |F_u(x) - G_{\xi,\beta(u)}(x)| = 0. \tag{C.6}$$

The EVT uses PBD theorem in order to find an approximation to the tail of the loss data. Given a loss dataset, one selects a threshold u for which a GPD is a good approximation of the tail. The losses below the threshold are modeled using an empirical distribution; the losses above the threshold are modeled using the GPD.

Now we proceed to discuss the convergence required for quantiles and shortfall estimation. For a cdf F, denote by EVT$[F, u]$ the approximation to F obtained by using the EVT tail:

$$\mathrm{EVT}[F,u](x) = \begin{cases} F(x) & x \leq u \\ F(u) + (1 - F(u)) G_{\xi,\beta(u)}(x - u) & x > u \end{cases}$$

EVT approximation leads to uniform relative quantile (URQ) convergence if, for any $\epsilon > 0$, there exists a threshold u_0 such that for any $u \geq u_0$ and any quantile q,

$$\begin{aligned} &(1 - \epsilon)\mathrm{Quantile}(\mathrm{EVT}[F,u], q) \\ &\quad \leq \mathrm{Quantile}(F, q) \leq (1 + \epsilon)\mathrm{Quantile}(\mathrm{EVT}[F,u], q). \end{aligned} \tag{C.7}$$

For $F \in \mathrm{RV}(\xi)$, EVT approximation leads to the URQ convergence only if

$$0 < \lim_{x \to \infty} \overline{F}(x) x^{1/\xi} < \infty. \tag{C.8}$$

URQ convergence implies convergence in mean. That is, URQ convergence of EVT approximation implies

$$(1 - \epsilon)\mathrm{Mean}(\mathrm{EVT}[F, u]) \leq \mathrm{Mean}(F) \leq (1 + \epsilon)\mathrm{Mean}(\mathrm{EVT}[F, u]). \tag{C.9}$$

Bibliography

Andersen, T. G., Bollerslev, T., Diebold, F. X., & Labys, P. (2000). Great realizations. *Risk, 13*, 105–108.

Artzner, P., Delbaen, F., Eber, J.-M., & Heath, D. (1999). Coherent measures of risk. *Mathematical Finance, 9*(3), 203–228.

Bacon, C. R. (2008). *Practical portfolio performance measurement and attribution, with CD-ROM*. John Wiley & Sons.

Barndorff-Nielsen, O. E., Hansen, P. R., Lunde, A., & Shephard, N. (2008). Designing realized kernels to measure the ex post variation of equity prices in the presence of noise. *Econometrica, 76*(6), 1481–1536.

Berger, J. O. (1993). *Statistical decision theory and Bayesian analysis* (2nd ed.). Springer Series in Statistics.

Boudt, K., Peterson, B. G., & Croux, C. (2008). Estimation and decomposition of downside risk for portfolios with non-normal returns. *Journal of Risk, 11*(2), 79–103.

Bousquet, O., Boucheron, S., & Lugosi, G. (2004). Introduction to statistical learning theory. In *Advanced lectures on machine learning* (pp. 169–207). Springer.

Box, G. E. P. (1958). A note on the generation of random normal deviates. *Annual Mathematical Statistics, 29*, 610–611.

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32.

Breiman, L., Friedman, J., Stone, C. J., & Olshen, R. A. (1984). *Classification and regression trees*. CRC Press.

Christoffersen, P. F. (1998). Evaluating interval forecasts. *International Economic Review*, 841–862.

Cont, R. (2001). Empirical properties of asset returns: Stylized facts and statistical issues. *Quantitative Finance, 1*, 223–236.

Cont, R., Deguest, R., & Scandolo, G. (2010). Robustness and sensitivity analysis of risk measurement procedures. *Quantitative Finance, 10*(6), 593–606.

Das, S., & Dey, D. (2006). On Bayesian analysis of generalized linear models using jacobian technique. *The American Statistician, 60*(3), 265–268.

Das, S., & Dey, D. (2013). On dynamic generalized linear models with applications. *Methodology and Computing in Applied Probability, 15*, 407–421.

Das, S., & Dey, D. K. (2010). On Bayesian inference for generalized multivariate gamma distribution. *Statistics & Probability Letters, 80*(19):1492–1499. ISSN: 0167-7152.

Das, S., Halder, A., & Dey, D. K. (2017). Regularizing portfolio risk analysis: A Bayesian approach. *Methodology and Computing in Applied Probability, 19*, 865–889.

Dowd, K. (2007). *Measuring market risk*. John Wiley & Sons.

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

R. Sen and S. Das, *Computational Finance with R*, Indian Statistical Institute Series,
https://doi.org/10.1007/978-981-19-2008-0

Embrechts, P., Klüppelberg, C., & Mikosch, T. (2013). *Modelling extremal events: For insurance and finance* (Vol. 33). Springer Science & Business Media.

Epps, T. W. (1979). Comovements in stock prices in the very short run. *Journal of the American Statistical Association, 74*(366a), 291–298.

Fishman, G. S., & Moore, L. R., III. (1986). An exhaustive analysis of multiplicative congruential random number generators with modulus 2^{31}-1. *SIAM Journal on Scientific and Statistical Computing, 7*(1), 24–45.

Friedman, J., Hastie, T., & Tibshirani, R. (2009). *The elements of statistical learning* (2nd ed.). New York: Springer Series in Statistics.

Hastings, C. (1955). *Approximations for digital computers*. Princeton University.

Head, M. L., Holman, L., Lanfear, R., Kahn, A. T., & Jennions, M. D. (2015). The extent and consequences of p-hacking in science. *PLOS Biology, 13*(3).

Hein, T., & Hofmann, B. (2003). On the nature of ill-posedness of an inverse problem arising in option pricing. *Inverse Problems, 19*(6), 1319.

Huang, W., & Zhang, Y. (2014). Asymmetry index of stock price fluctuations. *Journal of Global Economics*.

Jacod, J., & Shiryaev, A. N. (2003). *Limit theorems for stochastic processes* (2nd ed.). Grundlehren der Mathematischen Wissenschaften.

Jacod, J., Li, Y., Mykland, P. A., Podolskij, M., & Vetter, M. (2009). Microstructure noise in the continuous case: The pre-averaging approach. *Stochastic Processes and Their Applications, 119*(7), 2249–2276.

James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *The elements of statistical learning* (2nd ed.). Springer.

Karatzas, I., & Shreve, S. (2012). *Brownian motion and stochastic calculus* (Vol. 113). Springer Science & Business Media.

Krishnan, C. N. V., Petkova, R., & Ritchken, P. (2009). Correlation risk. *Journal of Empirical Finance, 16*(3), 353–367.

Kupiec, P. (1995). Techniques for verifying the accuracy of risk measurement models. *The Journal of Derivatives, 3*(2), 73–84.

L'ecuyer, P. (1988). Efficient and portable combined random number generators. *Communications of the ACM, 31*(6), 742–751.

Litterman, R., Gumerlock, R., & Warburg Dillon Read, S. B. C. (1998). *The practice of risk management: Implementing processes for managing firm-wide market risk*. Euromoney Institutional Investor Plc.

Markowitz, H. (1952). Portfolio selection. The. *Journal of Finance, 7*(1), 77–91.

Marsaglia, G., & Bray, T. A. (1964). A convenient method for generating normal variables. *SIAM Review, 6*(3), 260–264.

Matsumoto, M., & Nishimura, T. (1998). Mersenne twister: a 623-dimensionally equidistributed uniform pseudo-random number generator. *ACM Transactions on Modeling and Computer Simulation (TOMACS), 8* (1), 3–30.

McCullagh, P., & Nelder, J. (1989). *Generalized linear models* (2nd ed.). Chapman and Hall/CRC.

Merton, R. C. (1980). On estimating the expected return on the market: An exploratory investigation. *Journal of Financial Economics, 8*(4), 323–361.

Michaud, R. O. (1989). The markowitz optimization enigma: Is 'optimized' optimal? *Financial Analysts Journal, 45*(1), 31–42.

Mitchell, T. M. (2006). *The discipline of machine learning* (Vol. 9). School of Computer Science, Machine Learning Department, Carnegie Mellon University.

Pérignon, C., & Smith, D. R. (2010). The level and quality of value-at-risk disclosure by commercial banks. *Journal of Banking & Finance, 34*(2), 362–377.

Philippe, J. (2001). *Value at risk: The new benchmark for managing financial risk*. McGraw-Hill Professional.

Renò, R. (2003). A closer look at the epps effect. *International Journal of theoretical and applied finance, 6*(01), 87–102.

Roy, A. D. (1952). Safety first and the holding of assets. *Econometrica: Journal of the Econometric Society*, 431–449.

Sambasivan, R., Das, S., & Sahu, S. K. (2020). A Bayesian perspective of statistical machine learning for big data. *Computational Statistics, 35*, 893–930.

Sen, R., & Subramaniam, M. (2019). Stylized Facts of the Indian Stock Market. Asia-Pacific Financial Markets, 26(4), 479–493. Springer.

Sharpe, W. F. (1994). The sharpe ratio. *Journal of Portfolio Management, 21*(1), 49–58.

Shinal, J. (2017). Google CEO Sundar PIchai: Moving all directions at once. https://www.cnbc.com/2017/05/18/google-ceo-sundar-pichai-machine-learning-big-data.html, May 2017.

Shreve, S. E. (Ed.). (2004). *Stochastic calculus for finance II continuous time model*. Springer.

Thompson, S. (2013). The stylised facts of stock price movements. *The New Zealand Review of Economics and Finance, 1*, 50–77.

UC Berkeley (2018). *Statistical machine learning*. Univeristy of California at Berkeley. https://www.stat.berkeley.edu/~statlearning/.

Van Loan, C. F., & Golub, G. (1996). *Matrix computations (Johns Hopkins studies in mathematical sciences)*.

Wichmann, B. A., & Hill, I. D. (1982). An efficient and portable pseudo-random number generator. *Journal of the Royal Statistical Society: Series C (Applied Statistics), 31*(2), 188–190.

Wichura, M. J. (1988). Algorithm as 241: The percentage points of the normal distribution. *Journal of the Royal Statistical Society. Series C (Applied Statistics), 37*(3), 477–484.

Xiu, D. (2010). Quasi-maximum likelihood estimation of volatility with high frequency data. *Journal of Econometrics, 159*(1), 235–250.

Yeh, I.-C. (2009). *Default of credit card clients data set*. https://archive.ics.uci.edu/ml/datasets/default+of+credit+card+clients

Zangari, P. (1996). A var methodology for portfolios that include options. *RiskMetrics Monitor, 1*, 4–12.

Zhang, L. (2011). Estimating covariation: Epps effect, microstructure noise. *Journal of Econometrics, 160*(1), 33–47.

Zhang, L., Mykland, P. A., & Aït-Sahalia, Y. (2005). A tale of two time scales: Determining integrated volatility with noisy high-frequency data. *Journal of the American Statistical Association, 100*(472), 1394–1411.

Žiković, S., & Aktan, B. (2011). Decay factor optimisation in time weighted simulation-evaluating var performance. *International Journal of Forecasting, 27*(4), 1147–1159.

Index

Symbols
`c(...)`, 331
`data.frame`, 338
`for`, 339
`function`, 333, 341
`list`, 336
`repeat`, 339
`while`, 339

A
Adjusted R-square, 264
Alpha, 269
Asset Pricing, 257

B
Beta, 261, 266
Beta Hedging, 267
Beta Negative, 266

C
Capital Market Line, 258
CAPM, 257, 258

D
Diversification, 272

F
Factor Models, 270

L
Looping in `R`, 339

O
Outlier, 268

S
Security Market Line, 265
Systematic Risk, 261

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

R. Sen and S. Das, *Computational Finance with R*, Indian Statistical Institute Series,
https://doi.org/10.1007/978-981-19-2008-0